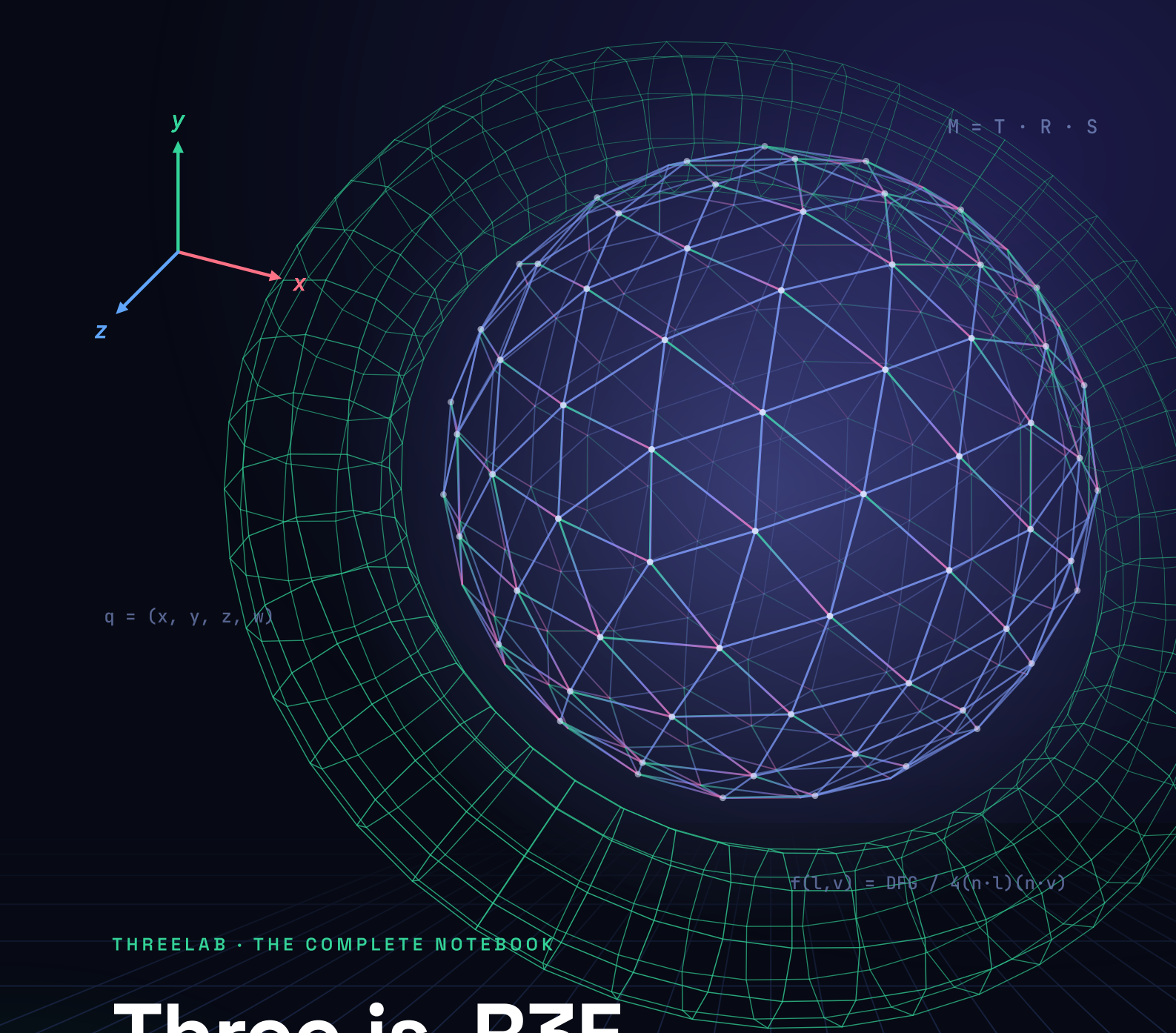


THREELAB · THE COMPLETE NOTEBOOK

Three.js, R3F & 3D Graphics

Concepts, definitions, the mathematics with worked calculations, three.js and React Three Fiber implementations, modern GPU techniques, and a full interview question bank, in one study notebook.

Maths · Rendering · PBR · Shaders · WebGPU · TSL · R3F · Performance · Interviews

Edition 1 · October 2026 · three.js r186 · React Three Fiber 9

How to use this notebook

This is a study notebook, not a tutorial. Each chapter explains one idea from first principles: what it is, why it exists, the maths behind it with numbers worked out, and then how to build it in plain three.js and in React Three Fiber. Read it front to back once, then use it as a reference and a revision guide before interviews.

Definition
The precise meaning of a term. Learn these word for word.

Formula
The equation, with every symbol explained.

Worked example
The same formula with real numbers, step by step.

Intuition
A picture or analogy that makes the idea stick.

Tip
What experienced developers do in practice.

Pitfall
A common bug and how to avoid it.

Deep dive
Extra depth for advanced readers. Safe to skip at first.

New in recent three.js
Changes from r180 to r186 that older tutorials miss.

Interview question
A question you may be asked, with a model answer.

Conventions

- **Coordinates.** three.js is right-handed with **+Y up**; the camera looks down its own **−Z**. One unit is treated as one metre.
- **Vectors and matrices.** Bold lower-case letters are vectors ($\mathbf{v}$), capitals are matrices (M). Vectors are columns, so a matrix transforms a point as $M\mathbf{p}$ and transforms read **right to left**.
- **Angles.** three.js uses **radians** everywhere except `PerspectiveCamera.fov`, which is in degrees.
- **Code.** Code labelled `three.js` is plain JavaScript with `import * as THREE from "three"`. Code labelled `R3F` is a React component inside a `<Canvas>`. Shader code is labelled GLSL, WGSL or TSL.
- **Versions.** Written against three.js r186, React Three Fiber 9, drei 10 and React 19. Where an API changed recently, a *New in recent three.js* box says so.

Companion to the ThreeLab course, where every idea here has a live, editable 3D example. Notebook text is in Indian English; API names keep their original spelling (for example `color`).

Contents

FOUNDATIONS OF 3D GRAPHICS

01 The 3D Mental Model 6
02 The GPU Rendering Pipeline 15
03 Vectors 25
04 Matrices and Transforms 35
05 Rotations and Quaternions 45
06 Cameras and Projection 54

BUILDING SCENES WITH THREE.JS

07 Scene Graph, Renderer and the Frame Loop 67
08 Geometry and Meshes 77
09 Materials and Classic Lighting Models 87
10 Lights and Shadows 96
11 Textures and UV Mapping 106
12 Colour, HDR and Tone Mapping 117
13 Physically Based Rendering 125

MOTION AND INTERACTION

14 Raycasting, Picking and Collision Maths 139
15 Animation and Skinning 149
16 Assets — glTF and the Loading Pipeline 159
17 Physics and Simulation 167

SHADERS AND THE GPU

18 Shaders and GLSL 177
19 Render Targets and Post-processing 187
20 WebGPU, TSL and Compute 196

REACT THREE FIBER

21 React Three Fiber in Depth 206
22 The R3F Ecosystem and App Architecture 215

ADVANCED RENDERING AND PRODUCTION

23 Performance and Optimisation 222
24 Advanced Rendering Techniques 230
25 WebXR — VR and AR on the Web 242

INTERVIEW PREPARATION

26 The Interview Question Bank 250
27 Cheat Sheets and Formula Sheet 263

PART

Foundations of 3D Graphics

The 3D Mental Model

01

What real-time rendering actually is, the handful of objects every 3D app is built from, the coordinate systems that a point travels through, and how three.js and React Three Fiber map onto all of it.

IN THIS CHAPTER

1.1 What real-time rendering is
1.2 The building blocks of every 3D app
1.3 Coordinate systems
1.4 Your first scene
1.5 How three.js is organised
1.6 What React Three Fiber is
1.7 Interview questions

1.1 What real-time rendering is

A 3D scene on a screen is an illusion made of flat pictures. Many times per second the computer takes a mathematical description of a world (shapes, surfaces, lights, a camera) and **renders** it into a grid of coloured pixels. Show those pictures fast enough and the brain sees smooth motion.

DEFINITION · **Rendering**

The process of turning a description of a 3D scene into a 2D image. **Real-time** rendering must finish each image inside a fixed time budget (for example 16.7 ms) so the result can be shown interactively. **Offline** rendering (films, product shots) may spend minutes or hours per image.

DEFINITION · **Frame and frame rate**

A **frame** is one finished image. The **frame rate** is how many frames are produced per second (FPS). The **frame time** is the time one frame takes, $t = 1000/\mathrm{FPS}$ milliseconds.

Displays refresh at a fixed rate: most laptops at 60 Hz, many phones at 90–120 Hz, VR headsets at 72–120 Hz. The browser calls your render function once per refresh through `requestAnimationFrame` (three.js wraps this as `renderer.setAnimationLoop`). If your work for one frame takes longer than the refresh interval, a refresh is missed and the animation stutters.

Σ FORMULA · **Frame budget**

$$t_{\text{budget}} = \frac{1000\ \text{ms}}{f_{\text{refresh}}}$$

f_{refresh} is the display refresh rate in hertz. Everything you do in a frame (JavaScript, physics, the GPU's drawing) must fit inside t_{budget}.

WORKED EXAMPLE · **Budgets you should know by heart**

- 60 Hz: $1000/60 = 16.67$ ms per frame.
- 90 Hz (Quest-class headsets): $1000/90 = 11.11$ ms.
- 120 Hz (ProMotion iPad, gaming phones): $1000/120 = 8.33$ ms.

If your JavaScript takes 6 ms and the GPU needs 9 ms, the CPU and GPU work in parallel (the GPU draws frame n while JavaScript prepares frame $n+1$), so the frame time is roughly $\max(6, 9) = 9$ ms. That fits 60 Hz and 90 Hz, but not 120 Hz.

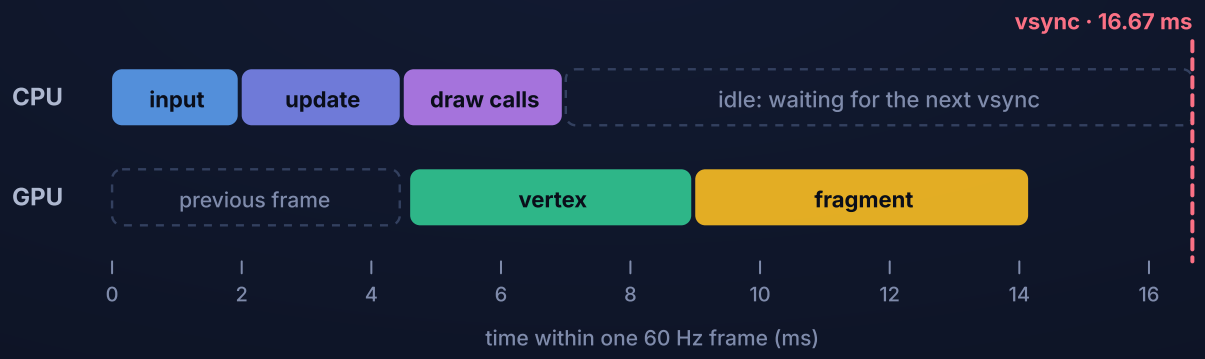


Figure 1.1 One frame at 60 Hz. The CPU (JavaScript, scene updates, issuing draw calls) and the GPU (vertex and fragment work) overlap. The slower of the two decides the frame rate, so you must first find out which side is the bottleneck.

INTUITION · **A flip book**

Rendering is a flip book that draws its own pages. Each page must be drawn in the time it takes to flip to it. If one page takes too long, the book visibly stutters.

1.2 The building blocks of every 3D app

Every 3D engine, from three.js to Unreal, is built from the same small set of ideas. Learn these words precisely: interviews start here.

DEFINITION · **Scene**

A container (a tree, called the **scene graph**) holding everything that can be drawn or that affects drawing: meshes, lights, cameras, helpers and groups. In three.js, `THREE.Scene`.

DEFINITION · **Camera**

The viewpoint. It defines where the eye is, where it looks, and how 3D is flattened into 2D (the **projection**: perspective or orthographic). In three.js, `PerspectiveCamera` or `OrthographicCamera`.

DEFINITION · **Renderer**

The object that talks to the GPU. Each frame it walks the scene, works out what is visible from the camera, and issues the GPU commands that draw it into a canvas. In three.js, `WebGLRenderer` or `WebGPURenderer`.

DEFINITION · **Mesh**

A drawable object made of a **geometry** (the shape: vertex positions, normals, UVs, triangle indices) and a **material** (how the surface reacts to light: colour, roughness, textures, the shader program).

DEFINITION · **Light**

A source of illumination used by lit materials: ambient, hemisphere, directional (sun), point (bulb), spot and rect-area lights in three.js, plus **image-based lighting** from an environment map.

The relationship is easy to remember: **the renderer draws the scene as seen by the camera**, which in code is literally `renderer.render(scene, camera)`.

Concept	Responsibility	three.js	R3F (JSX)
Scene	Holds the world	`new THREE.Scene()`	created by `<Canvas>`
Camera	Eye + projection	`new THREE.PerspectiveCamera(50, aspect, 0.1, 100)`	`<Canvas camera={{ fov: 50, position: [0, 1, 5] }}>`
Renderer	Talks to the GPU	`new THREE.WebGLRenderer({ antialias: true })`	created by `<Canvas gl={...}>`
Geometry	Shape data	`new THREE.BoxGeometry(1, 1, 1)`	`<boxGeometry args={[1, 1, 1]} />`
Material	Surface look	`new THREE.MeshStandardMaterial({ color: "orange" })`	`<meshStandardMaterial color="orange" />`
Mesh	Geometry + material	`new THREE.Mesh(geo, mat)`	`<mesh>…</mesh>`
Light	Illumination	`new THREE.DirectionalLight("white", 2)`	`<directionalLight intensity={2} />`

1.3 Coordinate systems

A 3D world needs agreed directions. three.js uses a **right-handed** coordinate system with **+Y up**. Point your right hand's thumb along +X (right), index finger along +Y (up); your middle finger points along +Z, **towards you**, out of the screen. A camera with no rotation therefore looks down **−Z**.

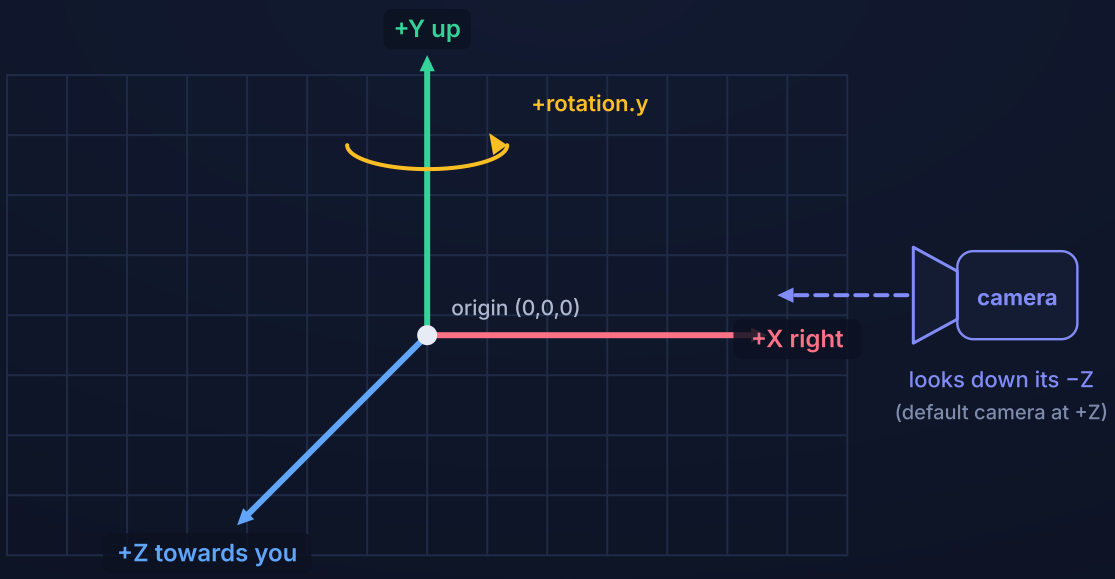


Figure 1.2 The three.js world axes. The default camera sits on +Z and looks towards −Z. Positive rotation about an axis is anticlockwise when you look down that axis towards the origin (the right-hand rule).

DEFINITION · **Right-hand rule**

For a right-handed system, $\mathbf{x} \times \mathbf{y} = \mathbf{z}$. Curl the fingers of your right hand in the direction of rotation and your thumb points along the axis; a **positive** angle turns anticlockwise when seen from the tip of the axis.

PITFALL · **Other tools use other conventions**

Blender is right-handed but **Z-up**; Unity is **left-handed** Y-up; Unreal is left-handed Z-up. glTF is right-handed Y-up like three.js, which is one reason glTF is the preferred format: models arrive the right way up. Since r184, `FBXLoader` converts Z-up models to Y-up automatically.

Units. three.js has no built-in unit, but the convention (and what WebXR, physics engines and glTF assume) is **1 unit = 1 metre**. Keep it: lights with physical intensities, physics gravity of $9.81\ \mathrm{m/s^2}$ and VR scale only look right when the world is in metres.

1.3.1 The journey of a vertex through spaces

A single vertex passes through a chain of coordinate systems before it becomes a pixel. Each arrow is a matrix multiplication, which the next chapters derive one by one.

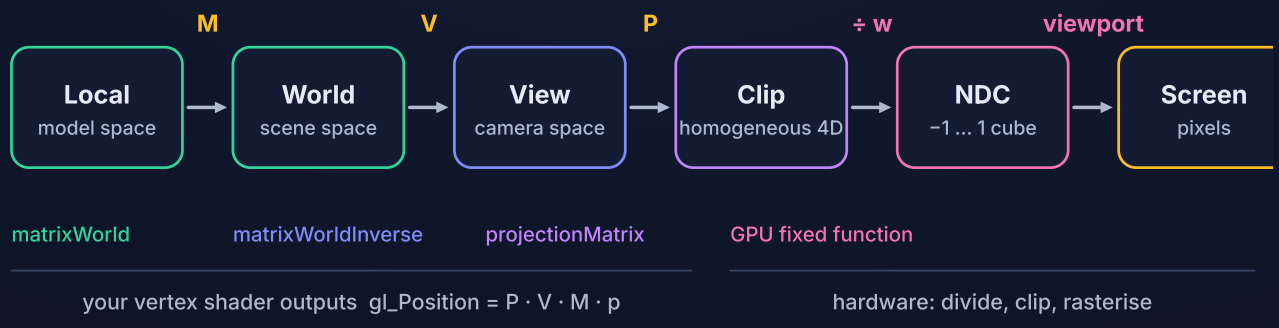


Figure 1.3 The coordinate spaces of the rendering pipeline. three.js builds the model, view and projection matrices for you; the GPU performs the divide by w and the viewport mapping in fixed hardware.

Space	What the numbers mean	Who computes it
Local (object / model) space	Relative to the object's own origin, as authored	The geometry's data
World space	Relative to the scene origin	`object.matrixWorld` (model matrix)
View (camera / eye) space	Relative to the camera; camera at origin looking down −Z	`camera.matrixWorldInverse` (view matrix)
Clip space	4D homogeneous coordinates after projection	`camera.projectionMatrix`
NDC	Clip ÷ w: a cube from −1 to 1 (z from 0 to 1 in WebGPU)	GPU (perspective divide)
Screen (window) space	Pixels, origin top-left in CSS, bottom-left in GL	GPU (viewport transform)

Σ FORMULA · **The whole chain in one line**

$$\mathbf{p}_{\text{clip}} = P \cdot V \cdot M \cdot \mathbf{p}_{\text{local}}$$

M is the model (world) matrix, V the view matrix (inverse of the camera's world matrix), P the projection matrix. In a three.js vertex shader this is `projectionMatrix * modelViewMatrix * vec4(position, 1.0)`, where `modelViewMatrix` $= VM$ is precomputed on the CPU per object.

1.4 Your first scene

Here is the smallest complete program in both styles. Read them side by side: R3F does exactly what the plain three.js code does, only declaratively.

three.js — a lit, spinning cube JAVASCRIPT

```javascript
import * as THREE from "three";

const scene = new THREE.Scene();
scene.background = new THREE.Color("#0b1020");

const camera = new THREE.PerspectiveCamera(50, innerWidth / innerHeight, 0.1, 100);
camera.position.set(0, 1.2, 4);
camera.lookAt(0, 0, 0);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setPixelRatio(Math.min(devicePixelRatio, 2)); // cap: 3x screens cost 2.25x the pixels of 2x
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);

const cube = new THREE.Mesh(
  new THREE.BoxGeometry(1, 1, 1),
  new THREE.MeshStandardMaterial({ color: "#34d399", roughness: 0.4 })
);
scene.add(cube);
scene.add(new THREE.HemisphereLight("#dbeafe", "#1e293b", 1.5));
const sun = new THREE.DirectionalLight("#ffffff", 2.5);
sun.position.set(3, 4, 2);
scene.add(sun);

const timer = new THREE.Timer();       // core since r179; replaces the deprecated Clock
timer.connect(document);               // pauses cleanly when the tab is hidden
renderer.setAnimationLoop((time) => {
  timer.update(time);
  cube.rotation.y += timer.getDelta() * 0.8;   // radians per second: frame-rate independent
  renderer.render(scene, camera);
});

addEventListener("resize", () => {
  camera.aspect = innerWidth / innerHeight;
  camera.updateProjectionMatrix();       // the aspect lives inside the projection matrix
  renderer.setSize(innerWidth, innerHeight);
});
```

R3F — the same scene JSX

```jsx
import { Canvas, useFrame } from "@react-three/fiber";
import { useRef } from "react";

function Cube() {
  const ref = useRef();
  useFrame((state, delta) => { ref.current.rotation.y += delta * 0.8; });
  return (
    <mesh ref={ref}>
      <boxGeometry args={[1, 1, 1]} />
      <meshStandardMaterial color="#34d399" roughness={0.4} />
    </mesh>
  );
}
```

```jsx
export default function App() {
  return (
    <Canvas camera={{ fov: 50, position: [0, 1.2, 4] }} dpr={[1, 2]}>
      <color attach="background" args={["#0b1020"]} />
      <hemisphereLight args={["#dbeafe", "#1e293b", 1.5]} />
      <directionalLight position={[3, 4, 2]} intensity={2.5} />
      <Cube />
    </Canvas>
  );
}
```

What `<Canvas>` did for you: created the scene, a perspective camera, a `WebGLRenderer` with sensible defaults (antialiasing, sRGB output, ACES Filmic tone mapping in R3F), a render loop, resize handling and pointer events. `dpr={[1, 2]}` clamps the device pixel ratio between 1 and 2.

> TIP · **Two rules from the very first scene**
>
> 1. **Animate with time, not frames.** `rotation.y += 0.01` spins twice as fast at 120 Hz as at 60 Hz. Multiply by the frame's delta time instead.
> 2. **Cap the pixel ratio.** Pixel count grows with the square of the ratio: a 3× phone screen at full ratio costs $3^2/2^2 = 2.25$ times the fragment work of 2×, for little visible gain.

1.5 How three.js is organised

three.js ships as one npm package, `three`, versioned `0.NNN.x` where NNN is the release number (r186 is `three@0.186.0`). Releases land every two to three months. It has a small number of entry points:

Import	What you get
`three`	Core: maths, scene graph, geometries, materials, lights, loaders' base classes, `WebGLRenderer`
`three/addons/...`	Everything optional: `OrbitControls`, `GLTFLoader`, `DRACOLoader`, `EffectComposer`, `RGBELoader`'s successor `HDRLoader`, `SunLight`, ...
`three/webgpu`	The same core plus `WebGPURenderer`, node materials and `RenderPipeline`
`three/tsl`	TSL, the three.js shading language (shader code written as JavaScript functions)

> NEW IN RECENT THREE.JS · **The modern three.js baseline (r180–r186)**
>
> - `THREE.Timer` is in core (r179) and `Clock` is deprecated (r183).
> - `RGBELoader` is now `HDRLoader` (r180).
> - The `PostProcessing` class of the WebGPU renderer is now `RenderPipeline` (r183).
> - `PCFSoftShadowMap` is gone (r186); `PCFShadowMap` is soft since r182.
> - `SunLight` gives cascaded shadows for both renderers (r186), and a Gaussian-splatting addon arrived in r186.
> - The minified builds were removed and the CommonJS build deprecated (r186): use ES modules. Upgrade in steps of at most ten releases, because deprecation warnings only last ten releases.

1.6 What React Three Fiber is

DEFINITION • **React Three Fiber (R3F)**

A **React renderer** for three.js. Just as `react-dom` turns JSX into DOM nodes, `@react-three/fiber` turns JSX into three.js objects. `<mesh>` becomes `new THREE.Mesh()`, props become property assignments, and children are added with `.add()` (or attached to a property with `attach`). It adds no wrapper layer at render time: every frame is still plain three.js.

Three facts make R3F click:

1. **Every three.js class is available as lower-camel-case JSX.** `THREE.MeshPhysicalMaterial` → `<meshPhysicalMaterial>`. Constructor arguments go in `args={[...]}`.
2. **Props set properties, with dashes for nesting.** `position={[1, 2, 3]}` calls `position.set(1, 2, 3)`; `rotation-y={Math.PI}` sets `rotation.y`; `material-color="red"` reaches into the material.
3. **React state is for things that change rarely.** Per-frame animation goes in `useFrame` by mutating refs, never through `setState` (Chapter 21 explains why).

R3F version 9 pairs with React 19 (R3F 8 pairs with React 18). Its `gl` prop accepts an **async factory**, which is how you plug in `WebGPURenderer`.

KEY POINTS

- ✓ A frame must finish inside $1000/f$ ms: 16.67 ms at 60 Hz, 11.11 ms at 90 Hz, 8.33 ms at 120 Hz.
- ✓ Scene, camera, renderer, mesh (geometry + material), light: `renderer.render(scene, camera)`.
- ✓ three.js is right-handed and Y-up; the camera looks down −Z; 1 unit = 1 metre by convention.
- ✓ A vertex travels local → world → view → clip → NDC → screen via $P\,V\,M\,\mathbf{p}$.
- ✓ Animate by delta time, cap the pixel ratio, update the projection matrix on resize.
- ✓ R3F maps JSX to three.js objects one-to-one; per-frame changes are mutations in `useFrame`.

1.7 Interview questions

Q **What are the minimum objects you need to show something with three.js, and what does each do?** JUNIOR}

A A **scene** (the container of everything), a **camera** (the viewpoint and projection), a **renderer** (draws the scene from the camera into a canvas with the GPU), and at least one **mesh** (geometry + material). If the material is lit (`MeshStandardMaterial`), you also need light: a light object or an environment map, otherwise the mesh renders black. Then call `renderer.render(scene, camera)`, usually every frame inside `renderer.setAnimationLoop`.

Q **Why must animation use delta time? Give the numbers.** JUNIOR}

A Because frame rates differ between devices. `rotation += 0.01` per frame gives 0.6 rad/s at 60 Hz but 1.2 rad/s at 120 Hz. Multiplying a speed in units per second by the frame's delta, `rotation += speed * delta`, gives the same motion on every device. In three.js use `THREE.Timer` (`update()` then `getDelta()`); in R3F, `useFrame((state, delta) ⇒ …)` passes delta directly.

Q **Which coordinate system does three.js use, and why does it matter when importing models?** MID}

A Right-handed, +Y up, the camera looks down −Z, and by convention 1 unit is 1 metre. Tools differ: Blender is Z-up, Unity is left-handed. A model exported with another convention arrives lying on its side or mirrored, and at the wrong scale (centimetres give a model 100× too big). glTF fixes the convention (right-handed, Y-up, metres), which is why it is the recommended format; exporters convert on export.

Q **What happens on a window resize, and what goes wrong if you forget it?** JUNIOR}

A Two things must change: the camera's `aspect` (then `camera.updateProjectionMatrix()`, because aspect is baked into the projection matrix) and the renderer's drawing-buffer size (`renderer.setSize`). Forget the first and the image stretches; forget the second and the canvas is blurry (upscaled) or crops. R3F's `<Canvas>` handles both automatically by observing its container's size.

Q **How does React Three Fiber relate to three.js? Is it slower?** MID}

A R3F is a React renderer (a custom reconciler) whose host objects are three.js objects. JSX elements are converted to `new THREE.X(...args)`, props to property sets, children to `add()` or `attach`. Rendering is still three.js's own `renderer.render`, so there is no per-frame overhead from React. React only does work when the component tree changes. It becomes slow only if you misuse React, for example calling `setState` every frame, which re-renders components 60 times a second; the correct pattern is mutating refs in `useFrame`.

The GPU Rendering Pipeline

02

How triangles become pixels. The stages of the graphics pipeline, rasterisation and barycentric interpolation, the depth buffer, blending maths, draw calls, and what three.js does inside renderer.render() every frame.

IN THIS CHAPTER

2.1 CPU and GPU: two processors, two jobs
2.2 The stages of the pipeline
2.3 Rasterisation and barycentric coordinates
2.4 The depth buffer
2.5 Blending maths
2.6 What renderer.render() actually does
2.7 WebGL and WebGPU
2.8 Interview questions

2.1 CPU and GPU: two processors, two jobs

A computer that renders 3D has two very different processors working together.

	CPU	GPU
Cores	A few (4–16) very fast, clever cores	Thousands of simple cores
Good at	Branching logic, game rules, JavaScript	The same small program on millions of items at once
In a three.js app	Your code, scene-graph updates, culling, sorting, issuing commands	Transforming vertices, shading pixels, blending
Talks via	A command buffer (WebGL calls / WebGPU command encoders)	Reads buffers and textures in its own memory (VRAM)

DEFINITION · **Shader**

A small program that runs **on the GPU**, once per item, massively in parallel. A **vertex shader** runs once per vertex; a **fragment shader** (pixel shader) runs once per fragment (a candidate pixel). WebGL shaders are written in GLSL, WebGPU shaders in WGSL; three.js can generate both from its node system (TSL).

DEFINITION · **Draw call**

One command from the CPU telling the GPU “draw this geometry with this shader and these settings”. In three.js, each visible `Mesh` normally costs at least one draw call (more with multi-materials, shadows or multiple passes). Draw calls have a fixed CPU cost, so their **count** is often the first performance limit.

INTUITION • **A kitchen**

The CPU is the head chef reading orders and shouting instructions. The GPU is a kitchen with a thousand cooks who can only do simple steps, but all at once. Shouting one instruction ("chop all these 10,000 onions") is fast. Shouting 10,000 separate instructions ("chop this onion", "chop this onion"...) wears the chef out long before the cooks are busy. That is why draw calls matter more than triangle counts in many scenes.

2.2 The stages of the pipeline

Every draw call flows through the same fixed sequence. Some stages are **programmable** (you write the shader), the rest are **fixed-function** hardware that you only configure.

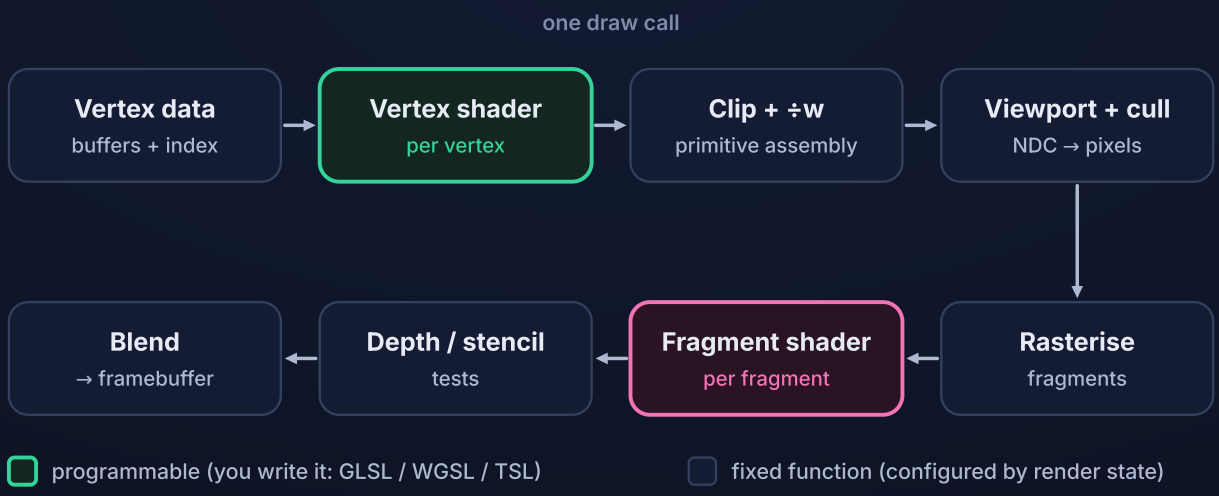


Figure 2.1 The real-time graphics pipeline. Programmable stages are highlighted; the others are fixed hardware configured by render state (culling, depth test, blending).

1. **Input assembly.** The GPU reads vertex attributes (position, normal, uv, ...) from buffers, and the **index buffer** that says which vertices form each triangle.
2. **Vertex shader** (programmable). Runs once per vertex. Its main job is to output the vertex's **clip-space position** (`gl_Position`), usually $PVM\mathbf{p}$, and any **varyings** (values to be interpolated across the triangle, such as normals and UVs).
3. **Primitive assembly.** Groups transformed vertices into triangles (or lines, points).
4. **Clipping.** Discards triangles fully outside the view volume and cuts the ones that cross its edges, in clip space, so nothing behind the camera is drawn.
5. **Perspective divide.** Divides x, y, z by w, giving **normalised device coordinates** (NDC).
6. **Viewport transform.** Maps NDC to pixel coordinates in the framebuffer.
7. **Face culling.** Drops triangles facing away from the camera (`material.side = THREE.FrontSide` is the default) using the winding order: counter-clockwise on screen counts as front-facing.
8. **Rasterisation.** Works out which pixel centres each triangle covers, creating a **fragment** for each, and interpolates the varyings for it.

9. **Fragment shader** (programmable). Runs once per fragment and outputs a colour (lighting, textures, fog).
10. **Per-fragment tests.** Scissor, stencil and **depth** tests decide whether the fragment survives.
11. **Blending.** Combines the fragment's colour with what is already in the framebuffer (for transparency).
12. **Output.** The colour lands in the **framebuffer** (the canvas, or an offscreen render target).

DEFINITION · **Fragment**

A candidate pixel produced by rasterising a triangle: it has a screen position, a depth, and interpolated varyings. It only becomes a visible pixel if it passes the depth and stencil tests and survives blending. Several fragments can compete for the same pixel; that waste is called **overdraw**.

TIP · **Early-z**

GPUs try to run the depth test **before** the fragment shader ("early-z"), so hidden fragments are rejected cheaply. Writing `gl_FragDepth`, using `discard` or alpha testing can disable it. This is why three.js draws opaque objects **front to back**: near objects fill the depth buffer first and the far ones behind them are rejected before shading.

2.3 Rasterisation and barycentric coordinates

To fill a triangle, the GPU must know, for each pixel centre $\mathbf{p}$, whether it is inside and how much each of the three vertices contributes to it. Both answers come from **barycentric coordinates**.

DEFINITION · **Barycentric coordinates**

For a triangle A, B, C, any point $\mathbf{p}$ in its plane can be written $\mathbf{p} = \lambda_A A + \lambda_B B + \lambda_C C$ with $\lambda_A + \lambda_B + \lambda_C = 1$. The point is **inside** the triangle exactly when all three λ are between 0 and 1. Each λ is the area of the sub-triangle opposite its vertex divided by the whole triangle's area.

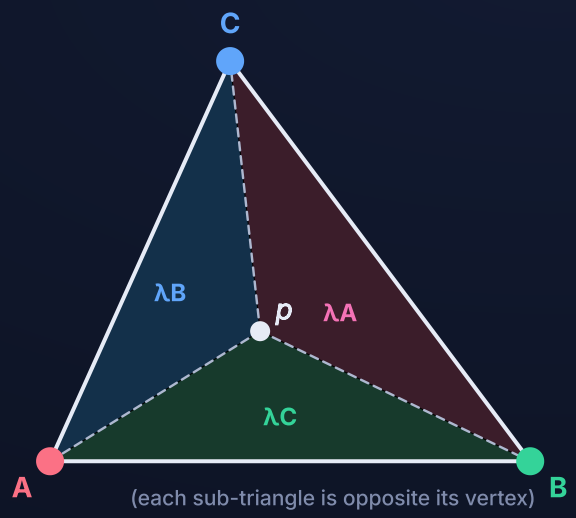


Figure 2.2 Barycentric weights are area ratios. The weight of A is the area of the triangle PBC (opposite A) divided by the area of ABC. A varying such as colour is blended with these weights.

Σ FORMULA · **Edge function and barycentrics**

$$E_{AB}(\mathbf{p}) = (B_x - A_x)(p_y - A_y) - (B_y - A_y)(p_x - A_x)$$

E_{AB} is twice the signed area of triangle $AB\mathbf{p}$: positive on one side of edge AB, negative on the other, zero on it. With $\text{Area} = E_{AB}(C)$:

$$\lambda_A = \frac{E_{BC}(\mathbf{p})}{\text{Area}}, \quad \lambda_B = \frac{E_{CA}(\mathbf{p})}{\text{Area}}, \quad \lambda_C = \frac{E_{AB}(\mathbf{p})}{\text{Area}}$$

WORKED EXAMPLE · **Is the pixel inside, and what colour is it?**

Triangle $A = (0, 0)$ red, $B = (4, 0)$ green, $C = (0, 4)$ blue. Pixel centre $\mathbf{p} = (1, 1)$.

1. $\text{Area} = E_{AB}(C) = (4 - 0)(4 - 0) - (0 - 0)(0 - 0) = 16$.
2. $E_{BC}(\mathbf{p}) = (0 - 4)(1 - 0) - (4 - 0)(1 - 4) = -4 + 12 = 8$, so $\lambda_A = 8/16 = 0.5$.
3. $E_{CA}(\mathbf{p}) = (0 - 0)(1 - 4) - (0 - 4)(1 - 0) = 0 + 4 = 4$, so $\lambda_B = 4/16 = 0.25$.
4. $E_{AB}(\mathbf{p}) = (4 - 0)(1 - 0) - (0 - 0)(1 - 0) = 4$, so $\lambda_C = 0.25$.
5. All three are in $[0, 1]$ and sum to 1: the pixel is **inside**.
6. Interpolated colour $= 0.5\,(1, 0, 0) + 0.25\,(0, 1, 0) + 0.25\,(0, 0, 1) = (0.5, 0.25, 0.25)$, a muted red.

2.3.1 Perspective-correct interpolation

Interpolating linearly in **screen** space is wrong for 3D triangles: a texture on a floor would bend. The fix is to interpolate a/w and $1/w$ linearly (they are linear in screen space) and divide.

Σ FORMULA · **Perspective-correct varying**

$$a(\mathbf{p}) = \frac{\lambda_A \frac{a_A}{w_A} + \lambda_B \frac{a_B}{w_B} + \lambda_C \frac{a_C}{w_C}}{\lambda_A \frac{1}{w_A} + \lambda_B \frac{1}{w_B} + \lambda_C \frac{1}{w_C}}$$

a is any varying (UV, normal, colour), w the clip-space w of each vertex (its distance in front of the camera) and λ the screen-space barycentrics. GPUs do this for every varying automatically; GLSL's `flat` and `noperspective` qualifiers turn it off.

WORKED EXAMPLE · **Why the correction matters**

A line from a near vertex ($w = 1$, $u = 0$) to a far vertex ($w = 3$, $u = 1$). At the screen-space midpoint ($\lambda = 0.5$ each), plain interpolation gives $u = 0.5$. Perspective-correct: numerator $0.5 \cdot 0/1 + 0.5 \cdot 1/3 = 0.1667$, denominator $0.5 \cdot 1 + 0.5 \cdot 1/3 = 0.6667$, so $u = 0.25$. The screen midpoint is much closer to the near vertex in 3D, so it really is only a quarter of the way along the texture.

2.4 The depth buffer

DEFINITION · **Depth buffer (z-buffer)**

A per-pixel image that stores the depth of the nearest fragment drawn so far. When a new fragment arrives, the **depth test** compares its depth with the stored one; by default (`LessEqualDepth`) it passes only if it is nearer, and then it writes its own depth (if `depthWrite` is true). This solves visibility without sorting triangles.

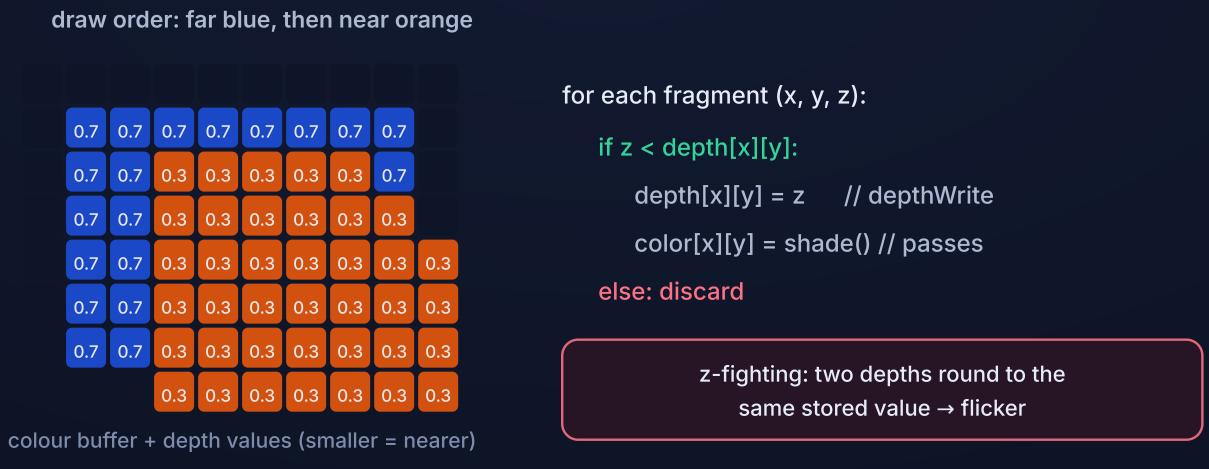


Figure 2.3 The z-buffer algorithm. Whichever triangle is drawn first, each pixel keeps the nearest fragment. Where two surfaces have almost equal depth, rounding makes them flicker against each other (z-fighting).

⚠ PITFALL · **Z-fighting**

When two surfaces are almost coplanar (a decal on a wall, two overlapping planes), their depths round to the same stored value and the winner changes pixel by pixel and frame by frame. Fixes, in order of preference: separate the surfaces a little; push the near plane **out** (`camera.near` of 0.1, not 0.0001, because precision depends mostly on near); use `material.polygonOffset`; or use a reversed or logarithmic depth buffer (Chapter 6).

2.4.1 Transparency and sorting

The depth buffer only works for opaque surfaces. A transparent surface must show what is behind it, so what is behind must be drawn **first**. three.js therefore renders in two lists:

1. **Opaque** objects, sorted **front to back** (to make early-z reject hidden work).
2. **Transparent** objects (`material.transparent = true`), sorted **back to front** by the distance of each object's centre to the camera, drawn with blending and usually `depthWrite: false`.

Sorting is per **object**, not per triangle, so intersecting transparent objects, or a single concave transparent mesh, can still draw in the wrong order. Workarounds: split meshes, set `renderOrder`, use `alphaTest` or `alphaHash` for cut-outs, or use order-independent transparency (Chapter 24).

2.5 Blending maths

Σ FORMULA · **The blend equation**

$$C_{\text{out}} = C_{\text{src}} \cdot F_{\text{src}} \;\oplus\; C_{\text{dst}} \cdot F_{\text{dst}}$$

C_{src} is the incoming fragment colour, C_{dst} the colour already in the framebuffer, F the blend factors and $\oplus$ the blend operation (usually add).

Mode	Factors	Result	three.js
Normal (straight alpha)	$F_s = \alpha_s,\ F_d = 1 - \alpha_s$	$\alpha_s C_s + (1-\alpha_s) C_d$	`NormalBlending`
Premultiplied alpha	$F_s = 1,\ F_d = 1 - \alpha_s$	$C'_s + (1-\alpha_s) C_d$ where $C'_s = \alpha_s C_s$	`premultipliedAlpha: true`
Additive	$F_s = \alpha_s$ (or 1), $F_d = 1$	$C_s + C_d$ (only brightens)	`AdditiveBlending`
Multiply	$F_s = C_d,\ F_d = 0$	$C_s \cdot C_d$ (only darkens)	`MultiplyBlending`

WORKED EXAMPLE · **Two panes of glass**

Background $C_d = (0.2, 0.2, 0.2)$. A red pane $C = (1, 0, 0)$ with $\alpha = 0.5$ is drawn, then a blue pane $C = (0, 0, 1)$ with $\alpha = 0.4$ in front of it.

1. After red: $0.5(1,0,0) + 0.5(0.2, 0.2, 0.2) = (0.6, 0.1, 0.1)$.

2. After blue: $0.4(0, 0, 1) + 0.6(0.6, 0.1, 0.1) = (0.36, 0.06, 0.46)$.

Draw blue first and red second and you get $0.5(1, 0, 0) + 0.5(0.12, 0.12, 0.52) = (0.56, 0.06, 0.26)$ instead: blending is **not commutative**, which is exactly why transparent objects must be sorted.

INTUITION · **Why premultiplied alpha is preferred**

Filtering (mipmaps, bilinear sampling) averages neighbouring texels. With straight alpha, a fully transparent texel's meaningless colour (often black) bleeds into the edge, creating dark halos. Premultiplied colours are already weighted by alpha, so a transparent texel is $(0, 0, 0, 0)$ and averages correctly. Compositing also becomes associative, which matters for layered effects.

2.6 What renderer.render() actually does

Knowing the steps inside `renderer.render(scene, camera)` explains most performance advice.

1. **Update matrices.** If `scene.matrixWorldAutoUpdate` is true, walk the tree and recompute each object's `matrixWorld` from its position, quaternion and scale (and the parent's matrix).
2. **Update the camera.** Compute the view matrix (`camera.matrixWorldInverse`) and the combined view-projection.
3. **Frustum culling.** For each object with `frustumCulled = true`, test its **bounding sphere** against the six frustum planes; skip objects fully outside.
4. **Build render lists.** Split visible objects into opaque, transmissive and transparent lists; sort opaque front to back (grouped by program and material to reduce state changes) and transparent back to front.
5. **Shadows.** For each shadow-casting light, render the scene's depth from the light into its shadow map.
6. **Programs.** For each material, find or compile the shader program for its current settings. A new combination of features (fog, number of lights, textures used, skinning) means a new program to compile, which causes a hitch the first time.
7. **Draw.** For each object: bind the program, upload changed uniforms, bind buffers and textures, set state (blending, depth, culling) and issue the draw call.

TIP · **Read the numbers**

`renderer.info.render.calls`, `.triangles`, `.points`, `.lines` and `renderer.info.memory.geometries` / `.textures` / `renderer.info.programs.length` tell you what the last frame cost. For `WebGPURenderer` the draw-call count is `renderer.info.render.drawCalls`. Watch them while you optimise.

2.7 WebGL and WebGPU

DEFINITION · **WebGL 2**

A JavaScript binding of OpenGL ES 3.0, available in every browser since around 2021. It is a **global state machine**: you bind things, set states, then draw. Shaders are GLSL ES 3.0. No compute shaders.

DEFINITION · **WebGPU**

The modern web graphics and compute API, modelled on Vulkan, Metal and Direct3D 12. State is baked into immutable **pipeline** objects, commands are recorded into **command encoders** and submitted in batches, and **compute shaders** run general GPU programs. Shaders are WGSL. Enabled by default in Chrome and Edge (113+), Safari 26 and Firefox on Windows and Apple Silicon.

	WebGL 2	WebGPU
Model	Global state machine	Explicit pipelines, bind groups, command encoders
Shader language	GLSL ES 3.0	WGSL
Compute shaders	No (tricks with render targets)	Yes
CPU overhead per draw	Higher (validation per call)	Lower (validation at pipeline creation)
NDC depth range	−1 to 1	0 to 1
three.js renderer	`WebGLRenderer` (mature, recommended for pure WebGL)	`WebGPURenderer` (falls back to WebGL 2 automatically)
Custom shaders	`ShaderMaterial`, `onBeforeCompile`	Node materials and TSL only

NOTE · **Which renderer should I use?**

`WebGLRenderer` is still maintained and the safe choice for an existing WebGL project, but new big features go into `WebGPURenderer`. New projects can start on `WebGPURenderer`: it uses WebGPU where available and falls back to WebGL 2 elsewhere, with one codebase. Profile both, since WebGPU is not faster for every scene.

✓ KEY POINTS

- ✓ The pipeline: vertex shader → clip → divide by w → viewport → rasterise → fragment shader → depth/stencil test → blend.
- ✓ Barycentric weights are area ratios; varyings are interpolated perspective-correctly via $1/w$.
- ✓ The depth buffer solves opaque visibility; transparent objects must be sorted back to front and usually do not write depth.
- ✓ Normal blending: $\alpha C_s + (1-\alpha)C_d$; order matters.
- ✓ Draw calls cost CPU time; overdraw and heavy fragment shaders cost GPU time.
- ✓ three.js per frame: update matrices → cull → sort → shadows → compile programs if needed → draw.

2.8 Interview questions

Q **Walk me through what happens between a vertex in a buffer and a pixel on the screen.** MID}

A The vertex shader transforms each vertex to clip space ($PVM\mathbf{p}$) and outputs varyings. Triangles are assembled, clipped against the view volume, divided by w into NDC, and mapped to the viewport. Back faces are culled by winding order. The rasteriser finds covered pixel centres and creates fragments, interpolating varyings perspective-correctly. The fragment shader computes each fragment's colour. Depth and stencil tests discard hidden fragments (ideally before shading, early-z), blending combines the survivors with the framebuffer, and the result is shown at the next vsync.

Q **Why is a scene with 2,000 separate small meshes slow even though it has few triangles?** MID}

A Each mesh is at least one draw call, and every draw call has a fixed CPU cost (binding programs, uploading uniforms, setting state, driver validation) independent of how small the mesh is. 2,000 calls can cost several milliseconds of CPU, while the GPU sits idle. Fixes: `InstancedMesh` for repeated geometry, `BatchedMesh` for different geometries sharing a material, merging static meshes with `BufferGeometryUtils.mergeGeometries`, and sharing materials. Measure with `renderer.info.render.calls`.

Q **Why do transparent objects sometimes disappear or draw in the wrong order?** MID}

A The depth buffer cannot handle partial coverage, so three.js sorts transparent objects back to front by object centre and draws them after the opaque ones. Sorting per object fails for intersecting or concave transparent meshes, and if a transparent material writes depth it can hide objects drawn after it. Fixes: `depthWrite: false` on transparent materials, splitting meshes, `renderOrder`, using `alphaTest` / `alphaHash` for cut-out foliage, or order-independent transparency.

Q **What is z-fighting and how do you fix it?** JUNIOR}

A Flickering where two surfaces have nearly the same depth, because the depth buffer's limited precision rounds them to the same value. Perspective depth precision is concentrated near the near plane, so the first fix is to increase `camera.near` (and reduce `far`). Others: offset the geometry slightly, `polygonOffset` on the decal material, or a logarithmic or reversed depth buffer (`reversedDepthBuffer: true` on `WebGLRenderer`, `WebGPURenderer` supports reversed depth since r183).

Q **What are barycentric coordinates used for in rendering?** SENIOR}

A They express a point inside a triangle as a weighted mix of its three vertices, with weights equal to sub-triangle area ratios. The rasteriser uses them (via edge functions) to test whether a pixel centre is inside a triangle and to interpolate varyings such as UVs, normals and colours, with a $1/w$ correction for perspective. They are also used in ray–triangle intersection (Möller–Trumbore returns barycentrics), in picking (three.js returns `intersection.barycoord` in recent versions) and in wireframe shaders that measure distance to edges.

Q **Compare WebGL and WebGPU. Would you move an existing project?** SENIOR}

A WebGL 2 is a global state machine with per-call validation and no compute. WebGPU uses pre-validated pipeline objects, bind groups and command encoders, which lowers CPU cost per draw, and adds compute shaders and storage buffers, enabling GPU particle simulation, culling and more. Its NDC depth range is 0–1. In three.js, moving means switching to `WebGPURenderer`, porting `ShaderMaterial` / `onBeforeCompile` code to node materials and TSL, and replacing `EffectComposer` with `RenderPipeline`. I would move when the project needs compute or is draw-call bound, after profiling, because WebGPU is not automatically faster for every scene.

Vectors

03

The single most used maths in 3D. Points and directions, length and normalisation, the dot and cross products and what they mean geometrically, projection, reflection, interpolation and frame-rate-independent damping, with three.js and R3F code for each.

IN THIS CHAPTER

3.1 Points and directions
3.2 Length and normalisation
3.3 The dot product
3.4 The cross product
3.5 Projection and reflection
3.6 Interpolation and damping
3.7 Useful distance tests
3.8 Interview questions

3.1 Points and directions

DEFINITION · **Vector**

A quantity with a **direction** and a **magnitude** (length), written as components $\mathbf{v} = (v_x, v_y, v_z)$. In 3D graphics the same three numbers can mean two different things: a **point** (a position) or a **direction** (a displacement, a velocity, a normal).

The difference matters when transforming. Moving an object should move its points but must not change a direction such as “up”. Homogeneous coordinates encode this with a fourth component: points have $w = 1$, directions have $w = 0$, so translation (which lives in the fourth column of a matrix) affects only points. Chapter 4 shows the matrix.

Operation	Result	Meaning
point − point	vector	the displacement from one point to the other
point + vector	point	a point moved by a displacement
vector + vector	vector	combined displacement
point + point	(meaningless)	except in weighted averages whose weights sum to 1

In three.js all of these are `THREE.Vector3`. The class does not know whether it holds a point or a direction; you decide, and you choose the right method (`applyMatrix4` for points, `transformDirection` for directions).

⚠ PITFALL · **three.js vector methods change the vector itself**

`a.add(b)` modifies `a` and returns `a` (for chaining); it does **not** return a new vector. To keep `a`, write `a.clone().add(b)` or `result.addVectors(a, b)`. Forgetting this is the most common maths bug in three.js code: for example `mesh.position.add(velocity)` moves the mesh, which is right, but `const p = mesh.position; p.add(offset)` also moves the mesh, which usually is not intended.

3.2 Length and normalisation

Σ FORMULA · **Length (magnitude)**

$$\|\mathbf{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

The distance between points A and B is $\|B - A\|$.

DEFINITION · **Unit vector and normalisation**

A **unit vector** has length 1 and represents a pure direction. **Normalising** divides a vector by its length: $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$. Normals, light directions and camera axes are always unit vectors, because the formulas that use them assume it.

WORKED EXAMPLE · **Distance and direction to a target**

Player at $P = (1, 0, 2)$, enemy at $E = (4, 0, 6)$.

1. Displacement $\mathbf{d} = E - P = (3, 0, 4)$.
2. Distance $\|\mathbf{d}\| = \sqrt{9 + 0 + 16} = 5$ metres.
3. Direction $\hat{\mathbf{d}} = (3/5, 0, 4/5) = (0.6, 0, 0.8)$.
4. Moving at 2 m/s for one 60 Hz frame ($\Delta t = 0.0167$ s): step $= 2 \times 0.0167 \times (0.6, 0, 0.8) = (0.02, 0, 0.0267)$.

TIP · **Compare squared lengths**

A square root is not free. To compare distances ("is the enemy within 10 m?") compare **squared** lengths: `p.distanceToSquared(e) < 100`. The answer is the same and no square root is needed.

three.js — distance, direction and movement JAVASCRIPT

```javascript
const toEnemy = new THREE.Vector3();              // allocate once, reuse every frame

function update(dt) {
  toEnemy.subVectors(enemy.position, player.position);  // E - P
  const dist = toEnemy.length();
  if (dist > 0.001) {
    toEnemy.divideScalar(dist);                    // normalise (same as .normalize())
    player.position.addScaledVector(toEnemy, Math.min(speed * dt, dist));
  }
}
```

3.3 The dot product

Σ FORMULA · **Dot product**

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta$$

θ is the angle between the vectors. For unit vectors, $\mathbf{a} \cdot \mathbf{b} = \cos\theta$.

The dot product answers "**how much do two directions agree?**". For unit vectors:

$\hat{\mathbf{a}} \cdot \hat{\mathbf{b}}$	Angle	Meaning
1	0°	same direction
between 0 and 1	under 90°	roughly the same way ("in front")
0	90°	perpendicular
below 0	over 90°	roughly opposite ("behind")
−1	180°	exactly opposite

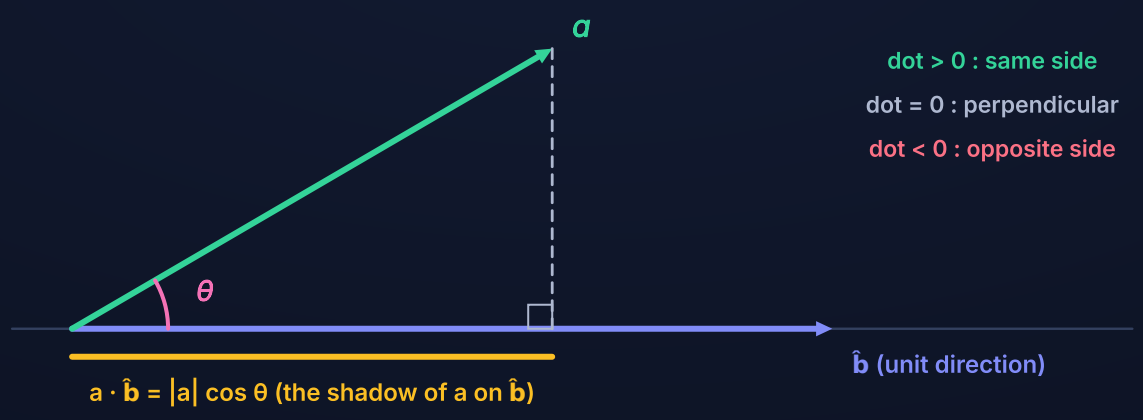


Figure 3.1 The dot product as projection. $\mathbf{a} \cdot \hat{\mathbf{b}}$ is the length of $\mathbf{a}$'s shadow on the line of $\hat{\mathbf{b}}$. Its sign tells you which side of the perpendicular $\mathbf{a}$ points to.

Uses you will meet throughout this notebook:

- **Diffuse lighting.** Lambert's law: brightness $\propto \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$ (Chapter 9).
- **Facing tests.** Is the enemy in front of the player? $\text{forward} \cdot (E - P) > 0$. Within a 60° vision cone? $\hat{\text{forward}} \cdot \hat{\mathbf{d}} > \cos 30^\circ = 0.866$.
- **Back-face culling** and **Fresnel** effects use $\hat{\mathbf{n}} \cdot \hat{\mathbf{v}}$.
- **Projection.** The component of $\mathbf{a}$ along a unit $\hat{\mathbf{b}}$ is $(\mathbf{a} \cdot \hat{\mathbf{b}})\,\hat{\mathbf{b}}$.

WORKED EXAMPLE · Is the enemy inside the guard's view cone?

Guard faces $\hat{\mathbf{f}} = (0, 0, -1)$. Enemy direction from the guard (normalised) $\hat{\mathbf{d}} = (0.6, 0, -0.8)$. Half-angle of the cone: 45°.

1. $\hat{\mathbf{f}} \cdot \hat{\mathbf{d}} = 0(0.6) + 0(0) + (-1)(-0.8) = 0.8$.
2. Angle $= \arccos(0.8) = 36.87^\circ$.
3. Threshold $\cos 45^\circ = 0.7071$. Since $0.8 > 0.7071$, the enemy is **inside** the cone. No `acos` is needed at runtime: compare the dot product with a precomputed cosine.

three.js — view-cone test JAVASCRIPT

```javascript
const forward = new THREE.Vector3();
const toTarget = new THREE.Vector3();
const COS_HALF = Math.cos(THREE.MathUtils.degToRad(45));

function canSee(guard, target) {
  guard.getWorldDirection(forward);   // unit vector of the object's local +Z in world space
  forward.negate();                    // characters usually "face" -Z; flip if yours face +Z
  toTarget.subVectors(target.position, guard.position).normalize();
  return forward.dot(toTarget) > COS_HALF;
}
```

PITFALL · getWorldDirection points along +Z

`Object3D.getWorldDirection()` returns the object's local **+Z** axis in world space. For a camera, `camera.getWorldDirection()` is overridden to return the direction the camera **looks** (its −Z). Know which one you are calling.

3.4 The cross product

FORMULA · Cross product

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y,\;\; a_z b_x - a_x b_z,\;\; a_x b_y - a_y b_x)$$

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\|\,\|\mathbf{b}\| \sin\theta$$

The result is perpendicular to both inputs, its direction follows the right-hand rule, and its length is the area of the parallelogram the two vectors span. It is **anti-commutative**: $\mathbf{b} \times \mathbf{a} = -(\mathbf{a} \times \mathbf{b})$.

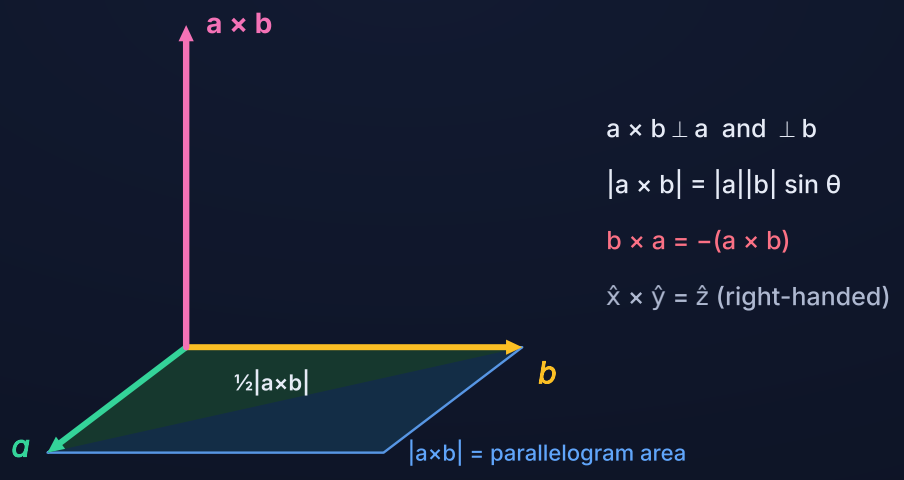


Figure 3.2 The cross product is perpendicular to both inputs. Its length equals the parallelogram's area, so half of it is the area of the triangle formed by the two edges.

Uses:

- **Surface normals.** For a triangle A, B, C with counter-clockwise winding, $\hat{\mathbf{n}} = \text{normalise}((B - A) \times (C - A))$. This is what `geometry.computeVertexNormals()` does per face before averaging.
- **Building a coordinate frame.** Given a forward direction and world up, $\text{right} = \text{forward} \times \text{up}$, then $\text{up}' = \text{right} \times \text{forward}$. This is how `lookAt` builds a rotation (Chapter 5).
- **Areas.** Triangle area $= \frac{1}{2}\|(B - A) \times (C - A)\|$, used for uniform random sampling on meshes (`MeshSurfaceSampler`).
- **Which side?** The sign of $(\mathbf{a} \times \mathbf{b})_y$ tells whether $\mathbf{b}$ is to the left or right of $\mathbf{a}$ on the ground plane, which steering behaviours use.

WORKED EXAMPLE · Normal and area of a triangle

$A = (0, 0, 0)$, $B = (2, 0, 0)$, $C = (0, 0, -3)$ (a triangle lying on the ground).

1. Edges: $\mathbf{e}_1 = B - A = (2, 0, 0)$, $\mathbf{e}_2 = C - A = (0, 0, -3)$.
2. $\mathbf{e}_1 \times \mathbf{e}_2 = (0 \cdot (-3) - 0 \cdot 0,\ \ 0 \cdot 0 - 2 \cdot (-3),\ \ 2 \cdot 0 - 0 \cdot 0) = (0, 6, 0)$.
3. Normal $= (0, 1, 0)$: it points **up**, as expected for a counter-clockwise triangle seen from above.
4. Area $= \frac{1}{2} \times 6 = 3\ \text{m}^2$. Check: a right triangle with legs 2 and 3 has area 3.

3.5 Projection and reflection

FORMULA · Projection onto a direction and onto a plane

$$\text{proj}_{\hat{\mathbf{n}}}\mathbf{v} = (\mathbf{v} \cdot \hat{\mathbf{n}})\,\hat{\mathbf{n}} \qquad \text{proj}_{\text{plane}}\mathbf{v} = \mathbf{v} - (\mathbf{v} \cdot \hat{\mathbf{n}})\,\hat{\mathbf{n}}$$

Projecting onto a plane removes the part of $\mathbf{v}$ along the plane's normal. Characters use this to slide along walls and slopes.

Σ FORMULA · **Reflection**

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \hat{\mathbf{n}})\,\hat{\mathbf{n}}$$

$\mathbf{d}$ is the incoming direction, $\hat{\mathbf{n}}$ the unit surface normal. Used for bouncing balls, mirror and environment reflections (`reflect()` in GLSL) and specular highlights.

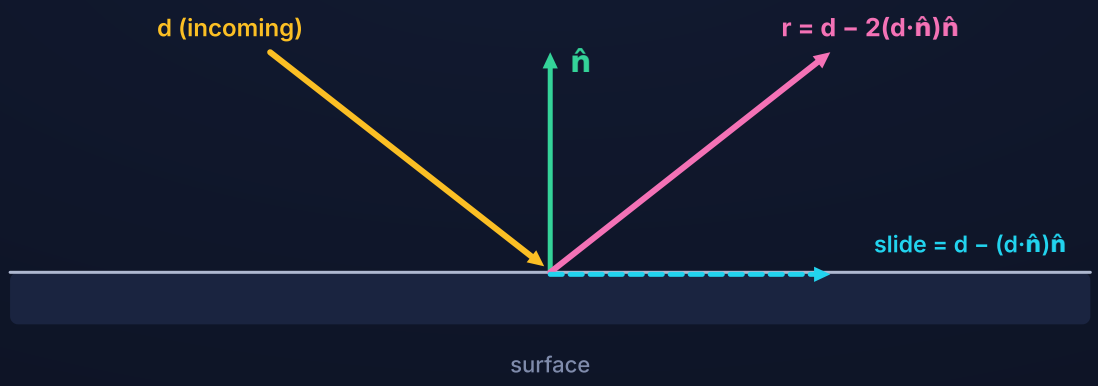


Figure 3.3 Reflection keeps the part of the direction along the surface and flips the part along the normal. Sliding (projection onto the plane) keeps only the along-surface part.

WORKED EXAMPLE · **A ball bouncing off the floor**

Velocity $\mathbf{d} = (3, -4, 0)$ m/s hits the floor with normal $\hat{\mathbf{n}} = (0, 1, 0)$.

1. $\mathbf{d} \cdot \hat{\mathbf{n}} = -4$.
2. $\mathbf{r} = (3, -4, 0) - 2(-4)(0, 1, 0) = (3, -4, 0) + (0, 8, 0) = (3, 4, 0)$.
3. With a restitution (bounciness) of 0.8 applied to the normal part only: $(3, 0.8 \times 4, 0) = (3, 3.2, 0)$.

three.js — reflect, slide and project JAVASCRIPT

```javascript
const n = new THREE.Vector3(0, 1, 0);                 // must be unit length
velocity.reflect(n);                                   // r = d - 2(d·n)n, in place
const slide = velocity.clone().projectOnPlane(n);      // movement along the surface
const along = velocity.clone().projectOnVector(n);     // movement into the surface
```

3.6 Interpolation and damping

Σ FORMULA · **Linear interpolation (lerp)**

$$\operatorname{lerp}(\mathbf{a}, \mathbf{b}, t) = \mathbf{a} + t\,(\mathbf{b} - \mathbf{a}) = (1 - t)\,\mathbf{a} + t\,\mathbf{b}$$

$t = 0$ gives $\mathbf{a}$, $t = 1$ gives $\mathbf{b}$, $t = 0.5$ the midpoint.

Σ FORMULA · **Smoothstep**

$$s(t) = 3t^2 - 2t^3 \qquad (0 \le t \le 1)$$

An S-shaped curve with zero slope at both ends: motion eases in and out. GLSL's `smoothstep(e0, e1, x)` first maps x to $t = (x - e_0)/(e_1 - e_0)$ and clamps it to $[0, 1]$.

A very common camera trick is "move 10% of the way towards the target every frame": `pos.lerp(target, 0.1)`. It looks smooth, but it is **frame-rate dependent**: at 120 Hz it runs that step twice as often and arrives faster. The fix is **exponential damping**.

Σ FORMULA · **Frame-rate-independent damping**

$$x \leftarrow \operatorname{lerp}\left(x,\ \text{target},\ 1 - e^{-\lambda\,\Delta t}\right)$$

λ is a smoothing rate (per second); the remaining distance shrinks by a factor $e^{-\lambda \Delta t}$ each frame, so after one second it has shrunk by $e^{-\lambda}$ whatever the frame rate. three.js provides `THREE.MathUtils.damp(x, target, lambda, dt)`; drei's companion library `maath` provides `easing.damp3` for vectors.

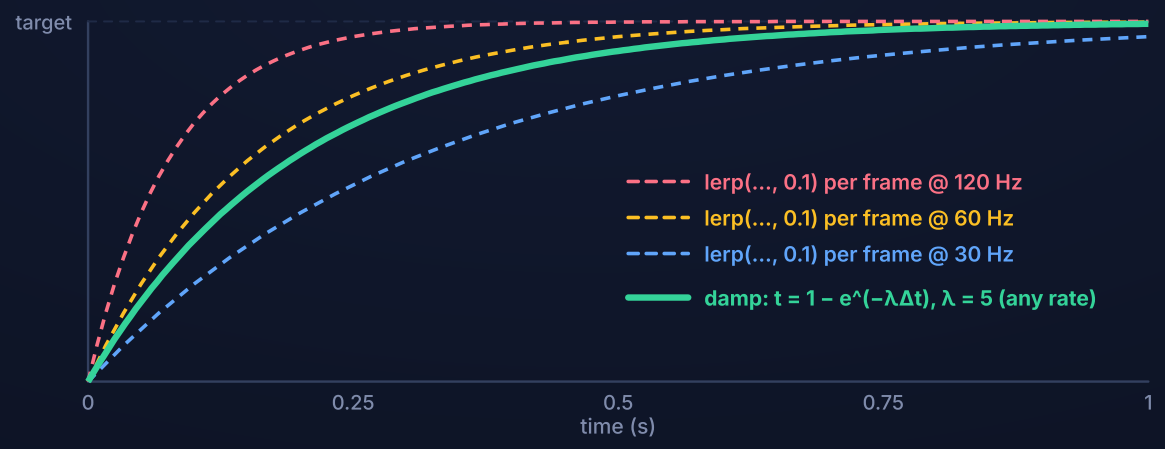


Figure 3.4 Following a target with a fixed per-frame lerp (dashed) depends on the frame rate; exponential damping (solid) gives the same curve at 30, 60 and 120 Hz.

WORKED EXAMPLE · **Same feel at 60 Hz and 120 Hz**

With $\lambda = 5$:

- At 60 Hz, $\Delta t = 1/60$: $t = 1 - e^{-5/60} = 1 - e^{-0.0833} = 0.0800$ per frame. After 60 frames the remaining fraction is $(1 - 0.08)^{60} = e^{-5} = 0.0067$.
- At 120 Hz, $\Delta t = 1/120$: $t = 1 - e^{-0.04167} = 0.0408$ per frame. After 120 frames: $e^{-5} = 0.0067$. Identical.
- The naive `lerp(…, 0.08)` at 120 Hz would leave $(0.92)^{120} = 0.000045$ after one second: much faster.

three.js — damped follow camera JAVASCRIPT

```
const desired = new THREE.Vector3();
renderer.setAnimationLoop((time) => {
  timer.update(time);
  const dt = Math.min(timer.getDelta(), 0.1);         // clamp: a background tab can return a
huge delta
  desired.copy(player.position).add(offset);          // where the camera wants to be
  const k = 1 - Math.exp(-5 * dt);
  camera.position.lerp(desired, k);
  camera.lookAt(player.position);
  renderer.render(scene, camera);
});
```

R3F — the same with maath JSX

```
import { useFrame } from "@react-three/fiber";
import { easing } from "maath";

function FollowCamera({ target, offset = [0, 3, 6] }) {
  useFrame((state, dt) => {
    const p = target.current.position;
    easing.damp3(state.camera.position, [p.x + offset[0], p.y + offset[1], p.z + offset[2]],
0.25, dt);
    state.camera.lookAt(p);
  });
  return null;
}
```

ⓘ NOTE · **maath's smoothTime**

`easing.damp3(current, target, smoothTime, dt)` takes a **smooth time** in seconds (roughly the time to reach the target) instead of a rate; it uses a critically damped spring internally, so it also stays frame-rate independent.

3.7 Useful distance tests

Σ FORMULA · **Signed distance from a point to a plane**

A plane with unit normal $\hat{\mathbf{n}}$ through point Q can be written $\hat{\mathbf{n}} \cdot \mathbf{x} + d = 0$ with $d = -\hat{\mathbf{n}} \cdot Q$. For any point $\mathbf{p}$:

$$\text{dist}(\mathbf{p}) = \hat{\mathbf{n}} \cdot \mathbf{p} + d$$

Positive in front of the plane (the side the normal points to), negative behind, zero on it. `THREE.Plane.distanceToPoint()` computes it, and frustum culling is six of these tests.

WORKED EXAMPLE · **Which side of the water surface?**

Water plane: normal $(0, 1, 0)$ through $(0, 2, 0)$, so $d = -2$. A fish at $(5, 1.4, -3)$: $\text{dist} = 1.4 - 2 = -0.6$, so it is 0.6 m **under** water. A bird at $(1, 7, 0)$: $7 - 2 = 5$ m above.

three.js — Plane and Sphere helpers JAVASCRIPT

```
const water = new THREE.Plane(new THREE.Vector3(0, 1, 0), -2);   // n·x + d = 0
water.distanceToPoint(fish.position);          // -0.6
const bounds = new THREE.Sphere(new THREE.Vector3(0, 0, 0), 1.5);
bounds.containsPoint(p);                       // |p - c| ≤ r
const box = new THREE.Box3().setFromObject(model);   // world-space axis-aligned bounding box
box.getSize(new THREE.Vector3()); box.getCenter(new THREE.Vector3());
```

✓ KEY POINTS

- ✓ Points have $w = 1$ and are moved by translation; directions have $w = 0$ and are not.
- ✓ three.js maths methods mutate in place and return `this`; clone or use `…Vectors(a, b)` forms when you need to keep inputs.
- ✓ Dot product: agreement between directions, $\cos\theta$ for unit vectors; used for lighting, facing tests, projection.
- ✓ Cross product: a perpendicular vector whose length is the parallelogram area; used for normals and coordinate frames.
- ✓ Reflection $\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$; sliding is projection onto the plane.
- ✓ Use $1 - e^{-\lambda \Delta t}$ (or `MathUtils.damp`, `maath` `easing`) for frame-rate-independent smoothing.
- ✓ Avoid allocating vectors inside the frame loop: create temporaries once and reuse them.

3.8 Interview questions

Q How do you check whether one object is in front of another? JUNIOR}

A Take the first object's forward direction $\mathbf{f}$ in world space and the vector to the second, $\mathbf{d} = B - A$. If $\mathbf{f} \cdot \mathbf{d} > 0$ the target is in front (angle under 90°). For a vision cone with half-angle α, normalise both and compare the dot product with $\cos\alpha$ (precomputed), which avoids `acos`. In three.js: `obj.getWorldDirection(f)` (remember it returns local +Z for non-camera objects) and `f.dot(d.normalize())`.

Q How do you compute a triangle's normal, and why does winding order matter? JUNIOR}

A $\hat{\mathbf{n}} = \text{normalise}((B - A) \times (C - A))$. The cross product's direction follows the right-hand rule, so swapping the order of B and C flips the normal. three.js treats counter-clockwise triangles (as seen from the viewer) as front faces, so geometry with the wrong winding is culled or lit from the back. `computeVertexNormals()` averages face normals at shared vertices for smooth shading.

Q `camera.position.lerp(target, 0.1)` **in the render loop feels different on a 120 Hz screen. Why, and how do you fix it?** MID

A A constant per-frame factor runs more steps per second at higher refresh rates, so the camera converges faster (after one second the remaining distance is 0.9^{60} at 60 Hz but 0.9^{120} at 120 Hz). Use exponential damping with delta time: `lerp(target, 1 - Math.exp(-lambda * dt))`, or `THREE.MathUtils.damp`, or `maath`'s `easing.damp3`. Then the remaining fraction after time t is always $e^{-\lambda t}$. Also clamp `dt`, since a tab returning from the background can report a huge delta.

Q **Why is** `new THREE.Vector3()` **inside** `useFrame` **or an animation loop a problem?** MID

A It allocates an object every frame (60–120 times a second per call site). The garbage collector must later reclaim them, and GC pauses cause frame hitches. Create temporaries once (module scope, `useMemo`, or a closure) and reuse them with `.set()` / `.copy()`. This is listed in the official R3F performance pitfalls.

Q **How would you make a character slide along a wall instead of stopping?** MID

A When the movement vector $\mathbf{v}$ hits a wall with unit normal $\hat{\mathbf{n}}$, remove the component going into the wall: $\mathbf{v}' = \mathbf{v} - (\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$, which is `v.projectOnPlane(n)`. Only remove it when $\mathbf{v} \cdot \hat{\mathbf{n}} < 0$ (moving into the wall). Physics engines' character controllers (for example Rapier's `KinematicCharacterController`) do this with several iterations to handle corners.

Matrices and Transforms

04

How position, rotation and scale become one 4×4 matrix, why the order of operations matters, how the scene graph multiplies parent and child matrices, inverses and the view matrix, and the normal matrix that keeps lighting correct under non-uniform scale.

IN THIS CHAPTER

4.1 What a matrix does
4.2 Homogeneous coordinates and 4×4 matrices
4.3 Composition and the order of operations
4.4 The scene graph: local and world matrices
4.5 Inverses and the view matrix
4.6 The normal matrix
4.7 Matrix4 in three.js
4.8 Interview questions

4.1 What a matrix does

DEFINITION · **Linear transformation**

A function on vectors that keeps straight lines straight and the origin fixed: rotations, scales, shears and reflections. Every linear transformation of 3D space can be written as a 3×3 matrix A, and applied to a vector by multiplication: $\mathbf{v}' = A\mathbf{v}$.

The single most useful fact about matrices in graphics: **the columns of the matrix are where the basis vectors land.** If a transformation sends $\hat{\mathbf{x}}$ to $\mathbf{c}_1$, $\hat{\mathbf{y}}$ to $\mathbf{c}_2$ and $\hat{\mathbf{z}}$ to $\mathbf{c}_3$, then its matrix is $[\,\mathbf{c}_1\ \mathbf{c}_2\ \mathbf{c}_3\,]$, and any vector (x, y, z) maps to $x\mathbf{c}_1 + y\mathbf{c}_2 + z\mathbf{c}_3$.

Figure 4.1 A matrix is a new set of axes. Its first column is where the x-axis goes and its second column where the y-axis goes. Here the columns are $(1.5, 0.5)$ and $(-0.5, 1)$, which rotates, stretches and shears the grid.

Σ FORMULA · **Matrix times vector**

$$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} ax + by + cz \\ dx + ey + fz \\ gx + hy + iz \end{pmatrix}$$

Each output component is the dot product of one **row** with the vector.

4.2 Homogeneous coordinates and 4×4 matrices

Translation ("move by $\mathbf{t}$") is not linear: it moves the origin. A 3×3 matrix cannot express it. The trick is to add a fourth coordinate w and use 4×4 matrices.

DEFINITION · **Homogeneous coordinates**

A 3D point (x, y, z) is written $(x, y, z, 1)$; a direction is written $(x, y, z, 0)$. A 4×4 matrix can then hold a 3×3 linear part plus a translation in its fourth column. After a perspective projection w can become any value, and the actual 3D point is recovered by dividing by it: $(x/w, y/w, z/w)$.

Σ FORMULA · **Translation, scale and rotation matrices**

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix} \quad R_y(\theta) = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix} \quad R_z(\theta) = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

with $c = \cos\theta$, $s = \sin\theta$ (rotation matrices shown as their 3×3 part). Positive θ rotates anticlockwise when looking down the axis towards the origin.

WORKED EXAMPLE · **Translation affects points but not directions**

T with $\mathbf{t} = (5, 0, 2)$.

- Point $(1, 1, 1, 1)$: $(1 + 5,\ 1 + 0,\ 1 + 2,\ 1) = (6, 1, 3)$. Moved.
- Direction $(1, 1, 1, 0)$: the translation column is multiplied by $w = 0$, so the result is $(1, 1, 1)$. Unchanged, as a direction should be.

WORKED EXAMPLE · **Rotating a point 90° about Y**

$\theta = 90°$: $c = 0$, $s = 1$. Point $(1, 0, 0)$:

$$R_y\,(1, 0, 0)^T = (c \cdot 1 + 0 + s \cdot 0,\ 0,\ -s \cdot 1 + 0 + c \cdot 0) = (0, 0, -1)$$

A point on +X swings round to −Z. Seen from above (+Y looking down), that is an anticlockwise quarter turn, which matches the right-hand rule. In three.js, `mesh.rotation.y = Math.PI / 2` does exactly this to every vertex.

4.3 Composition and the order of operations

Multiplying matrices composes transformations. Because vectors are columns and multiply on the right, **the transformation nearest the vector happens first**: in $TRS\,\mathbf{p}$, the point is scaled, then rotated, then translated.

Σ FORMULA · **The model matrix**

$$M = T\,R\,S$$

three.js builds every object's local matrix this way from `position` (T), `quaternion` / `rotation` (R) and `scale` (S): scale about the object's origin, rotate about its origin, then move it into place. This is `Matrix4.compose(position, quaternion, scale)`.

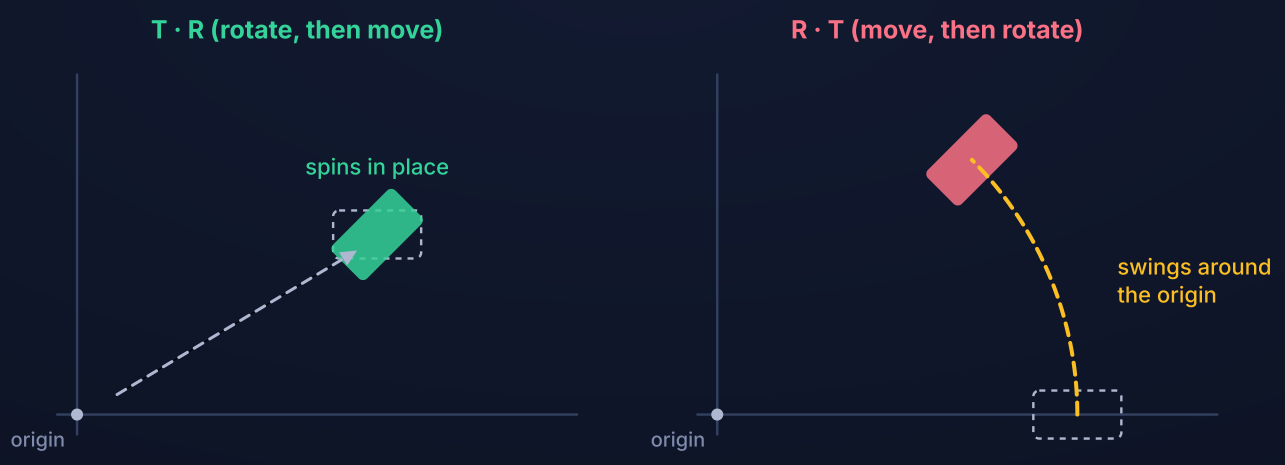


Figure 4.2 Order matters. Rotating then translating (left) moves the object to the target and keeps it there. Translating then rotating (right) swings the object around the world origin, like a ball on a string.

WORKED EXAMPLE · **Same pieces, different order**

$\mathbf{p} = (1, 0, 0)$, S = uniform scale 2, R = 90° about Z ($c = 0$, $s = 1$), T = translate by $(5, 0, 0)$.

1. $TRS\,\mathbf{p}$: scale gives $(2, 0, 0)$; rotate gives $(0, 2, 0)$; translate gives $(5, 2, 0)$.
2. $RTS\,\mathbf{p}$: scale gives $(2, 0, 0)$; translate gives $(7, 0, 0)$; rotate gives $(0, 7, 0)$.
3. $SRT\,\mathbf{p}$: translate gives $(6, 0, 0)$; rotate gives $(0, 6, 0)$; scale gives $(0, 12, 0)$.

Three different answers: **matrix multiplication is not commutative**. When something orbits the origin instead of spinning in place, the order is wrong.

TIP · Rotating around a pivot

To spin a door around its hinge rather than its centre, either translate the geometry so the hinge is at the origin (`geometry.translate(0.5, 0, 0)`), or nest it: a `Group` placed at the hinge, with the door as a child offset by half its width. Rotate the group. The same idea, $T_{\text{pivot}} R T_{\text{pivot}}^{-1}$, appears whenever you rotate about an arbitrary point.

three.js — a door on a hinge JAVASCRIPT

```
const hinge = new THREE.Group();
hinge.position.set(-0.5, 1, 0);                 // where the hinge is in the world
scene.add(hinge);
const door = new THREE.Mesh(new THREE.BoxGeometry(1, 2, 0.05), wood);
door.position.x = 0.5;                          // the door's centre is 0.5 m from the hinge
hinge.add(door);
hinge.rotation.y = -Math.PI / 3;                // opens 60° about the hinge, not the centre
```

R3F — the same nesting JSX

```
<group position={[-0.5, 1, 0]} rotation-y={open ? -Math.PI / 3 : 0}>
  <mesh position-x={0.5}>
    <boxGeometry args={[1, 2, 0.05]} />
    <meshStandardMaterial color="#8b5a2b" />
  </mesh>
</group>
```

4.4 The scene graph: local and world matrices

DEFINITION · Scene graph

A tree of `Object3D` s. Each node's transform is **relative to its parent**. Its **local matrix** (`object.matrix`) comes from its own position, rotation and scale; its **world matrix** (`object.matrixWorld`) is its parent's world matrix times its local matrix.

FORMULA · World matrix

$$M_{\text{world}}(\text{child}) = M_{\text{world}}(\text{parent}) \cdot M_{\text{local}}(\text{child})$$

For a chain root → a → b → c this unrolls to $M_{\text{root}} M_a M_b M_c$. three.js recomputes it in `updateMatrixWorld()` , which `renderer.render()` calls on the scene each frame.

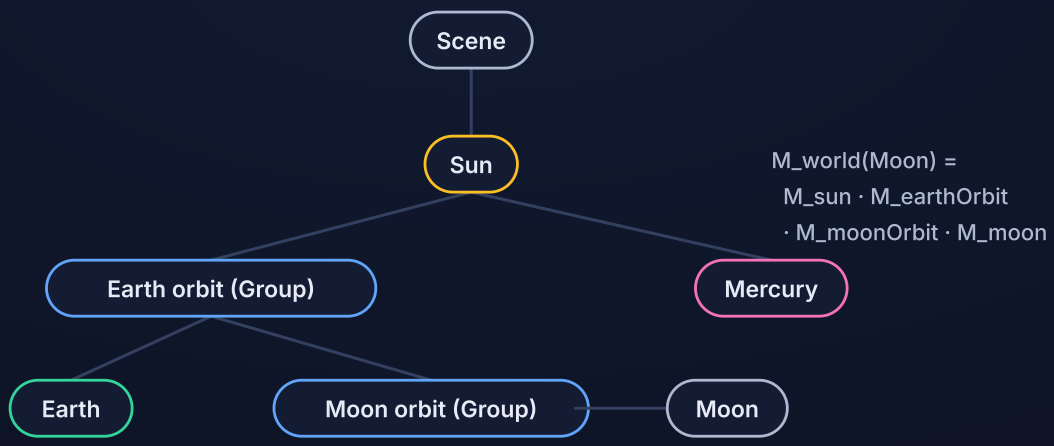


Figure 4.3 A solar system as a scene graph. The Moon only knows its offset from the Earth; it orbits the Sun automatically because its world matrix includes the Earth's and the Sun's transforms.

WORKED EXAMPLE · Where does the child end up?

A parent at $(10, 0, 0)$ rotated 90° about Y. Its child has local position $(0, 0, 2)$.

1. Rotate the child's offset by the parent's rotation: $R_y(90^\circ)(0, 0, 2) = (0 + 1 \cdot 2,\ 0,\ -1 \cdot 0 + 0 \cdot 2) = (2, 0, 0)$.
2. Add the parent's position: $(10, 0, 0) + (2, 0, 0) = (12, 0, 0)$.

`child.getWorldPosition(v)` returns $(12, 0, 0)$, while `child.position` still reads $(0, 0, 2)$.

three.js — local ↔ world conversions — JAVASCRIPT

```
const v = new THREE.Vector3();
child.getWorldPosition(v);                 // world position (updates world matrices first)
child.getWorldQuaternion(q);               // world orientation
child.localToWorld(v.set(0, 0.5, 0));      // a point in the child's space → world space
child.worldToLocal(v.copy(hitPoint));      // a world point → the child's space

newParent.attach(child);   // re-parent but KEEP the world transform (e.g. picking up an object)
newParent.add(child);      // re-parent and keep the LOCAL transform (the object jumps)
```

PITFALL · matrixAutoUpdate and static objects

Every frame three.js recomputes `matrix` from position/rotation/scale for every object with `matrixAutoUpdate = true` (the default). For thousands of static objects, set `matrixAutoUpdate = false` and call `object.updateMatrix()` once after placing them. Since r185, if you then write `object.matrix` yourself you must also set `object.matrixWorldNeedsUpdate = true`, because `updateWorldMatrix()` now honours that flag.

4.5 Inverses and the view matrix

DEFINITION · **Inverse matrix**

M^{-1} undoes M: $M^{-1}M = I$. It exists when the determinant is non-zero (the transformation does not squash space flat).

FORMULA · **Inverse of a TRS matrix**

$$(TRS)^{-1} = S^{-1}\,R^{-1}\,T^{-1} = S^{-1}\,R^{T}\,T(-\mathbf{t})$$

The order reverses. A rotation matrix is **orthonormal**, so its inverse is its transpose, which is much cheaper than a general inverse. S^{-1} just uses $1/s_x$, $1/s_y$, $1/s_z$.

The camera is an object like any other, with a world matrix that places it in the world. Rendering needs the opposite: positions **relative to the camera**. So the **view matrix** is the inverse of the camera's world matrix, `camera.matrixWorldInverse`.

WORKED EXAMPLE · **Camera at z = 5**

The camera sits at $(0, 0, 5)$ with no rotation. Its world matrix is a translation by $(0, 0, 5)$, so the view matrix is a translation by $(0, 0, -5)$.

- World origin $(0, 0, 0)$ → view space $(0, 0, -5)$: 5 m in front of the camera (the camera looks down −Z).
- A point at $(1, 2, 3)$ → $(1, 2, -2)$: 2 m in front, 1 m right, 2 m up.

4.6 The normal matrix

Normals are not ordinary directions. Under a **non-uniform** scale, transforming a normal with the model matrix tilts it, and it is no longer perpendicular to its surface. Lighting then goes wrong.

FORMULA · **Normal matrix**

$$N = \left(M_{3\times3}^{-1}\right)^{T}$$

Transform normals with the inverse-transpose of the upper-left 3×3 of the model (or model-view) matrix, then renormalise. For rotations and uniform scales N is just M up to a scale factor, which is why the problem only appears with non-uniform scale. three.js computes `normalMatrix` per object and the built-in shaders use it.

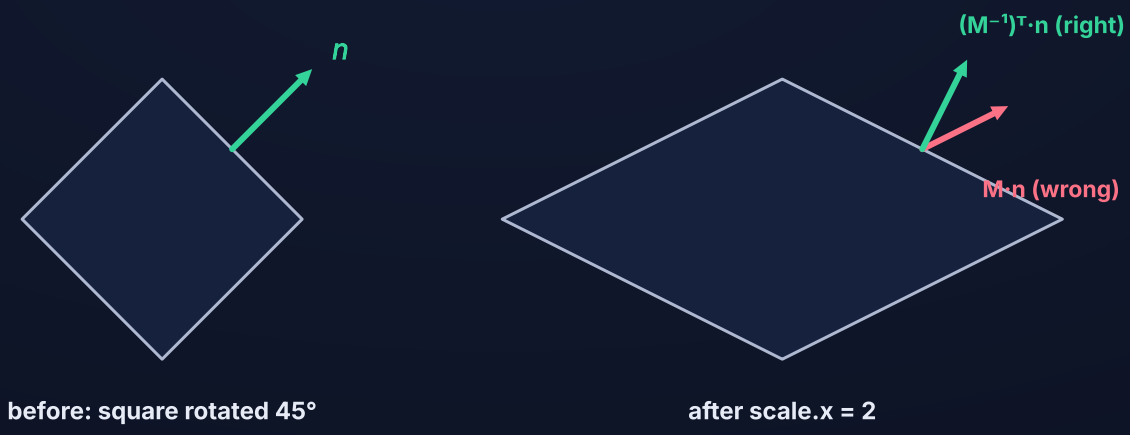


Figure 4.4 Squashing an object. Transforming the normal with the model matrix (red) leaves it tilted; the inverse-transpose (green) stays perpendicular to the surface.

WORKED EXAMPLE · Proving the inverse-transpose works

Surface with normal $\mathbf{n} = (1, 1, 0)$ (unnormalised) and a tangent lying in it $\mathbf{t} = (1, -1, 0)$; check $\mathbf{n} \cdot \mathbf{t} = 0$. Apply $M = \text{diag}(2, 1, 1)$ (stretch x by 2).

1. The tangent is a real direction on the surface, so it transforms by M: $\mathbf{t}' = (2, -1, 0)$.
2. Wrong: $M\mathbf{n} = (2, 1, 0)$; $\mathbf{t}' \cdot M\mathbf{n} = 4 - 1 = 3 \neq 0$. Not perpendicular.
3. Right: $N = (M^{-1})^T = \text{diag}(0.5, 1, 1)$; $N\mathbf{n} = (0.5, 1, 0)$; $\mathbf{t}' \cdot N\mathbf{n} = 1 - 1 = 0$. Perpendicular.

GLSL — what three.js's vertex shader does with normals GLSL

```glsl
// built-in uniforms injected by three.js for ShaderMaterial
vec3 transformedNormal = normalMatrix * normal;          // inverse-transpose of modelView (3x3)
vec3 vNormal = normalize(transformedNormal);             // view-space normal for lighting
vec4 mvPosition = modelViewMatrix * vec4(position, 1.0);
gl_Position = projectionMatrix * mvPosition;
```

4.7 Matrix4 in three.js

Task	Code
Build from parts	`m.compose(position, quaternion, scale)`
Split into parts	`m.decompose(position, quaternion, scale)`
Transform a point	`v.applyMatrix4(m)` (divides by w too)
Transform a direction	`v.transformDirection(m)` (ignores translation, normalises)
Combine	`a.multiply(b)` gives $a\,b$; `a.premultiply(b)` gives $b\,a$; `c.multiplyMatrices(a, b)`
Invert	`m.clone().invert()`
Normal matrix	`new THREE.Matrix3().getNormalMatrix(m)`
Bake into geometry	`geometry.applyMatrix4(m)` (changes vertices permanently)

⚠ PITFALL · **Column-major storage, row-major arguments**

`Matrix4.elements` is a flat array stored **column by column** (as WebGL expects): `elements[12], [13], [14]` hold the translation. But `Matrix4.set(n11, n12, n13, n14, …)` takes its sixteen arguments **row by row**, as you would write the matrix on paper. Mixing these up transposes the matrix.

4.7.1 Determinant and mirroring

DEFINITION · **Determinant**

For a 3×3 matrix, the signed factor by which it scales volume. $\det = s_x s_y s_z$ for a scale matrix, 1 for a rotation. A **negative** determinant means the transformation mirrors space (an odd number of negative scale factors), which also reverses triangle winding.

three.js checks the sign of each object's world-matrix determinant and flips its front-face winding automatically, so `scale.x = -1` mirrors a mesh without turning it inside out. A determinant of zero (a scale of 0 on any axis) makes the matrix non-invertible: normals and raycasting break, so use a tiny value like `1e-5` when you animate scale to zero.

✓ KEY POINTS

- ✓ Matrix columns are where the basis axes land; 4×4 matrices with w add translation.
- ✓ Points use $w = 1$, directions $w = 0$; `applyMatrix4` vs `transformDirection`.
- ✓ $M = TRS$, applied right to left: scale, then rotate, then translate. Order changes the result.
- ✓ World matrix = parent world × local. Use `attach` to re-parent without a jump.
- ✓ View matrix = inverse of the camera's world matrix; $(TRS)^{-1} = S^{-1}R^{T}T^{-1}$.
- ✓ Normals use the inverse-transpose; it differs from M only under non-uniform scale.
- ✓ `Matrix4.elements` is column-major; `Matrix4.set()` is row-major.

4.8 Interview questions

Q **Explain the difference between `object.matrix` and `object.matrixWorld`.** JUNIOR}

A `matrix` is the local transform relative to the parent, composed from `position`, `quaternion` and `scale` (TRS). `matrixWorld` is the transform relative to the scene root: the parent's `matrixWorld` multiplied by `matrix`. three.js updates both in `updateMatrixWorld()` during `render()`. If you need a world position in the middle of a frame (before rendering), call `getWorldPosition()`, which updates the chain first.

Q **An object orbits around the origin instead of spinning in place. What is wrong?** JUNIOR}

A The rotation is being applied after the translation (RT instead of TR), so it rotates the already-moved position around the world origin. In three.js this usually comes from rotating a parent group that sits at the origin while the mesh is offset inside it, or from baking a translation into the geometry. Fix: rotate the object itself (or a group placed at the object's position), or move the geometry's pivot with `geometry.center()` / `geometry.translate()`.

Q **Why can't you transform normals with the model matrix?** MID}

A Normals must stay perpendicular to the surface. Tangent directions transform with M, so for $\mathbf{n}$ to stay perpendicular to every transformed tangent ($\mathbf{n}'^T M\mathbf{t} = 0$ whenever $\mathbf{n}^T\mathbf{t} = 0$), we need $\mathbf{n}' = (M^{-1})^T\mathbf{n}$. Under rotation and uniform scale the inverse-transpose equals M up to scale, so the mistake is invisible; with non-uniform scale lighting looks wrong. three.js provides `normalMatrix` for this.

Q **How would you place 10,000 static trees efficiently, transform-wise?** MID}

A Use one `InstancedMesh` so all trees are one draw call, and compose each instance matrix once: `matrix.compose(position, quaternion, scale)` then `setMatrixAt(i, matrix)` and `instanceMatrix.needsUpdate = true`. Do not create 10,000 `Object3D`s. If they were separate meshes, `matrixAutoUpdate = false` and one `updateMatrix()` call would at least avoid recomputing local matrices every frame. Call `computeBoundingSphere()` on the instanced mesh so frustum culling works for the whole set.

Q What does `attach()` do differently from `add()` ? MID}

A `add()` keeps the object's local transform, so its world transform changes to follow the new parent (it visually jumps). `attach()` computes the local transform that keeps the **world** transform the same under the new parent, using the inverse of the new parent's world matrix. Use `attach` for picking up and dropping objects, moving items between rooms, or grabbing in VR.

Q What does a negative determinant tell you, and how does three.js handle it? SENIOR}

A The transformation mirrors space, for example `scale.x = -1`. Mirroring reverses triangle winding, so front faces would become back faces and be culled. three.js computes the determinant of each object's world matrix and flips the GPU's front-face setting for negative values, so mirrored meshes render correctly. A zero determinant (zero scale) is singular: no inverse, so normal matrices and raycasting fail.

Rotations and Quaternions

05

Four ways to describe an orientation, why Euler angles suffer gimbal lock, the axis–angle formula, quaternions from first principles with hand-worked rotations, slerp for smooth turning, and how lookAt builds a rotation from a direction.

IN THIS CHAPTER

5.1 Four ways to describe a rotation
5.2 Euler angles and gimbal lock
5.3 Axis–angle and Rodrigues' formula
5.4 Quaternions
5.5 Slerp: smooth interpolation between orientations
5.6 Useful quaternion recipes
5.7 How lookAt builds a rotation
5.8 Interview questions

5.1 Four ways to describe a rotation

Representation	Numbers	Strengths	Weaknesses	three.js
Euler angles	3 (+ an order)	Human-readable; easy UI sliders	Gimbal lock; order-dependent; poor interpolation	`object.rotation` (`THREE.Euler`)
Axis–angle	4 (unit axis + angle)	Intuitive for one rotation	Composing two is awkward	`setFromAxisAngle`, `rotateOnAxis`
Rotation matrix	9	Directly usable by the GPU; composes by multiplication	Redundant; drifts from orthonormal under repeated maths	`Matrix4` / `Matrix3`
Quaternion	4	Compact; no gimbal lock; smooth interpolation (slerp); cheap to compose	Not intuitive to read	`object.quaternion` (`THREE.Quaternion`)

`object.rotation` and `object.quaternion` are **kept in sync** by three.js: change one and the other updates. Use Euler angles to set simple orientations and quaternions for anything animated, accumulated or interpolated.

5.2 Euler angles and gimbal lock

DEFINITION · Euler angles

An orientation given as three successive rotations about coordinate axes, plus the **order** in which they are applied. three.js's default order is `"XYZ"`: the rotation matrix is $R = R_x(\alpha)\,R_y(\beta)\,R_z(\gamma)$. Read as rotations about the object's own (moving) axes, it turns about X first, then about its new Y, then its new Z.

TIP · Use "YXZ" for cameras and characters

For a first-person camera you want yaw (turn left/right about world Y) to be independent of pitch (look up/down). With order `"YXZ"` the object yaws first, then pitches about its own X, then rolls. Set `camera.rotation.order = "YXZ"`; `PointerLockControls` does exactly this.

DEFINITION · Gimbal lock

The loss of one degree of freedom when the middle rotation of an Euler sequence reaches ±90°, which lines up the first and third rotation axes. Two of the three angles then turn the object about the same axis, so one direction of rotation becomes impossible to reach with small changes, and interpolation takes strange paths.

Figure 5.1 Three nested gimbals. Normally each ring adds an independent axis (left). When the middle ring turns 90°, the outer and inner axes coincide (right): two angles now do the same thing.

WORKED EXAMPLE · Seeing gimbal lock in the matrix

Order `"XYZ"` with $\beta = 90^\circ$ ($\cos\beta = 0$, $\sin\beta = 1$). Multiplying out $R_x(\alpha)R_y(90^\circ)R_z(\gamma)$ gives a matrix whose entries depend only on $\alpha + \gamma$, for example its first row is $(0,\ 0,\ 1)$ and the second is $(\sin(\alpha+\gamma),\ \cos(\alpha+\gamma),\ 0)$. So $(\alpha, \gamma) = (30^\circ, 0^\circ)$ and $(0^\circ, 30^\circ)$ give the **same** orientation: one degree of freedom has gone.

5.3 Axis–angle and Rodrigues' formula

Any 3D rotation, however complicated, is a single turn by some angle θ about some unit axis $\hat{\mathbf{k}}$ (Euler's rotation theorem).

Σ FORMULA · Rodrigues' rotation formula

$$\mathbf{v}_{\text{rot}} = \mathbf{v}\cos\theta + (\hat{\mathbf{k}} \times \mathbf{v})\sin\theta + \hat{\mathbf{k}}\,(\hat{\mathbf{k}} \cdot \mathbf{v})(1 - \cos\theta)$$

The part of $\mathbf{v}$ along the axis stays put; the perpendicular part turns in the plane perpendicular to $\hat{\mathbf{k}}$.

WORKED EXAMPLE · 90° about Y with Rodrigues

$\mathbf{v} = (1, 0, 0)$, $\hat{\mathbf{k}} = (0, 1, 0)$, $\theta = 90^\circ$.

1. $\mathbf{v}\cos\theta = (0, 0, 0)$.
2. $\hat{\mathbf{k}} \times \mathbf{v} = (1\cdot 0 - 0\cdot 0,\ 0\cdot 1 - 0\cdot 0,\ 0\cdot 0 - 1\cdot 1) = (0, 0, -1)$, times $\sin\theta = 1$.
3. $\hat{\mathbf{k}} \cdot \mathbf{v} = 0$, so the last term vanishes.
4. Result $(0, 0, -1)$, the same answer as the matrix $R_y(90^\circ)$ in Chapter 4.

5.4 Quaternions

DEFINITION · Quaternion

A four-component number $q = (x, y, z, w) = (\mathbf{u}, w)$ with a vector part $\mathbf{u}$ and a scalar part w. A **unit** quaternion ($x^2 + y^2 + z^2 + w^2 = 1$) represents a rotation by angle θ about unit axis $\hat{\mathbf{k}}$:

$$q = \left(\hat{\mathbf{k}}\sin\tfrac{\theta}{2},\ \cos\tfrac{\theta}{2}\right)$$

The identity (no rotation) is $(0, 0, 0, 1)$. three.js stores quaternions in the order `x, y, z, w`.

Why the half angle? A quaternion rotates a vector by "sandwiching" it, $q\,\mathbf{v}\,q^*$, and each side contributes half of the turn. A side effect: q and $-q$ describe the **same** rotation (a turn of θ and of $\theta + 360^\circ$), which is called the **double cover**.

Σ FORMULA · Quaternion product (composition)

$$q_1 q_2 = \left(w_1\mathbf{u}_2 + w_2\mathbf{u}_1 + \mathbf{u}_1 \times \mathbf{u}_2,\ \ w_1 w_2 - \mathbf{u}_1 \cdot \mathbf{u}_2\right)$$

$q_1 q_2$ means "rotate by q_2, then by q_1" (like matrices, the right-most acts first). It is **not commutative**. The inverse of a unit quaternion is its conjugate $q^* = (-\mathbf{u}, w)$.

Σ FORMULA · Rotating a vector with a quaternion

$$\mathbf{v}' = q\,\mathbf{v}\,q^* = \mathbf{v} + 2w\,(\mathbf{u}\times\mathbf{v}) + 2\,\mathbf{u}\times(\mathbf{u}\times\mathbf{v})$$

The right-hand form is what `Vector3.applyQuaternion` effectively computes: two cross products, no trigonometry.

WORKED EXAMPLE · Building and applying a quaternion by hand

Rotate $\mathbf{v} = (1,0,0)$ by 90° about Y.

1. $q = (\hat{\mathbf{k}}\sin 45^\circ, \cos 45^\circ) = (0,\ 0.7071,\ 0,\ 0.7071)$; so $\mathbf{u} = (0, 0.7071, 0)$, $w = 0.7071$.
2. $\mathbf{u}\times\mathbf{v} = (0.7071\cdot 0 - 0\cdot 0,\ 0\cdot 1 - 0\cdot 0,\ 0\cdot 0 - 0.7071\cdot 1) = (0, 0, -0.7071)$.
3. $2w(\mathbf{u}\times\mathbf{v}) = 2(0.7071)(0,0,-0.7071) = (0,0,-1)$.
4. $\mathbf{u}\times(\mathbf{u}\times\mathbf{v}) = (0.7071\cdot(-0.7071) - 0,\ 0-0,\ 0-0) = (-0.5,0,0)$; doubled: $(-1,0,0)$.
5. $\mathbf{v}' = (1,0,0) + (0,0,-1) + (-1,0,0) = (0,0,-1)$. Matches Rodrigues and the matrix.

WORKED EXAMPLE · Composing: two quarter turns make a half turn

$q\,q$ with $q = (0, 0.7071, 0, 0.7071)$:

- Vector part: $w\mathbf{u} + w\mathbf{u} + \mathbf{u}\times\mathbf{u} = 2(0.7071)(0, 0.7071, 0) + \mathbf{0} = (0,1,0)$.
- Scalar part: $w^2 - \mathbf{u}\cdot\mathbf{u} = 0.5 - 0.5 = 0$.
- $(0,1,0,0) = (\hat{\mathbf{y}}\sin 90^\circ, \cos 90^\circ)$: a 180° turn about Y.

TIP · Renormalise accumulated quaternions

Multiplying many quaternions (for example applying a small rotation every frame) slowly drifts their length away from 1 through floating-point error, which skews the object. Call `q.normalize()` occasionally. Matrices drift too, and are much harder to fix (re-orthonormalisation), one more reason to accumulate rotations as quaternions.

5.5 Slerp: smooth interpolation between orientations

Interpolating Euler angles component by component gives wobbly, non-uniform paths and can take the long way round. Interpolating quaternions along the great circle of the 4D unit sphere gives the **shortest** rotation at **constant** angular speed.

Σ FORMULA · **Spherical linear interpolation (slerp)**

$$\operatorname{slerp}(q_0, q_1, t) = \frac{\sin\big((1-t)\Omega\big)}{\sin\Omega}\, q_0 + \frac{\sin(t\Omega)}{\sin\Omega}\, q_1, \qquad \cos\Omega = q_0 \cdot q_1$$

If $q_0 \cdot q_1 < 0$, negate q_1 first (allowed by the double cover) so the path takes the short way. When Ω is tiny, fall back to normalised lerp to avoid dividing by $\sin\Omega \approx 0$. three.js's `Quaternion.slerp` does all of this.

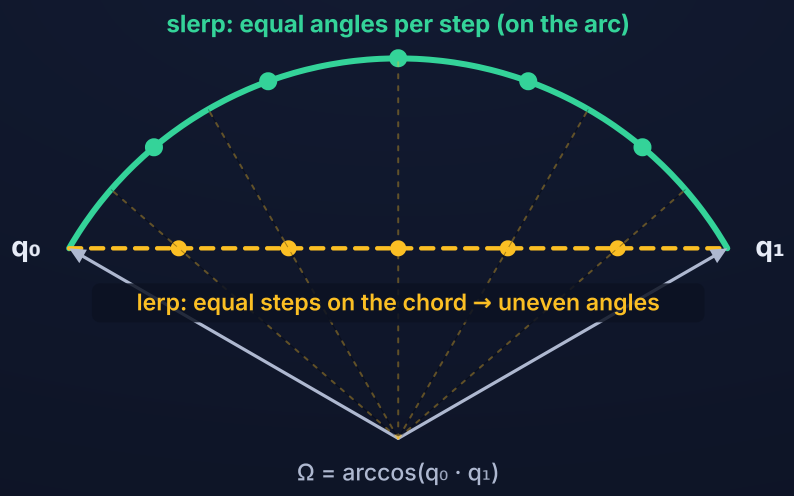


Figure 5.2 Lerp moves along the straight chord and must be renormalised, which makes it speed up in the middle. Slerp moves along the arc at constant angular speed. For small angles the two are almost identical, so nlerp (normalised lerp) is a common cheap substitute.

WORKED EXAMPLE · **Halfway between "no rotation" and "90° about Y"**

$q_0 = (0, 0, 0, 1)$, $q_1 = (0, 0.7071, 0, 0.7071)$, $t = 0.5$.

1. $\cos\Omega = q_0 \cdot q_1 = 0.7071$, so $\Omega = 45^\circ$ (in quaternion space, half the rotation angle).
2. Both weights $= \sin(22.5^\circ)/\sin(45^\circ) = 0.3827/0.7071 = 0.5412$.
3. $q = 0.5412\,(0, 0, 0, 1) + 0.5412\,(0, 0.7071, 0, 0.7071) = (0,\ 0.3827,\ 0,\ 0.9239)$.
4. That is $(\hat{\mathbf{y}} \sin 22.5^\circ, \cos 22.5^\circ)$: exactly **45°** about Y, as expected.

three.js — turn a turret smoothly towards a target JAVASCRIPT

```
const targetQ = new THREE.Quaternion();
const m = new THREE.Matrix4();
const UP = new THREE.Vector3(0, 1, 0);

function aim(turret, targetPos, dt) {
  // a rotation matrix that points the object's -Z at the target (camera-style lookAt)
  m.lookAt(turret.position, targetPos, UP);
  targetQ.setFromRotationMatrix(m);
  // Option 1: exponential smoothing (frame-rate independent)
  turret.quaternion.slerp(targetQ, 1 - Math.exp(-8 * dt));
  // Option 2: a fixed maximum turn speed of 90° per second
  // turret.quaternion.rotateTowards(targetQ, THREE.MathUtils.degToRad(90) * dt);
}
```

R3F — the same inside useFrame JSX

```
function Turret({ target }) {
  const ref = useRef();
  const tmp = useMemo(() => ({ m: new THREE.Matrix4(), q: new THREE.Quaternion(), up: new
THREE.Vector3(0, 1, 0) }), []);
  useFrame((_, dt) => {
    const t = ref.current;
    tmp.m.lookAt(t.position, target.current.position, tmp.up);
    tmp.q.setFromRotationMatrix(tmp.m);
    t.quaternion.slerp(tmp.q, 1 - Math.exp(-8 * dt));
  });
  return <mesh ref={ref}><coneGeometry args={[0.3, 1, 16]} /><meshStandardMaterial /></mesh>;
}
```

5.6 Useful quaternion recipes

Goal	three.js
Rotation of θ about an axis	`q.setFromAxisAngle(axis, theta)` (axis must be unit length)
The rotation that turns direction $\mathbf{a}$ into $\mathbf{b}$	`q.setFromUnitVectors(a, b)` (both unit length)
From Euler angles	`q.setFromEuler(new THREE.Euler(x, y, z, "YXZ"))`
Apply in world space	`object.rotateOnWorldAxis(axis, angle)` or `object.quaternion.premultiply(q)`
Apply in local space	`object.rotateOnAxis(axis, angle)` or `object.quaternion.multiply(q)`
Angle between two orientations	`q1.angleTo(q2)` (radians)
Rotate a vector	`v.applyQuaternion(q)`

WORKED EXAMPLE · **Aligning a model's up axis with a surface normal**

A tree should stand perpendicular to a slope with unit normal $\hat{\mathbf{n}} = (0.6, 0.8, 0)$. `q.setFromUnitVectors((0,1,0), n)` gives the rotation about the axis $\hat{\mathbf{y}} \times \hat{\mathbf{n}} = (0, 0, -0.6)$ (normalised $(0, 0, -1)$) by the angle $\arccos(\hat{\mathbf{y}} \cdot \hat{\mathbf{n}}) = \arccos(0.8) = 36.87^\circ$. The tree tilts 36.87° towards +X, matching the slope.

5.7 How lookAt builds a rotation

`lookAt` turns "look from here towards there" into an orientation by building three perpendicular unit axes, which become the columns of a rotation matrix.

Σ FORMULA · Look-at basis

For an eye at E looking at target T with a world up hint $\mathbf{up}$:

$$\hat{\mathbf{z}} = \text{normalise}(E - T), \quad \hat{\mathbf{x}} = \text{normalise}(\mathbf{up} \times \hat{\mathbf{z}}), \quad \hat{\mathbf{y}} = \hat{\mathbf{z}} \times \hat{\mathbf{x}}$$

$R = [\,\hat{\mathbf{x}}\ \hat{\mathbf{y}}\ \hat{\mathbf{z}}\,]$. The camera's local −Z then points at the target. three.js's `Object3D.lookAt` uses this for cameras and lights, and the reverse ($+Z$ towards the target) for other objects.

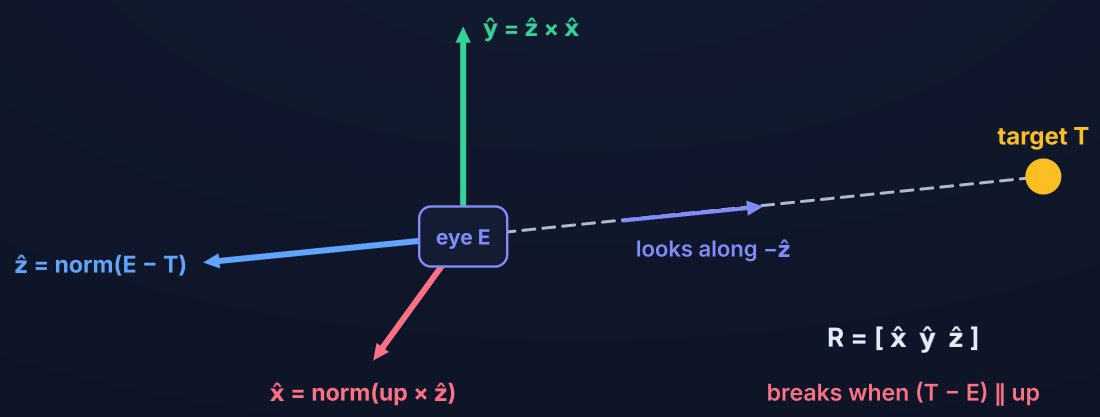


Figure 5.3 lookAt builds a frame from the view direction and the up hint. If the view direction is parallel to up (looking straight down), the cross product is zero and the frame is undefined, which causes a sudden flip.

WORKED EXAMPLE · A camera looking at the origin from (0, 0, 5)

$\hat{\mathbf{z}} = \text{normalise}((0,0,5) - (0,0,0)) = (0,0,1)$; $\hat{\mathbf{x}} = (0,1,0) \times (0,0,1) = (1,0,0)$; $\hat{\mathbf{y}} = (0,0,1) \times (1,0,0) = (0,1,0)$. The identity rotation, as expected for a camera on +Z looking down −Z.

⚠ PITFALL · Looking straight up or down

When the view direction is (almost) parallel to `object.up`, $\mathbf{up} \times \hat{\mathbf{z}} \approx \mathbf{0}$ and the orientation flips or spins. Top-down cameras should set `camera.up.set(0, 0, -1)` (or another axis not parallel to the view) before calling `lookAt`. `OrbitControls` clamps the polar angle slightly short of the poles for the same reason.

✓ KEY POINTS

- ✓ `rotation` (Euler) and `quaternion` are synced; default Euler order is `"XYZ"`, use `"YXZ"` for cameras.
- ✓ Gimbal lock: the middle Euler angle at ±90° aligns the other two axes.
- ✓ Unit quaternion $= (\hat{\mathbf{k}} \sin\frac{\theta}{2}, \cos\frac{\theta}{2})$; q and $-q$ are the same rotation.
- ✓ $q_1 q_2$ applies q_2 first; rotate a vector with $\mathbf{v} + 2w(\mathbf{u} \times \mathbf{v}) + 2\mathbf{u} \times (\mathbf{u} \times \mathbf{v})$.
- ✓ Slerp gives constant-speed, shortest-path interpolation; negate when the dot product is negative.
- ✓ `setFromUnitVectors`, `rotateTowards`, `slerp` with $1 - e^{-\lambda \Delta t}$ cover most gameplay rotation needs.
- ✓ lookAt builds $[\hat{\mathbf{x}}\ \hat{\mathbf{y}}\ \hat{\mathbf{z}}]$ with cross products; it breaks when looking along `up`.

5.8 Interview questions

Q What is gimbal lock and how do quaternions avoid it? MID}

A Euler angles apply three rotations in sequence. When the middle one reaches ±90°, the first and third axes line up, so two angles produce the same motion and a degree of freedom is lost; interpolation then follows odd paths. A quaternion represents the whole orientation as one rotation about one axis (a point on the 4D unit sphere), with no sequence of axes to collapse, so it has no gimbal lock. Gimbal lock is a problem of the **parametrisation**, not of rotations themselves.

Q Why does a unit quaternion use half the angle, and what is the double cover? SENIOR}

A A quaternion rotates a vector by conjugation, $q\mathbf{v}q^*$, and each multiplication contributes half the rotation, so q stores $\sin\frac{\theta}{2}$ and $\cos\frac{\theta}{2}$. Consequently q and $-q$ give the same rotation (adding 360° to θ flips both signs). When interpolating you must pick the representative closer to the start ($q_0 \cdot q_1 \geq 0$), otherwise slerp takes the long way round (a 350° turn instead of 10°).

Q How would you make an object turn towards a moving target at a limited speed? MID}

A Each frame compute the desired orientation (for example `Matrix4.lookAt` then `setFromRotationMatrix`, or `setFromUnitVectors` for a single axis), then move the current quaternion towards it: `quaternion.rotateTowards(target, maxAngularSpeed * dt)` for a hard speed limit, or `slerp(target, 1 - Math.exp(-k * dt))` for smooth exponential easing. Both are frame-rate independent. Avoid interpolating Euler angles, which can spin the long way when an angle wraps past ±180°.

Q What is the difference between `multiply` and `premultiply` on a quaternion? MID

A `q.multiply(r)` sets $q = q\,r$: r is applied first, in the object's **local** frame (a local-space rotation). `q.premultiply(r)` sets $q = r\,q$: r is applied after, in the **world** frame. So turning a spaceship about its own nose axis uses `multiply`, while rotating it about the world's vertical axis uses `premultiply` (or `rotateOnWorldAxis`).

Q A top-down camera flips randomly when you call lookAt. Why? MID

A `lookAt` builds the right axis as $\mathbf{up} \times \hat{\mathbf{z}}$. With the default up $(0, 1, 0)$ and a view straight down, the view direction is parallel to up, the cross product is (nearly) zero, and normalising it is unstable, so the orientation flips. Set a different `camera.up` (for example $(0, 0, -1)$) for top-down views, or use an orthographic camera with a fixed rotation.

Cameras and Projection

06

The pinhole camera, field of view and focal length, the perspective and orthographic projection matrices worked through with numbers, why depth precision is non-linear and how reversed-Z fixes it, frustum culling, converting between screen and world, and orbit-camera maths.

IN THIS CHAPTER

6.1 The pinhole camera
6.2 The perspective projection matrix
6.3 Depth precision is not linear
6.4 The orthographic camera
6.5 Frustum culling
6.6 From screen to world and back
6.7 Orbit cameras and spherical coordinates
6.8 Interview questions

6.1 The pinhole camera

A real camera, and our virtual one, works like a pinhole: light from a point in the world travels in a straight line through the pinhole onto the image plane. Objects twice as far away appear half as big. That one rule, **size divides by distance**, is perspective.

DEFINITION · **Field of view (FOV)**

The angle the camera can see. three.js's `PerspectiveCamera(fov, aspect, near, far)` takes the **vertical** FOV in **degrees**; the horizontal FOV follows from the aspect ratio: $\tan\frac{\text{fov}_h}{2} = \text{aspect} \cdot \tan\frac{\text{fov}_v}{2}$. A small FOV (20–30°) looks like a telephoto lens; 50–60° feels natural; above 90° distorts edges.

DEFINITION · **View frustum**

The truncated pyramid of space the camera can see, bounded by six planes: left, right, top, bottom, **near** and **far**. Anything outside it is not drawn. Its shape is fully described by the projection matrix.

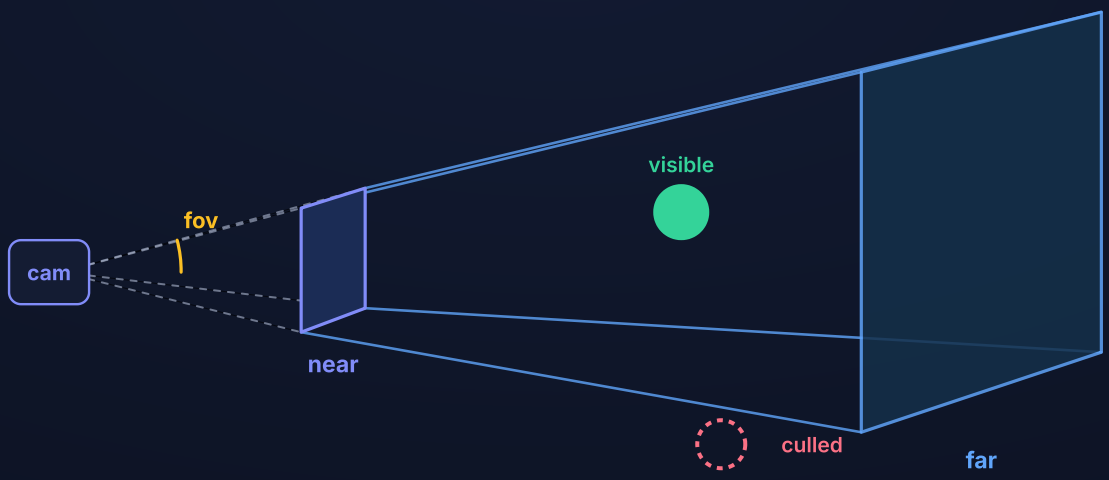


Figure 6.1 The perspective view frustum. Points between the near and far planes inside the pyramid are visible. The projection squeezes this pyramid into a cube (NDC).

Σ FORMULA · **Visible size at a distance**

$$h = 2\,d\,\tan\frac{\text{fov}}{2}, \qquad w = h \cdot \text{aspect}$$

h and w are the height and width of the slice of the world visible at distance d in front of the camera.

WORKED EXAMPLE · **How much of the world fits on screen?**

FOV 50°, camera 10 m from a wall, aspect 16:9.

1. $h = 2 \times 10 \times \tan 25^\circ = 2 \times 10 \times 0.4663 = 9.33$ m.
2. $w = 9.33 \times 1.778 = 16.58$ m.

A 1 m cube on that wall therefore covers about $1/9.33 \approx 10.7\%$ of the screen height: 116 px on a 1080 px tall canvas.

Σ FORMULA · **Fitting an object in view**

For a bounding sphere of radius r, the distance at which it just fits the vertical FOV is

$$d = \frac{r}{\sin(\text{fov}/2)}$$

(use the smaller of the vertical and horizontal FOV on narrow screens). Multiply by 1.1–1.3 for a margin.

WORKED EXAMPLE · **Framing a product**

A shoe with bounding-sphere radius $r = 0.15$ m and FOV 50°: $d = 0.15/\sin 25^\circ = 0.15/0.4226 = 0.355$ m; with a 1.2× margin, put the camera 0.43 m from the shoe's centre. On a portrait phone with aspect 0.5, the horizontal FOV is $2\arctan(0.5\tan 25^\circ) = 26.3^\circ$, so fit to that instead: $d = 0.15/\sin 13.1^\circ = 0.66$ m.

three.js — frame any object JAVASCRIPT

```
function frame(object, camera, controls, margin = 1.2) {
  const sphere = new THREE.Box3().setFromObject(object).getBoundingSphere(new THREE.Sphere());
  const vFov = THREE.MathUtils.degToRad(camera.fov);
  const hFov = 2 * Math.atan(Math.tan(vFov / 2) * camera.aspect);
  const dist = (sphere.radius / Math.sin(Math.min(vFov, hFov) / 2)) * margin;
  const dir = camera.position.clone().sub(controls.target).normalize();   // keep the current
view direction
  camera.position.copy(sphere.center).addScaledVector(dir, dist);
  camera.near = dist / 100; camera.far = dist * 100;                    // keep near/far
proportional
  camera.updateProjectionMatrix();
  controls.target.copy(sphere.center); controls.update();
}
```

R3F — drei does the same with <Bounds> JSX

```
import { Bounds, OrbitControls } from "@react-three/drei";
const Viewer = () => (
  <>
    <Bounds fit clip observe margin={1.2}>   {/* fit on load, adjust near/far, refit on resize
*/}
      <Shoe />
    </Bounds>
    <OrbitControls makeDefault />
  </>
);
```

6.1.1 Focal length

Photographers think in **focal length** (mm) instead of FOV. three.js models a 35 mm film (`camera.filmGauge = 35`), and `camera.setFocalLength(mm)` converts: $\text{fov}_v = 2\arctan\left(\frac{h_{\text{film}}/2}{\text{focal}}\right)$, where $h_{\text{film}} = 35/\text{aspect}$ for landscape canvases. A 50 mm lens on a 3:2 canvas gives $2\arctan(11.67/50) = 26.3^\circ$. Matching focal lengths makes three.js views match product photography or Blender cameras.

6.2 The perspective projection matrix

The projection matrix maps view-space positions into **clip space**. After the GPU divides by w, the visible frustum becomes the NDC cube. three.js (WebGL convention) maps x, y, z to $[-1, 1]$.

Σ FORMULA · Perspective matrix (three.js / OpenGL convention)

With $s = 1/\tan(\text{fov}/2)$, aspect a, near n and far f:

$$P = \begin{pmatrix} s/a & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & \dfrac{f+n}{n-f} & \dfrac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

The bottom row copies $-z_{\text{view}}$ (the distance in front of the camera) into w. Dividing by w is what makes far things small.

WORKED EXAMPLE · Projecting a point all the way to a pixel

FOV 60°, 1920×1080 canvas ($a = 1.778$), $n = 0.1$, $f = 100$. A point in view space $(1, 0.5, -5)$: 5 m in front, 1 m right, 0.5 m up.

1. $s = 1/\tan 30^\circ = 1.732$; $s/a = 0.974$.
2. Clip: $x_c = 0.974$, $y_c = 1.732 \times 0.5 = 0.866$, $z_c = \frac{100.1}{-99.9}(-5) + \frac{20}{-99.9} = 5.010 - 0.200 = 4.810$, $w_c = 5$.
3. NDC (divide by w): $(0.195,\ 0.173,\ 0.962)$. All within $[-1, 1]$, so it is visible.
4. Pixels: $x_{px} = \frac{0.195+1}{2} \times 1920 = 1147$; $y_{px} = \frac{1-0.173}{2} \times 1080 = 446$ (measured from the top).

NOTE · WebGPU uses a 0 to 1 depth range

WebGPU (and Direct3D, Metal, Vulkan) put NDC z in $[0, 1]$, not $[-1, 1]$. three.js handles this: each camera has a `coordinateSystem` (`WebGLCoordinateSystem` or `WebGPUCoordinateSystem`) that `WebGPURenderer` sets, and `makePerspective` builds the matching matrix. Remember it when you reconstruct positions from a depth texture in custom shaders.

6.3 Depth precision is not linear

Σ FORMULA · NDC depth as a function of distance

For a point at distance d in front of the camera:

$$z_{\text{ndc}}(d) = \frac{(f+n)\,d - 2fn}{(f-n)\,d}, \qquad \text{stored depth} = \frac{z_{\text{ndc}} + 1}{2}$$

This is a **hyperbola** in d: it changes quickly near the camera and hardly at all far away. Almost all of the depth buffer's resolution is spent close to the near plane.

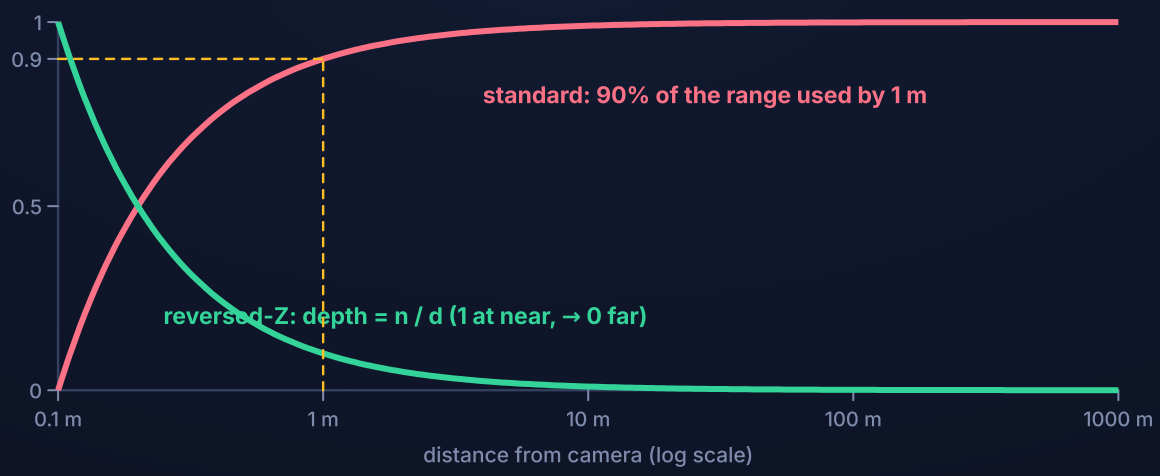


Figure 6.2 Stored depth against distance for near = 0.1 m and far = 1000 m. Ninety per cent of the range is used up in the first metre. Reversed-Z with a floating-point buffer spreads precision almost evenly by placing the dense part of float precision where the curve is flat.

WORKED EXAMPLE · Where did the precision go?

$n = 0.1$, $f = 1000$:

Distance	z_{ndc}	Stored depth
0.1 m	−1.000	0.0000
1 m	0.8002	0.9001
10 m	0.9802	0.9901
100 m	0.9982	0.9991
500 m	0.9998	0.9999

Everything from 1 m to 1 km shares the last 10% of the range. A 24-bit depth buffer has about 16.8 million steps, but only about 15,000 of them are left between 100 m and 1000 m, and about 1,700 between 500 m and 1000 m: one step every 30 cm, so distant surfaces close together z-fight. Increasing n from 0.1 to 1 helps far more than reducing f, because the curve is controlled mostly by n.

DEFINITION · Reversed-Z

Mapping the near plane to depth 1 and the far plane to 0, combined with a 32-bit **floating-point** depth buffer. Floats are densest near 0, which is exactly where the hyperbola is flattest (far away), so the two effects cancel and precision becomes nearly uniform. NVIDIA's analysis shows near-zero depth errors with float reversed-Z, even with an infinite far plane.

three.js — options for large scenes JAVASCRIPT

```
// 1) Reversed depth (needs the EXT_clip_control extension on WebGL; named reversedDepthBuffer since r179)
const renderer = new THREE.WebGLRenderer({ antialias: true, reversedDepthBuffer: true });
// 2) Logarithmic depth: writes gl_FragDepth per fragment, which disables early-z (slower)
const renderer2 = new THREE.WebGLRenderer({ logarithmicDepthBuffer: true });
// 3) Usually enough: a sensible near/far ratio
camera.near = 0.5; camera.far = 2000;      // ratio 4000:1 instead of 10,000,000:1
```

6.4 The orthographic camera

DEFINITION • **Orthographic projection**

A projection with no perspective: parallel lines stay parallel and size does not change with distance. The frustum is a box. Used for 2D games, UI overlays, CAD and architectural drawings, isometric games, and **shadow maps of directional lights**.

FORMULA • **Orthographic matrix**

For a box with left l, right r, top t, bottom b, near n, far f:

$$P_{\text{ortho}} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

w stays 1, so there is no perspective divide, and depth is **linear** in distance.

three.js — an orthographic camera that keeps 10 m of height visible JAVASCRIPT

```
const H = 10;
const a = innerWidth / innerHeight;
const ortho = new THREE.OrthographicCamera(-H * a / 2, H * a / 2, H / 2, -H / 2, 0.1, 100);
ortho.position.set(10, 10, 10); ortho.lookAt(0, 0, 0);   // a classic isometric-style view
ortho.zoom = 2; ortho.updateProjectionMatrix();          // zoom scales the box, not the position
```

R3F — orthographic canvas JSX

```
<Canvas orthographic camera={{ zoom: 80, position: [10, 10, 10] }}>   {/* zoom = pixels per unit */}
  <Scene />
</Canvas>
```

6.5 Frustum culling

Drawing an object that is off-screen wastes a draw call and all its vertex work. three.js tests every object's **bounding sphere** against the six frustum planes each frame, and skips it if it is entirely outside any plane.

Σ FORMULA · **Sphere–frustum test**

Each plane is $\hat{\mathbf{n}} \cdot \mathbf{x} + d = 0$ with its normal pointing **into** the frustum. A sphere (centre $\mathbf{c}$, radius r) is outside if, for any plane,

$$\hat{\mathbf{n}} \cdot \mathbf{c} + d < -r$$

The six planes can be read straight from the combined matrix PV (rows $\mathbf{r}_1..\mathbf{r}_4$): left $= \mathbf{r}_4 + \mathbf{r}_1$, right $= \mathbf{r}_4 - \mathbf{r}_1$, bottom $= \mathbf{r}_4 + \mathbf{r}_2$, top $= \mathbf{r}_4 - \mathbf{r}_2$, near $= \mathbf{r}_4 + \mathbf{r}_3$, far $= \mathbf{r}_4 - \mathbf{r}_3$ (the Gribb–Hartmann method). `Frustum.setFromProjectionMatrix` does this.

WORKED EXAMPLE · **Is the rock culled?**

Camera at the origin looking down −Z; the far plane at 100 m has normal $(0, 0, 1)$ (pointing back towards the camera, into the frustum) and $d = 100$. A rock's bounding sphere: centre $(3, 0, -104)$, radius 5.

$\hat{\mathbf{n}} \cdot \mathbf{c} + d = -104 + 100 = -4$. Since $-4 > -5$, the sphere still pokes 1 m inside the far plane, so it is **not** culled (and the GPU clips the part beyond). At centre $z = -106$: $-6 < -5$, culled.

⚠ PITFALL · **When culling goes wrong**

Culling uses the geometry's **bounding sphere** computed from the original vertices. If a vertex shader moves vertices (wind, waves, GPU particles, morph targets, skinning far from rest pose), the sphere is out of date and objects vanish near the screen edges. Fix: enlarge `geometry.boundingSphere` manually, or set `mesh.frustumCulled = false` for that object. After editing positions on the CPU, call `geometry.computeBoundingSphere()`.

6.6 From screen to world and back

Σ FORMULA · **Pointer position to NDC**

$$x_{\text{ndc}} = \frac{p_x - \text{rect.left}}{\text{rect.width}} \cdot 2 - 1, \qquad y_{\text{ndc}} = -\left(\frac{p_y - \text{rect.top}}{\text{rect.height}} \cdot 2 - 1\right)$$

y is flipped because the DOM measures from the top while NDC y points up. Use the canvas's `getBoundingClientRect()`, not the window size, so it works when the canvas is not full-screen.

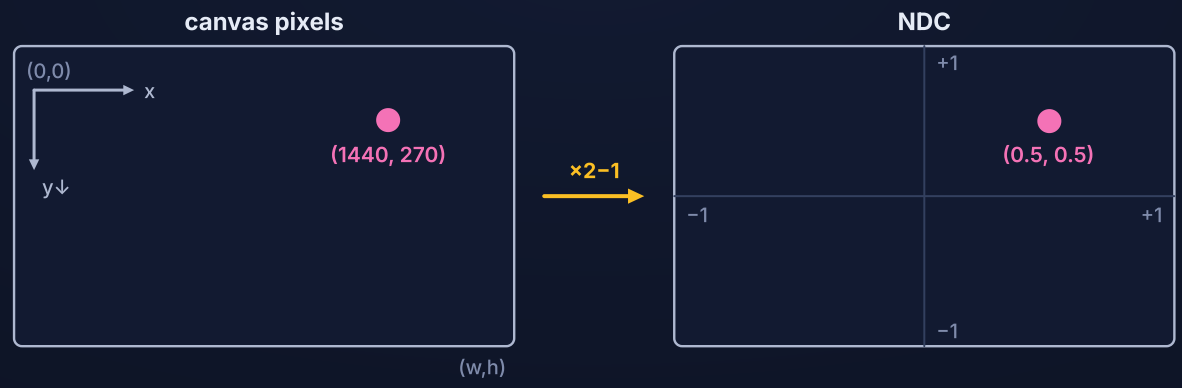


Figure 6.3 Pixel coordinates (origin top-left, y down) map to NDC (origin centre, y up). A ray from the camera through the NDC point is what the Raycaster casts.

WORKED EXAMPLE · **Clicking on a 1920×1080 canvas**

Click at $(1440, 270)$ px: $x = 1440/1920 \cdot 2 - 1 = 0.5$; $y = -(270/1080 \cdot 2 - 1) = -(0.5 - 1) = 0.5$. The click is half-way to the right edge and half-way to the top edge, in NDC $(0.5, 0.5)$.

three.js — world → screen (HTML labels) and screen → world (rays) JAVASCRIPT

```
const v = new THREE.Vector3();
function toScreen(object, camera, canvas) {
  object.getWorldPosition(v).project(camera);              // world → NDC (applies V then P,
divides by w)
  const behind = v.z > 1;                                  // beyond far / behind the camera
  return { x: (v.x + 1) / 2 * canvas.clientWidth, y: (1 - v.y) / 2 * canvas.clientHeight, behind
};
}

const raycaster = new THREE.Raycaster();
const ndc = new THREE.Vector2();
canvas.addEventListener("pointerdown", (e) => {
  const r = canvas.getBoundingClientRect();
  ndc.set(((e.clientX - r.left) / r.width) * 2 - 1, -((e.clientY - r.top) / r.height) * 2 + 1);
  raycaster.setFromCamera(ndc, camera);                    // origin = camera, direction through
the pixel
  const hits = raycaster.intersectObjects(scene.children, true);
  if (hits.length) console.log(hits[0].object.name, hits[0].point);
});
```

6.7 Orbit cameras and spherical coordinates

Σ FORMULA · Spherical coordinates (three.js `Spherical`)

A point at radius r, polar angle ϕ measured from +Y, and azimuth θ measured around Y from +Z:

$$x = r \sin\phi \sin\theta, \qquad y = r \cos\phi, \qquad z = r \sin\phi \cos\theta$$

`OrbitControls` stores the camera's offset from its target this way; dragging changes θ and ϕ, scrolling changes r, and it clamps ϕ to avoid the poles.

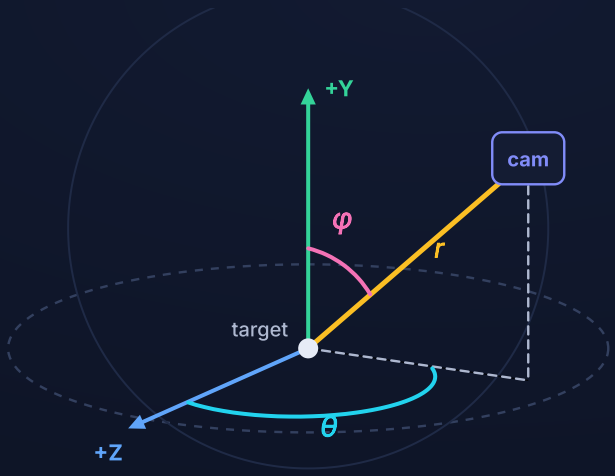


Figure 6.4 An orbit camera rides on a sphere around its target. Horizontal drags change θ (azimuth), vertical drags change φ (polar angle), the wheel changes r.

WORKED EXAMPLE · Placing an orbit camera

$r = 5$, $\phi = 60^\circ$, $\theta = 45^\circ$: $x = 5 \sin 60^\circ \sin 45^\circ = 3.06$, $y = 5 \cos 60^\circ = 2.5$, $z = 3.06$. The camera sits at $(3.06, 2.5, 3.06)$ relative to the target, 30° above the horizon, looking diagonally down at it.

Control	Feels like	Typical use
`OrbitControls`	Turntable around a target, Y always up	Product viewers, configurators, editors
`MapControls`	Pan with left drag, top-down	Maps, strategy games, floor plans
`TrackballControls`	Free tumble, no fixed up	Molecules, CAD inspection
`PointerLockControls`	Mouse-look first person	Walkthroughs, FPS games
`FlyControls`	Six degrees of freedom	Space scenes, drones
drei `CameraControls` (yomotsu)	Orbit with smooth transitions, `fitToBox`, `setLookAt`	Guided tours, animated camera moves

⚠ PITFALL · **Damping needs update()**

With `controls.enableDamping = true`, call `controls.update()` every frame (in the animation loop), otherwise the camera stops abruptly or drifts. In R3F, drei's `<OrbitControls enableDamping makeDefault />` does it for you; `makeDefault` also registers the controls in the store so other drei helpers (`Bounds`, `CameraShake`) can find them.

R3F — cameras with drei JSX

```jsx
import { PerspectiveCamera, OrbitControls } from "@react-three/drei";
import { useThree } from "@react-three/fiber";

function Rig() {
  const camera = useThree((s) => s.camera);       // the current default camera
  return (
    <>
      <PerspectiveCamera makeDefault fov={40} position={[4, 2, 6]} near={0.1} far={200} />
      <OrbitControls makeDefault enableDamping minPolarAngle={0.2} maxPolarAngle={Math.PI / 2.1}
minDistance={2} maxDistance={20} />
    </>
  );
}
```

DEEP DIVE · **The dolly zoom (Vertigo effect)**

Move the camera towards a subject while widening the FOV so the subject's on-screen size stays fixed: keep $d \cdot \tan(\text{fov}/2)$ constant. If the subject is 10 m away at FOV 30° ($10 \tan 15^\circ = 2.68$), at 5 m you need $\tan(\text{fov}/2) = 0.536$, so FOV $= 56.4^\circ$. The background appears to rush away. It is a neat demonstration that FOV controls perspective distortion while distance controls framing.

✓ KEY POINTS

- ✓ `PerspectiveCamera(fovVerticalDegrees, aspect, near, far)`; visible height at distance d is $2d\tan(\text{fov}/2)$.
- ✓ Fit a sphere: $d = r/\sin(\text{fov}/2)$ using the smaller of the vertical and horizontal FOV.
- ✓ The projection matrix puts distance into w; dividing by w creates perspective; NDC is $[-1,1]^3$ in WebGL, $z \in [0,1]$ in WebGPU.
- ✓ Depth is hyperbolic: precision lives near the near plane; raise `near` first, then consider reversed-Z or logarithmic depth.
- ✓ Frustum culling tests bounding spheres against six planes; shader-displaced geometry needs larger bounds or `frustumCulled = false`.
- ✓ Pointer → NDC: $x = 2u - 1$, $y = 1 - 2v$; `vector.project(camera)` goes the other way.
- ✓ Call `updateProjectionMatrix()` after changing `fov`, `aspect`, `near`, `far` or `zoom`.

6.8 Interview questions

Q **What is the difference between a perspective and an orthographic camera, and when would you use each?** JUNIOR}

A A perspective camera divides by distance, so far objects look smaller and parallel lines converge; it matches human vision and suits most 3D scenes. An orthographic camera projects along parallel lines, so size is independent of distance; use it for 2D and isometric games, UI layers, CAD and architectural views, technical diagrams, and the shadow cameras of directional lights. Orthographic depth is linear; perspective depth is hyperbolic.

Q **Distant objects flicker where they overlap. The camera has near = 0.001 and far = 100000. What is happening?** MID}

A Z-fighting caused by depth precision. Perspective depth is proportional to $1/d$, so precision is concentrated near the near plane; with a near:far ratio of 10^8, almost no precision is left for distant surfaces. The fix is to raise `near` (it matters far more than `far`), for example to 0.1 or 1 depending on the scene, and lower `far` to what is needed. For huge scenes (planets, cities) use `reversedDepthBuffer` (float reversed-Z) or `logarithmicDepthBuffer`, accepting the latter's cost of disabling early depth testing.

Q **How do you convert a mouse click to a 3D ray?** JUNIOR}

A Convert the pointer to NDC relative to the canvas rectangle ($x = 2u - 1$, $y = 1 - 2v$), then `raycaster.setFromCamera(ndc, camera)`. Internally, for a perspective camera, the ray origin is the camera position and the direction is the unprojected NDC point (`vector.set(x, y, 0.5).unproject(camera)`) minus the camera position, normalised. For an orthographic camera all rays are parallel, so the origin moves and the direction is the camera's forward axis.

Q **Objects near the edge of the screen disappear when a vertex shader animates them. Why?** MID}

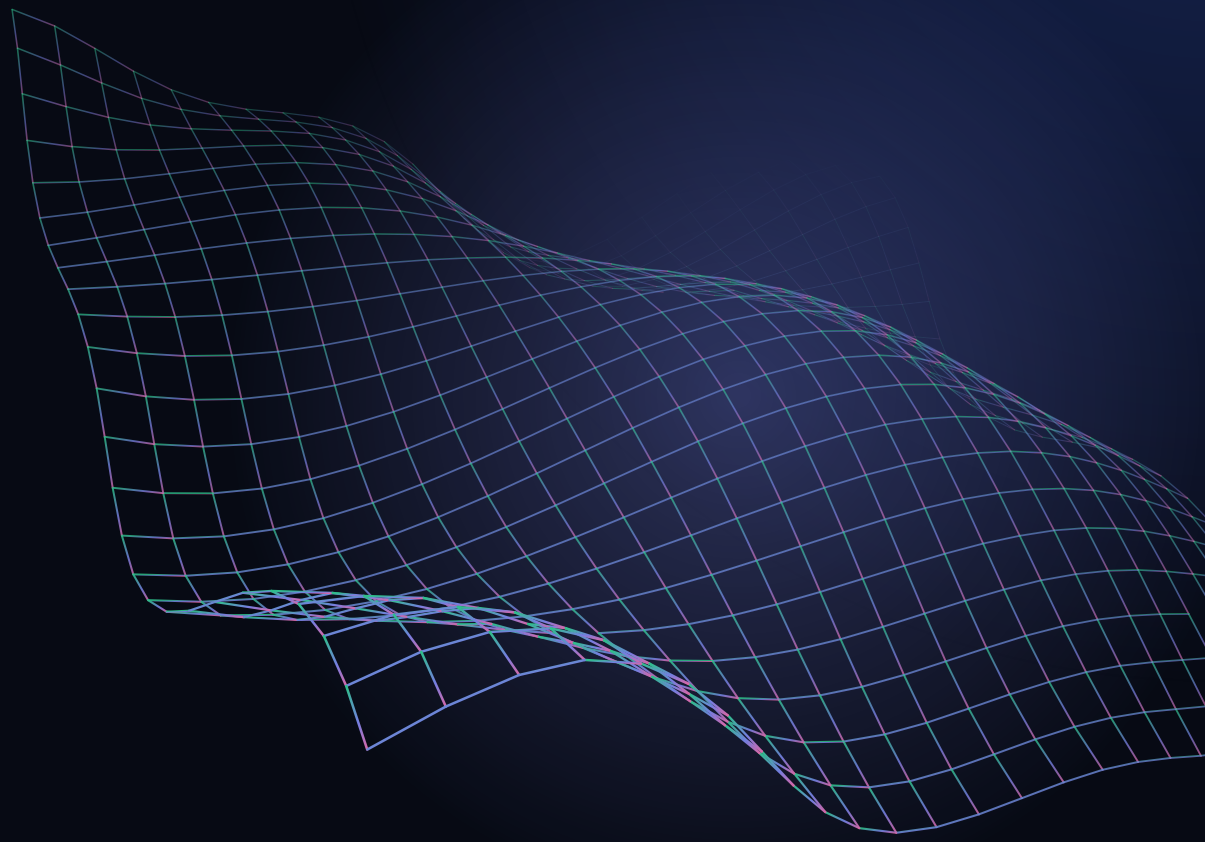
A Frustum culling uses the geometry's bounding sphere, computed from the CPU-side vertex positions. A vertex shader that displaces vertices (waves, wind, GPU particles) moves the visible shape outside that sphere, so three.js decides the object is off-screen and skips it. Fix by enlarging `geometry.boundingSphere` to cover the maximum displacement, or set `frustumCulled = false` on that mesh (accepting the cost of always drawing it).

Q **How would you place an HTML label over a 3D object?** MID

A Each frame take the object's world position, call `vector.project(camera)` to get NDC, convert to pixels ($x_{px} = (x+1)/2 \cdot w$, $y_{px} = (1-y)/2 \cdot h$) and position the element with a CSS transform. Hide it if NDC $z > 1$ (behind the camera or beyond far), and optionally raycast to hide labels behind other geometry. three.js's `CSS2DRenderer` and drei's `<Html>` component do exactly this (drei can also occlude labels).

Q **Derive the visible height of the scene at distance d and use it to size a full-screen plane.** MID

A The vertical FOV forms a triangle with the camera at its apex: half the visible height is $d\tan(\text{fov}/2)$, so $h = 2d\tan(\text{fov}/2)$ and $w = h \cdot \text{aspect}$. A plane of size $w \times h$ placed d units in front of the camera (and parented to it) exactly fills the screen. R3F exposes this as `useThree((s) ⇒ s.viewport)`, which gives the width and height in world units at the target distance (via `viewport.getCurrentViewport(camera, target)`).



PART

Building Scenes with three.js

Scene Graph, Renderer and the Frame Loop

07

Object3D in detail, configuring WebGLRenderer and WebGPURenderer, timing and fixed-timestep loops, render-on-demand, pixel ratio maths, multiple views, disposing GPU memory properly, and how R3F's Canvas wraps all of it.

IN THIS CHAPTER

7.1 Object3D: the base of everything
7.2 Configuring the renderer
7.3 The frame loop
7.4 Resizing correctly
7.5 Several views in one canvas
7.6 Disposing GPU memory
7.7 R3F's Canvas, in detail
7.8 Helpers for development
7.9 Interview questions

7.1 Object3D: the base of everything

Every thing you place in a scene (meshes, lights, cameras, groups, sprites, even the scene itself) extends `THREE.Object3D`. Knowing its properties saves a lot of searching.

Property	Type	What it does
`position`, `rotation`, `quaternion`, `scale`	Vector3, Euler, Quaternion, Vector3	The local transform (T, R, S)
`matrix`, `matrixWorld`	Matrix4	Local and world matrices (Chapter 4)
`matrixAutoUpdate`	boolean	Recompute `matrix` from TRS every frame (default true)
`parent`, `children`	Object3D, Object3D[]	The tree; use `add`, `remove`, `attach`, `clear`
`visible`	boolean	Skips rendering this object **and its children**
`castShadow`, `receiveShadow`	boolean	Shadow participation (also needs a shadow-casting light)
`frustumCulled`	boolean	Skip when the bounding sphere is off-screen (default true)
`renderOrder`	number	Overrides sorting; lower draws first
`layers`	Layers	32 bit flags; the camera only renders objects sharing a layer
`name`, `userData`	string, object	Your own labels and data (`getObjectByName`)

Property	Type	What it does
`up`	Vector3	The up hint for `lookAt` (default $(0, 1, 0)$)

three.js — walking the tree JAVASCRIPT

```
scene.traverse((o) => {                       // every descendant, depth first
  if (o.isMesh) { o.castShadow = true; o.receiveShadow = true; }
});
const wheel = car.getObjectByName("wheel_front_left");
scene.traverseVisible((o) => { /* skips hidden branches */ });
object.removeFromParent();                    // safe even if it has no parent
```

DEFINITION · **Layers**

A 32-bit mask on every object and camera. The renderer draws an object only if it shares at least one enabled layer with the camera; the `Raycaster` also has layers. Use them to render a "selection outline" set in a second pass, hide helpers from a reflection camera, or make some objects unclickable.

three.js — layers for selective rendering and picking JAVASCRIPT

```
const HELPERS = 1;
gizmo.layers.set(HELPERS);                    // only on layer 1
camera.layers.enable(HELPERS);                // main camera sees layers 0 and 1
mirrorCamera.layers.disable(HELPERS);         // reflections do not show gizmos
raycaster.layers.set(0);                      // picking ignores the helpers
```

7.2 Configuring the renderer

three.js — a production WebGLRenderer JAVASCRIPT

```
const renderer = new THREE.WebGLRenderer({
  canvas,                          // reuse an existing <canvas> (optional)
  antialias: true,                 // MSAA on the default framebuffer
  alpha: false,                    // opaque canvas: cheaper compositing; true to see the page
behind
  powerPreference: "high-performance",    // ask for the discrete GPU on dual-GPU laptops
  stencil: false,                  // skip the stencil buffer unless you need it
  preserveDrawingBuffer: false,    // true only for toDataURL screenshots taken outside the loop
});
renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
renderer.setSize(canvas.clientWidth, canvas.clientHeight, false);  // false: do not touch CSS
size
renderer.outputColorSpace = THREE.SRGBColorSpace;     // default; see Chapter 12
renderer.toneMapping = THREE.ACESFilmicToneMapping;   // or AgXToneMapping / NeutralToneMapping
renderer.toneMappingExposure = 1.0;
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFShadowMap;         // soft since r182; PCFSoftShadowMap
removed in r186
```

DEFINITION · **Device pixel ratio (DPR)**

The number of physical screen pixels per CSS pixel: 1 on old monitors, 2 on most laptops and iPads, 3 on many phones. The drawing buffer has (CSS width × DPR) × (CSS height × DPR) pixels, and fragment cost grows with that **pixel count**, so cost grows with DPR².

WORKED EXAMPLE · **What a pixel ratio costs**

An iPad Pro 11″ canvas of 1194×834 CSS pixels:

- DPR 1: $1194 \times 834 = 0.996$ M pixels.
- DPR 2 (its native ratio): $2388 \times 1668 = 3.98$ M pixels: **4×** the fragment work.
- A 3× phone (390×844 CSS) at full DPR: $1170 \times 2532 = 2.96$ M; capped at 2: $780 \times 1688 = 1.32$ M, a 56% saving that most users cannot see.

This is why `setPixelRatio(Math.min(devicePixelRatio, 2))` is standard, and why adaptive apps drop to 1–1.5 when the frame rate falls.

7.2.1 WebGPURenderer differences

three.js — WebGPURenderer setup JAVASCRIPT

```
import * as THREE from "three/webgpu";

const renderer = new THREE.WebGPURenderer({ antialias: true /*, forceWebGL: true for testing */
});
renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);
await renderer.init();                  // needed if you render or read capabilities before the
loop starts
renderer.setAnimationLoop(() => renderer.render(scene, camera));   // setAnimationLoop also
initialises
console.log(renderer.backend.isWebGPUBackend ? "WebGPU" : "WebGL 2 fallback");
```

NEW IN RECENT THREE.JS · **Async methods**

Since r181 the `renderAsync()` / `computeAsync()` family is deprecated: initialise once (`await renderer.init()` or implicitly via `setAnimationLoop`) and then call the synchronous `render()` / `compute()`. `compileAsync()` stopped blocking rendering in r184, and r186 added `compileComputeAsync()` to pre-compile compute shaders and avoid first-use hitches.

7.3 The frame loop

`renderer.setAnimationLoop(fn)` calls `fn` once per display refresh, using `requestAnimationFrame` normally and the XR session's own loop inside VR/AR. Always prefer it to calling `requestAnimationFrame` yourself, because WebXR **requires** it.

DEFINITION · **THREE.Timer**

The time-keeping class (core since r179, replacing the deprecated `Clock`). Call `timer.update(timestamp)` once per frame, then read `getDelta()` (seconds since the previous update) and `getElapsed()` (seconds since start). `timer.connect(document)` uses the Page Visibility API so that a long hidden period does not produce one giant delta when the tab returns.

7.3.1 Fixed timestep for physics and game logic

Variable delta time is fine for visual animation, but physics simulations become unstable or non-deterministic when the step size changes every frame. The standard pattern: advance the simulation in **fixed** steps, as many as fit into the elapsed time, and interpolate the rendering between the last two states.

Σ FORMULA · **The accumulator loop**

$$\text{acc} \leftarrow \text{acc} + \Delta t; \qquad \text{while acc} \geq h: \ \text{step}(h),\ \text{acc} \leftarrow \text{acc} - h; \qquad \alpha = \text{acc}/h$$

h is the fixed step (for example $1/60$ s). Render each object at $\text{lerp}(\text{previous}, \text{current}, \alpha)$. Cap the number of steps per frame (say 5) to avoid the "spiral of death" on slow devices.

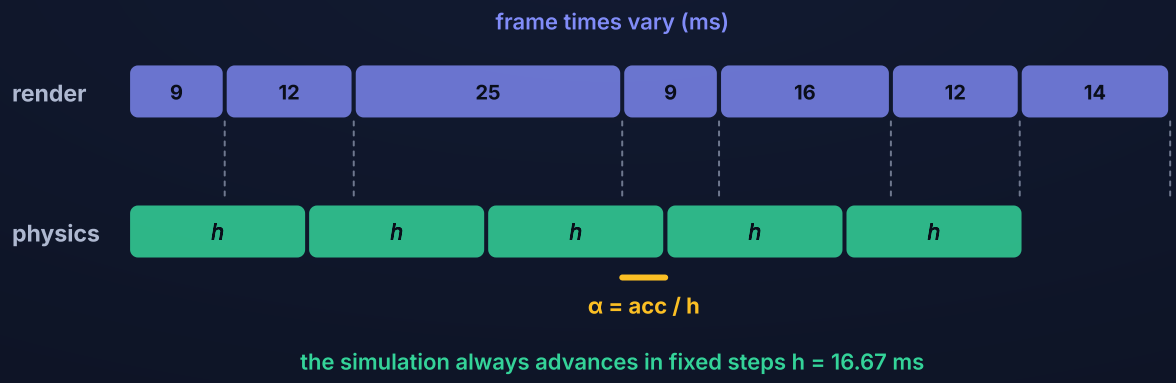


Figure 7.1 Rendering at a variable rate while the simulation advances in fixed 16.7 ms steps. Leftover time stays in the accumulator and becomes the interpolation factor α.

WORKED EXAMPLE · **One slow frame**

$h = 16.67$ ms. The accumulator holds 4 ms, and the next frame took $\Delta t = 25$ ms.

1. $\text{acc} = 4 + 25 = 29$ ms.
2. $29 \geq 16.67$: one step, $\text{acc} = 12.33$ ms. $12.33 < 16.67$: stop.
3. $\alpha = 12.33/16.67 = 0.74$: draw objects 74% of the way from their previous to their current physics state.

three.js — fixed-step simulation with interpolation JAVASCRIPT

```
const STEP = 1 / 60, MAX_STEPS = 5;
let acc = 0;
renderer.setAnimationLoop((t) => {
  timer.update(t);
  acc += Math.min(timer.getDelta(), 0.25);
  let steps = 0;
  while (acc >= STEP && steps++ < MAX_STEPS) {
    for (const b of bodies) b.prev.copy(b.pos);   // remember the last state
    world.step(STEP);                               // deterministic step
    acc -= STEP;
  }
  const alpha = acc / STEP;
  for (const b of bodies) b.mesh.position.lerpVectors(b.prev, b.pos, alpha);
  renderer.render(scene, camera);
});
```

7.3.2 Render on demand

A product viewer that is not moving does not need to redraw 60 times a second. Rendering only when something changes saves battery and keeps laptops cool.

three.js — render only when needed JAVASCRIPT

```
let pending = false;
function requestRender() {
  if (!pending) { pending = true; requestAnimationFrame(() => { pending = false;
controls.update(); renderer.render(scene, camera); }); }
}
controls.addEventListener("change", requestRender);   // OrbitControls fires change while moving
addEventListener("resize", () => { resize(); requestRender(); });
requestRender();
```

R3F — frameloop='demand' JSX

```
<Canvas frameloop="demand">   {/* renders on prop changes, events and invalidate() */}
  <Model />
  <OrbitControls />           {/* drei controls call invalidate() for you */}
</Canvas>

// from your own code, after mutating something outside React:
const invalidate = useThree((s) => s.invalidate);
useEffect(() => { ref.current.material.color.set(color); invalidate(); }, [color]);
```

⚠ PITFALL · **Damping and demand mode**

With `enableDamping`, OrbitControls keeps moving for a while after the pointer is released. In a hand-written on-demand loop, keep requesting frames while `controls.update()` returns true (it reports whether the camera changed). drei's `<OrbitControls>` handles this in R3F.

7.4 Resizing correctly

The canvas has two sizes: its **CSS size** (layout) and its **drawing-buffer size** (pixels). Keep the drawing buffer equal to CSS size × DPR, and the camera's aspect equal to CSS width ÷ height.

three.js — resize from the canvas's own size (works in any layout) JAVASCRIPT

```
const ro = new ResizeObserver(([entry]) => {
  const { width, height } = entry.contentRect;
  camera.aspect = width / height;
  camera.updateProjectionMatrix();
  renderer.setSize(width, height, false);    // false = leave the CSS size to the layout
});
ro.observe(renderer.domElement);
```

7.5 Several views in one canvas

`setViewport` and `setScissor` restrict drawing to a rectangle of the canvas, so one renderer can draw a minimap, a split screen or a picture-in-picture view in the same frame.

three.js — a minimap in the corner JAVASCRIPT

```
renderer.setScissorTest(true);
const w = renderer.domElement.clientWidth, h = renderer.domElement.clientHeight;
renderer.setViewport(0, 0, w, h); renderer.setScissor(0, 0, w, h);
renderer.render(scene, camera);                          // main view
renderer.setViewport(w - 220, h - 220, 200, 200);      // origin is bottom-left, in CSS pixels
renderer.setScissor(w - 220, h - 220, 200, 200);
renderer.render(scene, topDownCamera);                  // minimap
renderer.setScissorTest(false);
```

For a HUD that must draw on top of everything, render a second scene with `renderer.autoClear = false`, calling `renderer.clearDepth()` between the two renders so the HUD is never hidden by world geometry.

7.6 Disposing GPU memory

JavaScript's garbage collector frees JavaScript objects, but **not** the GPU buffers, textures and shader programs behind them. three.js keeps those alive until you call `dispose()`. Long-running apps (configurators, editors, single-page apps with routes) leak GPU memory without it, and mobile browsers kill the tab when memory runs out.

Owner	What `dispose()` frees
`BufferGeometry`	Vertex and index buffers
`Material`	The shader program reference (freed when no material uses it)
`Texture`	The GPU texture (and the `ImageBitmap` if you close it yourself)
`WebGLRenderTarget`	Its colour and depth textures and framebuffer
`Object3D` (r186)	A new `dispose()` hook for custom objects; call `super.dispose()` in subclasses
Renderer	Everything it owns; use when tearing down the whole canvas

three.js — dispose a subtree completely — JAVASCRIPT

```
function disposeDeep(root) {
  root.traverse((o) => {
    if (o.geometry) o.geometry.dispose();
    const mats = Array.isArray(o.material) ? o.material : o.material ? [o.material] : [];
    for (const m of mats) {
      for (const v of Object.values(m)) if (v && v.isTexture) v.dispose();   // map, normalMap,
...
      m.dispose();
    }
  });
  root.removeFromParent();
}
console.log(renderer.info.memory);   // { geometries, textures } should go back down afterwards
```

⚠ PITFALL · **Shared resources**

Only dispose what nobody else uses. If ten meshes share one material and one texture, disposing them for the first mesh breaks the other nine on the next frame (three.js re-uploads, causing hitches and possible errors). Keep shared assets in a cache with reference counts, or dispose them when the whole level unloads.

✳ TIP · **WebGL context loss**

The browser can take the GPU context away (driver reset, too many contexts, the phone reclaiming memory). Listen for `webglcontextlost` (call `event.preventDefault()` to allow restoration) and `webglcontextrestored`; three.js re-uploads resources after restoration. Browsers allow a limited number of live WebGL contexts per page (often 16), so reuse one renderer across views instead of creating one per component.

7.7 R3F's Canvas, in detail

Canvas prop	Meaning
`gl`	Renderer options object, an instance, or a (sync or async) factory `(props) ⇒ renderer`

Canvas prop	Meaning
`camera`	Default camera props, or an existing camera; `orthographic` switches type
`dpr`	Pixel ratio, or `[min, max]` clamped to the device (default `[1, 2]`)
`frameloop`	`"always"` (default), `"demand"` (render on change) or `"never"` (you call `advance`)
`shadows`	`true` enables `renderer.shadowMap` (or `"soft"`, `"basic"`, `"percentage"`, `"variance"`)
`flat`	Disables tone mapping (uses `NoToneMapping`)
`linear`	Disables automatic sRGB output
`events`, `eventSource`	Pointer-event system and the element it listens on
`performance`	`{ min, max, debounce }` for `regress()`-based adaptive quality
`onCreated`	Called with the store state once the renderer is ready

DEFINITION · **useThree**

A hook returning R3F's store: `gl` (renderer), `scene`, `camera`, `size` (CSS pixels), `viewport` (world units), `clock`, `invalidate`, `advance`, `setDpr`, `events`, `raycaster`, `pointer`. Pass a selector to subscribe to one slice and avoid re-renders: `useThree((s) ⇒ s.camera)`.

DEFINITION · **useFrame**

`useFrame((state, delta, xrFrame) ⇒ …, priority)` registers a callback that runs every frame **before** rendering. Callbacks run in ascending priority. Any callback with a **positive** priority tells R3F that you will render yourself (for example a post-processing pipeline), so R3F stops calling `gl.render` automatically.

NOTE · **Automatic disposal in R3F**

When a JSX-created object unmounts, R3F calls `dispose()` on it (geometries, materials, textures created through JSX). Objects you pass in with `<primitive object={…}>` are **not** disposed automatically, because you may share them; add `dispose={null}` to opt any element out. Assets loaded through `useLoader` / `useGLTF` are cached: call `useGLTF.clear(url)` to evict them.

7.8 Helpers for development

Helper	Shows
`AxesHelper(size)`	X (red), Y (green), Z (blue) axes
`GridHelper(size, divisions)`	A ground grid

Helper	Shows
`BoxHelper` / `Box3Helper`	An object's bounding box
`CameraHelper(light.shadow.camera)`	A camera's frustum: essential for tuning shadows
`DirectionalLightHelper`, `PointLightHelper`, `SpotLightHelper`	Light positions and directions
`SkeletonHelper(model)`	The bones of a skinned mesh
`VertexNormalsHelper` (addon)	Normals as small lines
stats-gl / `Stats`	FPS, CPU and GPU frame time
lil-gui (vanilla) / leva (R3F)	Sliders for live tuning
Spector.js, browser GPU profilers	Every WebGL call in a captured frame

✓ KEY POINTS

- ✓ Everything is an `Object3D`: transform, tree, visibility, shadows, culling, `renderOrder`, `layers`, `userData`.
- ✓ Configure the renderer once: antialias, pixel ratio ≤ 2, `SRGBColorSpace` output, a tone mapper, `PCFShadowMap`.
- ✓ Fragment cost scales with DPR²; capping at 2 saves more than half the work on 3× phones.
- ✓ `setAnimationLoop` for every loop (XR needs it); `Timer` for time; a fixed-step accumulator for physics.
- ✓ Render on demand for static viewers (`frameloop="demand"` + `invalidate()` in R3F).
- ✓ Dispose geometries, materials, textures and render targets you no longer need; watch `renderer.info.memory`.
- ✓ In R3F, `useFrame` with a positive priority takes over rendering; `<primitive>` objects are not auto-disposed.

7.9 Interview questions

Q Why use `renderer.setAnimationLoop` instead of `requestAnimationFrame`? JUNIOR}

A It does the same thing outside XR, but when a WebXR session starts the browser stops firing `window.requestAnimationFrame` for the immersive display and instead drives frames through `XRSession.requestAnimationFrame` at the headset's refresh rate. `setAnimationLoop` switches automatically. It also initialises `WebGPURenderer` before the first frame. Passing `null` stops the loop.

Q **A configurator's memory grows every time the user switches models, until mobile Safari reloads the page. What do you do?** MID

A GPU resources are not garbage-collected. When a model is removed, traverse it and call `dispose()` on its geometries, materials and textures (unless shared), and dispose any render targets created for it. Verify with `renderer.info.memory.geometries/textures` returning to the baseline after a switch, and with the browser's memory tools. Cache and reuse assets the user is likely to come back to rather than reloading them. In R3F, JSX objects dispose on unmount automatically, but `<primitive>` objects and cached `useGLTF` results need explicit handling.

Q **How do you keep physics stable when the frame rate varies?** MID

A Use a fixed timestep with an accumulator: add each frame's delta to an accumulator, run the physics in constant steps (for example 1/60 s) while the accumulator has enough time, and interpolate the rendered transforms with $\alpha = \mathrm{acc}/h$. Cap steps per frame to avoid a spiral of death, and clamp huge deltas after tab switches. Rapier and cannon-es support this pattern (`@react-three/rapier` exposes `timeStep` and `interpolate`).

Q **What does `frameloop="demand"` do in R3F and when would you use it?** MID

A R3F stops rendering continuously and renders only when something requests it: React props changing, pointer events, controls calling `invalidate()`, or your own `invalidate()` calls after imperative mutations. Use it for mostly static scenes (product viewers, maps, dashboards) to save battery and heat. Anything animated must call `invalidate()` each frame while it animates, or use `frameloop="always"`.

Q **Why is the canvas blurry on a retina screen, and why might you deliberately keep it slightly blurry?** JUNIOR

A If the drawing buffer is sized in CSS pixels, a DPR-2 screen stretches each rendered pixel over four physical pixels. `renderer.setPixelRatio(devicePixelRatio)` fixes it, but costs DPR² more fragment work. On 3× phones and in heavy scenes you cap it at 2 (or lower, adaptively), trading a small loss of sharpness for a large gain in frame rate and battery life.

Geometry and Meshes

08

How shapes are stored on the GPU. BufferGeometry and its attributes, indexed versus non-indexed meshes, normals and why a cube has 24 vertices, UV seams, built-in and procedural geometry, updating vertices efficiently, bounding volumes, merging, morph targets, points and lines.

IN THIS CHAPTER

8.1 BufferGeometry: shapes as arrays
8.2 Indexed and non-indexed geometry
8.3 Built-in and procedural geometry
8.4 Updating geometry at runtime
8.5 Bounding volumes
8.6 Merging geometry
8.7 Morph targets
8.8 Points, lines and sprites
8.9 Interview questions

8.1 BufferGeometry: shapes as arrays

DEFINITION · **BufferGeometry**

three.js's container for shape data. It holds named **attributes** (per-vertex arrays such as `position`, `normal`, `uv`, `color`) and an optional **index** (which vertices form each triangle). Each attribute is a `BufferAttribute` wrapping a typed array plus an **item size** (3 for positions and normals, 2 for UVs, 4 for tangents and skin weights). These arrays are uploaded to GPU buffers as they are.

Attribute	Item size	Used for
`position`	3	Vertex positions in local space (required)
`normal`	3	Lighting (Chapter 9)
`uv`	2	Texture coordinates (Chapter 11); `uv1`, `uv2` … for extra sets (light maps, AO)
`color`	3 or 4	Vertex colours (enable `material.vertexColors`)
`tangent`	4	Normal mapping with a tangent frame; `w` stores handedness
`skinIndex`, `skinWeight`	4	Skeletal animation (Chapter 15)

three.js — a triangle by hand JAVASCRIPT

```
const geometry = new THREE.BufferGeometry();
const positions = new Float32Array([
  -1, 0, 0,     // vertex 0
   1, 0, 0,     // vertex 1
   0, 1.5, 0,   // vertex 2  (counter-clockwise when seen from +Z, so it faces the default
camera)
]);
geometry.setAttribute("position", new THREE.BufferAttribute(positions, 3));
geometry.computeVertexNormals();            // (0, 0, 1) for all three
const tri = new THREE.Mesh(geometry, new THREE.MeshStandardMaterial({ color: "tomato" }));
```

R3F — the same geometry declared in JSX JSX

```
function Triangle() {
  const positions = useMemo(() => new Float32Array([-1, 0, 0, 1, 0, 0, 0, 1.5, 0]), []);
  return (
    <mesh>
      <bufferGeometry onUpdate={(g) => g.computeVertexNormals()}>
        <bufferAttribute attach="attributes-position" args={[positions, 3]} />
      </bufferGeometry>
      <meshStandardMaterial color="tomato" />
    </mesh>
  );
}
```

8.2 Indexed and non-indexed geometry

DEFINITION · **Index buffer**

An array of integers listing vertices three at a time, one triple per triangle. With an index, a vertex shared by several triangles is stored **once** and referenced several times. Without one (non-indexed), every triangle stores its own three vertices.

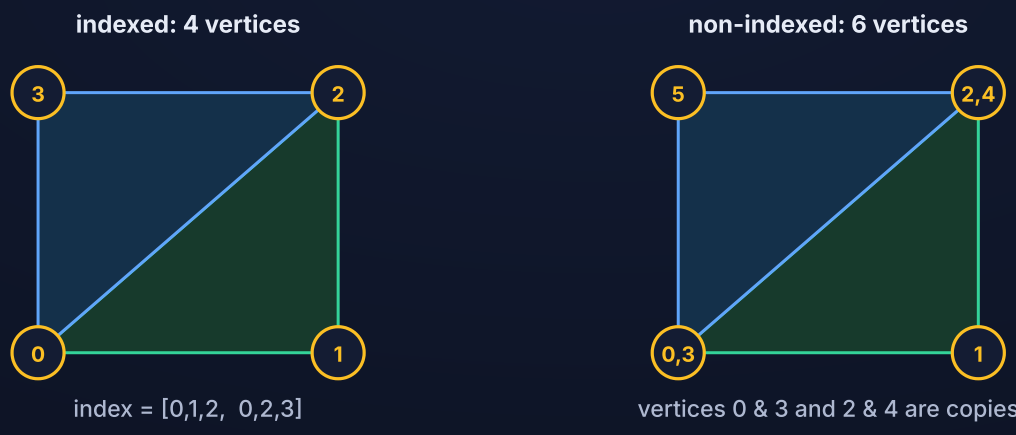


Figure 8.1 A quad as two triangles. Non-indexed it needs six vertices; indexed it needs four vertices plus six small integers, and the GPU's post-transform cache can reuse shaded vertices.

WORKED EXAMPLE · **Memory for a 100 × 100 terrain grid**

`PlaneGeometry(1, 1, 100, 100)`: $(100+1)^2 = 10{,}201$ vertices and $2 \times 100^2 = 20{,}000$ triangles. With position + normal + uv each vertex is $(3+3+2) \times 4 = 32$ bytes.

- Indexed: $10{,}201 \times 32 = 326$ KB of vertices, plus $60{,}000$ indices $\times$ 2 bytes (Uint16 suffices below 65,536 vertices) $= 120$ KB. Total about **446 KB**.
- Non-indexed: $60{,}000$ vertices $\times 32 = 1.92$ MB, about **4.3×** larger.

8.2.1 Why a cube has 24 vertices

A cube has 8 corners, yet `BoxGeometry` has 24 vertices. A vertex is not just a position: it is the **whole set of attributes**. Each corner touches three faces with three different normals (and different UVs), so it must be stored three times: $8 \times 3 = 24$.

DEFINITION · **Flat and smooth shading**

Smooth shading shares vertices between neighbouring faces and averages their normals, so lighting varies smoothly across edges (a sphere). **Flat** (faceted) shading gives each face its own vertices with the face normal, so edges look sharp (a cube, low-poly art). `material.flatShading = true` fakes flat shading in the shader using screen-space derivatives, without changing the geometry.

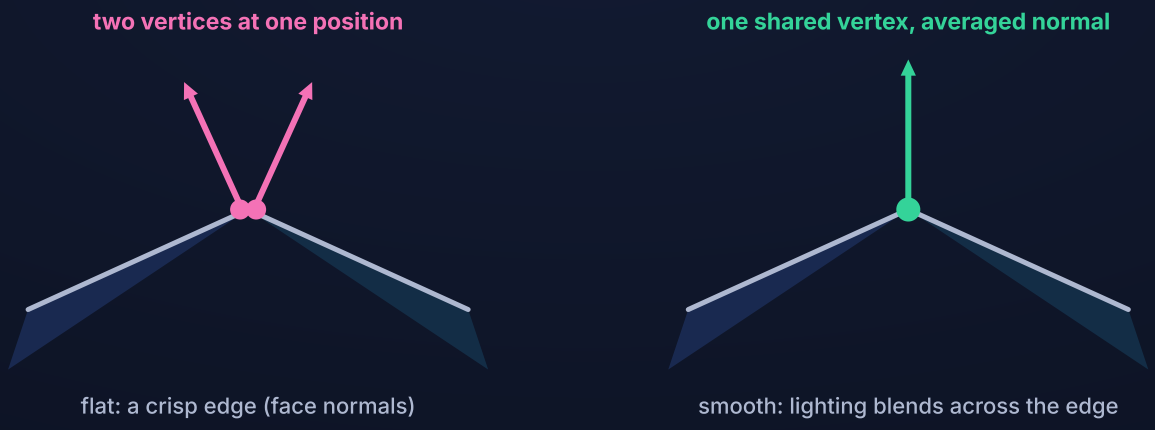


Figure 8.2 The same corner with split vertices (hard edge, separate face normals) and shared vertices (soft edge, averaged normal).

Geometry	Vertices	Triangles
`BoxGeometry(1,1,1)`	24	12
`PlaneGeometry(1,1)`	4	2
`SphereGeometry(1, 32, 16)`	561	960
`SphereGeometry(1, 64, 32)`	2,145	3,968
`CylinderGeometry(1,1,1,32)`	196	128
`TorusGeometry(1, 0.4, 12, 48)`	637	1,152
`TorusKnotGeometry()` (defaults)	585	1,024
`IcosahedronGeometry(1, 0)` (non-indexed)	60	20

TIP · Segment counts are a budget

Doubling both sphere segment counts roughly quadruples the triangles. A sphere filling a quarter of a 1080p screen looks perfectly round with 64×32; a background planet needs far fewer. Choose segments by on-screen size, or use LOD (Chapter 23).

8.2.2 Normals and UV seams

`geometry.computeVertexNormals()` averages, at each vertex, the normals of all triangles that share that vertex **through the index**. Vertices that are duplicated (for UV seams or hard edges) do not share, so the edge stays sharp. Two utilities change this:

- `BufferGeometryUtils.mergeVertices(geometry, tolerance)` welds duplicate vertices whose attributes all match, which makes a geometry indexed and smooth-shadeable.
- `geometry.toNonIndexed()` splits every triangle apart, for per-face effects (exploding meshes, per-face colours, flat normals).

PITFALL · UV seams split vertices too

A sphere's texture must wrap round, so the vertical seam where u jumps from 1 back to 0 needs two copies of each seam vertex. That is why `SphereGeometry(1, 32, 16)` has $33 \times 17 = 561$ vertices rather than 32×17. Welding such vertices would smear the texture across the whole sphere along the seam.

8.3 Built-in and procedural geometry

three.js ships with primitive geometries (box, sphere, plane, circle, ring, cylinder, cone, capsule, torus, torus knot, the platonic solids), shape-based ones (`ShapeGeometry`, `ExtrudeGeometry`, `LatheGeometry`, `TubeGeometry`) and, in addons, `TextGeometry`, `ConvexGeometry`, `RoundedBoxGeometry`, `DecalGeometry` and `ParametricGeometry`.

Σ FORMULA · A regular grid

A grid of $N_x \times N_z$ segments has $(N_x+1)(N_z+1)$ vertices and $2N_xN_z$ triangles. Vertex (i, j) has index $k = j\,(N_x+1) + i$, and cell (i, j) uses the two triangles $(k,\ k+N_x+1,\ k+1)$ and $(k+1,\ k+N_x+1,\ k+N_x+2)$.

three.js — terrain from a height function JAVASCRIPT

```
function terrain(size = 50, seg = 128, height = (x, z) ⇒ Math.sin(x * 0.3) * Math.cos(z * 0.3)
* 2) {
  const g = new THREE.PlaneGeometry(size, size, seg, seg);
  g.rotateX(-Math.PI / 2);                    // the plane is XY by default; lay it on XZ
  const pos = g.attributes.position;
  for (let i = 0; i < pos.count; i++) {
    pos.setY(i, height(pos.getX(i), pos.getZ(i)));
  }
  g.computeVertexNormals();                   // lighting must follow the new shape
  g.computeBoundingSphere();                  // culling must follow it too
  return g;
}
```

DEEP DIVE · Shapes, extrusion and lathe

`THREE.Shape` describes a 2D outline with `moveTo`, `lineTo`, `bezierCurveTo` and holes; `ExtrudeGeometry(shape, { depth, bevelEnabled, bevelSize, steps, extrudePath })` sweeps it into 3D (logos, floor plans from polygons, cookie cutters). `LatheGeometry(points, segments)` spins a 2D profile around the Y axis (vases, bottles, wine glasses), and `TubeGeometry(curve, tubularSegments, radius)` sweeps a circle along a 3D curve (cables, pipes, roller-coaster tracks), using the curve's Frenet frames to orient each ring.

8.4 Updating geometry at runtime

Changing a typed array on the CPU does nothing until three.js re-uploads it.

three.js — animating vertices on the CPU JAVASCRIPT

```
const pos = geometry.attributes.position;
pos.setUsage(THREE.DynamicDrawUsage);         // hint: this buffer changes often
function update(t) {
  for (let i = 0; i < pos.count; i++) {
    const x = pos.getX(i), z = pos.getZ(i);
    pos.setY(i, Math.sin(x * 2 + t) * 0.2);
  }
  pos.needsUpdate = true;                     // re-upload on the next render
  geometry.computeVertexNormals();            // costly; skip if the material ignores normals
}
// Only part of the array changed? Upload just that range:
pos.clearUpdateRanges();
pos.addUpdateRange(startIndex * 3, count * 3);
pos.needsUpdate = true;
```

TIP · **Move per-vertex animation to the GPU**

Looping over 100,000 vertices in JavaScript every frame costs milliseconds and a full buffer upload. The same wave in a vertex shader (Chapter 18) costs almost nothing on the CPU, because the GPU computes all vertices in parallel. Use CPU updates for occasional edits (sculpting, terrain painting) and shaders for continuous animation.

Other runtime tools: `geometry.setDrawRange(start, count)` draws only part of the buffer (growing trails, revealing a path), and pre-allocating a large buffer then increasing the draw range avoids reallocations.

8.5 Bounding volumes

DEFINITION · **Bounding box and bounding sphere**

`geometry.boundingBox` (a `Box3`) and `geometry.boundingSphere` (a `Sphere`) enclose all vertices in **local** space. three.js uses the sphere for frustum culling and as a quick rejection test before raycasting against triangles. They are computed lazily and are **not** updated automatically when vertices change.

three.js — bounds in local and world space JAVASCRIPT

```
geometry.computeBoundingBox();
const local = geometry.boundingBox;                    // local space
const world = new THREE.Box3().setFromObject(mesh);    // world space, follows transforms and
children
const size = world.getSize(new THREE.Vector3());       // e.g. to scale a model to fit 1 m
mesh.scale.setScalar(1 / Math.max(size.x, size.y, size.z));
geometry.center();                                      // move the geometry so its bbox
centre is the origin
```

8.6 Merging geometry

DEFINITION · **Merging**

Combining several geometries into one `BufferGeometry` (with `BufferGeometryUtils.mergeGeometries`) so they render in **one draw call** with one material. The result is static: you can no longer move the parts independently, and the merged mesh is culled as one object.

three.js — merging a static city block JAVASCRIPT

```
import { mergeGeometries } from "three/addons/utils/BufferGeometryUtils.js";

const parts = buildings.map((b) => {
  const g = new THREE.BoxGeometry(b.w, b.h, b.d);
  g.translate(b.x, b.h / 2, b.z);            // bake each building's position into its vertices
  return g;
});
const block = new THREE.Mesh(mergeGeometries(parts), concrete);   // 1 draw call instead of parts.length
parts.forEach((g) => g.dispose());
```

Technique	Draw calls	Per-object movement	Different shapes	Typical use
Separate meshes	N	Yes	Yes	Few interactive objects
Merged geometry	1	No	Yes	Static scenery
`InstancedMesh`	1	Yes (matrix per instance)	No (one geometry)	Trees, crowds, particles, grids
`BatchedMesh`	1 (multi-draw)	Yes (matrix per instance)	Yes (shared material)	Varied static or dynamic props

8.7 Morph targets

DEFINITION · **Morph targets (blend shapes)**

Alternative positions (and optionally normals) for every vertex, stored in `geometry.morphAttributes.position`. The mesh blends between the base shape and the targets with weights in `mesh.morphTargetInfluences`. Used for facial expressions, lip sync, muscle bulges and shape keys from Blender.

FORMULA · **Morph blending**

$$\mathbf{p} = \mathbf{p}_{\text{base}} + \sum_i w_i \, \Delta\mathbf{p}_i$$

With `geometry.morphTargetsRelative = true` (as glTF stores them) each target holds the **offset** $\Delta\mathbf{p}_i$; otherwise it holds absolute positions and three.js uses $\Delta\mathbf{p}_i = \mathbf{p}_i - \mathbf{p}_{\text{base}}$.

WORKED EXAMPLE · **A smile and a blink**

A mouth-corner vertex at $(0.03, -0.02, 0.09)$. Its "smile" offset is $(0.004, 0.006, 0)$ and its "jawOpen" offset is $(0, -0.01, 0.002)$. With weights smile $= 0.5$, jawOpen $= 0.25$: $\mathbf{p} = (0.03, -0.02, 0.09) + 0.5(0.004, 0.006, 0) + 0.25(0, -0.01, 0.002) = (0.032,\ -0.0195,\ 0.0905)$.

three.js — driving morph targets by name JAVASCRIPT

```javascript
const face = gltf.scene.getObjectByName("Head");
const i = face.morphTargetDictionary["mouthSmile"];   // name → index
face.morphTargetInfluences[i] = 0.5;
```

R3F — the same through props JSX

```jsx
<mesh geometry={nodes.Head.geometry} material={materials.Skin}
      morphTargetDictionary={nodes.Head.morphTargetDictionary}
      morphTargetInfluences={influences} />
```

8.8 Points, lines and sprites

Object	Draws	Notes
`Points` + `PointsMaterial`	One square sprite per vertex	`size` in pixels or world units (`sizeAttenuation`); millions are cheap
`Line` / `LineLoop` / `LineSegments`	1-pixel lines	WebGL ignores `linewidth` > 1 on most platforms
`Line2` (addon) / drei `<Line>`	Thick lines built from instanced quads	Real widths, dashes, world or pixel units
`Sprite`	A camera-facing quad	Labels, icons, impostors
`EdgesGeometry` / `WireframeGeometry`	Edges or all triangle edges as lines	Technical drawings, outlines

✓ KEY POINTS

- ✓ A vertex is the full set of its attributes; any difference (normal, uv) forces a separate vertex.
- ✓ Indexed geometry stores shared vertices once; a cube needs 24 vertices because of per-face normals.
- ✓ Grid: $(N+1)^2$ vertices and $2N^2$ triangles; segments are a triangle budget.
- ✓ After editing attributes set `needsUpdate = true`; recompute normals and bounds when the shape changes.
- ✓ Continuous vertex animation belongs in shaders; CPU edits are for occasional changes.
- ✓ Merge static geometry, instance repeated geometry, batch varied geometry: each becomes one draw call.
- ✓ Morph targets blend vertex offsets with weights; glTF stores them as relative offsets.

8.9 Interview questions

Q **Why does `BoxGeometry` have 24 vertices instead of 8?** JUNIOR}

A Because a vertex is a unique combination of all its attributes, not only a position. Each cube corner belongs to three faces that need different normals (for flat lighting) and different UVs (for per-face textures), so every corner is stored three times: $8 \times 3 = 24$. If you merged them into 8 vertices, normals would be averaged and the cube would be shaded like a blob.

Q **What is the difference between indexed and non-indexed geometry?** JUNIOR}

A Indexed geometry stores each unique vertex once and an index buffer lists which three vertices make each triangle; shared vertices save memory and let the GPU reuse already-processed vertices. Non-indexed geometry repeats vertices for every triangle: simpler, needed for per-face attributes (exploding faces, flat colours), but larger. `mergeVertices` converts to indexed, `toNonIndexed` the other way.

Q **You change vertex positions but nothing happens on screen, or the mesh disappears near screen edges. Why?** MID}

A Nothing happens: the attribute was not flagged, so set `geometry.attributes.position.needsUpdate = true` after editing (and recompute normals if lighting should change). Disappearing: the bounding sphere still describes the old shape, so frustum culling (and raycasting) use stale bounds; call `geometry.computeBoundingSphere()` (and `computeBoundingBox()`) after the change, or enlarge the bounds when a shader displaces vertices.

Q **When would you merge geometries, use InstancedMesh, or use BatchedMesh?** MID}

A Merge static objects that never move independently and share a material (buildings, rocks baked into a level): one draw call, but no per-object transforms and culling as a whole. Use `InstancedMesh` for many copies of the same geometry that need individual transforms or colours (trees, crowds, particles). Use `BatchedMesh` when objects have different geometries but share a material and still need individual transforms and visibility (varied props, CAD parts); it renders them with a single multi-draw call.

Q **Morph targets or skinning for a character's face?** SENIOR}

A Both are used together in production. Skinning (bones) is cheap to store and suits large rigid-ish motions like the jaw, neck and eyes. Morph targets store a full offset per vertex per shape, which costs memory but captures subtle, non-rigid detail (lip shapes, cheek bulges, ARKit's 52 blend shapes for face tracking). three.js supports both on the same `SkinnedMesh`, and glTF exports both.

Materials and Classic Lighting Models

09

What a material really is, the three.js material family and its cost, the Lambert, Phong, Blinn–Phong, hemisphere and toon lighting models derived and calculated by hand, and the material settings (side, transparency, depth, blending) that cause most visual bugs.

IN THIS CHAPTER

9.1 What a material is
9.2 How light meets a surface
9.3 Lambert diffuse
9.4 Phong and Blinn–Phong specular
9.5 Ambient and hemisphere light
9.6 Toon shading
9.7 Material settings that cause most bugs
9.8 Interview questions

9.1 What a material is

DEFINITION · **Material**

The description of how a surface looks: a **shader program** (vertex + fragment code) plus the **parameters** it reads (colour, roughness, textures, flags). three.js generates the shader from the material type and its settings; two materials of the same type with the same features share one compiled program and differ only in uniforms.

Material	Lighting	Typical use	Relative cost
`MeshBasicMaterial`	None (unlit)	UI, flat colour, baked lighting, video screens	Lowest
`MeshLambertMaterial`	Diffuse only, per fragment	Matte stylised scenes, low-end mobile	Low
`MeshPhongMaterial`	Diffuse + Blinn–Phong specular	Shiny plastic look, legacy content	Low–medium
`MeshToonMaterial`	Stepped diffuse (gradient map)	Cartoon / cel shading	Low
`MeshStandardMaterial`	PBR metal–roughness	The default for realistic scenes and glTF	Medium
`MeshPhysicalMaterial`	PBR + clearcoat, sheen, transmission, iridescence, anisotropy…	Car paint, glass, fabric, gems	High (transmission highest)

Material	Lighting	Typical use	Relative cost
`MeshMatcapMaterial`	Lighting baked in a sphere image	Sculpting previews, stylised looks, no lights needed	Low
`MeshNormalMaterial`, `MeshDepthMaterial`	Debug views of normals and depth	Debugging, depth pre-passes	Low
`ShadowMaterial`	Only receives shadows (transparent elsewhere)	Shadow catchers under AR or product models	Low
`ShaderMaterial` / `RawShaderMaterial`	Whatever you write (GLSL)	Custom effects on `WebGLRenderer`	Varies
`Mesh*NodeMaterial`	Node/TSL based	Custom effects on `WebGPURenderer` (and WebGL since r184)	Varies

⚠ PITFALL · **"My model is black"**

Lit materials (`Lambert`, `Phong`, `Standard`, `Physical`, `Toon`) need light. With no lights and no environment map they render black. Add lights, or set `scene.environment` to an environment map (Chapter 13), which is how glTF models are meant to be lit. Metallic surfaces (metalness 1) show almost nothing **but** reflections, so they look black without an environment even when lights exist.

9.2 How light meets a surface

Classic real-time shading splits the light leaving a surface point into parts, each a simple formula:

- **Emission**: light the surface gives off by itself (screens, lamps): `emissive`.
- **Ambient**: a constant guess for light bouncing around the room.
- **Diffuse**: light scattered equally in all directions (matte surfaces). Depends on the light direction only.
- **Specular**: the shiny highlight, a blurred reflection of the light. Depends on the light **and** the viewer.

All of them use the same unit vectors at the shaded point: the normal $\hat{\mathbf{n}}$, the direction **to** the light $\hat{\mathbf{l}}$, the direction **to** the viewer $\hat{\mathbf{v}}$, and derived vectors.

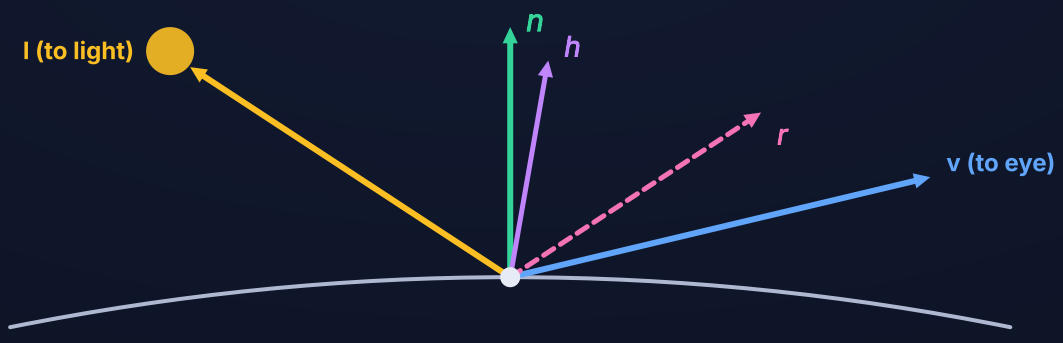


Figure 9.1 The vectors of local lighting: normal n, light direction l, view direction v, the mirror reflection r of l, and the half vector h halfway between l and v.

9.3 Lambert diffuse

Σ FORMULA · **Lambert's cosine law**

$$L_{\text{diffuse}} = k_d \, E_L \, \max(0, \, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$$

k_d is the surface's diffuse colour (albedo), E_L the light's colour times intensity. A physically based version divides by π (Chapter 13); three.js folds that constant into its light units.

Why a cosine? A beam of light of fixed width hitting a surface at an angle spreads over a larger area, so each unit of area receives less. The spread grows as $1/\cos\theta$, so the energy per area falls as $\cos\theta = \hat{\mathbf{n}} \cdot \hat{\mathbf{l}}$.

Figure 9.2 A tilted surface catches the same beam over a larger area, so it is lit more dimly in proportion to cos θ.

WORKED EXAMPLE · **Lighting one pixel by hand**

Albedo $k_d = (0.8, 0.3, 0.2)$ (terracotta). White light of intensity 2, so $E_L = (2, 2, 2)$, coming from $\hat{\mathbf{l}} = \text{normalise}(1, 2, 2) = (0.333, 0.667, 0.667)$. Normal $\hat{\mathbf{n}} = (0, 1, 0)$ (floor).

1. $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}} = 0.667$ (the light is about 48° from straight overhead).
2. $L = (0.8, 0.3, 0.2) \times 2 \times 0.667 = (1.067, 0.400, 0.267)$.
3. Red is above 1.0: in linear HDR that is fine; tone mapping (Chapter 12) maps it into displayable range rather than clipping.
4. For a wall facing away ($\hat{\mathbf{n}} = (0, 0, -1)$): $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}} = -0.667$, clamped to 0: no direct light.

9.4 Phong and Blinn–Phong specular

Σ FORMULA · **Phong specular**

$$\hat{\mathbf{r}} = 2(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}})\hat{\mathbf{n}} - \hat{\mathbf{l}}, \qquad L_{\text{spec}} = k_s \, E_L \, \max(0, \; \hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^{\alpha}$$

$\hat{\mathbf{r}}$ is the light direction mirrored about the normal. The highlight is brightest when you look straight along the reflection; the **shininess** exponent α sets how fast it falls off (10 = broad and dull, 200 = tight and glossy).

Σ FORMULA · **Blinn–Phong specular**

$$\hat{\mathbf{h}} = \text{normalise}(\hat{\mathbf{l}} + \hat{\mathbf{v}}), \qquad L_{\text{spec}} = k_s \, E_L \, \max(0, \; \hat{\mathbf{n}} \cdot \hat{\mathbf{h}})^{\alpha'}$$

The **half vector** $\hat{\mathbf{h}}$ lies halfway between the light and view directions. When the surface normal lines up with it, the surface acts like a mirror between the light and the eye. It is cheaper than computing $\hat{\mathbf{r}}$, behaves better at grazing angles, and is what `MeshPhongMaterial` uses. Roughly $\alpha' \approx 4\alpha$ gives a similar highlight size.

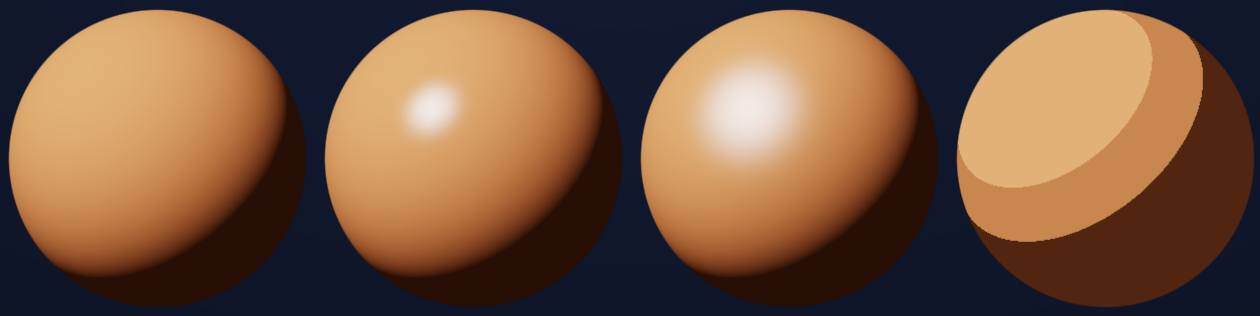


Figure 9.3 The same sphere and light rendered with four classic models, computed per pixel from the formulas in this chapter. Blinn–Phong's highlight is broader than Phong's at the same exponent; toon shading quantises n·l into bands.

WORKED EXAMPLE · **A highlight by hand**

$\hat{\mathbf{n}} = (0, 1, 0)$, light $\hat{\mathbf{l}} = (0.6, 0.8, 0)$, viewer $\hat{\mathbf{v}} = (-0.6, 0.8, 0)$ (the mirror position), shininess 32.

- Phong: $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}} = 0.8$, $\hat{\mathbf{r}} = 2(0.8)(0,1,0) - (0.6, 0.8, 0) = (-0.6, 0.8, 0)$, so $\hat{\mathbf{r}} \cdot \hat{\mathbf{v}} = 1$ and the specular term is $1^{32} = 1$: full highlight.
- Move the viewer to $\hat{\mathbf{v}} = (0,1,0)$: $\hat{\mathbf{r}} \cdot \hat{\mathbf{v}} = 0.8$, and $0.8^{32} = 0.00079$. Almost gone: shininess 32 is a tight highlight.
- Blinn–Phong for that second viewer: $\hat{\mathbf{h}} = \text{normalise}((0.6, 1.8, 0)) = (0.316, 0.949, 0)$; $\hat{\mathbf{n}} \cdot \hat{\mathbf{h}} = 0.949$; $0.949^{32} = 0.186$. Much broader at the same exponent, hence the $\approx 4\times$ rule ($0.949^{128} = 0.0012$).

three.js — classic materials JAVASCRIPT

```
const matte = new THREE.MeshLambertMaterial({ color: "#c2410c" });
const plastic = new THREE.MeshPhongMaterial({ color: "#2563eb", specular: "#ffffff", shininess:
80 });
const toon = new THREE.MeshToonMaterial({ color: "#f59e0b", gradientMap: threeTone });  // a 3-
pixel DataTexture
threeTone.minFilter = threeTone.magFilter = THREE.NearestFilter;   // keep the bands hard-edged
```

9.5 Ambient and hemisphere light

Σ FORMULA · **Hemisphere lighting**

$$L_{\text{ambient}} = k_d \cdot \text{mix}\big(C_{\text{ground}},\ C_{\text{sky}},\ \tfrac{1}{2}\hat{\mathbf{n}}_y + \tfrac{1}{2}\big)$$

Surfaces facing up get the sky colour, surfaces facing down the ground colour, and vertical ones a 50/50 mix. It is a two-colour approximation of an environment and much more convincing than a flat `AmbientLight`.

WORKED EXAMPLE · **Sky blue, grass green**

Sky $(0.6, 0.75, 1.0)$, ground $(0.25, 0.35, 0.15)$. A wall ($n_y = 0$) gets $t = 0.5$: $(0.425, 0.55, 0.575)$. A roof ($n_y = 1$) gets the full sky colour. An overhang's underside ($n_y = -1$) gets the green bounce.

9.6 Toon shading

Toon (cel) shading replaces the smooth $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}$ ramp with a few flat bands. `MeshToonMaterial` looks up $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}$ in a small `gradientMap` texture; with nearest filtering each texel becomes one band. Add an outline (the inverted-hull trick, a back-face-only scaled copy drawn in black, or the `OutlineEffect` addon) for the classic cartoon look.

Σ FORMULA · **Quantised diffuse**

$$d = \frac{\lfloor (\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) \cdot N \rfloor}{N - 1}$$

for N bands (with $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}$ clamped to $[0, 1)$). With $N = 3$, $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}} = 0.55$ gives $\lfloor 1.65 \rfloor / 2 = 0.5$: the middle band.

9.7 Material settings that cause most bugs

Setting	Default	What it controls	Common mistake
`side`	`FrontSide`	Which faces render (`BackSide`, `DoubleSide`)	Planes vanish when seen from behind; `DoubleSide` doubles fragment work on closed meshes
`transparent`	false	Puts the object in the sorted transparent list and enables blending	Setting `opacity` without `transparent: true` does nothing
`opacity`	1	Alpha multiplier	
`alphaTest`	0	Discard fragments with alpha below the threshold	Use for foliage/fences instead of `transparent` (no sorting problems)
`alphaHash`	false	Stochastic (dithered) transparency, no sorting needed	Noisy without TAA or multisampling
`depthWrite`	true	Write to the depth buffer	Transparent objects writing depth hide what is behind them
`depthTest`	true	Test against the depth buffer	Turn off for always-on-top gizmos
`blending`	`NormalBlending`	Additive, multiply, custom	Glows need `AdditiveBlending` + `depthWrite: false`
`vertexColors`	false	Multiply by the `color` attribute	Forgetting it, so vertex colours are ignored
`flatShading`	false	Per-face normals in the shader	
`toneMapped`	true	Apply the renderer's tone mapping	UI or emissive screens look washed out: set false
`polygonOffset`	false	Nudge depth to avoid z-fighting	Decals and outlines on surfaces
`wireframe`	false	Draw triangle edges	

⚠ PITFALL · **Changing features after the first render**

Some changes alter the **shader program** itself, not just a uniform: adding or removing a texture (`map` from null to a texture), switching `flatShading`, `vertexColors`, `fog`, or the number of lights in the scene. After such a change set `material.needsUpdate = true` (three.js detects many cases itself), and expect a one-off compile hitch. Swapping between pre-created materials (`mesh.material = variantB`) avoids recompiling.

R3F — materials as JSX and shared materials — JSX

```jsx
// props map to material properties; changing them later just sets the property
<mesh>
  <sphereGeometry args={[1, 64, 32]} />
  <meshPhongMaterial color="#2563eb" shininess={80} transparent opacity={0.85} side=
{THREE.DoubleSide} />
</mesh>

// one material shared by many meshes (one program, fewer state changes)
const glass = useMemo(() => new THREE.MeshPhysicalMaterial({ transmission: 1, roughness: 0.05,
thickness: 0.5 }), []);
return items.map((it) => <mesh key={it.id} geometry={it.geometry} material={glass} position=
{it.position} />);

// multi-material mesh: attach into the material array
<mesh>
  <boxGeometry />
  <meshStandardMaterial attach="material-0" color="red" />
  <meshStandardMaterial attach="material-1" color="green" />
  {/* … material-2 to material-5 for the other faces */}
</mesh>
```

DEEP DIVE · **Extending built-in materials**

With `WebGLRenderer` you can inject GLSL into a built-in material through `material.onBeforeCompile = (shader) => { shader.vertexShader = shader.vertexShader.replace("#include <begin_vertex>", "…") }`, keeping all of three.js's lighting while adding, for example, wind sway. The `three-custom-shader-material` library does this more robustly. With `WebGPURenderer`, `onBeforeCompile` is not supported; instead you assign TSL nodes to slots such as `positionNode`, `colorNode` or `normalNode` of a node material (Chapter 20).

✓ KEY POINTS

- ✓ A material = generated shader program + parameters; same type + same features = shared program.
- ✓ Diffuse (Lambert) $\propto \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$: cosine because a tilted surface spreads the beam.
- ✓ Phong uses $(\hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^{\alpha}$; Blinn–Phong uses $(\hat{\mathbf{n}} \cdot \hat{\mathbf{h}})^{\alpha'}$ with $\hat{\mathbf{h}} = \text{norm}(\hat{\mathbf{l}} + \hat{\mathbf{v}})$, $\alpha' \approx 4\alpha$.
- ✓ Hemisphere light mixes ground and sky colours by n_y; toon shading quantises $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}$.
- ✓ Black models: no light or environment. Metal needs an environment.
- ✓ `transparent` + `opacity` for blending, `alphaTest` / `alphaHash` for cut-outs, `depthWrite: false` for glows.
- ✓ Changes that alter the program need `needsUpdate`; prefer swapping pre-built materials.

9.8 Interview questions

Q **What is the difference between MeshBasic, MeshLambert, MeshPhong and MeshStandard materials?** JUNIOR}

A Basic is unlit: it shows its colour/texture regardless of lights. Lambert computes diffuse lighting only (matte). Phong adds a Blinn–Phong specular highlight controlled by `shininess` and `specular` (shiny plastic). Standard is physically based (metalness/roughness, energy conserving, works with environment maps and glTF), the default for realistic content. Cost rises in that order; Physical adds clearcoat, sheen, transmission and more on top of Standard.

Q **Explain the half vector in Blinn–Phong and why it is used.** MID}

A $\hat{\mathbf{h}} = \text{normalise}(\hat{\mathbf{l}} + \hat{\mathbf{v}})$ is the normal a perfect mirror would need to reflect the light into the eye. Specular intensity is $(\hat{\mathbf{n}} \cdot \hat{\mathbf{h}})^{\alpha}$: brightest when the surface normal matches it. It avoids computing the reflection vector, does not cut off abruptly when the angle between $\hat{\mathbf{r}}$ and $\hat{\mathbf{v}}$ exceeds 90°, and matches measured highlights better (it is the basis of microfacet models, where $\hat{\mathbf{h}}$ is the microfacet normal that reflects light to the viewer).

Q **Leaves of a tree show black or white halos and sort incorrectly. How do you fix it?** MID}

A The leaves use `transparent: true`, so they go through back-to-front object sorting and blending, which fails within a single mesh and causes sorting artefacts; halos come from straight-alpha colour bleeding at texture edges. Use `alphaTest: 0.5` (cut-out: fragments are kept or discarded, depth works normally, no sorting needed), or `alphaHash` with antialiasing, and keep `transparent: false`. Premultiplied alpha or padded texture edges (dilation) remove the halos.

Q **Why is `side: THREE.DoubleSide` not free?** MID}

A Back-face culling normally discards about half the triangles of a closed mesh before rasterisation. With `DoubleSide` they are rasterised and shaded too, doubling fragment work in the worst case and causing overdraw; lighting on back faces also needs flipped normals (three.js handles it). Use it only for thin open surfaces (leaves, cloth, paper); model thickness or use two one-sided meshes otherwise.

Q **What is a matcap and when is it a good choice?** JUNIOR}

A A “material capture”: an image of a lit sphere. The shader looks up the colour using the view-space normal’s x and y, so lighting and reflections are baked into the image. It needs no lights and is very cheap, so it suits sculpting previews, stylised products, medical or CAD viewers and low-end devices. Its limitation: the lighting moves with the camera (it is view-locked) and does not respond to scene lights or shadows.

Lights and Shadows

10

Every three.js light type, physical light units and the exact attenuation and spot-cone formulas three.js uses, then shadow mapping from first principles, shadow-camera fitting, acne and peter-panning, PCF and VSM filtering, cascaded shadows with the new SunLight, and cheaper alternatives.

IN THIS CHAPTER

10.1 The light types
10.2 Attenuation: how light fades with distance
10.3 Shadow mapping
10.4 Cheaper ways to get shadows
10.5 Interview questions

10.1 The light types

Light	Models	Casts shadows	Unit of `intensity`	Notes
`AmbientLight`	Uniform light from everywhere	No	—	Flattens shapes; prefer hemisphere or an environment map
`HemisphereLight`	Sky colour from above, ground colour from below	No	—	Cheap outdoor fill
`DirectionalLight`	Parallel rays from far away (the sun)	Yes (orthographic shadow camera)	lux-like	Direction = from `position` to `target.position`
`PointLight`	A bulb radiating in all directions	Yes (6 renders: cube)	candela	`power` = intensity × 4π lumens
`SpotLight`	A cone (torch, stage light)	Yes (perspective shadow camera)	candela	`power` = intensity × π; `angle`, `penumbra`, `map` (gobo)
`RectAreaLight`	A glowing rectangle (window, softbox)	No	nits (cd/m²)	Needs `RectAreaLightUniformsLib.init()`; Standard/Physical materials only
`LightProbe`	Spherical-harmonic	No	—	From `LightProbeGenerator` or baked

Light	Models	Casts shadows	Unit of `intensity`	Notes
	ambient light			
`SunLight` (addon, r186)	A sun with automatic **cascaded** shadows	Yes (2 cascades)	lux-like	Fits shadows to the view; no shadow-camera tuning
`scene.environment`	Image-based lighting from an HDR	No (but see contact/AO)	—	The main light source for PBR (Chapter 13)

DEFINITION · **Physically based light units**

Since r155 three.js uses physical light units by default (the legacy mode was removed in r165). Point and spot lights take **luminous intensity in candela**, `RectAreaLight` takes **luminance in nits**, and you can set `light.power` in **lumens** instead. Absolute values only matter relative to the renderer's exposure (`toneMappingExposure`), but keeping real ratios (a 60 W-equivalent bulb is about 800 lm) makes mixed lighting believable.

WORKED EXAMPLE · **Real bulbs in three.js units**

An 800 lumen household bulb as a `PointLight` : $I = 800/(4\pi) = 63.7$ cd, so `new THREE.PointLight("#ffd9a0", 63.7)` or `light.power = 800` . Conversely a point light of intensity 50 cd has `power` $= 50 \times 4\pi = 628$ lm.

10.2 Attenuation: how light fades with distance

Σ FORMULA · **Inverse-square law with a smooth cut-off (three.js)**

$$\text{falloff}(d) = \frac{1}{\max(d^{\,\text{decay}},\ 0.01)} \cdot \begin{cases} \text{saturate}\big(1 - (d/d_{\max})^4\big)^2 & d_{\max} > 0 \\ 1 & d_{\max} = 0 \end{cases}$$

d is the distance to the light, `decay` defaults to 2 (physically correct inverse square), and `distance` ($d_{\max}$) is an optional range after which the light is forced smoothly to zero (based on Frostbite's windowing function). The illuminance reaching a surface is $I \cdot \text{falloff}(d)$, then multiplied by $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}$.

WORKED EXAMPLE · **A lamp over a table**

`PointLight` with intensity 50 cd, decay 2.

- At 2 m: $50/2^2 = 12.5$. At 4 m: $50/16 = 3.125$. Doubling distance quarters the light.
- With `distance = 5` : at 4 m the window factor is $(1 - 0.8^4)^2 = (1 - 0.4096)^2 = 0.349$, so $3.125 \times 0.349 = 1.09$; at 5 m and beyond, exactly 0.

The window lets three.js (and clustered lighting) skip the light entirely beyond $d_{\max}$, at the price of a slightly unphysical fade.

10.2.1 The spotlight cone

Σ FORMULA · Spot attenuation

$$\text{spot}(\theta) = \text{smoothstep}\big(\cos(\text{angle}),\ \cos(\text{angle}\cdot(1-\text{penumbra})),\ \cos\theta\big)$$

θ is the angle between the spot's axis and the direction to the shaded point. Inside the inner cone the factor is 1, outside the outer cone 0, with a smooth edge between. `angle` is the half-angle of the cone in radians (maximum $\pi/2$).

WORKED EXAMPLE · A soft-edged spotlight

`angle` = 30°, `penumbra` = 0.5, so the outer cosine is $\cos 30^\circ = 0.8660$ and the inner cosine $\cos 15^\circ = 0.9659$. A point 20° off-axis: $\cos 20^\circ = 0.9397$.

$t = (0.9397 - 0.8660)/(0.9659 - 0.8660) = 0.737$; smoothstep $= 3t^2 - 2t^3 = 0.829$. The point receives 83% of the beam.

three.js — lights you will use most — JAVASCRIPT

```javascript
scene.add(new THREE.HemisphereLight("#cfe8ff", "#3b2f2f", 0.8));

const sun = new THREE.DirectionalLight("#fff4e0", 3);
sun.position.set(8, 12, 5);           // direction = position → target (default target at the origin)
scene.add(sun, sun.target);           // add the target if you move it

const spot = new THREE.SpotLight("#ffffff", 120, 0, THREE.MathUtils.degToRad(30), 0.5, 2);
spot.position.set(0, 4, 0);
spot.target.position.set(0, 0, 0);
scene.add(spot, spot.target);
```

R3F — the same lights — JSX

```jsx
const Lights = () => (
  <>
    <hemisphereLight args={["#cfe8ff", "#3b2f2f", 0.8]} />
    <directionalLight position={[8, 12, 5]} intensity={3} color="#fff4e0" />
    <spotLight position={[0, 4, 0]} angle={THREE.MathUtils.degToRad(30)} penumbra={0.5} intensity={120} />
  </>
);
```

10.3 Shadow mapping

Lighting formulas only ask "does this surface face the light?". They do not know whether something is **in the way**. Shadow mapping answers that with a second render, from the light's point of view.

DEFINITION · **Shadow map**

A depth texture rendered from the light's viewpoint. Each texel stores the distance from the light to the nearest surface in that direction. When shading a point, compare its own distance from the light with the stored value: if it is farther, something closer to the light blocks it, so it is in shadow.

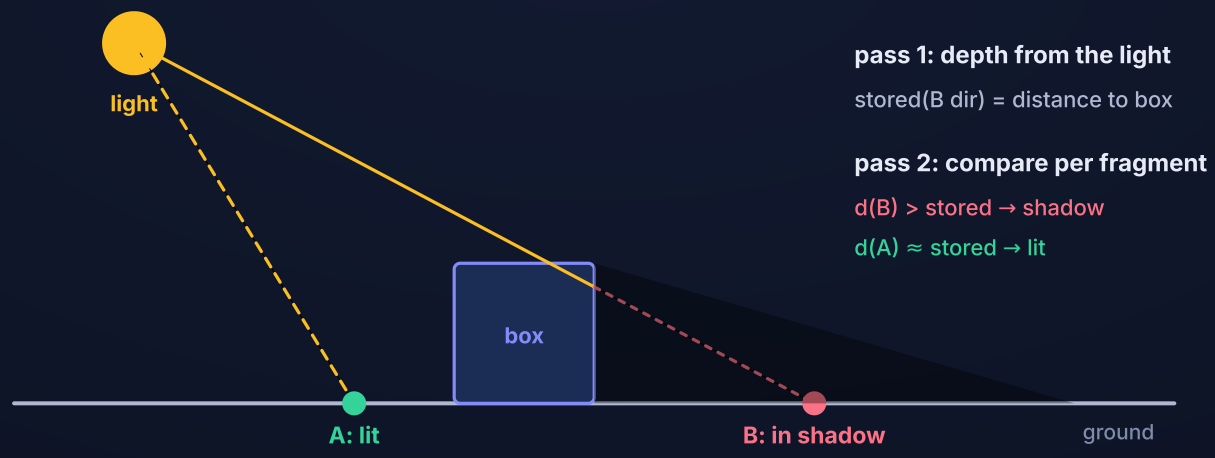


Figure 10.1 Shadow mapping in two passes. Pass 1 stores the nearest depth seen from the light. In pass 2, point A's depth matches the stored depth (lit), while point B is farther than the stored depth (the box is in the way), so it is in shadow.

FORMULA · **The shadow test**

For a fragment at world position $\mathbf{p}$:

$$\mathbf{q} = P_L\, V_L\, \mathbf{p}, \qquad (u, v, z) = \tfrac{1}{2}\left(\frac{\mathbf{q}_{xyz}}{q_w}\right) + \tfrac{1}{2}, \qquad \text{lit} = \left[\, z - \text{bias} \le \text{shadowMap}(u, v) \,\right]$$

$P_L V_L$ is the light camera's projection and view (orthographic for directional lights, perspective for spots, six cube faces for point lights). three.js stores this matrix per light as `light.shadow.matrix`.

10.3.1 Fitting the shadow camera

A directional light's shadow camera is an **orthographic box**. Everything outside the box casts and receives no shadow; everything inside shares the map's resolution. The tighter the box, the sharper the shadows.

WORKED EXAMPLE · **Shadow texel size**

`mapSize` 2048 × 2048.

- Box 40 m wide: $40/2048 = 0.0195$ m: each shadow texel covers about **2 cm**. A 1 cm cable casts a broken, blocky shadow.
- Box 10 m wide: $10/2048 = 0.0049$ m: about **5 mm** per texel, four times sharper.
- Memory: $2048^2 \times 4$ bytes $= 16.8$ MB per shadow map (32-bit depth). A 4096 map is 67 MB, and every shadow-casting point light needs six faces.

three.js — a well-fitted directional shadow JAVASCRIPT

```
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFShadowMap;      // soft PCF (r182+); VSMShadowMap is the
other option
sun.castShadow = true;
sun.shadow.mapSize.set(2048, 2048);
const cam = sun.shadow.camera;                      // an OrthographicCamera
cam.left = -6; cam.right = 6; cam.top = 6; cam.bottom = -6;   // just around the area of
interest
cam.near = 1; cam.far = 30;
sun.shadow.bias = -0.0005;                          // depth bias in shadow-map depth units
sun.shadow.normalBias = 0.02;                       // offset along the normal, in world units
cam.updateProjectionMatrix();
scene.add(new THREE.CameraHelper(cam));             // always look at the box while tuning
model.traverse((o) => { if (o.isMesh) { o.castShadow = true; o.receiveShadow = true; } });
```

10.3.2 Artefacts and how to fix them

Artefact	Looks like	Cause	Fix
Shadow acne	Stripes or moiré on lit surfaces	A surface compares against its own rounded depth and shadows itself	Small negative `bias`, or `normalBias` around one texel size
Peter-panning	Shadows detached from the object's base	Too much bias pushes the shadow away	Reduce bias; make thin objects thicker; prefer `normalBias`
Blocky edges	Stair-stepped shadow outlines	Too few texels per metre	Tighter shadow box, larger `mapSize`, cascades
Missing shadows	Some parts unshadowed	Objects outside the shadow box, or `castShadow` / `receiveShadow` not set	`CameraHelper`, set flags with `traverse`
Shimmering	Edges crawl as the camera moves	Shadow box moves by sub-texel amounts	Snap the box to texel increments; cascades do this

Figure 10.2 Shadow acne. The surface is sampled at texel centres, so on a sloped surface half the samples lie slightly below the stored depth and are wrongly shadowed. A bias moves the comparison just above the surface.

☆ NEW IN RECENT THREE.JS · **Shadows in r182–r186**

`PCFShadowMap` became soft in r182, so `PCFSoftShadowMap` was deprecated and then removed in r186 (renderers now warn and fall back to PCF). `WebGPURenderer`'s shadows improved in r183, so old projects should **reduce** their bias values after upgrading.

10.3.3 Soft shadows: PCF and VSM

DEFINITION · **Percentage-closer filtering (PCF)**

Instead of one depth comparison, take several comparisons around the lookup point and average the **results** (not the depths). The fraction that passes is the percentage of light, which blurs the shadow edge by the kernel's width.

WORKED EXAMPLE · **A 3×3 PCF kernel**

Nine neighbouring texels are compared with the fragment's depth; 4 say "lit" and 5 say "blocked". Light factor $= 4/9 = 0.44$: the fragment lies in the penumbra and is drawn 44% lit.

DEFINITION · **Variance shadow maps (VSM)**

Store the mean depth μ and the mean squared depth $E[d^2]$, which can be blurred like an ordinary image. At lookup, Chebyshev's inequality gives an upper bound on the lit fraction. Soft and smooth at any blur size, but prone to "light bleeding" where several occluders overlap.

Σ FORMULA · **Chebyshev's bound (VSM)**

$$\sigma^2 = E[d^2] - \mu^2, \qquad p_{\max}(t) = \frac{\sigma^2}{\sigma^2 + (t-\mu)^2} \quad (t > \mu)$$

t is the receiver's depth; if $t \le \mu$ the point is fully lit.

WORKED EXAMPLE · **VSM at a shadow edge**

Blurred moments $\mu = 10$, $E[d^2] = 100.25$, so $\sigma^2 = 0.25$. A receiver at $t = 11$: $p_{\max} = 0.25/(0.25 + 1) = 0.20$: 20% lit. At $t = 10.2$: $0.25/(0.25 + 0.04) = 0.86$.

10.3.4 Cascaded shadow maps and SunLight

One shadow map cannot cover a large outdoor scene sharply: near the camera you need centimetre texels, far away metres are fine. **Cascaded shadow maps** (CSM) split the view frustum into depth slices and give each slice its own shadow map, fitted tightly around it.

FORMULA · **The practical split scheme**

For N cascades between near n and far f, the i-th split distance blends logarithmic and uniform splits with $\lambda \in [0, 1]$:

$$z_i = \lambda\, n \left(\frac{f}{n}\right)^{i/N} + (1 - \lambda)\left(n + (f - n)\frac{i}{N}\right)$$

$\lambda = 1$ matches perspective's precision falloff; $\lambda \approx 0.5$ is a common compromise.

WORKED EXAMPLE · **Four cascades out to 100 m**

$n = 0.5$, $f = 100$, $\lambda = 0.5$: logarithmic splits $1.88,\ 7.07,\ 26.6,\ 100$; uniform $25.4,\ 50.3,\ 75.1,\ 100$; blended $z_1 = 13.6$, $z_2 = 28.7$, $z_3 = 50.9$, $z_4 = 100$ m. The first cascade covers only the nearest 13.6 m, so its 2048 texels give sub-centimetre shadows where the player looks.

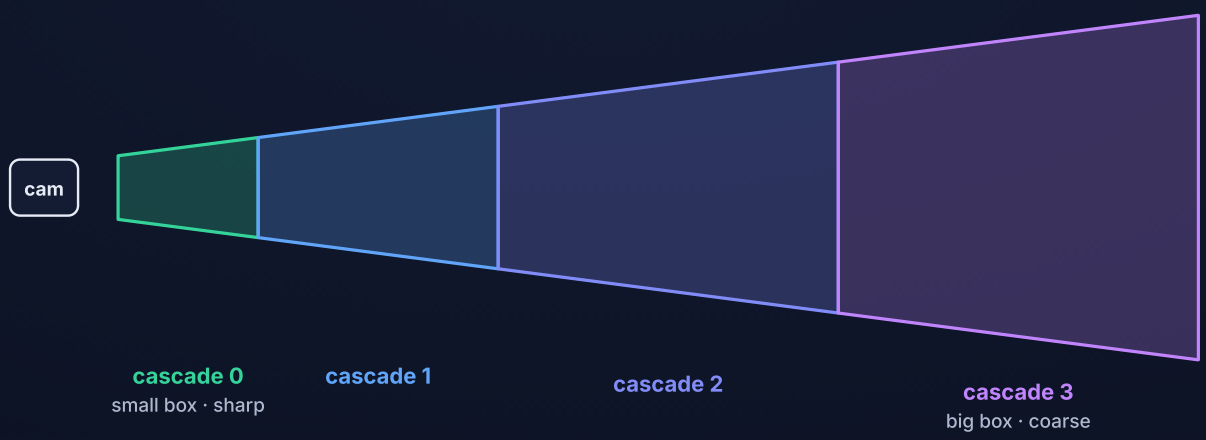


Figure 10.3 Cascaded shadows split the view frustum into slices; each slice gets its own shadow map fitted to it, so texel density is high near the camera and coarse far away.

three.js — SunLight (r186 addon), cascades without tuning JAVASCRIPT

```
import { SunLight } from "three/addons/lights/SunLight.js";

const sun = new SunLight(0xfff2e3, 3);
sun.position.set(1, 1, 1);                 // direction towards the sun
sun.castShadow = true;
sun.shadow.mapSize.set(2048, 2048);        // per cascade (default 1024); two cascades in one atlas
scene.add(sun);
// WebGPURenderer (r186): register its node once
// import { SunLightNode } from "three/addons/lights/SunLightNode.js";
// renderer.library.addLight(SunLightNode, SunLight);
```

TIP · When not to use cascades

Cascades fit the **view frustum**, not the scene. On a small, bounded stage (a product on a turntable) a single hand-fitted shadow box gives denser texels for less memory; the three.js maintainers switched their small-scene examples back to plain directional lights for this reason. Use `SunLight` (or the older `CSM` addon) for large worlds where the camera roams.

10.4 Cheaper ways to get shadows

Technique	Cost	Quality	Use when
Baked lightmaps (`aoMap`, `lightMap` from Blender)	Free at runtime	Excellent, soft, with bounce light	Static scenes: architecture, rooms
Blob shadow (a dark radial texture on a plane)	Tiny	Stylised	Characters in casual games
drei `<ContactShadows>`	One extra depth render, blurred	Soft grounding shadow	Products on a floor, configurators
drei `<AccumulativeShadows>` + `<RandomizedLight>`	Accumulates over many frames, then stops	Very soft, ray-traced look	Static product shots
`ShadowMaterial` on a plane	One plane	Receives real shadows only	AR, compositing over HTML
SSAO / GTAO (Chapter 19)	Post-process	Contact darkening, not cast shadows	Adds depth to any scene

TIP · Static shadows do not need re-rendering

If the light and the shadow casters do not move, render the shadow map once: `renderer.shadowMap.autoUpdate = false`, then set `renderer.shadowMap.needsUpdate = true` whenever something changes. Each shadow-casting light otherwise re-renders the casters every frame (point lights six times).

R3F — shadows with drei helpers JSX

```jsx
<Canvas shadows camera={{ position: [3, 2, 5], fov: 40 }}>
  <directionalLight castShadow position={[5, 8, 3]} intensity={3}
    shadow-mapSize={[2048, 2048]} shadow-bias={-0.0005} shadow-normalBias={0.02}
    shadow-camera-left={-5} shadow-camera-right={5} shadow-camera-top={5} shadow-camera-bottom=
{-5} />
  <mesh castShadow position-y={0.5}><boxGeometry /><meshStandardMaterial /></mesh>
  <mesh receiveShadow rotation-x={-Math.PI / 2}><planeGeometry args={[20, 20]} />
<meshStandardMaterial /></mesh>
  <ContactShadows position-y={0.001} opacity={0.6} scale={10} blur={2.5} far={4} />
</Canvas>
```

✓ KEY POINTS

- ✓ Directional (sun), point (bulb, candela), spot (cone, candela), rect-area (nits), hemisphere and environment lighting.
- ✓ three.js falloff: $1/d^{\text{decay}}$ with an optional smooth window to zero at `distance`.
- ✓ Spot factor: smoothstep between $\cos(\text{angle})$ and $\cos(\text{angle}(1-\text{penumbra}))$.
- ✓ Shadow mapping: render depth from the light, then compare each fragment's light-space depth with the map.
- ✓ Texel size = shadow box width ÷ `mapSize`: fit the box tightly; a 2048 map costs about 17 MB.
- ✓ Acne → bias / `normalBias`; too much bias → peter-panning.
- ✓ PCF averages comparisons; VSM uses moments + Chebyshev; cascades split the frustum; `SunLight` (r186) automates two cascades.
- ✓ Static scenes: bake lighting, or render the shadow map once with `autoUpdate = false`.

10.5 Interview questions

Q Explain how shadow mapping works. MID

A Two passes. First, render the scene's depth from the light's point of view into a texture (orthographic for a directional light, perspective for a spot, a cube for a point light). Second, when shading each fragment from the camera, transform its world position into the light's clip space, look up the stored depth at that position, and compare: if the fragment is farther from the light than the stored depth (plus a small bias), an occluder is in between and it is in shadow. Filtering (PCF, VSM) softens the edges.

Q **Your shadows look striped on lit surfaces. After adding bias, they float away from the objects. What is going on?** MID}

A The stripes are shadow acne: limited shadow-map resolution and depth precision make a surface compare against its own rounded depth and partially shadow itself, especially at grazing light angles. A large constant bias removes acne but separates shadows from their casters (peter-panning). Use a small bias plus `normalBias` (offsets the lookup along the surface normal, scaling naturally with slope), tighten the shadow camera so texels are smaller, and avoid paper-thin geometry. Tune with a `CameraHelper` on the shadow camera.

Q **How would you get good shadows for a large open world?** SENIOR}

A Use cascaded shadow maps: split the view frustum into several depth ranges (logarithmic/uniform blend), fit a light-space orthographic box around each, render a shadow map per cascade and pick (and blend) the right one per fragment. Stabilise each cascade by snapping its box to texel increments to stop shimmering. In three.js r186 `SunLight` does this with two cascades for both renderers; the `CSM` addon offers more control. Add baked or screen-space AO for contact detail, and limit shadow casters (skip small objects, use layers) to keep the extra renders cheap.

Q **A point light with shadows makes the scene much slower. Why, and what are the alternatives?** MID}

A A point light's shadow needs depth renders in six directions (a cube map), so every shadow caster is drawn six extra times per frame, plus memory for six faces. Alternatives: use a spot light if the light only needs to cover a cone (one render), disable `castShadow` on small objects, lower `mapSize`, render the shadow once (`autoUpdate = false`) when nothing moves, or bake the lighting into lightmaps for static scenes.

Q **What is the difference between `light.shadow.bias` and `light.shadow.normalBias` ?** SENIOR}

A `bias` is a constant offset added to the depth comparison in the shadow map's depth units: simple, but the right value depends on the slope of the surface relative to the light and on depth range, so one value rarely fits everywhere. `normalBias` moves the lookup position along the surface normal in world units before projecting into the shadow map; because the required offset grows with the slope, offsetting along the normal handles grazing angles much better and causes less peter-panning. A typical setup uses a tiny negative `bias` and a `normalBias` close to one shadow texel's world size.

Textures and UV Mapping

11

How images are wrapped onto surfaces. UV coordinates, wrapping and repeat, magnification and minification filters, mipmaps and anisotropy with the maths of choosing a mip level, GPU memory budgets, every PBR map type, normal mapping and the TBN matrix, compressed KTX2 textures, and special texture types.

IN THIS CHAPTER

11.1 UV coordinates
11.2 Wrapping, repeat and transforms
11.3 Filtering: magnification and minification
11.4 GPU memory budgets
11.5 The PBR texture set
11.6 Normal mapping and tangent space
11.7 Other kinds of textures
11.8 KTX2 and Basis Universal
11.9 Interview questions

11.1 UV coordinates

DEFINITION · **UV coordinates**

A 2D coordinate per vertex, (u, v), saying which point of a texture belongs to that vertex. u runs left to right and v bottom to top across the image, from 0 to 1. The rasteriser interpolates UVs across each triangle, and the fragment shader samples the texture at the interpolated UV.

DEFINITION · **UV unwrapping**

Cutting a 3D surface along **seams** and flattening it into the 0–1 square so a 2D image can be painted onto it, like flattening a cardboard box. Done in modelling tools (Blender's UV editor). Seams duplicate vertices (Chapter 8); stretching shows up as distorted textures.

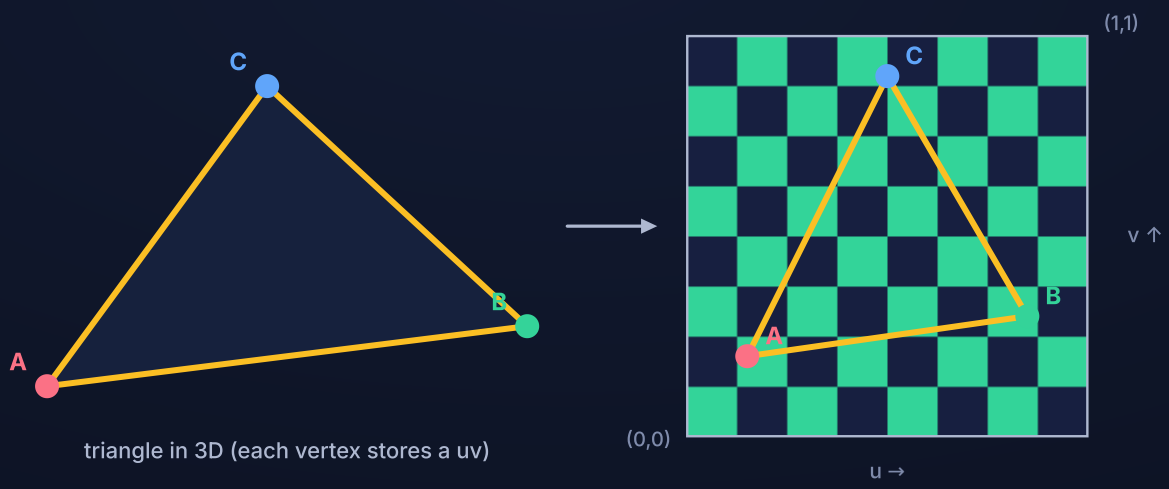


Figure 11.1 A UV map. Each vertex of the mesh points at a location in the texture; the triangle in 3D shows the triangle cut out of the image.

ⓘ NOTE · **flipY**

Images are stored top row first, but WebGL's texture origin is bottom-left. three.js flips images on upload (`texture.flipY = true`) so that $v = 0$ is the bottom of the picture. glTF uses the opposite convention, so `GLTFLoader` sets `flipY = false` on its textures. If you swap a texture on a glTF model by hand and it appears upside down, this is why.

11.2 Wrapping, repeat and transforms

Property	Values	Effect
`wrapS`, `wrapT`	`ClampToEdgeWrapping` (default), `RepeatWrapping`, `MirroredRepeatWrapping`	What happens outside 0–1 in u (S) and v (T)
`repeat`	`Vector2`	Scales UVs: `repeat.set(4, 4)` tiles the image 4 × 4 (needs `RepeatWrapping`)
`offset`	`Vector2`	Shifts UVs (scrolling water, sprite-sheet frames)
`rotation`, `center`	radians, `Vector2`	Rotates UVs about `center`
`channel`	0, 1, 2, 3	Which UV set to use (`uv`, `uv1`, …): light maps often use a second set

Σ FORMULA · **The UV transform three.js applies**

$$\mathbf{uv}' = R(\text{rotation})\,\big((\mathbf{uv} - \mathbf{c}) \odot \text{repeat}\big) + \mathbf{c} + \text{offset}$$

(three.js composes this into `texture.matrix`; call `texture.updateMatrix()` if you set `matrixAutoUpdate = false`.)

WORKED EXAMPLE · **Tiling floorboards to real size**

A floor plane is 8 m × 6 m and the plank texture shows 2 m × 2 m of real wood. To keep planks the right size: `repeat.set(8/2, 6/2) = (4, 3)` with `RepeatWrapping` on both axes. If the floor is resized to 10 m × 6 m, update repeat to $(5, 3)$, otherwise the planks stretch.

11.3 Filtering: magnification and minification

A texture is a grid of **texels**. They rarely line up one-to-one with screen pixels:

- **Magnification**: one texel covers many pixels (a close-up). `magFilter`: `NearestFilter` (blocky, for pixel art) or `LinearFilter` (bilinear blend of the 4 nearest texels, smooth).
- **Minification**: many texels fall inside one pixel (far away). Sampling just one of them causes **aliasing**: shimmering moiré as the camera moves. The fix is mipmapping.

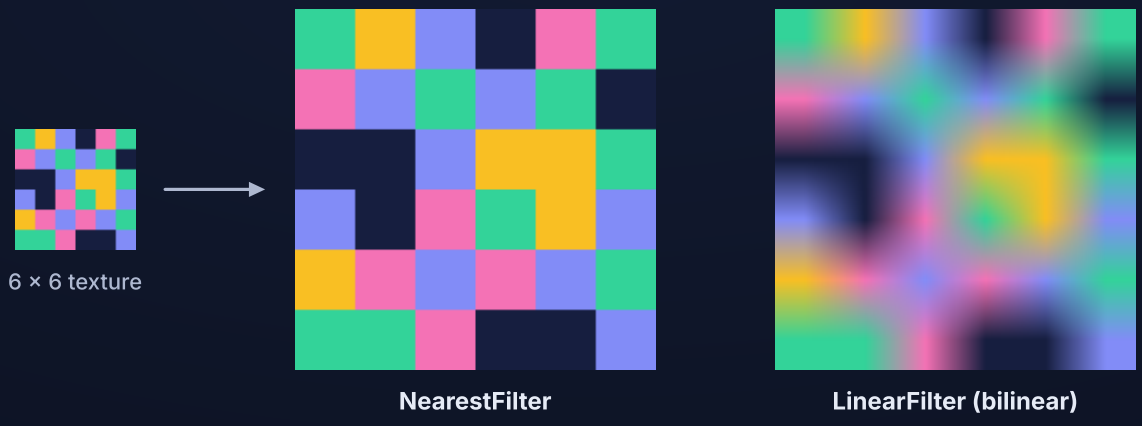


Figure 11.2 Magnifying a 6×6 texture: nearest filtering keeps hard texel edges (right for pixel art); bilinear filtering blends the four nearest texels (smooth, slightly blurry).

DEFINITION · **Mipmaps**

A chain of pre-shrunk copies of a texture, each half the width and height of the previous one, down to 1 × 1. When a texture is minified, the GPU samples the level whose texel size best matches the pixel size, so each sample already averages the right area. `LinearMipmapLinearFilter` (trilinear, the default `minFilter`) blends bilinear samples from the two nearest levels.

FORMULA · **Choosing a mip level**

$$\lambda = \log_2 \rho, \qquad \rho = \max\left(\left\|\frac{\partial (u,v)\, s}{\partial x}\right\|, \left\|\frac{\partial (u,v)\, s}{\partial y}\right\|\right)$$

ρ is the number of texels a one-pixel step covers (s is the texture size; the derivatives come from neighbouring pixels in a 2×2 quad). Level $\lambda = 0$ is the full texture, level 1 is half size, and so on.

WORKED EXAMPLE · **Which mip level is used?**

A 1024 × 1024 texture on a quad that covers 128 × 128 pixels on screen: each pixel spans $1024/128 = 8$ texels, so $\lambda = \log_2 8 = 3$: the GPU samples the 128×128 mip, which is exactly one texel per pixel.

Σ FORMULA · **Mipmap memory**

$$\text{total} = \text{base} \cdot \left(1 + \tfrac{1}{4} + \tfrac{1}{16} + \cdots\right) = \tfrac{4}{3}\,\text{base}$$

A full mip chain costs one third more memory.

DEFINITION · **Anisotropic filtering**

Mipmaps assume the pixel's footprint on the texture is square. On surfaces seen at a grazing angle (a road stretching away) the footprint is a long thin ellipse, so trilinear filtering picks a level that is too blurry. Anisotropic filtering takes several samples along the long axis. Set `texture.anisotropy = renderer.capabilities.getMaxAnisotropy()` (usually 16) on ground, road and floor textures; it is cheap on modern GPUs.

⚠ PITFALL · **Non-power-of-two textures**

WebGL 2 supports mipmaps for any size, but power-of-two sizes (512, 1024, 2048) still mip down cleanly and are required by several compressed formats and block sizes. Keep textures power-of-two and square where possible; three.js resizes images only in specific legacy cases.

11.4 GPU memory budgets

The file size of a PNG or JPEG says nothing about GPU memory: images are **decoded** to raw pixels before upload.

WORKED EXAMPLE · **What a 4K texture really costs**

4096 × 4096 RGBA8: $4096^2 \times 4$ bytes $= 64$ MiB, with mipmaps $\times 4/3 = 85.3$ MiB. A typical PBR material has base colour, normal and an ORM map: about **256 MiB** for one material. A 2.5 MB JPEG on disk becomes 85 MiB in VRAM.

The same texture in a GPU-compressed format: BC7 or ASTC 4×4 store 1 byte per texel ($16 \rightarrow 21.3$ MiB with mips); BC1/ETC1-class formats store 0.5 bytes per texel ($8 \rightarrow 10.7$ MiB). That is a **4–8× saving**, and the GPU samples compressed data directly.

Format	Bytes per texel	Notes
RGBA8 (PNG/JPEG/WebP decoded)	4	Universal, largest
RGBA16F (HDR, render targets)	8	Linear lighting data
BC7 / ASTC 4×4	1	High quality; desktop (BC) / mobile and Apple (ASTC)
BC1 / ETC1 / ETC2 RGB	0.5	Smaller, lower quality, no or 1-bit alpha
KTX2 + Basis Universal	transcodes to the above	One file transcoded at load time to what the GPU supports

11.5 The PBR texture set

Map	Material property	Colour space	Channel convention
Base colour (albedo)	`map`	**sRGB**	RGB colour, A alpha
Normal	`normalMap` (+ `normalScale`)	Linear (non-colour)	RGB = XYZ in tangent space
Roughness	`roughnessMap`	Linear	glTF: **G** channel
Metalness	`metalnessMap`	Linear	glTF: **B** channel
Ambient occlusion	`aoMap` (+ `aoMapIntensity`)	Linear	glTF: **R** channel (often a separate UV set)
Emissive	`emissiveMap` (+ `emissive`, `emissiveIntensity`)	sRGB	RGB glow colour
Alpha	`alphaMap`	Linear	G channel
Height	`displacementMap` (moves vertices) / `bumpMap` (fakes normals)	Linear	Greyscale
Light map	`lightMap`	Linear (HDR)	Baked lighting
Environment	`envMap` / `scene.environment`	Linear HDR	Equirect or cube (Chapter 13)

TIP · ORM packing

glTF packs occlusion, roughness and metalness into one texture's R, G and B channels ("ORM"), and three.js reads roughness from G and metalness from B automatically, so the same texture object can be assigned to `aoMap`, `roughnessMap` and `metalnessMap`. One texture fetch and one third of the memory instead of three textures.

⚠ PITFALL · **Colour space mistakes**

Only textures that hold **colours a human picked** (base colour, emissive) are sRGB. Data textures (normal, roughness, metalness, AO, height) are linear and must stay `NoColorSpace`. A base colour left as `NoColorSpace` looks washed out and too bright; a normal map marked sRGB gives wrong, bent lighting. `TextureLoader` leaves `colorSpace` as `NoColorSpace` by default, so set it yourself for colour maps. `GLTFLoader` sets it correctly for you.

three.js — loading a PBR texture set — JAVASCRIPT

```
const loader = new THREE.TextureLoader();
const load = (url, srgb = false) => {
  const t = loader.load(url);
  if (srgb) t.colorSpace = THREE.SRGBColorSpace;
  t.wrapS = t.wrapT = THREE.RepeatWrapping;
  t.repeat.set(4, 4);
  t.anisotropy = renderer.capabilities.getMaxAnisotropy();
  return t;
};
const orm = load("planks_orm.jpg");
const floorMat = new THREE.MeshStandardMaterial({
  map: load("planks_color.jpg", true),
  normalMap: load("planks_normal.png"),
  aoMap: orm, roughnessMap: orm, metalnessMap: orm,    // R, G, B channels of one texture
});
```

R3F — drei's useTexture — JSX

```
import { useTexture } from "@react-three/drei";

function Floor() {
  const maps = useTexture(
    { map: "planks_color.jpg", normalMap: "planks_normal.png", roughnessMap: "planks_orm.jpg" },
    (loaded) => {                                    // runs once the textures load
      const list = Array.isArray(loaded) ? loaded : Object.values(loaded);
      for (const t of list) { t.wrapS = t.wrapT = THREE.RepeatWrapping; t.repeat.set(4, 4); }
    }
  );
  maps.map.colorSpace = THREE.SRGBColorSpace;         // colour map only
  return (
    <mesh rotation-x={-Math.PI / 2} receiveShadow>
      <planeGeometry args={[8, 8]} />
      <meshStandardMaterial {...maps} />              {/* the object's keys match material
props */}
    </mesh>
  );
}
```

11.6 Normal mapping and tangent space

DEFINITION · Normal map

A texture whose RGB values encode a **surface normal** per texel, adding fine detail (bricks, scratches, pores) to lighting without adding triangles. Values are stored in **tangent space**, relative to the surface itself, so the same map works however the surface is oriented or deformed. A flat area is the familiar light blue $(128, 128, 255)$, meaning "straight out of the surface".

FORMULA · Decoding and transforming a tangent-space normal

$$\mathbf{n}_t = 2\,\mathbf{c} - 1, \qquad \mathbf{n}_{\text{world}} = \text{normalise}\big(\,[\,\hat{\mathbf{T}}\ \ \hat{\mathbf{B}}\ \ \hat{\mathbf{N}}\,]\,\mathbf{n}_t\big)$$

$\mathbf{c} \in [0,1]^3$ is the sampled colour. $\hat{\mathbf{T}}$ (tangent) follows the direction of increasing u, $\hat{\mathbf{B}}$ (bitangent) increasing v, $\hat{\mathbf{N}}$ is the geometric normal. The **TBN matrix** turns the map's normal into world (or view) space. `normalScale` scales the X and Y components to strengthen or weaken the effect.

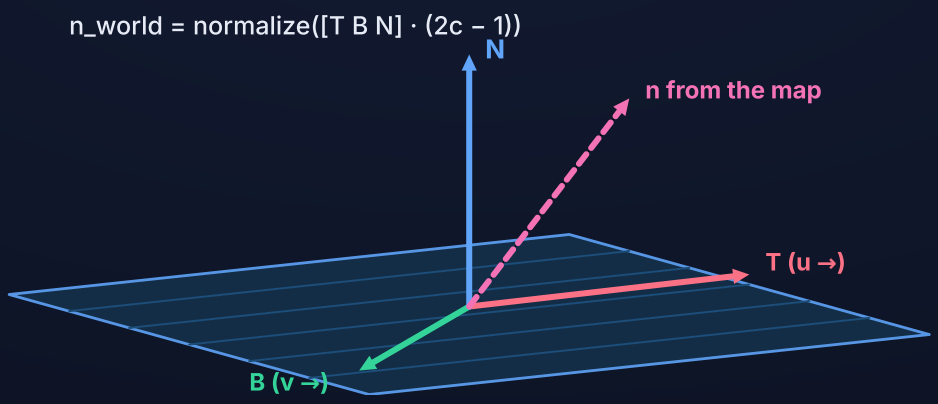


Figure 11.3 The tangent frame at a point on a surface. T follows the texture's u direction, B its v direction, N the surface normal. A normal-map texel is a direction in this frame.

WORKED EXAMPLE · Reading a normal-map texel

Texel colour $(200, 128, 220)$:

1. $\mathbf{c} = (200, 128, 220)/255 = (0.784, 0.502, 0.863)$.
2. $\mathbf{n}_t = 2\mathbf{c} - 1 = (0.569, 0.004, 0.725)$; length 0.922; normalised $(0.617, 0.004, 0.787)$.
3. The detail normal leans towards $+T$ by $\arccos(0.787) = 38^\circ$: a slope rising along the texture's u direction, the side of a groove.
4. The flat colour $(128, 128, 255)$ decodes to $(0.004, 0.004, 1)$: straight up, no change.

> ⚠ PITFALL · **OpenGL versus DirectX normal maps**
>
> There are two conventions for the green channel: OpenGL-style (+Y up, used by three.js, glTF and Blender) and DirectX-style (+Y down, common in game-engine exports). A DirectX map lit in three.js makes bumps look like dents in one direction. Fix it with `material.normalScale.y = -1` or by inverting the green channel of the image.

three.js derives the tangent frame per pixel from screen-space derivatives of the position and UV when the geometry has no `tangent` attribute. For mirrored UVs and skinned meshes, precomputed tangents are more reliable: glTF exporters can include them, or call `geometry.computeTangents()` (needs an index, normals and UVs).

Technique	Moves geometry	Changes silhouette	Cost	Notes
Bump map	No	No	Low	Greyscale height; normals from derivatives
Normal map	No	No	Low	Best detail per cost; the standard
Parallax / parallax occlusion mapping	No (fakes depth via UV shift)	No	Medium–high	Deep bricks and grates in custom shaders
Displacement map	Yes (per vertex)	Yes	Needs many vertices	Terrain, cloth; needs dense geometry

11.7 Other kinds of textures

Class	What it is	Typical use
`DataTexture`	A texture from a typed array you fill	Lookup tables, heightfields, GPGPU data, toon gradients
`DataArrayTexture`	Many same-size layers in one texture, sampled with a layer index	Terrain splat layers; per-instance textures with `BatchedMesh`
`Data3DTexture`	A true 3D volume	Medical volumes (CT/MRI), 3D noise, clouds
`CanvasTexture`	A 2D canvas you draw on	Dynamic labels, charts, signage (set `needsUpdate` after drawing)
`VideoTexture`	Frames of a `<video>`	Screens, 360° video, AR feeds
`CubeTexture`	Six images forming a cube	Skyboxes, reflections (legacy; HDR equirect + PMREM is preferred)
`CompressedTexture`	GPU-compressed blocks	Produced by `KTX2Loader`
`DepthTexture`	The depth buffer of a render target	Soft particles, SSAO, fog, outlines

Class	What it is	Typical use
`FramebufferTexture`	A copy of the current framebuffer	Refraction and distortion effects

DEFINITION · **Texture atlas**

Many small images packed into one big texture, with each object's UVs pointing to its own region. Objects that differ only in texture can then share one material, which enables merging, instancing or batching. Leave padding between regions, or mipmapping bleeds neighbouring images together at a distance.

11.8 KTX2 and Basis Universal

DEFINITION · **KTX2 / Basis Universal**

KTX2 is a container format for GPU textures. With **Basis Universal** supercompression, one `.ktx2` file can be transcoded at load time to whatever compressed format the device's GPU supports (BC on desktop, ASTC or ETC on mobile). Two modes: **ETC1S** (very small files, lower quality, good for colour) and **UASTC** (larger, high quality, good for normal maps).

three.js — KTX2 textures (also used by GLTFLoader) JAVASCRIPT

```
import { KTX2Loader } from "three/addons/loaders/KTX2Loader.js";
const ktx2 = new KTX2Loader()
  .setTranscoderPath("https://cdn.jsdelivr.net/npm/three@0.186.0/examples/jsm/libs/basis/")
  .detectSupport(renderer);          // after the renderer exists (await renderer.init() for WebGPU)
const albedo = await ktx2.loadAsync("rock_albedo.ktx2");
albedo.colorSpace = THREE.SRGBColorSpace;
gltfLoader.setKTX2Loader(ktx2);      // glTF files using KHR_texture_basisu now load too
```

TIP · **Convert with gltf-transform**

`npx @gltf-transform/cli etc1s in.glb out.glb` (or `uastc`, `optimize` for everything) converts all textures in a glTF to KTX2, typically cutting VRAM by 4–8× and download size substantially. Use UASTC for normal maps, which show ETC1S block artefacts badly.

✓ KEY POINTS

- ✓ UVs map vertices to texture positions; seams and stretching come from the unwrap.
- ✓ `RepeatWrapping` + `repeat` to tile; keep real-world scale by matching repeat to size.
- ✓ Mip level $\lambda = \log_2(\text{texels per pixel})$; mipmaps add $1/3$ memory; anisotropy fixes grazing-angle blur.
- ✓ VRAM = width × height × bytes per texel × 4/3; a 4K RGBA texture is 85 MiB, compressed 11–21 MiB.
- ✓ Only colour maps are sRGB; data maps stay linear. `TextureLoader` does not set sRGB for you; `GLTFLoader` does.
- ✓ Normal maps store tangent-space normals: $\mathbf{n} = 2\mathbf{c} - 1$, transformed by the TBN matrix.
- ✓ KTX2/Basis gives one file that transcodes to each GPU's compressed format.

11.9 Interview questions

Q **What are mipmaps and why do textures shimmer without them?** JUNIOR}

A Mipmaps are pre-filtered half-size copies of a texture down to 1 × 1. When a textured surface is far away, each pixel covers many texels; sampling only one of them picks an arbitrary texel each frame, causing shimmering and moiré. The GPU instead chooses the mip level where one texel is about one pixel ($\lambda = \log_2 \rho$) and samples it, which is already averaged. They cost one third more memory. Anisotropic filtering further improves surfaces seen at grazing angles.

Q **A texture looks washed out and too bright. Another model's bumps look inverted. What is wrong in each case?** MID}

A Washed out: a colour (base colour) texture is being treated as linear data. Set `texture.colorSpace = THREE.SRGBColorSpace` so the shader decodes sRGB to linear before lighting. Inverted bumps: the normal map uses the DirectX green-channel convention while three.js expects OpenGL (+Y), so flip it with `material.normalScale.y = -1` (or invert the green channel). Also check that normal maps are not marked sRGB.

Q **A product page loads ten 4K textures and crashes on older iPhones. What do you do?** MID}

A Decoded, ten 4K RGBA textures with mips need about 850 MiB of GPU memory, which exceeds what mobile Safari allows. Reduce resolution where the screen size does not need 4K (2K or 1K for most parts), pack ORM channels into one texture, convert to KTX2/Basis (4–8× less VRAM), share textures between materials, dispose textures of variants not on screen, and load progressively (low resolution first). Check with `renderer.info.memory` and the browser's memory tools.

Q How does normal mapping work? MID}

A Each texel stores a unit normal in the surface's tangent space, encoded as colour ($\mathbf{c} = 0.5\,\mathbf{n} + 0.5$). In the fragment shader the colour is decoded ($\mathbf{n} = 2\mathbf{c} - 1$), scaled by `normalScale`, and transformed to view or world space with the TBN matrix built from the tangent (direction of increasing u), bitangent (increasing v) and the interpolated normal. The perturbed normal is then used for lighting, so flat triangles light up as if they had detailed relief, but the silhouette does not change.

Q What is the difference between a bump map, a normal map and a displacement map? JUNIOR}

A A bump map is a greyscale height image from which the shader derives perturbed normals using derivatives; a normal map stores the perturbed normals directly (more accurate, the standard). Both change only lighting. A displacement map actually moves vertices along their normals in the vertex shader, changing the silhouette, but only as finely as the mesh is tessellated, so it needs dense geometry.

Colour, HDR and Tone Mapping

12

Why lighting must be computed in linear light, the exact sRGB transfer function, three.js colour management, high dynamic range and exposure, the Reinhard, ACES, AgX and Neutral tone mappers compared with numbers and pictures, banding and dithering, and the R3F defaults.

IN THIS CHAPTER

12.1 Light is linear, displays are not
12.2 Colour management in three.js
12.3 High dynamic range
12.4 Tone mapping
12.5 Banding and dithering
12.6 Interview questions

12.1 Light is linear, displays are not

Physical light adds up: two identical lamps give exactly twice the light of one. Lighting maths (sums of light contributions, blending, blurring, antialiasing) is only correct on numbers **proportional to light energy**, called **linear** values.

Human vision, however, is far more sensitive to differences between dark tones than between bright ones. Image files and displays therefore store colour **non-linearly** to spend their limited bits where the eye can see them: that encoding is the **sRGB transfer function**.

DEFINITION · **Colour space**

A colour space is defined by three choices: its **primaries** (which red, green and blue), its **white point** (sRGB uses D65), and its **transfer function** (how numbers map to light). *sRGB* and *Linear-sRGB* share primaries and white point and differ only in the transfer function: Linear-sRGB is proportional to light, sRGB is perceptually encoded.

FORMULA · **The sRGB transfer functions**

$$\text{encode}(L) = \begin{cases} 12.92\,L & L \le 0.0031308 \\ 1.055\,L^{1/2.4} - 0.055 & \text{otherwise} \end{cases} \qquad \text{decode}(V) = \begin{cases} V/12.92 & V \le \\ \left(\dfrac{V+0.055}{1.055}\right)^{2.4} & \text{othe} \end{cases}$$

L is linear light, V the encoded value, both in $[0, 1]$. The curve is close to a power of 2.2, which is why it is loosely called “gamma”, a term the three.js documentation avoids because it is ambiguous.

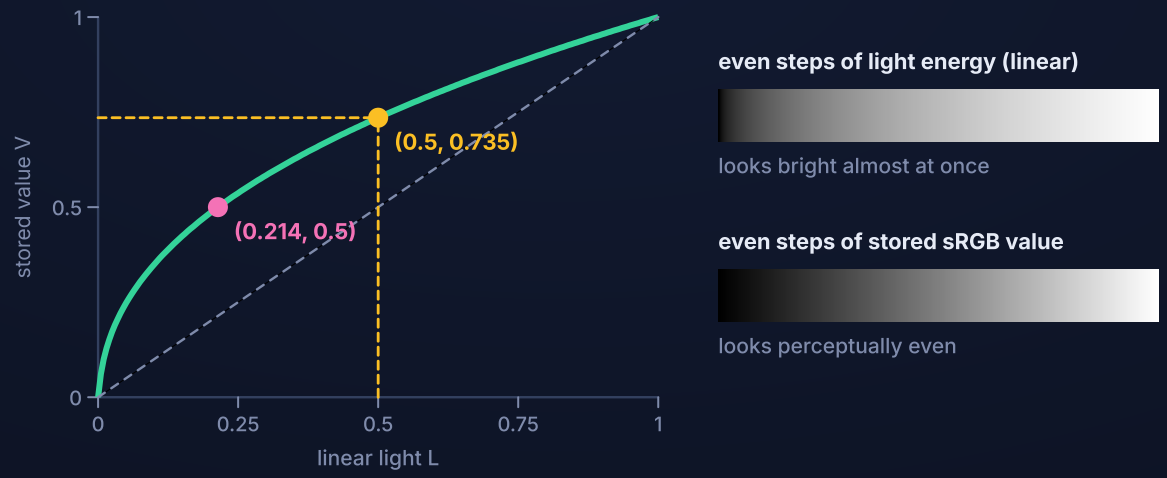


Figure 12.1 The sRGB transfer function. Half the light (linear 0.5) is stored as 0.735; the stored value 0.5 means only 21% of full light. Encoding spends more of the 256 levels on dark tones.

WORKED EXAMPLE · **Mid-grey is not 0.5**

1. Linear 0.5 (half the light of white) encodes to $1.055 \times 0.5^{1/2.4} - 0.055 = 0.735$, the hex value `#BCBCBC`.
2. The hex colour `#808080` (encoded $128/255 = 0.502$) decodes to linear 0.216: only about a fifth of white's light.
3. Photographic "18% grey" (linear 0.18) encodes to 0.461, close to the middle of the 0–255 range. That is the point of the encoding: perceptually even steps.

WORKED EXAMPLE · **Why blending must be linear**

An edge pixel half covered by a white line on a black background should emit half the light.

- In linear space: $(0 + 1)/2 = 0.5$ linear, displayed as encoded 0.735. Correct.
- Averaging the encoded values: $(0 + 1)/2 = 0.5$ encoded, which displays only 0.214 linear light. Edges look too dark and thin; gradients between colours pass through muddy, dark midpoints.

12.2 Colour management in three.js

three.js follows a **linear workflow**: convert every input to linear, compute everything in linear, convert once at the end.

Role	Colour space	three.js setting
Input: colour textures (base colour, emissive), CSS/hex colours	sRGB	`texture.colorSpace = SRGBColorSpace`; hex/CSS converted automatically
Input: data textures (normal, roughness, metalness, AO)	none (linear data)	`NoColorSpace` (default)
Input: HDR environment maps, EXR	Linear-sRGB	set by `HDRLoader` / `EXRLoader`
Working space (lighting, blending, render targets)	Linear-sRGB	`THREE.ColorManagement.workingColorSpace`
Output to the canvas	sRGB	`renderer.outputColorSpace = SRGBColorSpace` (default)

DEFINITION · **ColorManagement**

`THREE.ColorManagement.enabled` (true by default) makes `Color` setters convert sRGB inputs (hex numbers, CSS strings) to the linear working space, and getters convert back. So `new THREE.Color(0x808080).r` is 0.2159, while `getHex()` returns `0x808080` again. Plain `color.r = 0.5` is taken as already linear.

three.js — colours in and out JAVASCRIPT

```
const c = new THREE.Color("#808080");       // sRGB input, stored linear
console.log(c.r.toFixed(4));                // "0.2159"
console.log(c.getHexString());              // "808080" (converted back to sRGB)
c.setRGB(0.5, 0.5, 0.5, THREE.SRGBColorSpace);   // say explicitly which space the numbers are in
```

PITFALL · **Custom shaders and render targets**

`ShaderMaterial` writes whatever your fragment shader outputs. To get the renderer's tone mapping and sRGB conversion, add `#include <tonemapping_fragment>` and `#include <colorspace_fragment>` at the end of `main()`. Render targets stay linear: the sRGB conversion must happen exactly **once**, at the final output (with post-processing that is the `OutputPass`, or automatically in `RenderPipeline`). Converting twice gives a washed-out image; forgetting gives a dark, contrasty one.

12.3 High dynamic range

DEFINITION · **High dynamic range (HDR)**

Light values that are not limited to $[0, 1]$. Real scenes span huge ranges: a sunlit sky can be thousands of times brighter than a shadowed corner. Rendering in HDR (float buffers, values above 1) keeps that information: bright highlights stay distinguishable, bloom can find what is truly bright, and exposure can be adjusted afterwards.

HDR sources in three.js: `.hdr` (RGBE, `HDRLoader`, called `RGBELoader` before r180), `.exr` (`EXRLoader`), Ultra HDR JPEGs with gain maps (`UltraHDRLoader`, small files with HDR range), and half-float render targets (`HalfFloatType`) for post-processing.

12.3.1 Exposure

Σ FORMULA · Exposure in stops

$$L_{\text{exposed}} = L \cdot 2^{\text{EV}}$$

Each **stop** (EV) doubles or halves the light, as on a camera. three.js applies `renderer.toneMappingExposure` as the multiplier before the tone curve, so `toneMappingExposure = 2` is +1 stop and `0.5` is −1 stop.

12.4 Tone mapping

A display can only show $[0, 1]$. **Tone mapping** compresses the open HDR range into it with a smooth curve, instead of clipping everything above 1 to flat white.

DEFINITION · Tone mapping

A function $T : [0, \infty) \to [0, 1)$ applied to each linear HDR colour before sRGB encoding. Good tone mappers keep mid-tones roughly where they were, roll highlights off smoothly so detail survives, and (ideally) avoid shifting hues when bright saturated colours approach white.

Operator	three.js constant	Character	Use for
None	`NoToneMapping`	Values clip at 1	UI, unlit scenes, when post-processing tone-maps later
Linear	`LinearToneMapping`	Exposure scaling only, clips	Debugging
Reinhard	`ReinhardToneMapping`	$x/(1+x)$: soft but flat and greyish	Simple, predictable
Cineon	`CineonToneMapping`	Film-like	Stylised
ACES Filmic	`ACESFilmicToneMapping`	Contrasty, saturated, cinematic; bright colours skew (blue → purple, red → orange)	Games, cinematic scenes; R3F's default
AgX	`AgXToneMapping`	Desaturates bright highlights gracefully, no hue skew; flatter	Scenes with bright saturated lights; Blender's default since 4.0
Neutral (Khronos PBR Neutral)	`NeutralToneMapping`	Keeps base colours almost exact below the highlights	E-commerce and product configurators where colour accuracy matters
Custom	`CustomToneMapping`	Your own GLSL	Special looks

Σ FORMULA · Two classic curves

$$T_{\text{Reinhard}}(x) = \frac{x}{1+x} \qquad T_{\text{ACES fit}}(x) = \frac{x\,(2.51x+0.03)}{x\,(2.43x+0.59)+0.14}$$

The second is Krzysztof Narkowicz's popular approximation of the ACES filmic curve (it expects input pre-scaled by about 0.6). three.js uses a closer fit of the ACES reference transforms, but the shape is the same: a toe that deepens shadows and a shoulder that rolls off highlights.

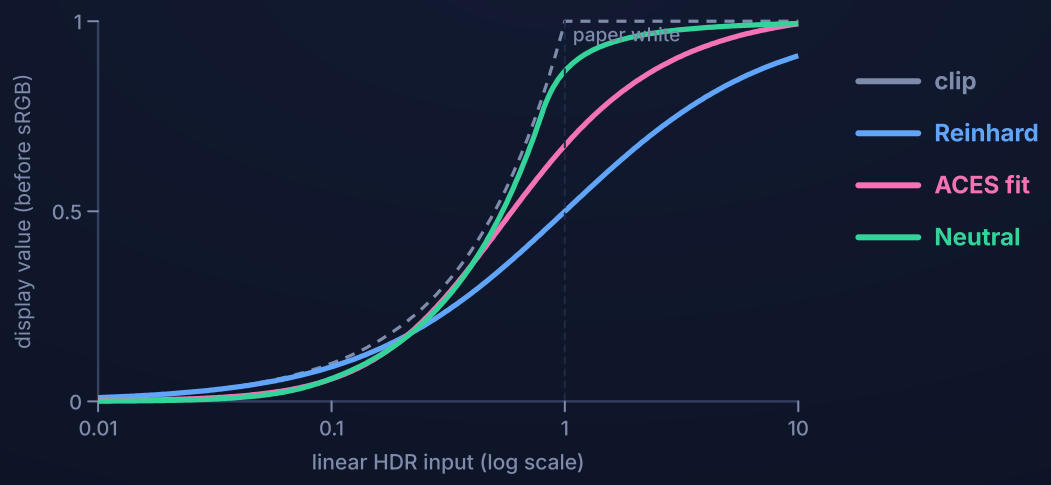


Figure 12.2 Tone-mapping curves for grey values (input on a log axis). Clipping loses everything above 1; Reinhard compresses gently but lowers mid-tones; ACES adds contrast; Neutral leaves values below about 0.76 almost untouched.

WORKED EXAMPLE · Mapping a few HDR values

Linear input	Reinhard	ACES fit (×0.6 pre-scale)
0.18 (mid-grey)	0.153	0.140
1.0	0.500	0.673
2.0	0.667	0.840
4.0	0.800	0.934
10.0	0.909	0.993

Reinhard maps 1.0 (paper white) to only 0.5, which is why Reinhard scenes look dim unless exposure is raised. ACES pushes 4.0 to 0.93: highlights are bright yet still not clipped.

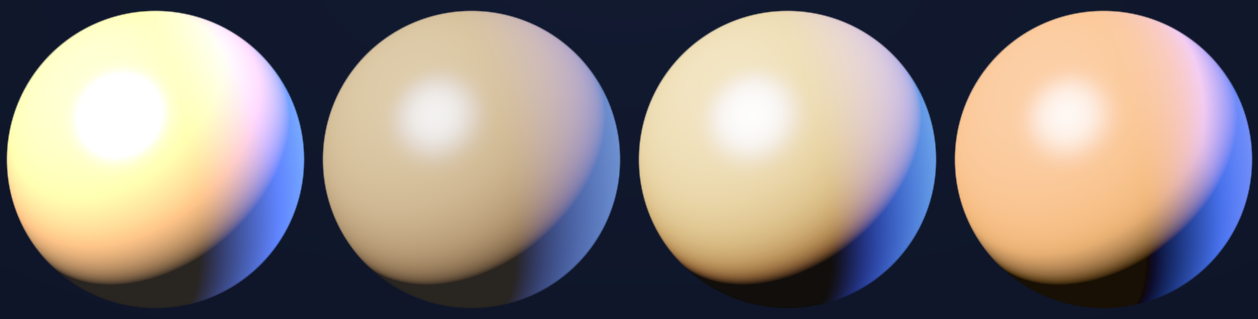


Figure 12.3 The same HDR render (a strong warm key light and a saturated blue rim light) through four tone mappers, computed per pixel. Without tone mapping the bright side clips to a flat yellow-white; Reinhard keeps detail but looks greyish and dim; ACES adds contrast and deep shadows; Neutral keeps the warm base colour most faithfully.

> TIP · **Choosing a tone mapper**
>
> For a product configurator, use **Neutral**: a sofa's fabric colour on screen should match the swatch. For games and cinematic scenes, **ACES** or **AgX** (AgX if you have bright coloured lights or neon). Whatever you choose, pick it early: artists tune materials and lights *through* the tone mapper, and switching later changes everything.

12.4.1 Where tone mapping happens

- **Without post-processing**, three.js applies tone mapping and sRGB conversion at the end of each material's fragment shader, but only when rendering to the canvas (not into render targets).
- **With post-processing**, the scene renders linear HDR into a half-float target, effects run in linear space, and tone mapping + sRGB happen once at the end: `OutputPass` for `EffectComposer`, built into `RenderPipeline` (its `outputColorTransform`) for WebGPU.
- Set `material.toneMapped = false` for things that must show their exact colour: UI panels, video screens, colour pickers.

12.5 Banding and dithering

An 8-bit output has 256 levels per channel. A slow gradient (a dark sky, fog, a vignette) that spans only a few levels shows visible **bands**. **Dithering** adds a tiny amount of noise before quantisation so the bands break up into an even texture the eye averages away. Enable it with `material.dithering = true`, or add noise in a final post-processing pass. Rendering into half-float targets keeps intermediate steps free of banding.

> DEEP DIVE · **Wide gamut: Display P3**

Many phones, Macs and iPads can show colours outside sRGB's gamut (Display P3). three.js supports P3 through `ColorManagement` definitions in the addons (`DisplayP3ColorSpace` , `LinearDisplayP3ColorSpace` in `three/addons/math/ColorSpaces.js`), and WebGPU canvases can be configured for P3 output. For most web 3D work sRGB remains the safe target; P3 matters for brand colours and premium product imagery.

R3F — colour defaults and how to change them JSX

```jsx
// R3F defaults: outputColorSpace = SRGBColorSpace, toneMapping = ACESFilmicToneMapping
const ProductViewer = () => (                          // accurate colours for products
  <Canvas gl={{ toneMapping: THREE.NeutralToneMapping, toneMappingExposure: 1.1 }}><Scene />
</Canvas>
);
const FlatScene = () => <Canvas flat><Scene /></Canvas>;      // NoToneMapping: UI-like scenes,
or post-processing tone-maps
const LinearOutput = () => <Canvas linear><Scene /></Canvas>; // no sRGB output conversion: only
if you handle it yourself
```

✓ KEY POINTS

- ✓ Lighting, blending and filtering must use linear values; files and displays use sRGB encoding.
- ✓ sRGB: linear 0.5 ↔ encoded 0.735; encoded 0.5 ↔ linear 0.214.
- ✓ three.js: colour textures `SRGBColorSpace` , data textures `NoColorSpace` , working space Linear-sRGB, output sRGB.
- ✓ HDR keeps values above 1; exposure multiplies by 2^{EV} before tone mapping.
- ✓ Reinhard $x/(1+x)$; ACES filmic for contrast; AgX for bright saturated light; Neutral for accurate product colours.
- ✓ Convert to sRGB exactly once; with post-processing, at the very end (`OutputPass` / `RenderPipeline`).
- ✓ Dither or use float targets to fight banding.

12.6 Interview questions

Q What is a linear workflow and why does it matter? MID}

A All colour inputs are converted to linear light (sRGB textures decoded, hex colours converted), all lighting, blending and filtering is done in linear space where values are proportional to energy, and the result is tone-mapped and encoded to sRGB once for display. It matters because the maths of light is additive only in linear space; working in sRGB makes lighting too contrasty, falloffs wrong, blends and antialiased edges dark, and PBR materials incorrect.

Q **My scene looks washed out after adding post-processing. What happened?** MID}

A Usually a double conversion: the scene was converted to sRGB (or tone-mapped) when rendering into the composer's target, and again by the final pass. With `EffectComposer` the scene should render linear HDR into a half-float target, and only the final `OutputPass` should apply tone mapping and sRGB conversion. The opposite symptom, a too dark and contrasty image, means the conversion is missing entirely.

Q **Which tone mapping would you choose for an e-commerce configurator and why?** MID}

A `NeutralToneMapping` (Khronos PBR Neutral). It was designed so that base colours under neutral lighting come out almost exactly as specified, with highlights compressed only near the top of the range. ACES Filmic adds contrast and shifts hues of bright, saturated colours, which would misrepresent a product's colour compared with its swatch or photograph. I would also use a neutral environment map and keep exposure fixed.

Q **What does `new THREE.Color(0x808080).r` return and why?** JUNIOR}

A About 0.216, not 0.5. Hex and CSS colours are sRGB-encoded, and with `ColorManagement.enabled` (the default) three.js converts them into its linear working colour space when you set them. `getHex()` converts back and returns `0x808080`. To store raw numbers without conversion, set components directly or pass `LinearSRGBColorSpace` to the setter.

Q **What is the difference between ACES and AgX?** SENIOR}

A Both are filmic curves with a toe and a shoulder. ACES Filmic (as approximated in real-time engines) is contrasty and saturated, but bright saturated colours skew in hue as they approach white (blues towards purple, reds towards orange or yellow) and can clip harshly. AgX compresses in a way that desaturates highlights smoothly towards white without hue skews, at the cost of a flatter, less punchy default look. AgX suits scenes with intense coloured lights; ACES suits a classic cinematic look. three.js supports both.

Physically Based Rendering

13

The rendering equation, BRDFs and energy conservation, the Cook–Torrance microfacet model term by term (GGX distribution, Smith geometry, Schlick Fresnel) evaluated by hand, the metal–roughness workflow, image-based lighting with PMREM and the split-sum approximation, and every MeshPhysicalMaterial feature.

IN THIS CHAPTER

13.1 Why physically based?
13.2 The rendering equation
13.3 Microfacet theory
13.4 The metal–roughness workflow
13.5 Image-based lighting
13.6 MeshPhysicalMaterial
13.7 Interview questions

13.1 Why physically based?

Classic models (Chapter 9) are tuned by eye: a Phong material that looks right under one light looks wrong under another. **Physically based rendering (PBR)** describes surfaces with measurable properties and follows the physics of light (energy conservation, Fresnel reflection, microscopic roughness), so one material looks correct under any lighting. That is what makes glTF assets portable between Blender, three.js, Unity and Unreal, and why `MeshStandardMaterial` is the default for realistic work.

DEFINITION · **Physically based rendering (PBR)**

A family of shading models built on (1) the rendering equation, (2) energy-conserving BRDFs, (3) microfacet theory for specular reflection, (4) Fresnel effects, and (5) real-world-like light units and environment lighting. Materials are described by a few intuitive, measurable parameters: base colour, metalness and roughness.

13.2 The rendering equation

FORMULA · **The rendering equation (Kajiya, 1986)**

$$L_o(\mathbf{p}, \boldsymbol{\omega}_o) = L_e(\mathbf{p}, \boldsymbol{\omega}_o) + \int_{\Omega} f_r(\mathbf{p}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o)\, L_i(\mathbf{p}, \boldsymbol{\omega}_i)\, (\hat{\mathbf{n}} \cdot \boldsymbol{\omega}_i)\, d\boldsymbol{\omega}_i$$

The light L_o leaving point $\mathbf{p}$ towards the viewer is the light it emits (L_e) plus, summed over every incoming direction $\boldsymbol{\omega}_i$ on the hemisphere Ω above the surface, the incoming light L_i weighted by the cosine term and by the **BRDF** f_r.

Real-time renderers approximate the integral in two parts: a **sum over punctual lights** (each light is one direction, so the integral collapses to one term per light) plus **image-based lighting** for light arriving from the whole environment.

Σ FORMULA · **The punctual-light form used in shaders**

$$L_o = \sum_k f_r(\boldsymbol{\omega}_k, \boldsymbol{\omega}_o)\; E_k\; (\hat{\mathbf{n}} \cdot \boldsymbol{\omega}_k) \;+\; L_{\text{IBL}}$$

E_k is light k's colour × intensity × attenuation (Chapter 10).

DEFINITION · **BRDF**

The **bidirectional reflectance distribution function** $f_r(\boldsymbol{\omega}_i, \boldsymbol{\omega}_o)$: the ratio of light reflected towards $\boldsymbol{\omega}_o$ to the light arriving from $\boldsymbol{\omega}_i$. A physically plausible BRDF is non-negative, symmetric in its two directions (Helmholtz reciprocity) and **energy conserving**: it never reflects more light than it receives, $\int_\Omega f_r\,(\hat{\mathbf{n}} \cdot \boldsymbol{\omega}_o)\, d\boldsymbol{\omega}_o \le 1$.

13.2.1 The Lambert BRDF and the mysterious π

Σ FORMULA · **Lambertian diffuse BRDF**

$$f_{\text{diffuse}} = \frac{\mathbf{c}_{\text{diff}}}{\pi}$$

WORKED EXAMPLE · **Where the π comes from**

A perfectly matte white surface reflects all light it receives, equally in every direction: $f = \rho$ constant. Energy conservation requires $\int_\Omega \rho \cos\theta \, d\omega = 1$. In spherical coordinates

$$\int_0^{2\pi} \int_0^{\pi/2} \cos\theta \sin\theta \, d\theta \, d\phi = 2\pi \cdot \tfrac{1}{2} = \pi,$$

so $\rho\,\pi = 1$ and $\rho = 1/\pi$. A white Lambertian BRDF is $1/\pi \approx 0.318$, not 1.

13.3 Microfacet theory

DEFINITION · **Microfacets**

At a microscopic scale every surface is a landscape of tiny perfect mirrors (microfacets) facing in slightly different directions. **Roughness** describes how spread out their orientations are. Only facets whose normal equals the half vector $\hat{\mathbf{h}}$ reflect light from $\hat{\mathbf{l}}$ into $\hat{\mathbf{v}}$; some of those are hidden by neighbouring bumps (shadowing and masking); and each reflects a fraction given by Fresnel.

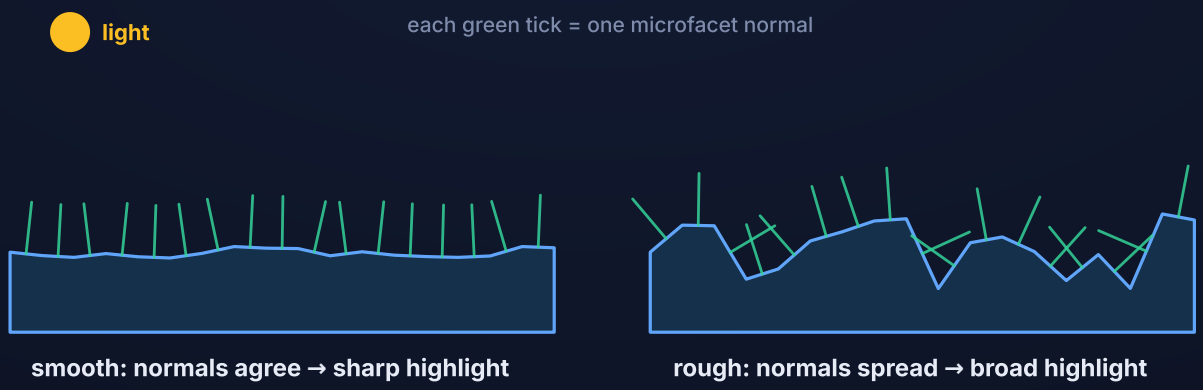


Figure 13.1 A smooth surface has facets aligned with the normal (a sharp highlight); a rough surface has facets pointing in many directions, so reflections spread into a broad, dim highlight. Bumps also block incoming light (shadowing) and outgoing light (masking).

Σ FORMULA · **Cook–Torrance specular BRDF**

$$f_{\text{spec}}(\hat{\mathbf{l}}, \hat{\mathbf{v}}) = \frac{D(\hat{\mathbf{h}})\, F(\hat{\mathbf{v}}, \hat{\mathbf{h}})\, G(\hat{\mathbf{l}}, \hat{\mathbf{v}})}{4\,(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}})\,(\hat{\mathbf{n}} \cdot \hat{\mathbf{v}})}$$

- D: the **normal distribution function**, the density of facets aligned with $\hat{\mathbf{h}}$ (size and shape of the highlight).
- F: **Fresnel**, the fraction of light each facet reflects (rises at grazing angles).
- G: **geometry (shadowing–masking)**, the fraction of those facets visible to both light and viewer.
- The denominator converts from facet space to the macro surface. It is often merged with G into a **visibility** term $V = G/(4\,(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}})(\hat{\mathbf{n}} \cdot \hat{\mathbf{v}}))$, so $f_{\text{spec}} = D\,V\,F$.

13.3.1 D: the GGX (Trowbridge–Reitz) distribution

Σ FORMULA · **GGX normal distribution**

$$D_{\text{GGX}}(\hat{\mathbf{h}}) = \frac{\alpha^2}{\pi\left((\hat{\mathbf{n}} \cdot \hat{\mathbf{h}})^2(\alpha^2 - 1) + 1\right)^2}, \qquad \alpha = \text{roughness}^2$$

Squaring the artist-facing roughness ($\alpha = r^2$, as three.js, glTF and Unreal do) makes the slider feel perceptually linear. GGX has a long “tail” that gives highlights a soft glow, matching measured materials better than Blinn–Phong.

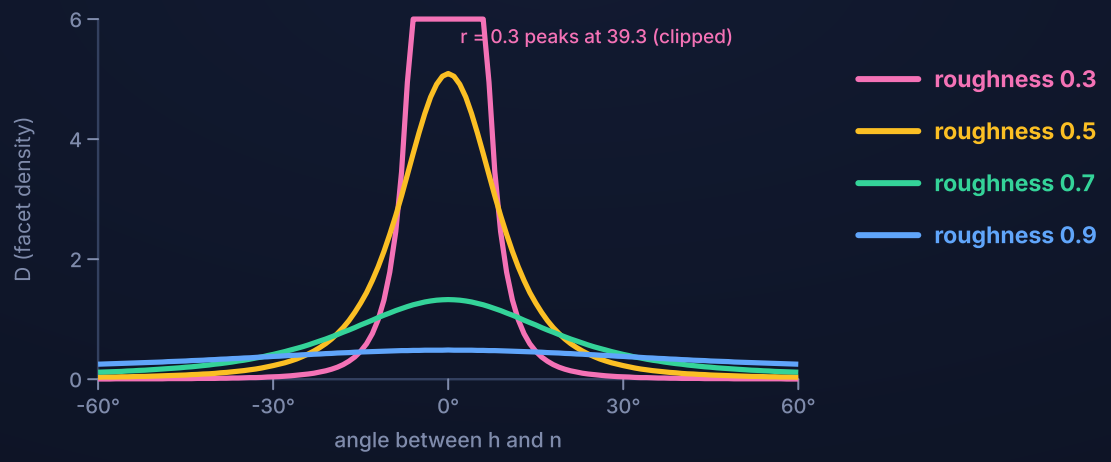


Figure 13.2 GGX distributions for several roughness values. Smooth surfaces concentrate facets around the normal (a tall, narrow peak: a small bright highlight); rough surfaces spread them out (a low, wide hump: a large dim highlight).

WORKED EXAMPLE · **How peaked is the highlight?**

Roughness 0.5: $\alpha = 0.25$, $\alpha^2 = 0.0625$.

- At the centre ($\hat{\mathbf{n}} \cdot \hat{\mathbf{h}} = 1$): $D = 0.0625/(\pi \cdot 0.0625^2) = 1/(0.0625\pi) = 5.09$.
- Slightly off ($\hat{\mathbf{n}} \cdot \hat{\mathbf{h}} = 0.95$, about 18°): denominator $(0.9025 \times (-0.9375) + 1)^2 = 0.1539^2 = 0.0237$, so $D = 0.0625/(\pi \times 0.0237) = 0.84$.

Moving the half vector 18° off the normal drops the facet density by a factor of six: that is the highlight's falloff.

13.3.2 G: Smith shadowing–masking

Σ FORMULA · **Smith G with the Schlick–GGX approximation**

$$G(\hat{\mathbf{l}}, \hat{\mathbf{v}}) = G_1(\hat{\mathbf{l}})\, G_1(\hat{\mathbf{v}}), \qquad G_1(\hat{\mathbf{x}}) = \frac{\hat{\mathbf{n}} \cdot \hat{\mathbf{x}}}{(\hat{\mathbf{n}} \cdot \hat{\mathbf{x}})(1-k) + k}, \qquad k = \frac{(r+1)^2}{8}$$

This version (from Unreal's 2013 course notes, also in LearnOpenGL) is the easiest to learn. three.js uses the more accurate **height-correlated** Smith visibility directly:

$$V = \frac{0.5}{(\hat{\mathbf{n}} \cdot \hat{\mathbf{v}})\sqrt{(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}})^2(1-\alpha^2) + \alpha^2} + (\hat{\mathbf{n}} \cdot \hat{\mathbf{l}})\sqrt{(\hat{\mathbf{n}} \cdot \hat{\mathbf{v}})^2(1-\alpha^2) + \alpha^2}}$$

13.3.3 F: Fresnel and the Schlick approximation

Look at a lake: straight down you see the bottom, at a shallow angle you see a mirror of the sky. All materials reflect more at grazing angles. This is the **Fresnel effect**.

Σ FORMULA · **Schlick's Fresnel approximation**

$$F(\hat{\mathbf{v}}, \hat{\mathbf{h}}) = F_0 + (1 - F_0)\,(1 - \hat{\mathbf{v}} \cdot \hat{\mathbf{h}})^5$$

F_0 is the reflectance looking straight at the surface. For a dielectric with index of refraction n (in air), $F_0 = \left(\frac{n-1}{n+1}\right)^2$. At grazing angles every material approaches 100% reflection.

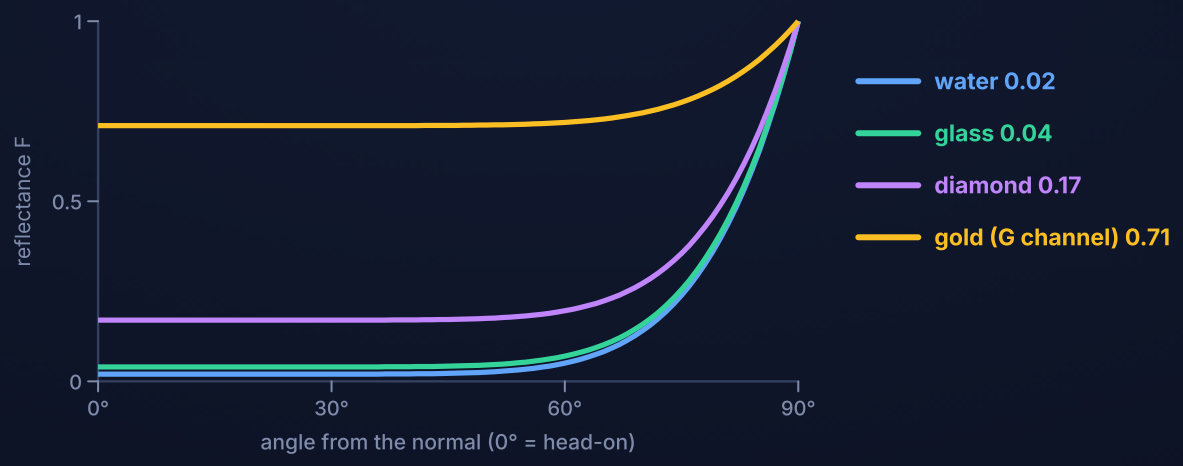


Figure 13.3 Schlick Fresnel against viewing angle. Water and glass reflect only 2–4% head-on but most of the light at grazing angles; gold reflects strongly at every angle and tints its reflections.

WORKED EXAMPLE · **Fresnel for glass and water**

- Glass, $n = 1.5$: $F_0 = (0.5/2.5)^2 = 0.04$. Water, $n = 1.33$: $F_0 = (0.33/2.33)^2 = 0.020$.
- Glass at a grazing view with $\hat{\mathbf{v}} \cdot \hat{\mathbf{h}} = 0.1$: $F = 0.04 + 0.96 \times 0.9^5 = 0.04 + 0.96 \times 0.590 = 0.607$, about fifteen times the head-on reflection.

13.3.4 Putting it together: one pixel by hand

WORKED EXAMPLE · **Cook–Torrance for a plastic surface**

Surface normal $\hat{\mathbf{n}} = (0, 1, 0)$, light from $\hat{\mathbf{l}} = (0.707, 0.707, 0)$ with $E = 3$, viewer at $\hat{\mathbf{v}} = (-0.6, 0.8, 0)$. Material: base colour 0.8 (grey plastic), roughness 0.5, metalness 0 (so $F_0 = 0.04$).

1. Half vector $\hat{\mathbf{h}} = \text{normalise}(\hat{\mathbf{l}} + \hat{\mathbf{v}}) = (0.071, 0.997, 0)$. Dots: $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}} = 0.707$, $\hat{\mathbf{n}} \cdot \hat{\mathbf{v}} = 0.800$, $\hat{\mathbf{n}} \cdot \hat{\mathbf{h}} = 0.997$, $\hat{\mathbf{v}} \cdot \hat{\mathbf{h}} = 0.755$.
2. D with $\alpha^2 = 0.0625$: $D = 4.40$ (near the peak of 5.09, since $\hat{\mathbf{h}}$ is close to $\hat{\mathbf{n}}$).
3. V (height-correlated): $\frac{0.5}{0.583 + 0.576} = 0.432$.
4. $F = 0.04 + 0.96\,(1 - 0.755)^5 = 0.0408$.
5. Specular BRDF $= D\,V\,F = 4.40 \times 0.432 \times 0.0408 = 0.0776$.
6. Diffuse BRDF $= (1 - F) \cdot 0.8/\pi = 0.959 \times 0.2546 = 0.244$.
7. $L_o = (0.244 + 0.078) \times E \times (\hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) = 0.322 \times 3 \times 0.707 = 0.683$.

The same geometry with **gold** (metalness 1, $F_0 = (1.00, 0.71, 0.29)$, no diffuse): $F \approx (1.00, 0.71, 0.29)$, specular $\approx (1.90, 1.35, 0.55)$, so $L_o \approx (4.03, 2.86, 1.17)$: a bright, gold-tinted highlight that tone mapping then compresses.

13.4 The metal–roughness workflow

Parameter	Range	Meaning
`color` / `map` (base colour)	sRGB colour	Albedo for dielectrics; **reflectance colour (F_0)** for metals
`metalness`	0 or 1 (in between only for transitions like rust or dust)	Dielectric (plastic, wood, skin, stone) vs conductor (metal)
`roughness`	0 (mirror) to 1 (chalk)	Microfacet spread; the most important artistic control
`ior` (Physical)	1.0–2.333, default 1.5	Sets F_0 for dielectrics
`emissive`	colour × intensity	Self-illumination, added after lighting

Σ FORMULA · **How three.js derives the BRDF inputs**

$$F_0 = \mathrm{mix}\big(0.04,\ \mathrm{baseColor},\ \mathrm{metalness}\big), \qquad \mathbf{c}_{\mathrm{diff}} = \mathrm{baseColor} \cdot (1 - \mathrm{metalness})$$

Dielectrics: white-ish 4% specular and coloured diffuse. Metals: **no diffuse at all**, coloured specular. (With `MeshPhysicalMaterial`, F_0 also follows `ior`, `specularIntensity` and `specularColor`.)

Material	F_0 (linear)	Notes
Water	0.02	
Plastic, glass, most dielectrics	0.04–0.05	Grey; colour comes from diffuse
Diamond	0.17	High IOR (2.42)
Iron	(0.56, 0.57, 0.58)	
Aluminium	(0.91, 0.92, 0.92)	
Silver	(0.95, 0.93, 0.88)	
Gold	(1.00, 0.71, 0.29)	
Copper	(0.95, 0.64, 0.54)	

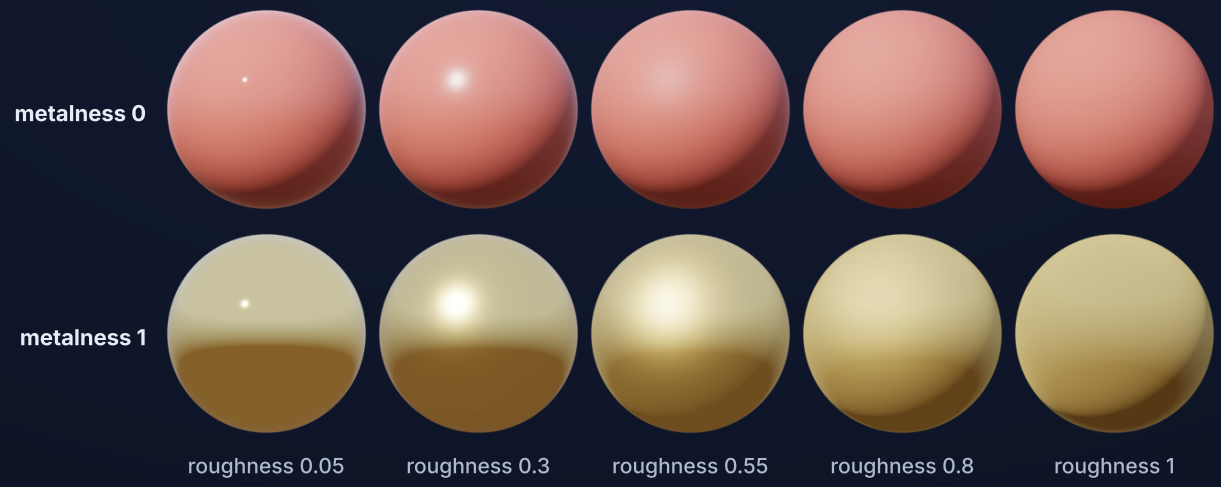


Figure 13.4 A metalness × roughness grid rendered per pixel with the Cook–Torrance formulas of this chapter (GGX, height-correlated Smith, Schlick Fresnel) under one light and a simple sky environment. Top row: dielectric (metalness 0); bottom row: metal (gold base colour).

TIP · **Plausible values**

Real albedos are never pure black or white: charcoal is about 0.04 and fresh snow about 0.9 (linear). Keep base colours roughly between 0.03 and 0.9 linear (about 50–240 in sRGB). Keep metalness at exactly 0 or 1 for clean materials. Most variety in real surfaces lives in the **roughness** map (fingerprints, scratches, wear), so spend texture effort there.

NEW IN RECENT THREE.JS · **Energy conservation in r181–r186**

A single-scattering microfacet BRDF loses energy on rough surfaces, because light that bounces between facets more than once is ignored, so rough metals look too dark. three.js adds **multiple-scattering compensation**; r181 improved it so rough materials (roughness > 0.5) are a bit brighter, and r186 reworked the multi-scatter and energy-conservation terms for diffuse and sheen again. Expect slightly different looks when upgrading, and compare screenshots.

13.5 Image-based lighting

Point lights produce highlights only in a few directions. Real surfaces reflect **everything around them**: the sky, walls, windows. **Image-based lighting (IBL)** uses a panoramic HDR image of the surroundings as a light source from every direction.

DEFINITION · **Environment map**

A 360° image of the surroundings, usually an HDR equirectangular panorama (2:1) or a cube map, used both as the background and as light. In three.js, assigning it to `scene.environment` lights every PBR material; `scene.background` displays it.

The IBL integral splits into a diffuse and a specular part, each precomputed so the shader needs only a texture lookup.

DEFINITION · **Irradiance (diffuse IBL)**

The cosine-weighted average of all incoming light around a normal: a very blurred version of the environment. three.js stores it in the lowest-detail levels of the PMREM (or as spherical harmonics in a `LightProbe`).

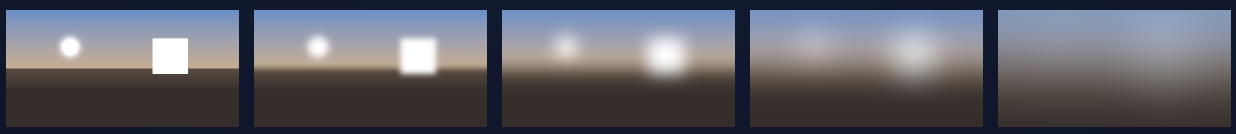
DEFINITION · **PMREM (prefiltered mipmapped radiance environment map)**

A version of the environment pre-blurred for a range of roughness values and packed into one texture. A mirror (roughness 0) samples the sharp level, a rough surface a blurrier one. `PMREMGenerator` builds it on the GPU; three.js does it automatically when you assign an equirectangular texture to `scene.environment`.

Σ FORMULA · **The split-sum approximation**

$$\int_{\Omega} L_i \, f_{\text{spec}} \, (\hat{\mathbf{n}} \cdot \boldsymbol{\omega}_i) \, d\boldsymbol{\omega}_i \;\approx\; \underbrace{\text{prefiltered}(\hat{\mathbf{r}}, \text{roughness})}_{\text{PMREM lookup}} \cdot \underbrace{(F_0 \cdot A + B)}_{\text{BRDF lookup}}$$

Brian Karis (Unreal, 2013) split the specular integral into a product of two separately precomputable integrals: the environment blurred by roughness (looked up along the reflection vector $\hat{\mathbf{r}}$), and the BRDF's response, which depends only on $\hat{\mathbf{n}} \cdot \hat{\mathbf{v}}$ and roughness and reduces to a scale A and bias B on F_0. three.js computes A and B with a fast analytic fit instead of a lookup texture.



roughness 0	roughness 0.25	roughness 0.5	roughness 0.75	roughness 1
mip level 0	mip level 1	mip level 2	mip level 3	mip level 4

one PMREM texture holds every level; the shader picks the level from roughness

Figure 13.5 The environment pre-blurred for increasing roughness. A shader picks the level matching its roughness and samples it in the reflection direction.

three.js — HDR environment lighting JAVASCRIPT

```javascript
import { HDRLoader } from "three/addons/loaders/HDRLoader.js";     // named RGBELoader before r180
import { RoomEnvironment } from "three/addons/environments/RoomEnvironment.js";

const hdr = await new HDRLoader().loadAsync("studio_small_08_2k.hdr");
hdr.mapping = THREE.EquirectangularReflectionMapping;   // three.js builds the PMREM automatically
scene.environment = hdr;                                // lights every PBR material
scene.background = hdr;                                 // optional
scene.backgroundBlurriness = 0.4;                       // blur only the background
scene.environmentIntensity = 1.0;                       // global IBL strength
scene.environmentRotation.y = Math.PI / 4;              // rotate the lighting (matches object rotation since r184)

// no HDR file? a procedural neutral studio:
const pmrem = new THREE.PMREMGenerator(renderer);
scene.environment = pmrem.fromScene(new RoomEnvironment(), 0.04).texture;
```

R3F — drei Environment JSX

```jsx
import { Environment, Lightformer } from "@react-three/drei";

// three alternatives: use one per scene
const City = () => <Environment preset="city" />;   // downloads a small HDR from a CDN
const Studio = () => <Environment files="/hdr/studio.hdr" background backgroundBlurriness={0.4} environmentIntensity={1} />;
const Custom = () => (
  <Environment resolution={256}>                  {/* build your own studio from glowing cards */}
    <Lightformer form="rect" intensity={4} position={[0, 3, -2]} scale={[6, 2, 1]} />
    <Lightformer form="ring" intensity={2} color="#ffd" position={[-3, 1, 2]} scale={2} />
  </Environment>
);
```

⚠ PITFALL · **Metals need an environment**

A metal has no diffuse colour; everything you see is reflection. Under only punctual lights a mirror-like metal is black except for tiny highlights. Always give PBR scenes an environment map, even a dim neutral one; for product shots it is the single biggest quality improvement.

13.6 MeshPhysicalMaterial

`MeshPhysicalMaterial` extends `MeshStandardMaterial` with layers that model specific real materials. Each feature costs shader time, so enable only what you need (three.js only compiles the enabled features into the program).

Property	Models	Typical values / notes	glTF extension
`clearcoat`, `clearcoatRoughness`, `clearcoatNormalMap`	A thin varnish layer	Car paint, lacquered wood,	`KHR_materials_clearcoat`

Property	Models	Typical values / notes	glTF extension
		carbon fibre	
`sheen`, `sheenColor`, `sheenRoughness`	Velvety back-scattering at grazing angles	Fabric, velvet, peach skin	`KHR_materials_sheen`
`transmission`, `thickness`, `attenuationColor`, `attenuationDistance`	Light passing through (refraction, absorption)	Glass, liquids, gems; renders a transmission pass	`KHR_materials_transmission`, `_volume`
`ior`	Index of refraction	1.0–2.333; glass 1.5, water 1.33, diamond 2.42	`KHR_materials_ior`
`dispersion`	Wavelength-dependent refraction (rainbow fringes)	Diamonds, prisms	`KHR_materials_dispersion`
`iridescence`, `iridescenceIOR`, `iridescenceThicknessRange`	Thin-film interference	Soap bubbles, oil slicks, beetle shells	`KHR_materials_iridescence`
`anisotropy`, `anisotropyRotation`	Stretched highlights along a direction	Brushed metal, hair, vinyl records	`KHR_materials_anisotropy`
`specularIntensity`, `specularColor`	Override dielectric F_0 strength and tint	Fine-tuning skin, fabric	`KHR_materials_specular`
`retroreflectivity` (r186)	Light reflected back towards its source (0–1)	Road signs, safety vests, cat's eyes	—

WORKED EXAMPLE · Thickness and absorption of tinted glass

`attenuationColor` is the colour white light becomes after travelling `attenuationDistance` through the material (Beer–Lambert absorption $T = c^{d/d_0}$ per channel). With `attenuationColor = (0.6, 0.9, 0.7)` and `attenuationDistance = 0.1` m, light crossing 0.2 m of glass keeps $(0.6^2, 0.9^2, 0.7^2) = (0.36, 0.81, 0.49)$: thick parts look a deeper green than thin edges, which is what makes glass look solid.

⚠ PITFALL · **Transmission is expensive**

`transmission > 0` makes three.js render the opaque scene into an extra render target every frame (and blur it by roughness) so transmissive objects can sample what is behind them. Several glass objects do not see each other through themselves. For many small glass items consider `opacity`-based fakes, or drei's `MeshTransmissionMaterial`, which adds chromatic aberration and supports seeing other transmissive objects at a further cost.

R3F — physical materials JSX

```jsx
const Showcase = () => (
  <>
    <mesh>  {/* car paint */}
      <sphereGeometry args={[1, 64, 32]} />
      <meshPhysicalMaterial color="#8b0000" metalness={0.6} roughness={0.35} clearcoat={1}
clearcoatRoughness={0.05} />
    </mesh>
    <mesh position-x={2.5}>  {/* tinted glass */}
      <torusKnotGeometry args={[0.6, 0.2, 200, 32]} />
      <meshPhysicalMaterial transmission={1} roughness={0.05} thickness={0.4} ior={1.5}
        attenuationColor="#9fe8c8" attenuationDistance={0.5} />
    </mesh>
  </>
);
```

✓ KEY POINTS

- ✓ Rendering equation: outgoing = emitted + ∫ BRDF × incoming × cosθ; real time = sum of lights + IBL.
- ✓ Lambert BRDF = albedo/π (the π makes it energy conserving).
- ✓ Cook–Torrance specular = $DFG/(4\,(\hat{\mathbf{n}}\cdot\hat{\mathbf{l}})(\hat{\mathbf{n}}\cdot\hat{\mathbf{v}}))$ with GGX D, Smith G, Schlick F; $\alpha = \text{roughness}^2$.
- ✓ $F_0 = 0.04$ for most dielectrics ($((n-1)/(n+1))^2$); metals use their base colour as F_0 and have no diffuse.
- ✓ IBL = irradiance (diffuse) + PMREM (prefiltered specular) × BRDF scale/bias (split sum).
- ✓ Always give PBR scenes an environment; metals depend on it.
- ✓ Physical extras (clearcoat, sheen, transmission, iridescence, anisotropy, dispersion) map to glTF `KHR_materials_*` extensions; transmission costs an extra pass.

13.7 Interview questions

Q **What does “physically based” mean, and what makes MeshStandardMaterial physically based?** MID}

A It means shading follows real light physics: energy conservation (never reflect more than received), a microfacet specular model (GGX distribution, Smith shadowing, Fresnel), Lambertian diffuse scaled so diffuse plus specular stays within the incoming energy, and physically meaningful parameters (base colour, metalness, roughness). `MeshStandardMaterial` implements exactly this Cook–Torrance metal–roughness model, the same as glTF’s core material, so assets look consistent across engines and under any lighting.

Q **Explain the D, F and G terms of the Cook–Torrance BRDF.** SENIOR}

A D (GGX) is the statistical density of microfacets whose normal equals the half vector, the ones that can reflect light into the eye; its width is set by roughness and it shapes the highlight. F (Fresnel, Schlick’s approximation) is how much of the light each facet reflects, rising from F_0 head-on to 1 at grazing angles; F_0 is about 0.04 for dielectrics and the base colour for metals. G (Smith) is the fraction of those facets not shadowed from the light or masked from the viewer by neighbouring bumps; it darkens rough surfaces at grazing angles. The $4(\hat{\mathbf{n}}\cdot\hat{\mathbf{l}})(\hat{\mathbf{n}}\cdot\hat{\mathbf{v}})$ denominator converts facet-space quantities to the macro surface.

Q **Why do metals look black in my scene?** JUNIOR}

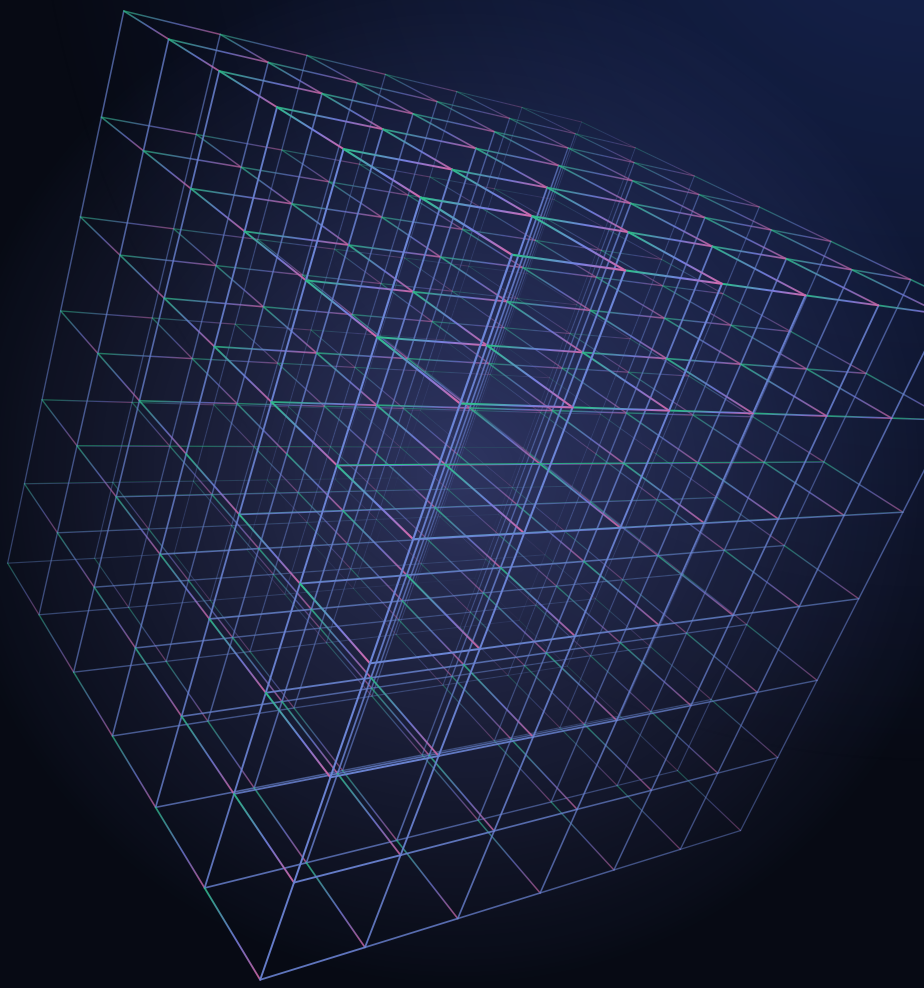
A In the metal–roughness model a metal has no diffuse component: its base colour becomes the specular reflectance. With only punctual lights it shows tiny highlights and otherwise reflects nothing, which reads as black. Give the scene an environment map (`scene.environment` with an HDR or `RoomEnvironment`, or drei’s `<Environment>`), so the metal has a world to reflect.

Q **What is the split-sum approximation?** SENIOR}

A The specular IBL integral of environment radiance times the BRDF over the hemisphere is too expensive per pixel. Karis’s split sum approximates it as the product of two integrals: (1) the environment convolved with the GGX lobe for a given roughness, precomputed per roughness level into a prefiltered mip chain (PMREM) and sampled along the reflection vector; and (2) the integral of the BRDF itself, which depends only on $\hat{\mathbf{n}}\cdot\hat{\mathbf{v}}$ and roughness, giving a scale and bias applied to F_0 (a 2D lookup table, or an analytic fit as in three.js). The approximation assumes $\hat{\mathbf{n}}=\hat{\mathbf{v}}=\hat{\mathbf{r}}$ when prefiltering, which loses the stretched reflections at grazing angles.

Q How would you make convincing glass in three.js, and what does it cost? MID}

A Use `MeshPhysicalMaterial` with `transmission: 1`, low `roughness`, `ior` 1.5, a `thickness` (and `attenuationColor` / `attenuationDistance` for tint), plus an environment map for reflections, since Fresnel makes glass mostly reflective at grazing angles. The cost: three.js renders the opaque scene into an extra transmission render target each frame and samples it blurred by roughness, and transmissive objects do not refract each other. For many glass objects or a stylised look, drei's `MeshTransmissionMaterial` or cheaper fakes (environment-mapped reflection with low opacity) may be better.



PART

Motion and Interaction

Raycasting, Picking and Collision Maths

14

Rays and the four intersection tests every 3D developer should be able to derive (plane, sphere, box and triangle), each worked by hand and checked against three.js. Then the Raycaster API, BVH acceleration, GPU picking, R3F pointer events, dragging, and the basics of collision detection.

IN THIS CHAPTER

14.1 Rays
14.2 Ray–plane
14.3 Ray–sphere
14.4 Ray–box (the slab method)
14.5 Ray–triangle (Möller–Trumbore)
14.6 The Raycaster
14.7 Pointer events in React Three Fiber
14.8 Collision detection basics
14.9 Interview questions

14.1 Rays

DEFINITION · **Ray**

A half-line with an origin $\mathbf{o}$ and a unit direction $\hat{\mathbf{d}}$: the set of points $\mathbf{r}(t) = \mathbf{o} + t\,\hat{\mathbf{d}}$ for $t \geq 0$. Because $\hat{\mathbf{d}}$ is unit length, t is the **distance** along the ray. Every intersection test below solves for t; the nearest positive t is the hit.

Rays answer most interaction questions: what is under the mouse, where does a bullet hit, is the ground below the character, can the guard see the player, where does the VR controller point. three.js's `Ray` class (`ray.intersectPlane`, `intersectSphere`, `intersectBox`, `intersectTriangle`) and the higher-level `Raycaster` implement them.

14.2 Ray–plane

FORMULA · **Ray–plane intersection**

Plane: $\hat{\mathbf{n}} \cdot \mathbf{x} + d = 0$. Substituting $\mathbf{x} = \mathbf{o} + t\hat{\mathbf{d}}$:

$$t = -\frac{\hat{\mathbf{n}} \cdot \mathbf{o} + d}{\hat{\mathbf{n}} \cdot \hat{\mathbf{d}}}$$

If $\hat{\mathbf{n}} \cdot \hat{\mathbf{d}} = 0$ the ray is parallel to the plane (no hit, or lying in it). A negative t means the plane is behind the origin.

WORKED EXAMPLE · Where on the floor did I click?

Camera ray: $\mathbf{o} = (0, 5, 10)$, $\hat{\mathbf{d}} = \text{normalise}(0, -0.5, -1) = (0, -0.447, -0.894)$. Floor $y = 0$: $\hat{\mathbf{n}} = (0, 1, 0)$, $d = 0$.

$t = -\dfrac{5 + 0}{-0.447} = 11.18$; point $= (0, 5, 10) + 11.18\,(0, -0.447, -0.894) = (0, 0, 0)$.

`ray.intersectPlane()` returns the same. This is how "click to move" and "drag on the ground" work, with no meshes needed at all.

14.3 Ray–sphere

FORMULA · Ray–sphere intersection

Sphere: centre $\mathbf{c}$, radius r. Let $\mathbf{m} = \mathbf{o} - \mathbf{c}$. Solving $\|\mathbf{m} + t\hat{\mathbf{d}}\|^2 = r^2$ gives a quadratic in t:

$$t^2 + 2bt + c = 0, \quad b = \mathbf{m} \cdot \hat{\mathbf{d}}, \quad c = \mathbf{m} \cdot \mathbf{m} - r^2, \qquad t = -b \pm \sqrt{b^2 - c}$$

The **discriminant** $b^2 - c$ decides: negative, a miss; zero, a graze; positive, two hits (entry and exit).

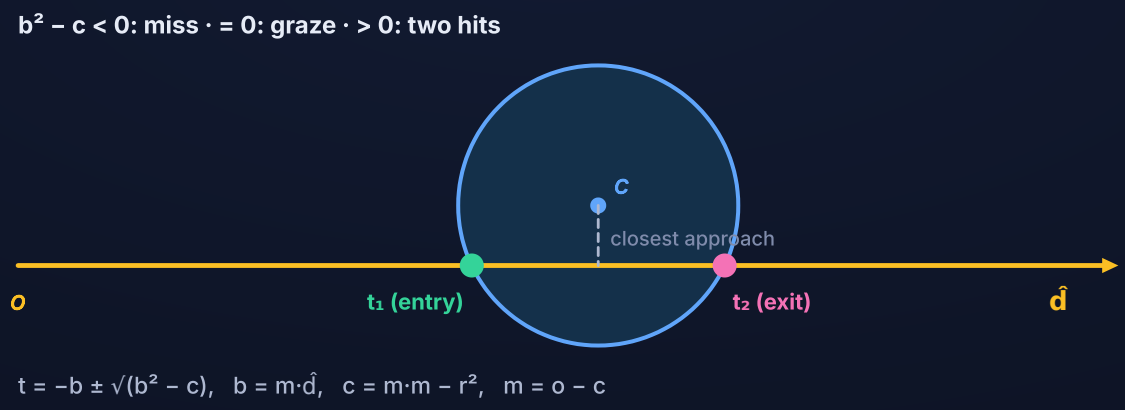


Figure 14.1 Ray–sphere geometry. The discriminant measures how close the ray passes to the centre compared with the radius.

WORKED EXAMPLE · A ray past a sphere

$\mathbf{o} = (0, 0, 0)$, $\hat{\mathbf{d}} = (0, 0, -1)$, sphere at $\mathbf{c} = (0.5, 0, -5)$, $r = 1$.

1. $\mathbf{m} = (-0.5, 0, 5)$, $b = \mathbf{m} \cdot \hat{\mathbf{d}} = -5$, $c = 25.25 - 1 = 24.25$.
2. Discriminant $= 25 - 24.25 = 0.75 > 0$: two hits.
3. $t = 5 \pm 0.866$: entry at $t = 4.134$, exit at 5.866.
4. Entry point $(0, 0, -4.134)$; check: its distance from the centre is $\sqrt{0.5^2 + 0.866^2} = 1$. three.js's `intersectSphere` gives $(0, 0, -4.134)$.

14.4 Ray–box (the slab method)

Σ FORMULA · Slab test for an axis-aligned box

An axis-aligned bounding box (AABB) is the intersection of three "slabs" $x_{\min} \le x \le x_{\max}$, etc. For each axis, compute where the ray enters and leaves its slab:

$$t_1 = \frac{x_{\min} - o_x}{d_x}, \qquad t_2 = \frac{x_{\max} - o_x}{d_x}$$

then $t_{\text{near}} = \max$ over axes of $\min(t_1, t_2)$ and $t_{\text{far}} = \min$ over axes of $\max(t_1, t_2)$. The ray hits the box if $t_{\text{near}} \le t_{\text{far}}$ and $t_{\text{far}} \ge 0$.

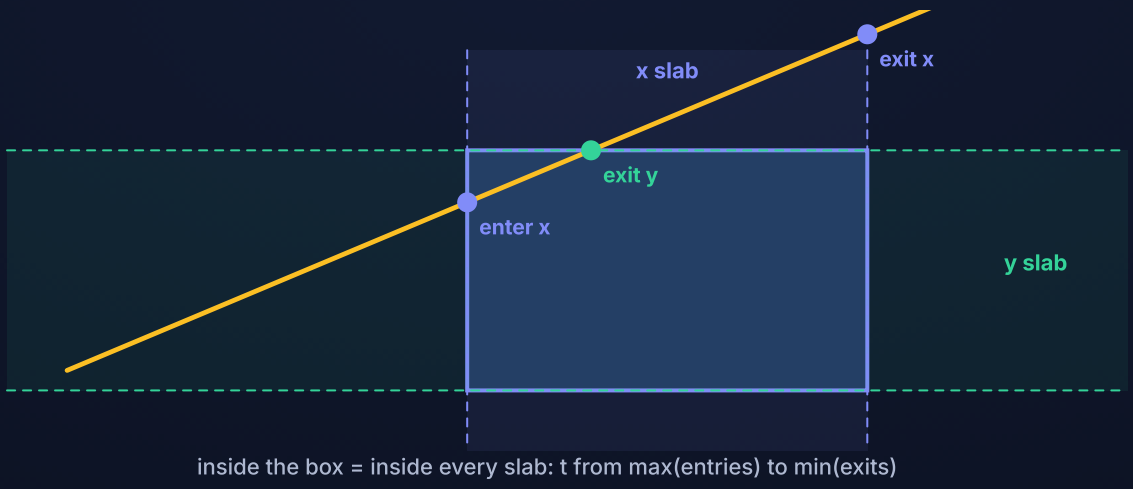


Figure 14.2 The slab method in 2D. The ray is inside the box only while it is inside every slab at once, from the latest entry to the earliest exit.

WORKED EXAMPLE · Hitting a crate

$\mathbf{o} = (0, 0, 0)$, $\hat{\mathbf{d}} = \text{normalise}(1, 0.25, 0) = (0.970, 0.243, 0)$, box from $(2, -1, -1)$ to $(4, 1, 1)$.

- x slab: $t = 2/0.970 = 2.06$ to $4/0.970 = 4.12$.
- y slab: $t = -1/0.243 = -4.12$ to $1/0.243 = 4.12$.
- z slab: $d_z = 0$ and $o_z = 0$ lies inside $[-1, 1]$, so the whole ray is inside it.
- $t_{\text{near}} = \max(2.06, -4.12) = 2.06$, $t_{\text{far}} = \min(4.12, 4.12) = 4.12$. Since $2.06 \le 4.12$: hit at $t = 2.06$, point $(2, 0.5, 0)$. `intersectBox` agrees.

The slab test needs only a few divisions (precompute $1/d$) and comparisons, which is why bounding boxes are the first test in every acceleration structure.

14.5 Ray–triangle (Möller–Trumbore)

Σ FORMULA · Möller–Trumbore

For a triangle $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2$ with edges $\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0$, $\mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$, solve $\mathbf{o} + t\hat{\mathbf{d}} = (1-u-v)\mathbf{v}_0 + u\mathbf{v}_1 + v\mathbf{v}_2$ with Cramer's rule:

$$\mathbf{p} = \hat{\mathbf{d}} \times \mathbf{e}_2, \quad \det = \mathbf{e}_1 \cdot \mathbf{p}, \quad \mathbf{s} = \mathbf{o} - \mathbf{v}_0, \quad u = \frac{\mathbf{s} \cdot \mathbf{p}}{\det}, \quad \mathbf{q} = \mathbf{s} \times \mathbf{e}_1, \quad v = \frac{\hat{\mathbf{d}} \cdot \mathbf{q}}{\det}, \quad t = \frac{\mathbf{e}_2 \cdot \mathbf{q}}{\det}$$

Hit if $\det \neq 0$, $u \geq 0$, $v \geq 0$, $u + v \leq 1$ and $t > 0$. The sign of $\det$ tells front face from back face (used for culling). u and v are barycentric coordinates, which give the hit's UV and interpolated normal.

WORKED EXAMPLE · A ray through a triangle

$\mathbf{v}_0 = (0,0,-2)$, $\mathbf{v}_1 = (2,0,-2)$, $\mathbf{v}_2 = (0,2,-2)$; ray $\mathbf{o} = (0.5, 0.5, 0)$, $\hat{\mathbf{d}} = (0,0,-1)$.

1. $\mathbf{e}_1 = (2,0,0)$, $\mathbf{e}_2 = (0,2,0)$.
2. $\mathbf{p} = \hat{\mathbf{d}} \times \mathbf{e}_2 = (2,0,0)$; $\det = \mathbf{e}_1 \cdot \mathbf{p} = 4$ (positive: front face).
3. $\mathbf{s} = (0.5, 0.5, 2)$; $u = (\mathbf{s} \cdot \mathbf{p})/4 = 1/4 = 0.25$.
4. $\mathbf{q} = \mathbf{s} \times \mathbf{e}_1 = (0,4,-1)$; $v = (\hat{\mathbf{d}} \cdot \mathbf{q})/4 = 1/4 = 0.25$.
5. $t = (\mathbf{e}_2 \cdot \mathbf{q})/4 = 8/4 = 2$.
6. All tests pass: hit at $(0.5, 0.5, -2)$ with barycentrics $(1-u-v, u, v) = (0.5, 0.25, 0.25)$, exactly what `Triangle.getBarycoord` returns.

14.6 The Raycaster

three.js — picking with Raycaster — JAVASCRIPT

```
const raycaster = new THREE.Raycaster();
raycaster.params.Points.threshold = 0.05;      // world-unit tolerance for Points
raycaster.params.Line.threshold = 0.05;        // and for Lines
raycaster.far = 100;                           // ignore hits beyond 100 m

function pick(ndc) {
  raycaster.setFromCamera(ndc, camera);
  const hits = raycaster.intersectObjects(clickables, true);   // recursive; sorted by distance
  if (!hits.length) return null;
  const h = hits[0];
  // h.distance, h.point (world), h.object, h.face, h.faceIndex, h.uv, h.barycoord,
  // h.normal (interpolated, LOCAL space: use transformDirection(h.object.matrixWorld) for
world),
  // h.instanceId (InstancedMesh), h.batchId (BatchedMesh)
  return h;
}
```

How `Mesh.raycast` works, which explains its performance:

1. Transform the ray into the mesh's **local space** with the inverse world matrix (cheaper than transforming every vertex).
2. Test the geometry's **bounding sphere**, then bounding box: most meshes are rejected here.
3. Otherwise test **every triangle** (Möller–Trumbore), respecting `material.side`.
4. Convert hits back to world space and sort by distance.

> ⚠ PITFALL · **Raycasting sees the CPU geometry, not the pixels**
>
> Vertex-shader displacement, skinning not applied on the CPU, morph targets, GPU particles and instanced transforms changed only in shaders are invisible to the `Raycaster`. The click lands where the original geometry is. For skinned meshes call `mesh.computeBoundingSphere()` / `computeBoundingBox()` after posing, or raycast against simple proxy shapes; for shader effects use GPU picking.

14.6.1 Accelerating raycasts with a BVH

> DEFINITION · **Bounding volume hierarchy (BVH)**
>
> A tree of nested bounding boxes over a mesh's triangles. A ray only descends into child boxes it actually hits, so instead of testing all N triangles it tests about $\log_2 N$ levels of boxes and a handful of triangles. Building it costs time once; queries become hundreds of times faster on big meshes.

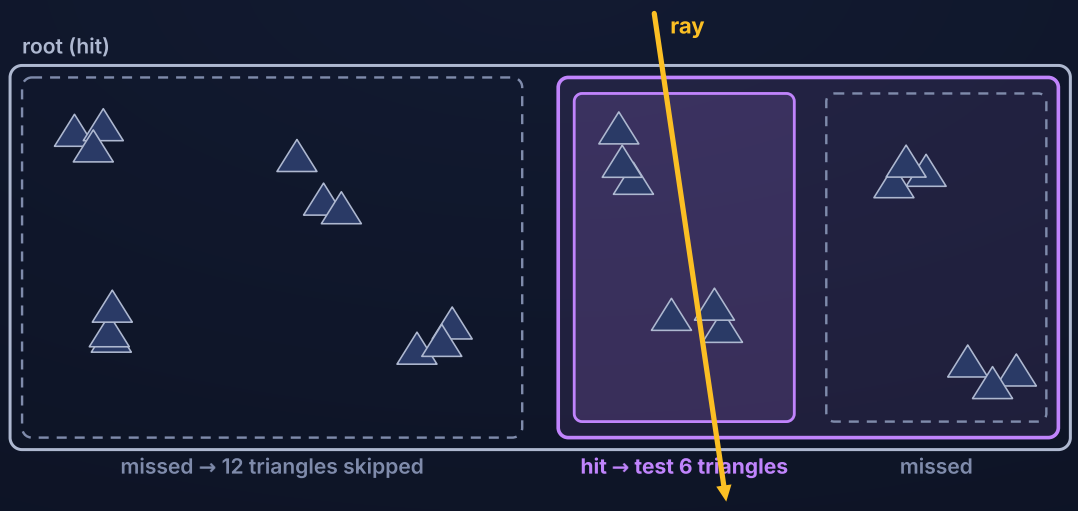


Figure 14.3 A BVH splits triangles into nested boxes. The ray (amber) only visits the boxes it crosses (highlighted), skipping every triangle in the other branches.

> WORKED EXAMPLE · **Brute force against a BVH**
>
> A 1,000,000-triangle scanned statue. Brute force: one million ray–triangle tests per pick (tens of milliseconds in JavaScript). A BVH with 10 triangles per leaf has about $\log_2(10^5) \approx 17$ levels; a ray typically visits a few dozen boxes and tests a few dozen triangles: well under a millisecond, which makes hover effects at 60 Hz and continuous VR pointer rays affordable.

three.js — three-mesh-bvh JAVASCRIPT

```javascript
import { computeBoundsTree, disposeBoundsTree, acceleratedRaycast } from "three-mesh-bvh";
THREE.BufferGeometry.prototype.computeBoundsTree = computeBoundsTree;
THREE.BufferGeometry.prototype.disposeBoundsTree = disposeBoundsTree;
THREE.Mesh.prototype.raycast = acceleratedRaycast;

statue.geometry.computeBoundsTree();          // once, after loading
raycaster.firstHitOnly = true;                // stop at the nearest hit (another big speed-up)
```

R3F — drei <Bvh> JSX

```jsx
import { Bvh } from "@react-three/drei";
<Bvh firstHitOnly>
  <Statue />          {/* every mesh inside gets a BVH and accelerated raycasting */}
</Bvh>
```

14.6.2 GPU picking

An alternative to rays: render the scene once more into a tiny off-screen target where each object is drawn in a **unique flat colour** that encodes its ID, then read back the pixel under the cursor.

	CPU raycasting	GPU picking
Cost	Grows with triangles (BVH helps)	One extra render (can be 1×1 pixel with a narrowed camera)
Shader-displaced or skinned geometry	Wrong	Correct (same vertex shader)
Returns	Exact point, normal, UV, all hits	Object/instance ID (depth needs a second channel)
Latency	Immediate	Reading pixels back can stall the GPU; use async reads
Good for	Most apps, physics-like queries	Millions of points, instanced crowds, CAD models

14.7 Pointer events in React Three Fiber

R3F runs the raycaster for you and gives meshes DOM-like event handlers.

R3F — events on meshes JSX

```jsx
function Box(props) {
  const [hovered, setHovered] = useState(false);
  return (
    <mesh {...props}
      onClick={(e) => { e.stopPropagation(); console.log(e.point, e.distance, e.object.name); }}
      onPointerOver={(e) => { e.stopPropagation(); setHovered(true); }}
      onPointerOut={() => setHovered(false)}
      onPointerMissed={() => console.log("clicked empty space")}>
      <boxGeometry />
      <meshStandardMaterial color={hovered ? "hotpink" : "orange"} />
    </mesh>
  );
}
```

Event field	Meaning
`e.object`	The object whose handler is running
`e.eventObject`	The object the handler is attached to (differs while bubbling)
`e.point`, `e.distance`	World-space hit point and distance from the ray origin
`e.face`, `e.uv`, `e.instanceId`	Triangle, texture coordinate, instance index
`e.intersections`	Every hit along the ray, nearest first
`e.ray`, `e.camera`, `e.pointer`	The ray, camera and NDC pointer used
`e.stopPropagation()`	Stop the event reaching objects **behind** (not only parents)

⚠ PITFALL · **Events go through objects**

Unlike the DOM, a ray passes **through** every object it hits. Without `e.stopPropagation()`, hovering a cube also triggers `onPointerOver` on the floor behind it. Call it in handlers on foreground objects. For cheap hit-testing on complex meshes, `raycast={meshBounds}` (drei) tests only the bounding sphere.

14.7.1 Dragging on a plane

Dragging an object across the floor is a ray–plane intersection each pointer move, with an offset so the object does not jump to the cursor.

three.js — drag on the ground plane JAVASCRIPT

```javascript
const ground = new THREE.Plane(new THREE.Vector3(0, 1, 0), 0);
const hit = new THREE.Vector3(), offset = new THREE.Vector3();
let dragging = null;

canvas.addEventListener("pointerdown", (e) => {
  const h = pick(toNdc(e));
  if (!h) return;
  dragging = h.object;
  raycaster.ray.intersectPlane(ground, hit);
```

```
  offset.copy(dragging.position).sub(hit);          // keep the grab point under the cursor
  canvas.setPointerCapture(e.pointerId);            // keep receiving moves outside the canvas
  controls.enabled = false;                         // stop OrbitControls fighting the drag
});
canvas.addEventListener("pointermove", (e) => {
  if (!dragging) return;
  raycaster.setFromCamera(toNdc(e), camera);
  if (raycaster.ray.intersectPlane(ground, hit)) dragging.position.copy(hit).add(offset);
});
canvas.addEventListener("pointerup", () => { dragging = null; controls.enabled = true; });
```

In R3F the same is a few lines with drei's `<DragControls>` or `<PivotControls>`, or by setting pointer capture in `onPointerDown` (`e.target.setPointerCapture(e.pointerId)`) and moving the object in `onPointerMove` with `e.ray.intersectPlane(...)`.

14.8 Collision detection basics

Physics engines (Chapter 17) handle collisions for you, but the core tests are short and worth knowing.

Σ FORMULA · Overlap tests

- **Sphere–sphere**: overlap if $\|\mathbf{c}_1 - \mathbf{c}_2\| < r_1 + r_2$ (compare squared values). Penetration depth $= r_1 + r_2 - \|\mathbf{c}_1 - \mathbf{c}_2\|$.
- **AABB–AABB**: overlap if, on **every** axis, $\min_1 \le \max_2$ and $\min_2 \le \max_1$.
- **Sphere–AABB**: clamp the centre to the box to find the closest point $\mathbf{q}$; overlap if $\|\mathbf{c} - \mathbf{q}\| < r$.
- **Oriented boxes, convex shapes**: the **separating axis theorem** (two convex shapes do not overlap if and only if some axis exists on which their projections do not overlap; for boxes, test 15 axes).

WORKED EXAMPLE · Two balls

Ball A at $(0, 1, 0)$ with $r = 0.5$; ball B at $(0.6, 1.2, 0.3)$ with $r = 0.4$. $\|\mathbf{c}_1 - \mathbf{c}_2\|^2 = 0.36 + 0.04 + 0.09 = 0.49$, so the distance is 0.7, and $r_1 + r_2 = 0.9$. They overlap by 0.2 m; push each apart by 0.1 m along the line between centres, $\text{normalise}(0.6, 0.2, 0.3) = (0.857, 0.286, 0.429)$.

DEFINITION · Broad phase and narrow phase

Testing every pair of N objects is $N(N-1)/2$ tests: 4,950 for 100 objects, about 500,000 for 1,000. The **broad phase** quickly finds candidate pairs with a cheap structure (a uniform grid or spatial hash, sweep-and-prune on sorted AABB edges, or a dynamic BVH); the **narrow phase** runs exact tests only on those candidates.

three.js maths classes for collision JAVASCRIPT

```
const a = new THREE.Box3().setFromObject(crate);
const b = new THREE.Sphere(ball.position, 0.5);
a.intersectsSphere(b);  a.intersectsBox(otherBox);  a.containsPoint(p);
const closest = a.clampPoint(ball.position, new THREE.Vector3());   // closest point on the box
const frustum = new THREE.Frustum().setFromProjectionMatrix(
  new THREE.Matrix4().multiplyMatrices(camera.projectionMatrix, camera.matrixWorldInverse));
frustum.intersectsObject(mesh);                                     // manual visibility test
```

✓ KEY POINTS

- ✓ Ray: $\mathbf{o} + t\hat{\mathbf{d}}$; with unit $\hat{\mathbf{d}}$, t is distance.
- ✓ Plane: $t = -(\hat{\mathbf{n}} \cdot \mathbf{o} + d)/(\hat{\mathbf{n}} \cdot \hat{\mathbf{d}})$. Sphere: quadratic with discriminant $b^2 - c$. Box: slab method. Triangle: Möller–Trumbore, which also yields barycentrics.
- ✓ `Mesh.raycast` : local-space ray → bounding sphere/box → every triangle; use three-mesh-bvh (`<Bvh>`) for big meshes and `firstHitOnly` .
- ✓ Raycasting ignores shader displacement; use proxies or GPU picking.
- ✓ R3F events pass through objects: call `stopPropagation()` ; `onPointerMissed` for empty clicks.
- ✓ Drag = ray–plane intersection + grab offset + pointer capture + disabling controls.
- ✓ Collision = broad phase (grid, sweep-and-prune, BVH) then narrow phase (exact shape tests, SAT).

14.9 Interview questions

Q How does three.js work out which object is under the mouse? JUNIOR}

A Convert the pointer to normalised device coordinates, build a ray from the camera through that point with `raycaster.setFromCamera(ndc, camera)` , and call `intersectObjects(objects, true)` . For each mesh three.js moves the ray into the mesh's local space, rejects it early with the bounding sphere and box, then tests triangles and returns hits sorted by distance, each with the point, face, UV, normal and (for instanced meshes) the instance ID. R3F does this automatically for its `onClick` / `onPointer*` handlers.

Q Derive the ray–sphere intersection. MID}

A Points on the ray are $\mathbf{o} + t\hat{\mathbf{d}}$; points on the sphere satisfy $\|\mathbf{x} - \mathbf{c}\|^2 = r^2$. Substituting with $\mathbf{m} = \mathbf{o} - \mathbf{c}$ gives $t^2 + 2(\mathbf{m} \cdot \hat{\mathbf{d}})t + (\mathbf{m} \cdot \mathbf{m} - r^2) = 0$ (the t^2 coefficient is 1 because $\hat{\mathbf{d}}$ is unit). So $t = -b \pm \sqrt{b^2 - c}$ with $b = \mathbf{m} \cdot \hat{\mathbf{d}}$ and $c = \mathbf{m} \cdot \mathbf{m} - r^2$. A negative discriminant is a miss; otherwise take the smallest positive root. If $c < 0$ the origin is inside the sphere.

Q **Hover detection on a 2-million-triangle CAD model is slow. What would you do?** MID}

A Build a BVH (three-mesh-bvh, `computeBoundsTree`, or drei's `<Bvh>`) so raycasts cost roughly logarithmic time, enable `firstHitOnly`, raycast only on pointer move (and throttle to once per frame), restrict the candidate list with layers or a smaller array of pickable objects, and test simplified proxy geometry if exact triangles are not needed. If the model is instanced or shader-deformed, use GPU picking (an ID render into a small target).

Q **Why does clicking a waving flag (vertex shader animation) hit the wrong spot?** MID}

A The raycaster tests the geometry stored on the CPU, which is the flag's rest pose; the waving happens only in the vertex shader on the GPU. Options: run the same displacement function on the CPU for the hit test, raycast against an approximate proxy, or use GPU picking, which renders with the same vertex shader and therefore sees the displaced surface.

Q **What is the separating axis theorem?** SENIOR}

A Two convex shapes are disjoint if and only if there is an axis onto which their projections (intervals) do not overlap. For convex polyhedra you only need to test a finite set of candidate axes: the face normals of each shape and the cross products of pairs of edges (for two oriented boxes: 3 + 3 + 9 = 15 axes). If every axis shows overlap, the shapes intersect, and the axis with the smallest overlap gives the minimum translation vector to separate them, which physics engines use for collision response.

Animation and Skinning

15

Keyframes and interpolation, easing curves, three.js's AnimationClip/Mixer/Action system with blending and cross-fades, skeletal animation and the linear blend skinning formula worked by hand, springs and two-bone IK for procedural motion, and R3F's useAnimations.

IN THIS CHAPTER

15.1 Kinds of animation
15.2 Keyframes and interpolation
15.3 The three.js animation system
15.4 Skeletal animation
15.5 Procedural animation
15.6 Animation in React Three Fiber
15.7 Interview questions

15.1 Kinds of animation

Kind	What changes	Where it runs	Examples
Procedural	Transforms or uniforms computed from time or input	JavaScript (`useFrame`) or shaders	Spinning logos, bobbing, wind, followers
Tweens	A property from A to B over a duration with easing	JavaScript (GSAP, react-spring, `MathUtils.damp`)	UI transitions, camera moves, configurator swaps
Keyframe clips	Many properties sampled from keyframes	three.js animation system	Doors, machines, cut-scenes from Blender
Skeletal	Bones drive a skinned mesh	Bones on CPU, skinning on GPU	Characters, creatures, robot arms
Morph targets	Vertex offsets blended by weights	GPU	Faces, lip sync, muscles
Physics	Forces and collisions	Physics engine (Chapter 17)	Falling, ragdolls, vehicles
Shader	Vertex or fragment maths over time	GPU	Water, flags, particles, dissolve

15.2 Keyframes and interpolation

DEFINITION · **Keyframe track**

A list of times and the values a property has at those times. Between keys the value is **interpolated**. In three.js a track names its target property with a path such as `"Arm.quaternion"` or `".material.opacity"` : `VectorKeyframeTrack` , `QuaternionKeyframeTrack` (interpolated with slerp), `NumberKeyframeTrack` , `ColorKeyframeTrack` , `BooleanKeyframeTrack` and `StringKeyframeTrack` (stepped).

Interpolation	three.js	Behaviour
Step (discrete)	`InterpolateDiscrete`	Holds a value until the next key
Linear	`InterpolateLinear` (default)	Straight lines between keys (slerp for quaternions)
Smooth (cubic)	`InterpolateSmooth`	A smooth curve through the keys (Catmull–Rom-like)
Cubic spline with tangents	glTF `CUBICSPLINE`	Hermite curve with stored in/out tangents per key

WORKED EXAMPLE · **Sampling a track**

Times $[0, 1, 2]$ s, values $y = [0, 2, 1]$. At $t = 1.25$ s the segment is from key 1 to key 2, $\alpha = (1.25 - 1)/(2 - 1) = 0.25$, so linear interpolation gives $y = 2 + 0.25\,(1 - 2) = 1.75$. With step interpolation it would still be 2.

15.2.1 Easing

DEFINITION · **Easing function**

A curve $e(t)$ that maps linear progress $t \in [0, 1]$ to eased progress, so motion speeds up and slows down instead of starting and stopping abruptly. Objects in the real world have inertia, so eased motion feels physical.

FORMULA · **Common easings**

$$\text{easeInQuad}(t) = t^2, \qquad \text{easeOutQuad}(t) = 1 - (1 - t)^2, \qquad \text{easeInOutCubic}(t) = \begin{cases} 4t^3 \\ 1 - \frac{(2-2t)^3}{2} \end{cases}$$

Apply as $\text{value} = \text{lerp}(a, b, e(t))$ with $t = \text{elapsed}/\text{duration}$ clamped to $[0, 1]$.

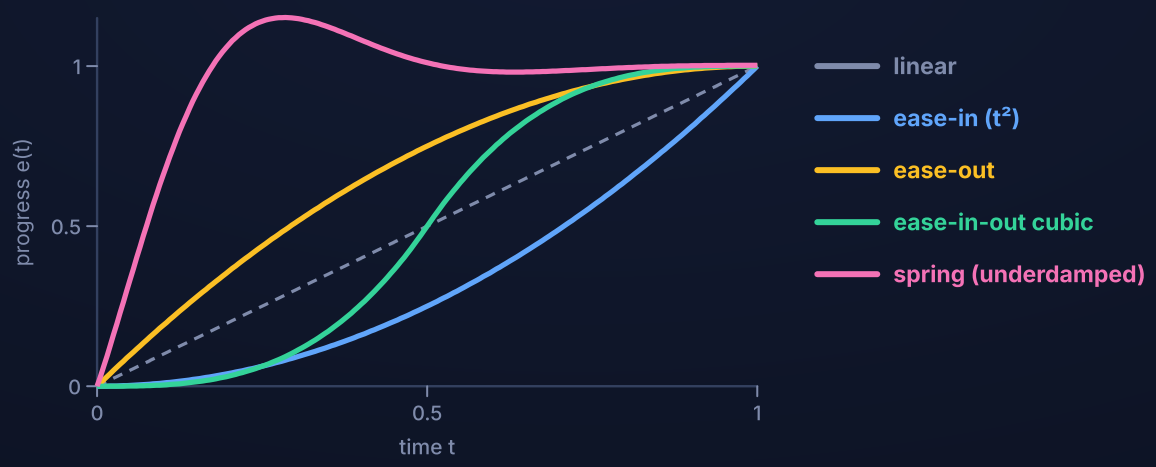


Figure 15.1 Easing curves. The slope is the speed: ease-in starts slowly, ease-out ends slowly, ease-in-out does both. A spring overshoots and settles.

WORKED EXAMPLE · **Ease-in-out cubic at quarters**

$e(0.25) = 4 \times 0.25^3 = 0.0625$; $e(0.5) = 0.5$; $e(0.75) = 1 - 0.5^3/2 = 0.9375$. A quarter of the way through the time, the object has covered only 6% of the distance: a gentle start.

15.3 The three.js animation system

DEFINITION · **AnimationClip, AnimationMixer, AnimationAction**

An **AnimationClip** is a named, reusable set of tracks (for example "Walk"). An **AnimationMixer** plays clips on one object hierarchy (the "root"). An **AnimationAction** is one clip scheduled on a mixer, with its own time, speed (`timeScale`), weight, loop mode and fades. Call `mixer.update(delta)` every frame.

three.js — playing and blending clips from a glTF JAVASCRIPT

```
const gltf = await new GLTFLoader().loadAsync("robot.glb");
scene.add(gltf.scene);
const mixer = new THREE.AnimationMixer(gltf.scene);
const clip = (name) => THREE.AnimationClip.findByName(gltf.animations, name);
const actions = {
  idle: mixer.clipAction(clip("Idle")),
  walk: mixer.clipAction(clip("Walk")),
  wave: mixer.clipAction(clip("Wave")),
};
actions.idle.play();

function switchTo(next, current, duration = 0.3) {
  next.reset().setEffectiveWeight(1).play();
  current.crossFadeTo(next, duration, true);   // true = warp time scales so the gaits stay in
step
}
actions.wave.setLoop(THREE.LoopOnce, 1);
actions.wave.clampWhenFinished = true;         // hold the last frame
```

```
mixer.addEventListener("finished", (e) => { if (e.action === actions.wave)
switchTo(actions.idle, actions.wave); });

renderer.setAnimationLoop((t) => { timer.update(t); mixer.update(timer.getDelta());
renderer.render(scene, camera); });
```

WORKED EXAMPLE · **Weights during a cross-fade**

A 0.3 s cross-fade from Idle to Walk. three.js fades weights linearly, so after 0.1 s Idle has weight $1 - 0.1/0.3 = 0.667$ and Walk 0.333. Each bone's final pose is the weighted blend: positions and scales lerped, rotations combined with weighted quaternion blending. The weights always sum to 1 during a plain cross-fade.

DEFINITION · **Additive animation**

A clip stored as the **difference** from a reference pose (`AnimationUtils.makeClipAdditive(clip)` and `blendMode = AdditiveAnimationBlendMode`). Additive layers are added on top of whatever else plays: a "breathing" or "look-left" layer over any walk, a recoil over any pose.

Action API	Effect
`play()` , `stop()` , `reset()`	Start, stop, rewind
`timeScale` , `setDuration(s)`	Speed (negative plays backwards)
`setLoop(LoopRepeat / LoopOnce / LoopPingPong, reps)`	Looping
`fadeIn(d)` , `fadeOut(d)` , `crossFadeTo(other, d, warp)`	Smooth weight changes
`setEffectiveWeight(w)` , `enabled`	Blend weight; disable without stopping
`clampWhenFinished`	Freeze on the last frame
`AnimationUtils.subclip(clip, name, startFrame, endFrame, fps)`	Cut a long clip into pieces

15.4 Skeletal animation

DEFINITION · **Skeleton and bones**

A **skeleton** is a hierarchy of **bones** (each an `Object3D` , so they have positions, rotations and parents). A `SkinnedMesh` 's vertices are attached to up to **four** bones each, with **weights** that sum to 1 (`skinIndex` and `skinWeight` attributes). Animating the bones deforms the mesh.

DEFINITION · **Bind pose and inverse bind matrices**

The **bind pose** is the pose in which the mesh was attached to the skeleton (usually a T- or A-pose). For each bone, the **inverse bind matrix** B_i^{-1} moves a vertex from model space into that bone's local space as it was at bind time. The bone's current world matrix M_i then carries the vertex to where the bone is now.

Σ FORMULA · Linear blend skinning (LBS)

$$\mathbf{v}' = \sum_{i=1}^{4} w_i \, M_i \, B_i^{-1} \, \mathbf{v}, \qquad \sum w_i = 1$$

Each influencing bone moves the vertex rigidly; the results are averaged by weight. three.js computes $M_i B_i^{-1}$ for every bone on the CPU each frame (`skeleton.update()`), uploads them in a bone texture, and the vertex shader does the weighted sum.

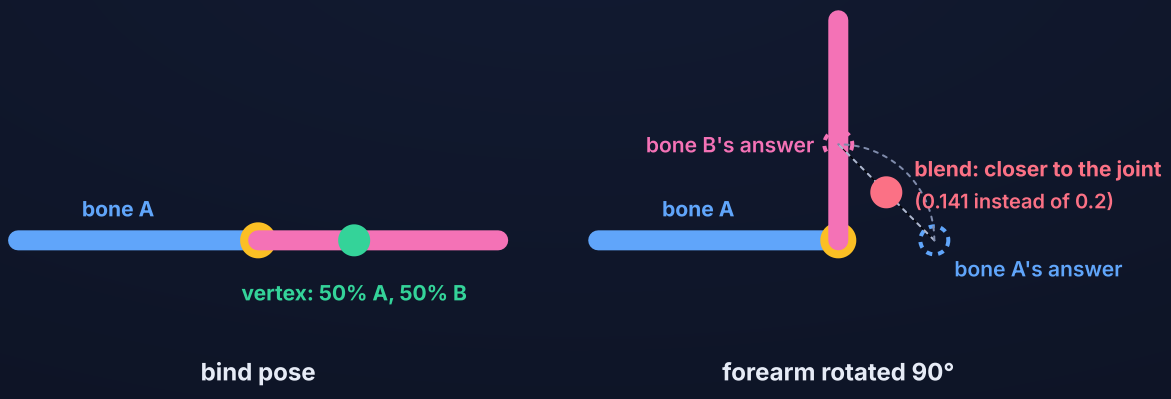


Figure 15.2 Linear blend skinning at an elbow. A vertex near the joint is moved by both bones; averaging their positions pulls it inwards, which is why bent elbows and twisted wrists lose volume.

WORKED EXAMPLE · Skinning a vertex by hand

An elbow joint at $\mathbf{j} = (1, 0, 0)$. Vertex $\mathbf{v} = (1.2, 0, 0)$ with weights 0.5 (upper arm, not moving) and 0.5 (forearm, rotated 90° about Z around the joint).

1. Upper arm: $M_A B_A^{-1} = I$, so it gives $(1.2, 0, 0)$.
2. Forearm: rotate the offset from the joint: $R_z(90^\circ)(0.2, 0, 0) = (0, 0.2, 0)$; add the joint: $(1, 0.2, 0)$.
3. Blend: $0.5\,(1.2, 0, 0) + 0.5\,(1, 0.2, 0) = (1.1, 0.1, 0)$.
4. Its distance from the joint is $\sqrt{0.1^2 + 0.1^2} = 0.141$ instead of 0.2: the skin has shrunk by 30% at the bend. Twisting 180° makes it collapse to the joint entirely (the "candy wrapper" artefact). Fixes: more bones and weight painting, corrective blend shapes, or dual-quaternion skinning (not built into three.js).

⚠ PITFALL · Cloning skinned meshes

`model.clone()` copies the meshes but the clones still point at the **original** skeleton's bones, so every copy moves with the first one. Use `SkeletonUtils.clone(model)` from `three/addons/utils/SkeletonUtils.js`, which rebuilds bone references; drei's `useGLTF` + `<Clone>` or `useMemo(() ⇒ SkeletonUtils.clone(scene), [scene])` do the same in R3F. Each clone also needs its own `AnimationMixer`.

TIP · **Crowds**

`InstancedMesh` cannot skin per instance. Dozens of characters: separate `SkinnedMesh` clones (share geometry and materials). Hundreds or thousands: bake the animation into a texture (a **vertex animation texture**, VAT) and play it in a custom instanced vertex shader, or use the `InstancedMesh` + morph texture approach; add LOD and update far characters' mixers less often.

15.5 Procedural animation

15.5.1 Springs

FORMULA · **Damped spring**

$$\ddot{x} = -k\,(x - x_{\text{target}}) - c\,\dot{x}$$

k is stiffness, c damping (unit mass). Critical damping $c = 2\sqrt{k}$ reaches the target as fast as possible without overshoot; less damping overshoots and wobbles. Integrate with **semi-implicit Euler**: update velocity first, then position with the new velocity (much more stable than explicit Euler).

WORKED EXAMPLE · **Three spring steps**

$k = 100$, critical $c = 2\sqrt{100} = 20$, $\Delta t = 1/60$, starting at $x = 1$, $v = 0$, target 0.

1. $a = -100(1) - 20(0) = -100$; $v = -1.667$; $x = 1 - 1.667/60 = 0.972$.
2. $a = -97.2 + 33.3 = -63.9$; $v = -2.732$; $x = 0.927$.
3. $v = -3.366$; $x = 0.871$, smoothly accelerating towards 0 and then settling without overshoot.

three.js — a springy follower (semi-implicit Euler) JAVASCRIPT

```
const k = 120, c = 2 * Math.sqrt(k) * 0.6;       // 0.6 × critical: a little overshoot feels
lively
const vel = new THREE.Vector3(), acc = new THREE.Vector3();
function updateSpring(obj, target, dt) {
  acc.subVectors(target, obj.position).multiplyScalar(k).addScaledVector(vel, -c);
  vel.addScaledVector(acc, dt);
  obj.position.addScaledVector(vel, dt);
}
```

15.5.2 Inverse kinematics

DEFINITION · **Inverse kinematics (IK)**

Forward kinematics computes where a hand ends up from joint angles. **Inverse** kinematics finds joint angles that put the hand (the end effector) on a target: feet on uneven ground, a hand on a door handle, a robot gripper on a part. Two-bone chains have an exact answer from the law of cosines; longer chains use iterative solvers such as CCD or FABRIK (three.js ships `CCDIKSolver`).

Σ FORMULA · Two-bone IK (law of cosines)

Upper bone length a, lower bone length b, distance from shoulder to target d (clamped to $|a-b| \le d \le a+b$):

$$\cos\theta_{\text{elbow}} = \frac{a^2 + b^2 - d^2}{2ab}, \qquad \cos\theta_{\text{shoulder}} = \frac{a^2 + d^2 - b^2}{2ad}$$

θ_{elbow} is the interior angle at the elbow; θ_{shoulder} is how far the upper bone turns away from the line to the target. A pole vector chooses which way the elbow bends.

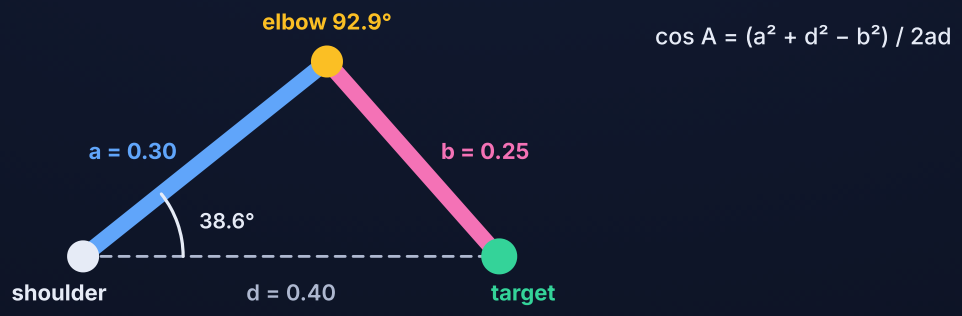


Figure 15.3 Two-bone IK. The triangle formed by the two bones and the shoulder–target line has known side lengths, so its angles follow from the law of cosines.

WORKED EXAMPLE · Reaching for a cup

Upper arm $a = 0.30$ m, forearm $b = 0.25$ m, target $d = 0.40$ m away.

- $\cos\theta_{\text{elbow}} = (0.09 + 0.0625 - 0.16)/(2 \times 0.3 \times 0.25) = -0.0075/0.15 = -0.05$, so $\theta_{\text{elbow}} = 92.9^\circ$.
- $\cos\theta_{\text{shoulder}} = (0.09 + 0.16 - 0.0625)/(2 \times 0.3 \times 0.4) = 0.78125$, so the upper arm points 38.6° off the shoulder–cup line, towards the pole.

15.6 Animation in React Three Fiber

R3F — drei useAnimations JSX

```jsx
import { useGLTF, useAnimations } from "@react-three/drei";

function Robot({ action = "Idle" }) {
  const group = useRef();
  const { scene, animations } = useGLTF("/robot.glb");
  const { actions } = useAnimations(animations, group);   // creates and updates the mixer for
you
  useEffect(() => {
    const a = actions[action];
    a.reset().fadeIn(0.3).play();
    return () => a.fadeOut(0.3);                          // cross-fade when `action` changes
  }, [action, actions]);
  return <primitive ref={group} object={scene} />;
}
```

R3F — springs with @react-spring/three JSX

```jsx
import { useSpring, animated } from "@react-spring/three";
function Card({ open }) {
  const { rotation } = useSpring({ rotation: open ? [0, Math.PI, 0] : [0, 0, 0], config: {
tension: 170, friction: 26 } });
  return <animated.mesh rotation={rotation}><boxGeometry args={[1, 1.4, 0.02]} />
<meshStandardMaterial /></animated.mesh>;
}
```

TIP · Which tool for which job

`useFrame` + damping for continuous follow behaviour; react-spring or `maath` easing for state-driven UI motion (open/closed, hover); GSAP timelines for choreographed sequences and scroll-driven stories; three.js clips for anything authored in Blender; shaders for thousands of moving things.

KEY POINTS

- ✓ Tracks store (time, value) keys; quaternion tracks slerp; glTF clips map to `AnimationClip` s.
- ✓ Easing maps linear time to eased progress: $4t^3$ / $1-(2-2t)^3/2$ for ease-in-out cubic.
- ✓ Mixer per character, `clipAction` per clip, `crossFadeTo` with warp for gait changes, `mixer.update(dt)` every frame.
- ✓ LBS: $\mathbf{v}' = \sum w_i M_i B_i^{-1} \mathbf{v}$; blending rotations linearly loses volume at bends.
- ✓ Clone skinned models with `SkeletonUtils.clone` ; crowds need VATs or similar.
- ✓ Springs: semi-implicit Euler, critical damping $c = 2\sqrt{k}$.
- ✓ Two-bone IK solves with the law of cosines; longer chains with CCD/FABRIK.

15.7 Interview questions

Q How do you play and blend animations from a glTF in three.js? JUNIOR}

A Create an `AnimationMixer` on the loaded scene, get an `AnimationAction` per clip with `mixer.clipAction(clip)` (find clips with `AnimationClip.findByName(gltf.animations, "Walk")`), call `play()`, and update the mixer every frame with the delta time. Blend with `fadeIn` / `fadeOut` or `crossFadeTo(next, duration, true)`, adjust `timeScale` for speed, and use `setLoop(LoopOnce)` plus `clampWhenFinished` for one-shot actions. In R3F, drei's `useAnimations` creates and updates the mixer for you.

Q Explain linear blend skinning and its main artefact. SENIOR}

A Each vertex is influenced by up to four bones with weights summing to one. For each bone, the inverse bind matrix takes the vertex from the bind pose into the bone's local space, and the bone's current matrix takes it to the animated position; the final position is the weighted sum of these rigidly transformed positions. Averaging positions (equivalently, linearly blending rotation matrices) does not preserve length, so at strongly bent or twisted joints vertices are pulled towards the joint: the elbow-collapse and "candy wrapper" artefacts. Remedies are better weights and extra twist bones, corrective morph targets, or dual-quaternion skinning.

Q I cloned a character five times and they all play the same animation in sync, in the same place. Why? MID}

A `Object3D.clone()` does not remap a `SkinnedMesh` 's skeleton: all clones still reference the original bones, so they all deform with the first character. Use `SkeletonUtils.clone()`, which clones the bone hierarchy and rebinds each skinned mesh to its own bones, and create one `AnimationMixer` per clone. Geometry and materials can still be shared.

Q How would you make a camera smoothly follow a target with a little bounce? MID}

A Use a damped spring on the camera position (and possibly its look-at target): acceleration $= k(\text{target} - x) - c\,v$, integrated with semi-implicit Euler using the frame delta, with c somewhat below the critical value $2\sqrt{k}$ for a slight overshoot. Clamp large deltas after tab switches. For no bounce, exponential damping (`MathUtils.damp` , maath `easing.damp3`) is simpler and frame-rate independent.

Q How do you place a character’s feet correctly on uneven ground? SENIOR}

A Foot IK: each frame, raycast down from each foot’s animated position to find the ground height and normal; move the foot target to the hit point (plus ankle height) and align the foot to the normal; solve two-bone IK (hip–knee–ankle) with the law of cosines and a pole vector in front of the knee; lower the pelvis by the larger foot offset so the legs can reach; blend the IK weight with the animation’s foot contact curve so feet only stick when planted.

Assets — glTF and the Loading Pipeline

16

The glTF format from the inside (scenes, nodes, meshes, accessors, buffers, extensions), loading with GLTFLoader plus Draco, Meshopt and KTX2, quantisation maths, an optimisation pipeline with gltf-transform and gltfjsx, other formats, and loading with Suspense in R3F.

IN THIS CHAPTER

16.1 Why glTF
16.2 Inside a glTF file
16.3 Loading glTF in three.js
16.4 The optimisation pipeline
16.5 Other formats
16.6 Loading in React Three Fiber
16.7 Interview questions

16.1 Why glTF

DEFINITION · **glTF 2.0**

The Khronos Group's open standard for delivering 3D scenes, often called "the JPEG of 3D". It stores meshes, PBR materials, textures, cameras, lights, skins and animations in a form that maps almost directly onto GPU buffers, so loading is mostly copying, not converting. It is right-handed, +Y up, in metres, and its core material is exactly the metal–roughness model of `MeshStandardMaterial`.

Variant	Contents	Use
`.gltf` + `.bin` + images	JSON description, binary buffers and textures as separate files	Debugging, hand editing
`.glb`	One binary file: a JSON chunk plus a binary chunk (textures embedded)	Delivery on the web

16.2 Inside a glTF file

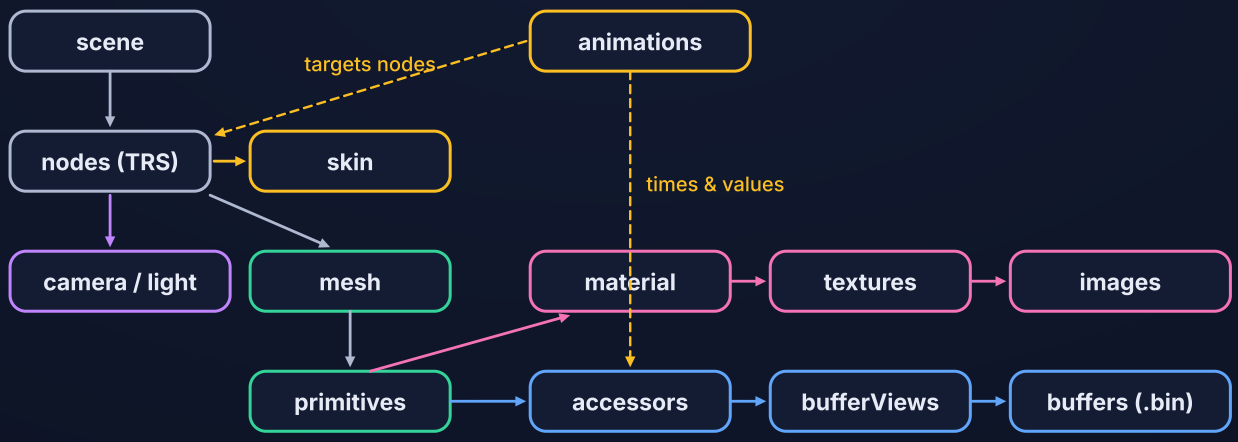


Figure 16.1 How a glTF file is organised. Nodes form the scene tree; a mesh's primitives point at accessors, which describe typed views into binary buffers; materials point at textures, which point at images and samplers.

Object	What it describes	Becomes in three.js
`scenes`, `nodes`	The hierarchy with TRS or matrix transforms	`Group` / `Object3D` tree (`gltf.scene`)
`meshes` → `primitives`	Geometry pieces, each with one material and a draw mode	`Mesh` per primitive (a `Group` if several)
`accessors`	Typed data: count, component type, `VEC3` / `VEC2` / `SCALAR`, min/max	`BufferAttribute`
`bufferViews`, `buffers`	Byte ranges inside binary blobs	`ArrayBuffer` slices
`materials`	PBR metal–roughness + extensions	`MeshStandardMaterial` / `MeshPhysicalMaterial`
`textures`, `images`, `samplers`	Image + filtering/wrapping	`Texture` (colour maps sRGB, `flipY = false`)
`skins`	Joints + inverse bind matrices	`Skeleton` + `SkinnedMesh`
`animations`	Channels (node + path) and samplers (input times, output values)	`AnimationClip`s (`gltf.animations`)
`cameras`, `KHR_lights_punctual`	Cameras and lights	`PerspectiveCamera`, `DirectionalLight`…
`extras`	Custom data from the exporter	`object.userData`

Σ FORMULA · **Accessor size**

$$\text{bytes} = \text{count} \times \text{components} \times \text{component size}$$

`VEC3` has 3 components; FLOAT is 4 bytes, UNSIGNED_SHORT 2, BYTE 1. (Vertex attributes are padded so each element starts on a 4-byte boundary.)

WORKED EXAMPLE · **What a 100k-vertex model weighs**

100,000 vertices with float positions (12 B), normals (12 B) and UVs (8 B), plus 200,000 triangles of 32-bit indices:

- Attributes: $100{,}000 \times 32 = 3.2$ MB. Indices: $600{,}000 \times 4 = 2.4$ MB. Total **5.6 MB** before textures.
- With `KHR_mesh_quantization`: positions as 16-bit integers (6 B, padded to 8), normals as 8-bit (3 B, padded to 4), UVs as 16-bit (4 B): 16 B per vertex, **1.6 MB**, and it stays that small **in GPU memory** too.
- Meshopt or Draco compression on top shrinks the download several times more.

Σ FORMULA · **Quantisation precision**

Storing a coordinate that spans an extent E with b bits gives a step of

$$\Delta = \frac{E}{2^b - 1}$$

WORKED EXAMPLE · **Is 16-bit enough?**

A 2 m tall statue with 16-bit positions: $\Delta = 2/65535 = 0.03$ mm, far below anything visible. A 2 km city in one mesh: $\Delta = 3$ cm, visible as wobbly edges up close, so split large worlds into tiles (each with its own offset and scale) before quantising.

16.3 Loading glTF in three.js

three.js — a complete, production-style loader — JAVASCRIPT

```
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";
import { DRACOLoader } from "three/addons/loaders/DRACOLoader.js";
import { KTX2Loader } from "three/addons/loaders/KTX2Loader.js";
import { MeshoptDecoder } from "three/addons/libs/meshopt_decoder.module.js";

const manager = new THREE.LoadingManager();
manager.onProgress = (url, loaded, total) ⇒ progressBar.style.width = `${(loaded / total) *
100}%`;

const draco = new
DRACOLoader(manager).setDecoderPath("https://www.gstatic.com/draco/versioned/decoders/1.5.7/");
const ktx2 = new KTX2Loader(manager)
  .setTranscoderPath("https://cdn.jsdelivr.net/npm/three@0.186.0/examples/jsm/libs/basis/")
```

```
  .detectSupport(renderer);
const loader = new GLTFLoader(manager)
  .setDRACOLoader(draco)
  .setKTX2Loader(ktx2)
  .setMeshoptDecoder(MeshoptDecoder);

const gltf = await loader.loadAsync("/models/chair.glb");
const model = gltf.scene;
model.traverse((o) => {
  if (o.isMesh) { o.castShadow = o.receiveShadow = true; }
});
// normalise size and position: 1 m tall, standing on the floor, centred
const box = new THREE.Box3().setFromObject(model);
const size = box.getSize(new THREE.Vector3()), centre = box.getCenter(new THREE.Vector3());
model.scale.setScalar(1 / size.y);
model.position.set(-centre.x / size.y, -box.min.y / size.y, -centre.z / size.y);
scene.add(model);
```

TIP · Where the decoders run

Draco and Basis transcoding run in **Web Workers** (WASM), so they do not freeze the page, but decoding still takes time on low-end phones. Host the decoder files yourself in production (same version as your three.js) instead of depending on a third-party CDN, and reuse **one** loader instance for all models so the workers are created once.

16.3.1 Draco versus Meshopt

	Draco (`KHR_draco_mesh_compression`)	Meshopt (`EXT_meshopt_compression`)
Download size	Smallest for geometry	Small, especially combined with gzip/Brotli
Decode speed	Slow (WASM, heavy)	Very fast (often near memcpy speed)
In GPU memory	Decompressed to full floats	Can stay quantised (with `KHR_mesh_quantization`)
Animations, morph targets	Geometry only	Also compresses animation and morph data
Best for	Huge static meshes where download dominates	Most web apps, mobile, animated characters

16.4 The optimisation pipeline

Exported files are rarely web-ready: duplicated materials, unused data, 4K PNGs, uncompressed geometry. Optimise as a build step.

1. A raw export from Blender, CAD or a scanner
2. `gltf-transform inspect model.glb` : see sizes per mesh, texture and animation
3. `dedup` , `prune` , `weld` , `join` : remove duplicates and unused data, merge compatible meshes
4. `simplify --ratio 0.5` : reduce triangles (meshoptimizer) where the detail is not visible
5. `resize --width 2048` then `etc1s` / `uastc` : smaller, GPU-compressed KTX2 textures
6. `meshopt` (or `draco`): compress geometry and animation
7. `gltfjsx model.glb --transform --types` : generate a typed R3F component (it runs similar optimisations)

one-line optimisation — SHELL

```
npx @gltf-transform/cli optimize in.glb out.glb --compress meshopt --texture-compress ktx2
npx gltfjsx out.glb --types --shadows   # → Model.tsx with typed nodes and materials
```

WORKED EXAMPLE · **A typical result**

A Blender export of a sofa: 38 MB (four 4K PNG textures, 450k triangles, float geometry). After `resize 2048` + KTX2 (ETC1S colour, UASTC normal) + `simplify 0.5` + meshopt: about 3–4 MB to download, and texture VRAM drops from about 340 MiB (four 85 MiB 4K maps) to under 25 MiB. Load time on 4G improves from tens of seconds to a couple of seconds.

16.5 Other formats

Format	Loader	Notes
FBX	`FBXLoader`	Common from Autodesk tools and Mixamo; converts Z-up to Y-up since r184; prefer converting to glTF
OBJ (+ MTL)	`OBJLoader` , `MTLLoader`	Geometry only, old materials, no animation; very common, very simple
USD / USDZ	`USDLoader` (`USDZLoader` deprecated in r179)	Apple and Pixar ecosystem; `USDZExporter` for AR Quick Look on iOS
STL	`STLLoader`	3D printing; no UVs or materials
PLY	`PLYLoader`	Scanners and point clouds; honours data types since r185
3DM	`3DMLoader`	Rhino (architecture, jewellery)
VOX	`VOXLoader`	MagicaVoxel voxel art

Format	Loader	Notes
Gaussian splats	`GaussianSplatPLYLoader`, `SPLATLoader`, `KSPLATLoader`, glTF extension (r186)	Photoreal captured scenes (Chapter 24)
IFC (BIM)	`web-ifc` / That Open Engine	Building information models

16.6 Loading in React Three Fiber

DEFINITION · **Suspense loading**

R3F's loaders (`useLoader`, drei's `useGLTF`, `useTexture`) **suspend** the component until the asset is ready, and cache the result by URL. Wrap them in React `<Suspense fallback={…}>` and the fallback shows while loading; every other component that asks for the same URL gets the cached result instantly.

R3F — loading with Suspense, preload and progress JSX

```jsx
import { Suspense } from "react";
import { useGLTF, useProgress, Html, Clone } from "@react-three/drei";

function Chair(props) {
  const { scene } = useGLTF("/models/chair.glb");   // Draco handled automatically (decoder from a CDN by default)
  return <Clone object={scene} {...props} />;        // safe to render many times (deep clone, skinning-aware)
}
useGLTF.preload("/models/chair.glb");               // start downloading before the component mounts

function Loader() {
  const { progress } = useProgress();
  return <Html center>{progress.toFixed(0)} %</Html>;
}

export default function App() {
  return (
    <Canvas shadows>
      <Suspense fallback={<Loader />}>
        <Chair position={[-1, 0, 0]} />
        <Chair position={[1, 0, 0]} rotation-y={Math.PI} />
      </Suspense>
    </Canvas>
  );
}
```

R3F — what gltfjsx generates (abridged) TSX

```tsx
type GLTFResult = GLTF & {
  nodes: { Seat: THREE.Mesh; Legs: THREE.Mesh };
  materials: { Fabric: THREE.MeshStandardMaterial; Oak: THREE.MeshStandardMaterial };
};
export function Model(props: ThreeElements["group"]) {
  const { nodes, materials } = useGLTF("/chair-transformed.glb") as unknown as GLTFResult;
  return (
    <group {...props} dispose={null}>
      <mesh castShadow geometry={nodes.Seat.geometry} material={materials.Fabric} />
      <mesh castShadow geometry={nodes.Legs.geometry} material={materials.Oak} position={[0,
-0.21, 0]} />
    </group>
  );
}
useGLTF.preload("/chair-transformed.glb");
```

TIP · Why gltfjsx is worth it

The generated component makes the model's structure explicit JSX: you can swap a material per instance (`material-color` overrides), add events to individual parts, animate a door node, or render many instances with drei's `<Merged>` for one draw call per part. `dispose={null}` keeps cached geometry alive when one instance unmounts.

KEY POINTS

- ✓ glTF = JSON scene graph + binary buffers + PBR materials; `.glb` packs everything in one file.
- ✓ Accessor bytes = count × components × component size; quantisation step = extent / $(2^b - 1)$.
- ✓ `GLTFLoader` + `DRACOLoader` + `KTX2Loader` + `MeshoptDecoder`, one shared instance, decoders self-hosted.
- ✓ Meshopt for fast decode and animated content; Draco for the smallest static geometry; KTX2 for textures.
- ✓ Optimise in the build: `gltf-transform optimize`, then `gltfjsx --transform` for R3F.
- ✓ R3F: `useGLTF` suspends and caches; preload; `<Clone>` for multiple copies; `useProgress` for loading UI.

16.7 Interview questions

Q Why is glTF the preferred format for the web? JUNIOR}

A It is an open standard designed for runtime delivery: its buffers map directly onto GPU vertex and index buffers, its core material is the same PBR metal–roughness model three.js uses, it carries skins, morph targets, animations, cameras and lights, it has a fixed convention (right-handed, Y-up, metres), and it has compression extensions for geometry (Draco, Meshopt, quantisation) and textures (KTX2/Basis). FBX and OBJ need more conversion, lose information and are larger.

Q **A 40 MB model takes too long to load on mobile. Walk me through optimising it.** MID

A Inspect it first (`gltf-transform inspect`) to see whether textures or geometry dominate. Usually textures: resize to what the screen needs (often 1–2K), convert to KTX2 (ETC1S for colour, UASTC for normals), pack ORM channels. Geometry: deduplicate, weld, simplify where detail is invisible, quantise and compress with Meshopt (or Draco for very large static meshes). Remove unused nodes, materials and animations. Serve with Brotli, preload, show progress, and consider splitting into a quick low-resolution first load plus streamed detail.

Q **What is the trade-off between Draco and Meshopt?** MID

A Draco produces the smallest geometry downloads but decoding is CPU-heavy (a WASM worker) and the result is expanded to full floats in memory; it covers geometry only. Meshopt compresses slightly less on its own (very well with gzip/Brotli), decodes extremely fast, can keep data quantised in GPU memory with `KHR_mesh_quantization`, and also compresses animations and morph targets. For most interactive web apps, especially on mobile or with animated characters, Meshopt is the better default.

Q **How does Suspense-based loading work in R3F?** MID

A Hooks like `useLoader` and drei's `useGLTF` start the load and throw a promise if the asset is not ready, which suspends the component; React shows the nearest `<Suspense>` fallback and re-renders once the promise resolves. Results are cached by URL, so other components using the same asset get it immediately, and `useGLTF.preload` starts loading early. Because the cache is shared, rendering the same model twice needs `<Clone>` (or gltfjsx's component, which reuses geometry and materials) rather than adding the same `scene` object twice.

Q **Why does adding the same `gltf.scene` to the scene twice only show one model?** JUNIOR

A An `Object3D` can have only one parent; adding it again moves it. You need separate objects: `gltf.scene.clone()` for static models (geometry and materials are shared, which is efficient), `SkeletonUtils.clone()` for skinned ones, or drei's `<Clone>` in R3F. For many copies of the same part, instancing (`InstancedMesh` or drei's `<Merged>` / `<Instances>`) is better still.

Physics and Simulation

17

Numerical integration (explicit and semi-implicit Euler, Verlet) compared on the same throw, forces and torque, impulse-based collision response with restitution and friction worked by hand, constraints and solvers, tunnelling and CCD, Verlet cloth, and practical physics with Rapier in three.js and @react-three/rapier.

IN THIS CHAPTER

17.1 Simulating motion
17.2 Collision response
17.3 Constraints, solvers and stability
17.4 Physics engines for the web
17.5 Verlet cloth and soft constraints
17.6 Interview questions

17.1 Simulating motion

A physics engine moves objects by repeatedly applying Newton's second law over small time steps.

Σ FORMULA · **Newton's second law, linear and angular**

$$\mathbf{F} = m\,\mathbf{a}, \qquad \boldsymbol{\tau} = \mathbf{r} \times \mathbf{F} = I\,\boldsymbol{\alpha}$$

Force $\mathbf{F}$ accelerates mass m. A force applied at offset $\mathbf{r}$ from the centre of mass also creates a **torque** $\boldsymbol{\tau}$, which spins the body according to its **inertia tensor** I (how its mass is spread out).

WORKED EXAMPLE · **Pushing a door**

A force of 10 N pushing in $-Z$ at the door's edge, 0.5 m from the hinge along $+X$: $\boldsymbol{\tau} = (0.5, 0, 0) \times (0, 0, -10) = (0 \cdot (-10) - 0 \cdot 0,\ 0 \cdot 0 - 0.5 \cdot (-10),\ 0) = (0, 5, 0)$ N·m. A torque about $+Y$: the door swings anticlockwise seen from above. Push at 0.25 m and the torque halves, which is why handles sit far from hinges.

17.1.1 Integrators

DEFINITION · **Numerical integration**

Turning accelerations into velocities and positions one time step h at a time. The choice of method decides accuracy and, more importantly for games, **stability**: whether errors stay bounded or grow until objects explode.

Σ FORMULA · **Three integrators**

$$\textbf{Explicit Euler:}\quad \mathbf{x}_{n+1} = \mathbf{x}_n + h\,\mathbf{v}_n, \quad \mathbf{v}_{n+1} = \mathbf{v}_n + h\,\mathbf{a}_n$$

$$\textbf{Semi-implicit (symplectic) Euler:}\quad \mathbf{v}_{n+1} = \mathbf{v}_n + h\,\mathbf{a}_n, \quad \mathbf{x}_{n+1} = \mathbf{x}_n + h\,\mathbf{v}_{n+1}$$

$$\textbf{Position Verlet:}\quad \mathbf{x}_{n+1} = 2\mathbf{x}_n - \mathbf{x}_{n-1} + h^2\,\mathbf{a}_n$$

Explicit Euler adds energy to oscillating systems (springs grow until they explode). Semi-implicit Euler is just as cheap but keeps energy bounded, which is why most game engines use it. Verlet needs no velocity at all (it is implied by the last two positions), which makes constraints easy (cloth, ropes, ragdolls).

WORKED EXAMPLE · **The same throw, three integrators**

A ball thrown straight up at 10 m/s, $g = -9.81$ m/s², step $h = 0.1$ s, ten steps (1 s). Exact height: $10(1) - \frac{1}{2}(9.81)(1)^2 = 5.095$ m.

- Explicit Euler: $y = 5.586$ m (too high, it uses the old, faster velocity).
- Semi-implicit Euler: $y = 4.605$ m (too low by the same amount, but stable).
- Verlet: $y = 5.095$ m (exact for constant acceleration).

With realistic steps ($h = 1/60$) the errors are about six times smaller; what matters more is that springs and stacked contacts stay stable.

Figure 17.1 Explicit Euler on a spring spirals outwards (gains energy); semi-implicit Euler stays on a stable orbit. Same step size, same cost.

17.2 Collision response

Detecting a collision (Chapter 14) gives a contact point, a normal $\hat{\mathbf{n}}$ and a penetration depth. The response changes velocities so the bodies separate, using an **impulse**: a large force over a tiny time, an instant change of momentum.

Σ FORMULA · **Impulse for a collision (no rotation)**

Relative velocity along the normal $v_n = (\mathbf{v}_1 - \mathbf{v}_2)\cdot\hat{\mathbf{n}}$ (with $\hat{\mathbf{n}}$ pointing from body 1 to body 2; $v_n > 0$ means approaching). With restitution e (0 = dead stop, 1 = perfectly bouncy):

$$j = \frac{(1+e)\,v_n}{\dfrac{1}{m_1} + \dfrac{1}{m_2}}, \qquad \mathbf{v}_1' = \mathbf{v}_1 - \frac{j}{m_1}\hat{\mathbf{n}}, \qquad \mathbf{v}_2' = \mathbf{v}_2 + \frac{j}{m_2}\hat{\mathbf{n}}$$

A static body has $1/m = 0$ (infinite mass). With rotation, the denominator gains terms $(I^{-1}(\mathbf{r}\times\hat{\mathbf{n}})\times\mathbf{r})\cdot\hat{\mathbf{n}}$ for each body.

WORKED EXAMPLE · **Two carts collide**

Cart 1: $m_1 = 2$ kg, $\mathbf{v}_1 = (3,0,0)$. Cart 2: $m_2 = 1$ kg, $\mathbf{v}_2 = (-1,0,0)$. Normal $\hat{\mathbf{n}} = (1,0,0)$, restitution $e = 0.5$.

1. $v_n = (3-(-1)) = 4$ m/s (approaching).
2. $j = (1.5\times 4)/(0.5+1) = 4$ N·s.
3. $\mathbf{v}_1' = 3 - 4/2 = 1$ m/s; $\mathbf{v}_2' = -1 + 4/1 = 3$ m/s.
4. Checks: momentum before $2(3)+1(-1) = 5$, after $2(1)+1(3) = 5$ (conserved). Separation speed $3-1 = 2 = e\times 4$ (restitution respected).

Σ FORMULA · **Coulomb friction**

$$\|\mathbf{F}_t\| \le \mu\,\|\mathbf{F}_n\|$$

Friction opposes sliding with at most μ times the normal force. In impulse form, the tangential impulse is clamped to $\mu\,j$. A block rests on a slope while $\tan\theta \le \mu_s$.

WORKED EXAMPLE · **When does the crate slide?**

With $\mu = 0.5$: $\tan\theta = 0.5$, so $\theta = 26.6^\circ$. Tilt the ramp beyond 26.6° and the crate slides. In Rapier, contact friction combines the two colliders' coefficients (the average by default).

17.3 Constraints, solvers and stability

DEFINITION · **Constraint and joint**

A rule that removes freedom of motion: a contact (no penetration), a **fixed** joint (weld), a **revolute** joint (hinge: doors, wheels), a **prismatic** joint (slider: pistons, drawers), a **spherical** joint (ball and socket: ragdoll shoulders), a **rope** joint (maximum distance). Motors drive joints towards a target speed or position.

DEFINITION · **Iterative solver**

Engines solve all contacts and joints together by **iteration** (sequential impulses / projected Gauss–Seidel): fix each constraint in turn, repeat a few times per step. More iterations give stiffer stacks and chains but cost CPU; too few and tall stacks jitter and sink.

Problem	Cause	Fix
Objects pass through thin walls	**Tunnelling**: one step moves farther than the wall is thick	Continuous collision detection (`ccd`), thicker colliders, smaller step
Stacks jitter and topple	Too few solver iterations; huge mass ratios	More iterations, similar masses, sleeping
Things explode apart	Deep initial overlap, explicit integration, giant forces	Spawn without overlap, clamp forces, fixed step
Slow-motion or speed-up	Simulating with the variable frame delta	Fixed timestep + interpolation (Chapter 7)
Ragdolls feel floaty	Wrong scale (centimetres instead of metres)	Model in metres; gravity −9.81

WORKED EXAMPLE · **Tunnelling a bullet**

A projectile at 100 m/s simulated at 60 Hz moves $100/60 = 1.67$ m per step. A 0.1 m wall lies between two sampled positions and is never touched. **Continuous collision detection** sweeps the shape between steps and finds the time of impact; enable it only for small fast objects, because it costs more.

DEFINITION · **Sleeping**

Bodies that have been almost motionless for a while are put to **sleep**: skipped by the solver until something touches them. It is why a pile of 1,000 resting boxes costs little. Calling `applyImpulse(…, true)` (the `true` wakes the body) or moving a neighbour wakes them.

17.4 Physics engines for the web

Engine	Language	Strengths	three.js integration
Rapier	Rust → WASM	Fast, robust, deterministic option, character controller, CCD, joints	`@dimforge/rapier3d-compat`; `RapierPhysics` addon; `@react-three/rapier`
Jolt (JoltPhysics.js)	C++ → WASM	AAA engine (Horizon Forbidden West), very fast with many bodies, vehicles, soft bodies	`JoltPhysics` addon
Ammo.js	Bullet (C++) → asm.js/WASM	Mature, soft bodies	`AmmoPhysics` addon (older, larger)
cannon-es	JavaScript	Tiny and easy, pure JS	`@react-three/cannon`

Physics runs in its own **world**, separate from the three.js scene. Each frame you step the world and copy every dynamic body's position and rotation onto its mesh (the addons and `@react-three/rapier` do this for you).

three.js — Rapier directly JAVASCRIPT

```
import RAPIER from "@dimforge/rapier3d-compat";
await RAPIER.init();
const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });

world.createCollider(RAPIER.ColliderDesc.cuboid(10, 0.1, 10));                // static ground
(half extents)
const body = world.createRigidBody(RAPIER.RigidBodyDesc.dynamic().setTranslation(0, 4,
0).setCcdEnabled(true));
world.createCollider(RAPIER.ColliderDesc.ball(0.5).setRestitution(0.6).setFriction(0.5), body);

renderer.setAnimationLoop(() => {
  world.step();                                    // default fixed step of 1/60 s
  const p = body.translation(), q = body.rotation();
  ball.position.set(p.x, p.y, p.z);
  ball.quaternion.set(q.x, q.y, q.z, q.w);
  renderer.render(scene, camera);
});
body.applyImpulse({ x: 0, y: 5, z: 0 }, true);    // a jump; true wakes the body
```

R3F — @react-three/rapier JSX

```
import { Physics, RigidBody, CuboidCollider } from "@react-three/rapier";

function Ball() {
  const ref = useRef();
  return (
    <RigidBody ref={ref} colliders="ball" position={[0, 4, 0]} restitution={0.6} ccd
      onCollisionEnter={({ other }) => console.log("hit", other.rigidBodyObject?.name)}>
      <mesh castShadow onClick={() => ref.current.applyImpulse({ x: 0, y: 5, z: 0 }, true)}>
        <sphereGeometry args={[0.5, 32, 16]} />
        <meshStandardMaterial color="#f472b6" />
      </mesh>
```

```
    </RigidBody>
  );
}

export default function App() {
  return (
    <Canvas shadows camera={{ position: [6, 5, 8] }}>
      <Physics gravity={[0, -9.81, 0]} debug={false}>
        <Ball />
        <RigidBody type="fixed" colliders={false}>
          <CuboidCollider args={[10, 0.1, 10]} position={[0, -0.1, 0]} />
        </RigidBody>
        <CuboidCollider sensor args={[1, 1, 1]} position={[4, 1, 0]} onIntersectionEnter={() =>
console.log("goal!")} />
      </Physics>
    </Canvas>
  );
}
```

Body type	Moved by	Use
`dynamic`	Forces, impulses, collisions	Balls, crates, ragdolls
`fixed`	Nothing	Ground, walls
`kinematicPosition`	You, via `setNextKinematicTranslation/Rotation`	Moving platforms, doors, robot arms, tilting mazes
`kinematicVelocity`	You, via a velocity	Conveyor belts, elevators

Collider shapes	Cost	Notes
ball, cuboid, capsule, cylinder, cone	Cheapest	Approximate everything you can with these
convex hull (`colliders="hull"`)	Moderate	A tight convex wrapper; good for props
trimesh (`colliders="trimesh"`)	Expensive, static-only in practice	Exact level geometry; avoid on dynamic bodies
heightfield	Cheap for terrain	Grid of heights

TIP · Character controllers

Players should usually **not** be dynamic rigid bodies (they tip over, slide and bounce). Use a kinematic capsule with Rapier's `KinematicCharacterController` (`world.createCharacterController(0.01)` , then `computeColliderMovement(collider, desired)` and read `computedMovement()`), which slides along walls, climbs steps and snaps to slopes, then move the body with `setNextKinematicTranslation` .

17.5 Verlet cloth and soft constraints

Cloth, ropes and jelly are particles joined by distance constraints, simulated with Verlet integration and **position-based** constraint solving (Jakobsen's method, the basis of position-based dynamics).

FORMULA · Distance constraint relaxation

For two particles $\mathbf{p}_1, \mathbf{p}_2$ that should be L apart, with $\mathbf{d} = \mathbf{p}_2 - \mathbf{p}_1$:

$$\Delta = \frac{\|\mathbf{d}\| - L}{\|\mathbf{d}\|}\,\mathbf{d}, \qquad \mathbf{p}_1 \leftarrow \mathbf{p}_1 + \tfrac{1}{2}\Delta, \qquad \mathbf{p}_2 \leftarrow \mathbf{p}_2 - \tfrac{1}{2}\Delta$$

Repeat over all constraints several times per step; pinned particles do not move (give their share to the other particle).

WORKED EXAMPLE · Relaxing one stretched link

Rest length $L = 1$, particles at $(0,0,0)$ and $(1.2,0,0)$: $\|\mathbf{d}\| = 1.2$, $\Delta = (0.2/1.2)(1.2,0,0) = (0.2,0,0)$. Each moves 0.1 m towards the other: $(0.1,0,0)$ and $(1.1,0,0)$, exactly 1 m apart. With many links, satisfying one disturbs its neighbours, which is why several iterations are needed.

three.js — Verlet cloth core loop JAVASCRIPT

```
for (const p of particles) {                    // integrate
  if (p.pinned) continue;
  const vx = (p.x - p.px) * 0.99, vy = (p.y - p.py) * 0.99, vz = (p.z - p.pz) * 0.99;  // 1% damping
  p.px = p.x; p.py = p.y; p.pz = p.z;
  p.x += vx; p.y += vy + GRAVITY * dt * dt; p.z += vz;
}
for (let it = 0; it < 8; it++) for (const c of links) satisfy(c);   // relax constraints
for (let i = 0; i < particles.length; i++) pos.setXYZ(i, particles[i].x, particles[i].y, particles[i].z);
pos.needsUpdate = true; geometry.computeVertexNormals();
```

DEEP DIVE · GPU simulation

Thousands of particles or cloth vertices move to the GPU: in WebGL with ping-pong float render targets (`GPUComputationRenderer`), in WebGPU with compute shaders writing storage buffers (Chapter 20). The GPU updates every particle in parallel; the CPU never touches the data, and the vertex shader reads positions straight from the buffer or texture.

✓ KEY POINTS

- ✓ $\mathbf{F} = m\mathbf{a}$, $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$; engines integrate in fixed steps.
- ✓ Semi-implicit Euler (velocity first) is stable; explicit Euler gains energy; Verlet suits constraints.
- ✓ Impulse $j = (1 + e)v_n/(1/m_1 + 1/m_2)$ conserves momentum and applies restitution; friction clamps the tangential impulse to μj.
- ✓ Tunnelling: CCD for fast small objects; iterations and similar masses for stable stacks; sleeping for resting piles.
- ✓ Rapier (WASM) is the default web choice; `@react-three/rapier` gives `<Physics>`, `<RigidBody>`, colliders, sensors and events.
- ✓ Prefer primitive colliders; trimesh only for static level geometry; kinematic character controllers for players.

17.6 Interview questions

Q **Why do physics engines use a fixed time step?** MID}

A Integration error and constraint behaviour depend on the step size: a variable step changes how bouncy, stiff and stable the simulation is from frame to frame, makes results non-deterministic and can explode after a long frame. A fixed step (often 1/60 s) with an accumulator, plus interpolation of the rendered transforms between the last two states, keeps simulation consistent while rendering at any frame rate.

Q **What is the difference between explicit and semi-implicit Euler, and which would you use?** MID}

A Both update position and velocity from the acceleration once per step. Explicit Euler updates position with the **old** velocity; for oscillating systems this injects energy each step, so springs and orbits spiral outwards and eventually explode. Semi-implicit Euler updates velocity first and then position with the **new** velocity; it is symplectic, so energy stays bounded at the same cost. Game and real-time engines use semi-implicit Euler (or Verlet for constraint-heavy systems).

Q **A fast bullet passes straight through a wall. Why, and how do you fix it?** JUNIOR}

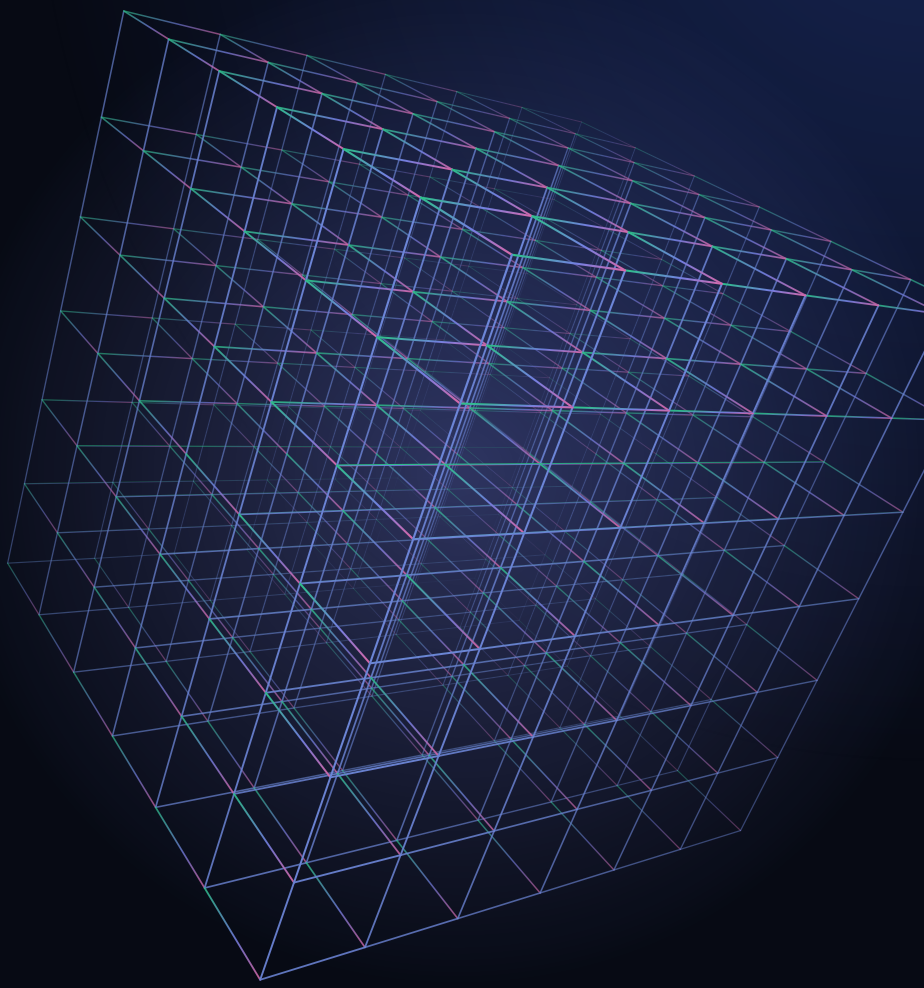
A Discrete collision detection only checks positions at each step. At 100 m/s and 60 Hz the bullet jumps 1.67 m per step, so a thin wall can lie entirely between two samples. Fixes: continuous collision detection (sweep the shape between steps and find the time of impact; `ccd` in Rapier), thicker colliders, a smaller step, or a raycast from the previous to the current position for bullets.

Q How would you build a player character with physics? SENIOR}

A Use a kinematic capsule rather than a dynamic body, driven by a character controller (Rapier's `KinematicCharacterController`): each step compute the desired movement from input, gravity and jumping, let the controller resolve collisions (slide along walls, climb steps up to a height, limit slope angles, snap to the ground), and apply the corrected translation with `setNextKinematicTranslation`. Interactions with dynamic objects can apply impulses explicitly. This gives responsive, predictable movement that a dynamic body (which tips, bounces and slides) cannot.

Q When would you use a trimesh collider, and why not always? MID}

A For static, complex level geometry where exact collision matters (terrain-like floors, architectural interiors). Trimesh collisions are expensive, are hollow (no inside), and are unreliable between two moving trimeshes, so dynamic objects should use primitives, compounds of primitives, or convex hulls. Simplified collision geometry is also much cheaper than the render mesh.



PART

Shaders and the GPU

Shaders and GLSL

18

Writing your own GPU programs. GLSL types and built-ins, how attributes, uniforms and varyings flow, ShaderMaterial versus RawShaderMaterial, a complete animated shader in three.js and R3F, the classic toolbox (smoothstep, fract, SDFs, smooth minimum, hash and fBm noise, Fresnel rim, dissolve, anti-aliased lines), extending built-in materials, and shader performance.

IN THIS CHAPTER

18.1 The shader mindset
18.2 How data reaches a shader
18.3 A complete shader
18.4 The shader toolbox
18.5 Extending built-in materials
18.6 Shader performance
18.7 Interview questions

18.1 The shader mindset

A fragment shader is a function that runs **for every pixel at once**, and cannot see its neighbours' results. You do not draw a circle by looping over pixels; you write, for one pixel, "how far am I from the centre, and therefore what colour am I?". Thinking per pixel (or per vertex) is the whole trick.

DEFINITION · **GLSL**

The OpenGL Shading Language, a C-like language for GPU programs. WebGL 2 uses **GLSL ES 3.00** (`#version 300 es` , `in` / `out` instead of `attribute` / `varying` ; three.js adds compatibility defines so `attribute` , `varying` and `gl_FragColor` still work in `ShaderMaterial`). WebGPU uses WGSL; three.js's TSL generates either (Chapter 20).

Type	Meaning
`float` , `int` , `uint` , `bool`	Scalars (write floats with a dot: `1.0` , not `1`)
`vec2` , `vec3` , `vec4` (`ivec` , `uvec` , `bvec`)	Vectors; components `.xyzw` , `.rgba` or `.stpq`
`mat2` , `mat3` , `mat4`	Column-major matrices; `M * v`
`sampler2D` , `samplerCube` , `sampler2DArray` , `sampler3D`	Textures, read with `texture(s, uv)`

Swizzling rearranges components freely: `v.xy` , `v.zyx` , `c.rgb * 0.5` , `vec4(p.xz, 0.0, 1.0)` . Operations are component-wise: `vec3 * vec3` multiplies each component.

Built-in	Does	Built-in	Does
`mix(a, b, t)`	Lerp	`step(e, x)`	0 if $x < e$, else 1
`smoothstep(e0, e1, x)`	Smooth 0→1 between edges	`clamp(x, lo, hi)`	Limit
`fract(x)`	$x - \lfloor x \rfloor$ (repeat)	`mod(x, y)`	Remainder
`length`, `distance`, `dot`, `cross`, `normalize`	Vector maths	`reflect(i, n)`, `refract(i, n, eta)`	Optics
`sin`, `cos`, `atan(y, x)`, `pow`, `exp`, `sqrt`	Maths	`dFdx`, `dFdy`, `fwidth`	Screen-space derivatives

18.2 How data reaches a shader

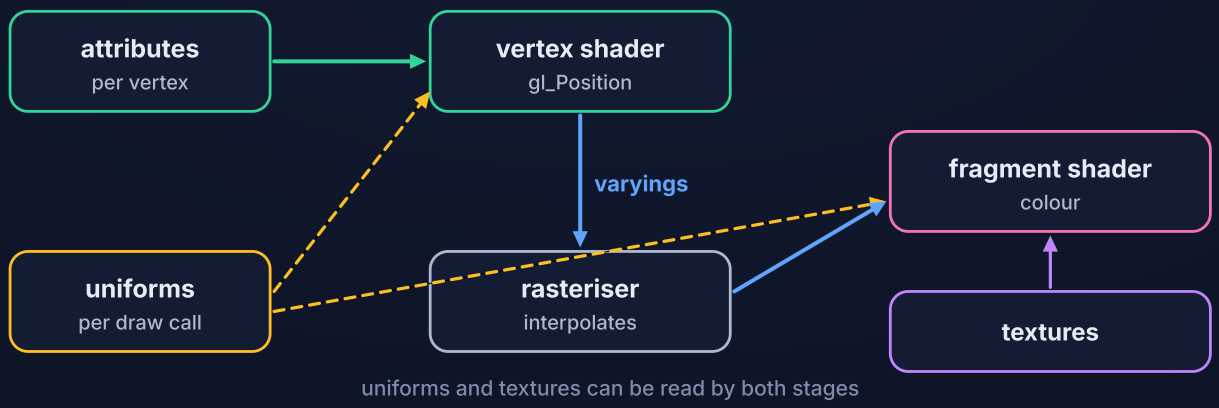


Figure 18.1 Data flow through a shader program. Attributes are per vertex, uniforms are the same for the whole draw call, varyings carry per-vertex values to the fragment shader (interpolated across the triangle), and textures are sampled anywhere.

DEFINITION · **Attribute, uniform, varying**

An **attribute** is per-vertex input from the geometry (`position`, `normal`, `uv`, your custom ones). A **uniform** is a value constant for a whole draw call, set from JavaScript (time, colours, matrices, textures). A **varying** is written by the vertex shader and read, **interpolated**, by the fragment shader.

	`ShaderMaterial`	`RawShaderMaterial`
Built-in declarations	Injected: `position`, `normal`, `uv`, `modelMatrix`, `modelViewMatrix`, `projectionMatrix`, `viewMatrix`, `normalMatrix`, `cameraPosition`	None: declare everything yourself
Version and precision	Added for you	You write `#version 300 es` and `precision`
Includes (`#include <common>`, fog, tone mapping chunks)	Available	Not available
Use	Most custom effects	Full control, minimal overhead

18.3 A complete shader

three.js — an animated wave with height-based colour JAVASCRIPT

```
const material = new THREE.ShaderMaterial({
  uniforms: {
    uTime: { value: 0 },
    uLow: { value: new THREE.Color("#1e3a8a") },
    uHigh: { value: new THREE.Color("#34d399") },
  },
  vertexShader: /* glsl */ `
    uniform float uTime;
    varying float vHeight;
    varying vec2 vUv;
    void main() {
      vec3 p = position;
      float h = sin(p.x * 3.0 + uTime) * 0.15 + sin(p.y * 4.0 + uTime * 1.3) * 0.1;
      p.z += h;                                    // the plane lies in XY before rotation
      vHeight = h;
      vUv = uv;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(p, 1.0);
    }`,
  fragmentShader: /* glsl */ `
    uniform vec3 uLow, uHigh;
    varying float vHeight;
    varying vec2 vUv;
    void main() {
      float t = smoothstep(-0.25, 0.25, vHeight);
      vec3 col = mix(uLow, uHigh, t);
      gl_FragColor = vec4(col, 1.0);
      #include <tonemapping_fragment>
      #include <colorspace_fragment>        // convert linear → output colour space like built-
ins
    }`,
});
const plane = new THREE.Mesh(new THREE.PlaneGeometry(4, 4, 128, 128), material);
plane.rotation.x = -Math.PI / 2;
renderer.setAnimationLoop((t) => { material.uniforms.uTime.value = t / 1000;
renderer.render(scene, camera); });
```

R3F — drei's shaderMaterial helper JSX

```jsx
import { shaderMaterial } from "@react-three/drei";
import { extend, useFrame } from "@react-three/fiber";

const WaveMaterial = shaderMaterial(
  { uTime: 0, uLow: new THREE.Color("#1e3a8a"), uHigh: new THREE.Color("#34d399") },
  vertexShaderSource,        // the same GLSL strings as above
  fragmentShaderSource
);
extend({ WaveMaterial });    // now usable as <waveMaterial />

function Wave() {
  const mat = useRef();
  useFrame((_, dt) => { mat.current.uTime += dt; });   // uniforms become properties
  return (
    <mesh rotation-x={-Math.PI / 2}>
      <planeGeometry args={[4, 4, 128, 128]} />
      <waveMaterial ref={mat} key={WaveMaterial.key} />   {/* key: hot-reload when the shader changes */}
    </mesh>
  );
}
```

⚠ PITFALL · **The wave and the raycaster disagree**

The displaced surface exists only on the GPU. Clicks hit the flat plane, and frustum culling uses the flat plane's bounding sphere (Chapter 14 and Chapter 6). Enlarge `geometry.boundingSphere` by the maximum displacement if edges disappear.

18.4 The shader toolbox

18.4.1 Shaping with step and smoothstep

Σ FORMULA · **smoothstep**

$$\text{smoothstep}(e_0, e_1, x) = t^2(3 - 2t), \qquad t = \text{clamp}\left(\frac{x - e_0}{e_1 - e_0}, 0, 1\right)$$

A soft threshold. `step` is the hard version and produces aliased (jagged) edges; `smoothstep` over about one pixel gives anti-aliased edges.

WORKED EXAMPLE · **smoothstep(0.4, 0.6, x)**

At $x = 0.45$: $t = 0.25$, so $0.25^2(3 - 0.5) = 0.156$. At $x = 0.5$: $t = 0.5$, result 0.5. At $x \geq 0.6$: 1.

18.4.2 Repetition with fract

`fract(uv * 10.0)` turns one 0–1 square into ten repeated squares; `floor(uv * 10.0)` gives each cell an integer ID (to randomise per cell). Combined with a shape function this gives grids, tiles, dots and stripes in one line each.

18.4.3 Signed distance functions

DEFINITION · Signed distance function (SDF)

A function returning the distance from a point to a shape's surface: negative inside, positive outside, zero on the boundary. Shapes become formulas; combining shapes becomes `min` (union), `max` (intersection) and `max(a, -b)` (subtraction). A smooth edge comes from `smoothstep` on the distance.

FORMULA · Basic 2D SDFs and smooth union

$$d_{\text{circle}}(\mathbf{p}) = \|\mathbf{p}\| - r \qquad d_{\text{box}}(\mathbf{p}) = \|\max(\mathbf{q}, 0)\| + \min(\max(q_x, q_y), 0), \quad \mathbf{q} = |\mathbf{p}| - \mathbf{b}$$

$$\text{smin}(a, b, k) = \min(a, b) - \frac{h^2 k}{4}, \qquad h = \frac{\max(k - |a - b|, 0)}{k}$$

$\mathbf{b}$ is the box's half size. The smooth minimum (Inigo Quilez's polynomial version) blends two shapes like melted wax within distance k.

Figure 18.2 Signed distance fields rendered per pixel: a circle, a box, their hard union, and their smooth union. Bands show distance contours; the white line is the zero contour (the surface).

WORKED EXAMPLE · Distances by hand

Circle of radius 0.25 at the origin, point $(0.3, 0.4)$: $d = 0.5 - 0.25 = 0.25$ (outside by 0.25). Smooth union of distances $a = 0.05$ and $b = 0.08$ with $k = 0.1$: $h = (0.1 - 0.03)/0.1 = 0.7$, so $\text{smin} = 0.05 - 0.49 \times 0.1/4 = 0.0378$: the blended surface bulges towards the point.

GLSL — an anti-aliased SDF badge GLSL

```glsl
float sdCircle(vec2 p, float r) { return length(p) - r; }
float sdBox(vec2 p, vec2 b) { vec2 q = abs(p) - b; return length(max(q, 0.0)) + min(max(q.x, q.y), 0.0); }
float smin(float a, float b, float k) { float h = max(k - abs(a - b), 0.0) / k; return min(a, b) - h * h * k * 0.25; }

void main() {
  vec2 p = vUv * 2.0 - 1.0;                          // centre the coordinates
  float d = smin(sdCircle(p - vec2(-0.3, 0.0), 0.35), sdBox(p - vec2(0.35, 0.0), vec2(0.25)), 0.2);
  float aa = fwidth(d);                              // one pixel's worth of distance
  float shape = 1.0 - smoothstep(-aa, aa, d);        // crisp at any zoom
  gl_FragColor = vec4(mix(vec3(0.05), vec3(0.2, 0.83, 0.6), shape), 1.0);
}
```

18.4.4 Noise

DEFINITION · **Hash, value noise, gradient noise and fBm**

A **hash** turns a coordinate into a pseudo-random number (`fract(sin(dot(p, vec2(12.9898, 78.233))) * 43758.5453)` is the classic one-liner). **Value noise** interpolates random values at grid points; **gradient (Perlin/simplex) noise** interpolates random gradients and looks less blocky. **Fractal Brownian motion (fBm)** sums several octaves of noise, each at double the frequency and half the amplitude, giving natural detail: clouds, terrain, marble, fire.

Σ FORMULA · **fBm**

$$\mathrm{fbm}(\mathbf{p}) = \sum_{i=0}^{N-1} a^i \,\mathrm{noise}\left(\lambda^i \mathbf{p}\right)$$

Usually gain $a = 0.5$ and lacunarity $\lambda = 2$. With 4 octaves the amplitudes are $1 + 0.5 + 0.25 + 0.125 = 1.875$, so divide by 1.875 to keep the result in range.

random value per cell | value noise (smoothed) | fBm, 4 octaves

Figure 18.3 From white noise to fBm. Random values per cell (blocky), smoothly interpolated value noise, and four octaves of fBm with natural multi-scale detail.

18.4.5 Fresnel rim and dissolve

Σ FORMULA · Fresnel-style rim light

$$\text{rim} = \left(1 - \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{v}})\right)^{p}$$

Bright where the surface turns away from the viewer: holograms, force fields, x-ray and selection highlights. With $\hat{\mathbf{n}} \cdot \hat{\mathbf{v}} = 0.2$ (near the silhouette) and $p = 3$: $0.8^3 = 0.51$.

GLSL — rim glow and noise dissolve (fragment) GLSL

```glsl
uniform float uProgress;           // 0 = solid, 1 = fully dissolved
varying vec3 vNormal;              // view-space normal from the vertex shader
varying vec3 vViewDir;             // view-space direction to the camera
void main() {
  float n = fbm(vUv * 6.0);
  if (n < uProgress) discard;                                  // burn away
  float edge = 1.0 - smoothstep(0.0, 0.05, n - uProgress);      // glowing edge band
  float rim = pow(1.0 - max(dot(normalize(vNormal), normalize(vViewDir)), 0.0), 3.0);
  vec3 col = vec3(0.1, 0.12, 0.2) + rim * vec3(0.3, 0.6, 1.0) + edge * vec3(1.0, 0.5, 0.1) *
4.0;
  gl_FragColor = vec4(col, 1.0);
}
```

18.4.6 Derivatives and anti-aliasing

DEFINITION · Screen-space derivatives

`dFdx(v)` and `dFdy(v)` give how much a value changes from one pixel to the next horizontally and vertically (the GPU shades pixels in 2×2 quads and differences neighbours). `fwidth(v) = |dFdx(v)| + |dFdy(v)|` is "one pixel's worth of `v`". Using it as the width of a `smoothstep` edge gives lines and shapes that stay one pixel soft at any distance.

GLSL — an infinite anti-aliased grid GLSL

```glsl
vec2 g = abs(fract(vWorldPos.xz - 0.5) - 0.5) / fwidth(vWorldPos.xz);   // distance to nearest
line, in pixels
float line = 1.0 - min(min(g.x, g.y), 1.0);                            // 1 on a line, fading
over 1 px
```

18.5 Extending built-in materials

Writing lighting yourself is a lot of work. Often you only want to change one thing (bend vertices, tint by height) and keep three.js's PBR lighting and shadows.

three.js — onBeforeCompile (WebGLRenderer) JAVASCRIPT

```
const mat = new THREE.MeshStandardMaterial({ color: "#3b7d4f" });
const uTime = { value: 0 };
mat.onBeforeCompile = (shader) => {
  shader.uniforms.uTime = uTime;
  shader.vertexShader = "uniform float uTime;\n" + shader.vertexShader.replace(
    "#include <begin_vertex>",
    `#include <begin_vertex>
     transformed.x += sin(uTime * 2.0 + position.y * 3.0) * 0.05 * position.y;   // sway grass
by height`
  );
};
mat.customProgramCacheKey = () => "grass-sway";   // so three.js does not mix it up with plain
Standard materials
```

TIP · CustomShaderMaterial and node materials

The `three-custom-shader-material` library extends any built-in material with your own GLSL in a supported way (and has an R3F component). On `WebGPURenderer`, the equivalent is assigning TSL nodes to `positionNode`, `colorNode`, `normalNode`, `emissiveNode` and friends of a node material; `onBeforeCompile` is not supported there.

18.6 Shader performance

Cost	Why	What to do
Fragment work × pixels	Runs for every covered pixel, including overdraw	Cheaper maths, lower resolution passes, fewer full-screen layers
Texture fetches	Memory bandwidth and latency	Pack channels, fewer lookups, mipmaps for cache efficiency
Divergent branches	A GPU runs groups of threads in lockstep; if neighbours take different branches, both run	Prefer `mix` / `step` arithmetic over `if` when cheap; branch on uniforms freely
`discard` / writing depth	Can disable early depth testing	Use only where needed (cut-outs, dissolve)
Precision	`highp` everywhere costs on some mobile GPUs	`mediump` for colours on mobile; `highp` for positions and UVs of large textures
Compile time	Big shaders stall the first frame they appear	`renderer.compile()` / `compileAsync()` during loading

TIP · Debugging shaders

Output intermediate values as colour: `gl_FragColor = vec4(vec3(vHeight * 0.5 + 0.5), 1.0);`, or normals as `n * 0.5 + 0.5`. Compile errors appear in the browser console with the full generated source and line numbers. Spector.js captures a frame and shows every draw call, its shader and its uniforms.

KEY POINTS

- ✓ Think per pixel/vertex: compute a value from position, never loop over neighbours.
- ✓ Attributes per vertex, uniforms per draw, varyings interpolated vertex → fragment.
- ✓ `ShaderMaterial` injects three.js built-ins; `RawShaderMaterial` gives you nothing.
- ✓ Add `#include <tonemapping_fragment>` and `<colorspace_fragment>` to match built-in output.
- ✓ Toolbox: `smoothstep` edges, `fract` repetition, SDFs with `min` / `max` / `smin`, hash/noise/fBm, Fresnel rims, `discard` dissolves, `fwidth` anti-aliasing.
- ✓ Extend built-ins with `onBeforeCompile` (WebGL) or node slots (WebGPU/TSL).
- ✓ Performance: pixels × instructions; avoid overdraw, divergent branches and unnecessary `discard`.

18.7 Interview questions

Q What is the difference between attributes, uniforms and varyings? JUNIOR}

A Attributes are per-vertex inputs read by the vertex shader from geometry buffers (position, normal, uv). Uniforms are constants for a whole draw call, set from JavaScript (time, colours, matrices, textures), readable in both shaders. Varyings are outputs of the vertex shader that the rasteriser interpolates across each triangle and passes to the fragment shader (UVs, normals, world positions).

Q How would you animate 50,000 grass blades swaying in the wind? MID}

A One `InstancedMesh` of a blade geometry (one draw call) with per-instance transforms, and the sway computed in the vertex shader from a time uniform, the instance's world position (so neighbours move together) and the vertex height (so roots stay fixed), for example a sum of sines or noise. Keep lighting by extending `MeshStandardMaterial` with `onBeforeCompile` or CustomShaderMaterial (or `positionNode` in TSL). The CPU only updates one uniform per frame.

Q **Why are edges drawn with step() jagged, and how do you fix it?** MID

A `step` switches from 0 to 1 between two adjacent pixels, so every pixel is either fully in or out: aliasing. Replace it with `smoothstep(-w, w, d)` where `w = fwidth(d)` is about one pixel's change of the distance value, giving a one-pixel soft edge at any zoom level. MSAA does not help here because it only antialiases triangle edges, not edges computed inside a fragment shader.

Q **What is a signed distance function and why are SDFs useful?** MID

A A function giving the distance from a point to a shape's surface, negative inside and positive outside. Shapes become small formulas; boolean operations become `min` / `max`; rounding, outlines, glows and shadows come from thresholds on the distance; smooth unions blend shapes organically; and the distance gives perfect resolution-independent anti-aliasing. In 3D, SDFs are rendered by raymarching (sphere tracing), and SDF text (MSDF fonts, troika-three-text) renders crisp text at any size.

Q **Your custom shader looks darker and more contrasty than the built-in materials next to it. Why?** JUNIOR

A It outputs linear colour directly to an sRGB canvas, skipping the renderer's tone mapping and output colour-space conversion that built-in materials apply. Add `#include <tonemapping_fragment>` and `#include <colorspace_fragment>` at the end of the fragment shader (or set `toneMapped: false` deliberately), and make sure input colours from `THREE.Color` uniforms are already linear (they are, with colour management on).

Render Targets and Post-processing

19

Rendering into textures, ping-pong buffers and multiple render targets, then the post-processing pipeline with EffectComposer (WebGL), pmndrs postprocessing (R3F) and RenderPipeline (WebGPU), and how the classic effects work inside: bloom, separable blur, SSAO/GTAO, anti-aliasing (MSAA, FXAA, SMAA, TAA), depth of field with the circle-of-confusion formula, outlines and colour grading.

IN THIS CHAPTER

19.1 Rendering into a texture
19.2 The post-processing pipeline
19.3 Inside the classic effects
19.4 Interview questions

19.1 Rendering into a texture

DEFINITION · **Render target**

An off-screen framebuffer (`WebGLRenderTarget` / `RenderTarget`) with a colour texture and optionally a depth (and stencil) buffer. Calling `renderer.setRenderTarget(rt)` makes the next `render()` draw into it instead of the canvas; the result `rt.texture` can then be used like any texture.

Use	How
Mirrors, water, portals	Render the scene from a mirrored or virtual camera into a target, map it onto a surface (`Reflector` , `Water` addons)
Security-camera screens, minimaps	A second camera into a target shown on a mesh
Shadow maps, transmission	three.js does it internally (Chapters 10 and 13)
Post-processing	Render the scene into a target, process it with full-screen passes
GPGPU (WebGL)	Store simulation state in float textures; update with shaders
Picking	Render object IDs into a small target (Chapter 14)

three.js — a TV screen showing another camera JAVASCRIPT

```
const rt = new THREE.WebGLRenderTarget(512, 512, {
  type: THREE.HalfFloatType,      // HDR-capable; UnsignedByteType for plain 8-bit
  samples: 4,                     // MSAA inside the target (WebGL 2)
  depthBuffer: true,
});
const tv = new THREE.Mesh(new THREE.PlaneGeometry(1.6, 0.9), new THREE.MeshBasicMaterial({ map:
rt.texture }));

renderer.setAnimationLoop(() => {
  tv.visible = false;                       // avoid rendering the screen into itself (a
feedback loop)
  renderer.setRenderTarget(rt);
  renderer.render(scene, securityCam);
  renderer.setRenderTarget(null);           // back to the canvas
  tv.visible = true;
  renderer.render(scene, camera);
});
```

⚠ PITFALL · **Reading and writing the same texture**

A shader may not sample the texture it is currently rendering into; the result is undefined (and WebGL reports a feedback loop). Effects that need the previous result, such as motion trails, blurs and simulations, use **two** targets and swap them every frame: **ping-pong** buffers.

DEFINITION · **Multiple render targets (MRT)**

Writing several outputs from one fragment shader in a single pass (colour, normals, depth, velocity, emissive). It is the basis of deferred shading's G-buffer and makes post-effects cheaper because the scene is drawn once. `WebGLRenderTarget` accepts a `count` option; in WebGPU, `pass()` combined with `mrt({ output, normal, velocity })` does this in TSL.

19.2 The post-processing pipeline

DEFINITION · **Post-processing**

Rendering the scene into a render target and then running one or more **full-screen passes** (fragment shaders over a screen-sized triangle) that read the image (and often the depth buffer) and write a modified image, until the last pass writes to the canvas.

Figure 19.1 A typical post-processing chain. The scene renders once into a half-float target; effects read colour and depth; tone mapping and the sRGB conversion happen once, at the end.

three.js — EffectComposer (WebGLRenderer) JAVASCRIPT

```
import { EffectComposer } from "three/addons/postprocessing/EffectComposer.js";
import { RenderPass } from "three/addons/postprocessing/RenderPass.js";
import { UnrealBloomPass } from "three/addons/postprocessing/UnrealBloomPass.js";
import { GTAOPass } from "three/addons/postprocessing/GTAOPass.js";
import { SMAAPass } from "three/addons/postprocessing/SMAAPass.js";
import { OutputPass } from "three/addons/postprocessing/OutputPass.js";

const composer = new EffectComposer(renderer);          // uses a HalfFloat target by default
composer.addPass(new RenderPass(scene, camera));
composer.addPass(new GTAOPass(scene, camera, innerWidth, innerHeight));
composer.addPass(new UnrealBloomPass(new THREE.Vector2(innerWidth, innerHeight), 0.8, 0.4,
0.9)); // strength, radius, threshold
composer.addPass(new SMAAPass());
composer.addPass(new OutputPass());                      // tone mapping + sRGB, once, at the
end
renderer.setAnimationLoop(() => composer.render());      // instead of renderer.render
addEventListener("resize", () => composer.setSize(innerWidth, innerHeight));
```

three.js — RenderPipeline (WebGPURenderer, TSL) JAVASCRIPT

```
import * as THREE from "three/webgpu";
import { pass } from "three/tsl";
import { bloom } from "three/addons/tsl/display/BloomNode.js";

const pipeline = new THREE.RenderPipeline(renderer);    // named PostProcessing before r183
const scenePass = pass(scene, camera);
const sceneColor = scenePass.getTextureNode("output");
pipeline.outputNode = sceneColor.add(bloom(sceneColor, 1.2, 0.4, 0.6));   // strength, radius,
threshold
renderer.setAnimationLoop(() => pipeline.render());     // tone mapping + sRGB applied
automatically at the end
```

R3F — @react-three/postprocessing (WebGL) JSX

```jsx
import { EffectComposer, Bloom, N8AO, SMAA, Vignette, ToneMapping } from "@react-three/postprocessing";

<Canvas gl={{ antialias: false }}>        {/* the composer does its own anti-aliasing */}
  <Scene />
  <EffectComposer multisampling={4}>
    <N8AO aoRadius={0.6} intensity={3} />
    <Bloom mipmapBlur luminanceThreshold={1} intensity={1.2} />
    <Vignette offset={0.25} darkness={0.6} />
    <SMAA />
  </EffectComposer>
</Canvas>
```

TIP · **Why pmndrs postprocessing is faster**

The pmndrs `postprocessing` library (behind `@react-three/postprocessing`) **merges** compatible effects into a single full-screen pass where possible, instead of one pass per effect as `EffectComposer` does. Every full-screen pass reads and writes every pixel, so at 4K with 6 passes you move gigabytes of memory per second; merging saves most of that.

WORKED EXAMPLE · **The cost of full-screen passes**

A 2560 × 1440 canvas at DPR 1 is 3.69 M pixels. Each pass reads and writes a half-float RGBA texel (8 bytes each way): about $3.69\text{M} \times 16 = 59$ MB of memory traffic per pass per frame. Six passes at 60 Hz: about **21 GB/s**, a large share of an integrated GPU's bandwidth. Hence: fewer passes, merged effects, and half-resolution buffers for blurry effects (bloom, AO, DOF).

19.3 Inside the classic effects

19.3.1 Gaussian and separable blur

FORMULA · **Gaussian weights**

$$w(x) = \frac{1}{\sqrt{2\pi}\,\sigma}\, e^{-x^2/(2\sigma^2)}$$

A 2D Gaussian is **separable**: blurring horizontally then vertically equals one 2D blur. An $N \times N$ kernel then costs $2N$ samples instead of N^2.

WORKED EXAMPLE · **A 9-tap blur**

$\sigma = 2$, taps $x = -4 \ldots 4$, normalised weights:
$0.028,\ 0.066,\ 0.124,\ 0.180,\ \mathbf{0.204},\ 0.180,\ 0.124,\ 0.066,\ 0.028$ (they sum to 1). A full 9×9 kernel needs 81 texture reads per pixel; two separable passes need $9 + 9 = 18$. Linear filtering can halve that again by sampling between two texels with a combined weight.

19.3.2 Bloom

DEFINITION · Bloom

The glow that spreads around very bright light (from lens and eye imperfections). Algorithm: (1) **bright pass**: keep only pixels brighter than a threshold (in HDR, above 1.0 means "brighter than white"); (2) **downsample** into a mip chain (½, ¼, ⅛... resolution), blurring each level; (3) **upsample** and add the levels back together; (4) **add** the result to the original image.

smaller levels cover larger screen distances for the same cost

Figure 19.2 Bloom as a mip chain. Each level is half the size of the previous, so blurring it covers twice the screen distance for the same cost. Adding the levels back together gives a wide, natural glow.

TIP · Selective glow without masks

In an HDR pipeline only pixels above the threshold glow. Set `luminanceThreshold` to 1 and give the things that should glow `emissiveIntensity` above 1 (with `toneMapped: true`): neon tubes bloom, white paper does not, with no layers or selection masks needed.

19.3.3 Ambient occlusion: SSAO and GTAO

DEFINITION · Screen-space ambient occlusion

An approximation of how much ambient light reaches each point, darkening creases, corners and contact areas. For each pixel, sample points in a hemisphere around its normal (reconstructed from the depth buffer) and count how many are hidden behind nearby geometry in the depth buffer: $\mathrm{AO} = 1 - \frac{\text{occluded}}{N}$. **GTAO** (ground-truth AO) instead searches horizon angles in a few directions and integrates the visible arc analytically, giving more physically correct results for similar cost. Both need a blur or denoise because they use few samples per pixel.

WORKED EXAMPLE · AO from samples

16 hemisphere samples, 6 of them end up behind the depth buffer: $\mathrm{AO} = 1 - 6/16 = 0.625$, so ambient lighting at that pixel is multiplied by 0.625. Since r181, three.js's GTAO stores AO in the red channel, and r185 made it more physically correct (darker and wider), so lower `radius` and `scale` after upgrading.

19.3.4 Anti-aliasing

Method	How	Cost	Quality and caveats
MSAA (`antialias: true`, `samples`)	Several coverage samples per pixel at triangle edges; shading once per pixel	Memory × samples; cheap shading	Great geometric edges; does nothing for shader aliasing or specular shimmer
FXAA	One pass finds high-contrast edges in the final image and blurs along them	Very cheap	Slightly blurry, misses subpixel detail
SMAA	Detects edge shapes (lines, corners) with lookup textures, blends precisely	Cheap–moderate	Sharper than FXAA; no temporal stability
TAA / TRAA	Jitter the camera by a subpixel offset each frame; blend with the reprojected history	Moderate (needs velocity buffer)	Best quality and stability; risk of ghosting and blur on motion
Supersampling (SSAA)	Render at higher resolution and downsample	Very expensive	Reference quality

DEFINITION · **Temporal anti-aliasing**

Each frame the projection is shifted by a different subpixel offset (from a low-discrepancy sequence such as Halton), so over several frames every pixel is sampled at many positions. The current frame is blended with the previous result, **reprojected** using per-pixel motion vectors; history colours are clamped to the current neighbourhood's colour range to reject stale data (otherwise you see ghosting). TAA also cleans up noisy effects such as `alphaHash`, SSR and AO. three.js provides `TRAANode` for WebGPU, and r184 added FSR 1 and TAAU upscaling nodes.

19.3.5 Depth of field

FORMULA · **Circle of confusion**

$$c = \frac{A\, f\, |S_2 - S_1|}{S_2\,(S_1 - f)}, \qquad A = \frac{f}{N}$$

A point at distance S_2 when the lens of focal length f and f-number N is focused at S_1 images not as a point but as a blur disc of diameter c on the sensor. DOF effects compute c per pixel from the depth buffer and blur by that amount (bokeh).

WORKED EXAMPLE · **How blurry is the background?**

A 50 mm lens at f/1.8 ($A = 27.8$ mm), focused on a face at $S_1 = 2$ m; the wall behind is at $S_2 = 4$ m. $c = \frac{0.0278 \times 0.05 \times 2}{4 \times 1.95} = 0.000356$ m $= 0.36$ mm on a 36 mm-wide sensor. Rendered 1920 px wide, that is $0.356/36 \times 1920 \approx 19$ px of blur diameter. Stopping down to f/8 shrinks it to about 4 px.

19.3.6 Outlines and edge detection

Selection outlines are drawn by (a) rendering the selected objects' mask or depth, then (b) running an edge detector such as the **Sobel** operator, which convolves the image with two 3×3 kernels to find horizontal and vertical gradients:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad G_y = G_x^T, \quad \|G\| = \sqrt{G_x^2 + G_y^2}$$

Applying Sobel to depth and normals of the whole scene gives a hand-drawn "ink" look. drei's `<Outlines>` takes a different route: the inverted hull (a slightly enlarged back-face copy of the mesh).

19.3.7 Colour grading and finishing touches

- **LUT colour grading**: a 3D lookup texture (exported from Photoshop, DaVinci Resolve) maps every input colour to a graded colour in one texture fetch (`LUTPass`, `lut3D` node, `<LUT>` in R3F).
- **Vignette**, **film grain**, **chromatic aberration**, **lens flares**: cheap single-pass effects; apply after tone mapping where they are meant to look photographic.
- **Motion blur**: blur along the per-pixel velocity buffer.
- **SSR** (screen-space reflections): march the reflected ray through the depth buffer; misses anything off-screen.

✓ KEY POINTS

- ✓ Render targets: `setRenderTarget(rt)` → render → `setRenderTarget(null)`; never sample the target you write; ping-pong instead.
- ✓ MRT writes several buffers in one pass (G-buffer, velocity, normals).
- ✓ WebGL: `EffectComposer` + `RenderPass` + effects + `OutputPass`; R3F: `@react-three/postprocessing` merges effects; WebGPU: `RenderPipeline` + `pass()` + TSL effect nodes.
- ✓ Each full-screen pass costs bandwidth: merge passes, use half resolution for blurry effects.
- ✓ Blur is separable ($2N$ vs N^2 samples); bloom = bright pass + mip-chain blur + add.
- ✓ AO: hemisphere samples (SSAO) or horizon search (GTAO), then denoise.
- ✓ AA: MSAA for edges, FXAA/SMAA post, TAA for the best stability (watch ghosting).
- ✓ DOF blur diameter from the circle of confusion $c = Af|S_2 - S_1|/(S_2(S_1 - f))$.

19.4 Interview questions

Q How would you make only neon signs glow, not the whole scene? MID}

A Render in HDR (half-float targets, tone mapping at the end) and make the neon materials emissive with intensity well above 1, then set the bloom threshold around 1. Only pixels brighter than white pass the bright filter, so ordinary white surfaces do not bloom. Alternatives: selective bloom with layers or `<SelectiveBloom>`, rendering the glowing objects into a separate mask, at the cost of extra passes.

Q Why does adding EffectComposer make my edges jagged, and how do you fix it? MID}

A `antialias: true` on the renderer only applies MSAA to the default framebuffer, but with post-processing the scene is drawn into an off-screen target that has no multisampling by default. Fix it by giving the composer's render target `samples: 4` (or `multisampling` on the R3F `EffectComposer`), or by adding an SMAA/FXAA pass at the end, or TAA for the best quality.

Q Explain how a Gaussian blur is made cheap. MID}

A The 2D Gaussian kernel is separable into the product of two 1D kernels, so a blur is done as a horizontal pass followed by a vertical pass: $2N$ samples per pixel instead of N^2. Further savings: use bilinear filtering to fetch two weighted texels per sample, blur at half or quarter resolution (blurry results do not need full resolution), and for very wide blurs use a mip chain (as in bloom) or a dual-filter downsample/upsample.

Q What is TAA and what are its trade-offs? SENIOR}

A Temporal anti-aliasing jitters the projection by a different subpixel offset every frame and accumulates samples over time, reprojecting the history with motion vectors so moving objects stay aligned. It produces very smooth, stable edges and also denoises stochastic effects (AO, SSR, dithered transparency). Its costs are a velocity buffer, a history buffer, and artefacts: ghosting on disoccluded areas or fast motion (reduced with neighbourhood colour clamping), and a slightly soft image (often countered with sharpening). It also requires every effect to provide correct motion vectors, including skinned and shader-animated objects.

Q How does SSAO work, and why is it noisy? SENIOR}

A For each pixel it reconstructs the view-space position and normal from the depth (and normal) buffer, then tests a small number of sample points in a hemisphere oriented around the normal: each sample is projected to the screen and compared with the depth buffer; samples behind the stored depth (within a range check) count as occluded. AO is one minus the occluded fraction. With only 8–32 samples per pixel the estimate is noisy, so implementations randomise sample rotations per pixel and then blur or denoise (depth-aware), or accumulate over time with TAA.

WebGPU, TSL and Compute

20

How WebGPU is organised (adapter, device, pipelines, bind groups, command encoders), what WebGPURenderer changes in three.js, writing materials and effects in TSL (three.js's shading language in JavaScript), compute shaders with storage buffers and workgroup maths, a 100,000-particle simulation, a migration checklist from WebGL, and WebGPU in R3F.

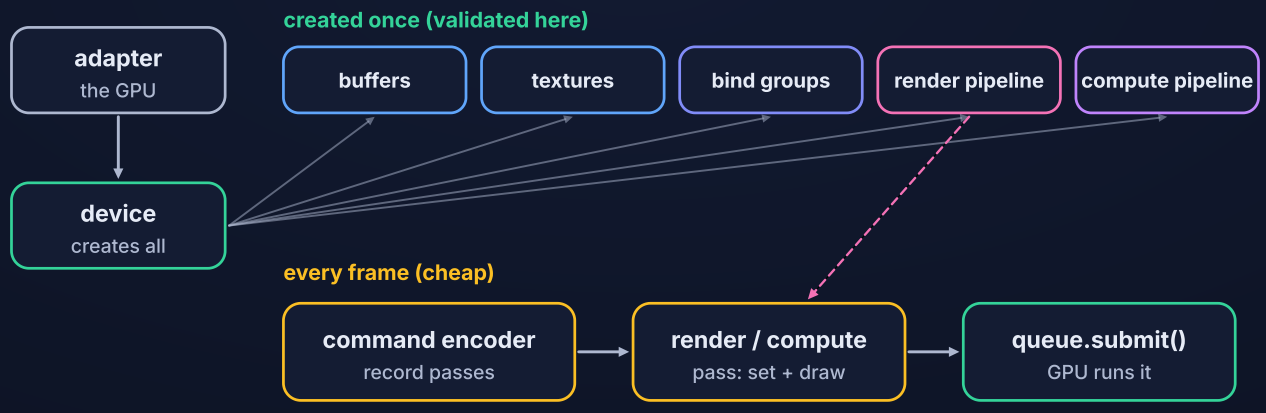
IN THIS CHAPTER

20.1 How WebGPU is organised
20.2 WebGPURenderer in three.js
20.3 TSL: the three.js shading language
20.4 Compute shaders
20.5 Migrating a project from WebGL
20.6 WebGPU in React Three Fiber
20.7 Interview questions

20.1 How WebGPU is organised

DEFINITION · **WebGPU objects**

Adapter: a physical GPU (and driver). **Device**: your logical connection to it, which creates everything else. **Buffers** and **textures**: GPU memory. **Bind groups**: bundles of resources (uniform buffers, storage buffers, textures, samplers) handed to a shader together. **Pipelines**: immutable, pre-validated objects combining shaders with fixed state (blending, depth, vertex layout), one per material variant. **Command encoder**: records render and compute passes; the finished command buffer is submitted to the device's **queue**.



three.js creates and caches pipelines per material variant; you never touch these objects directly

Figure 20.1 The WebGPU object model. Expensive validation happens once when pipelines and bind groups are created; per frame, the CPU only records and submits cheap commands.

The design moves work from every draw call (WebGL validates state on each call) to creation time, which is why WebGPU can issue more draw calls per millisecond, and why **pipeline creation** (shader compilation) is the moment to watch for hitches.

WGSL — a compute shader, for reference WGSL

```wgsl
@group(0) @binding(0) var<storage, read_write> pos : array<vec3f>;
@group(0) @binding(1) var<uniform> dt : f32;

@compute @workgroup_size(64)
fn main(@builtin(global_invocation_id) id : vec3u) {
  if (id.x ≥ arrayLength(&pos)) { return; }       // the last workgroup may run past the end
  pos[id.x].y -= 9.81 * dt;
}
```

You rarely write WGSL by hand with three.js: TSL generates it (or GLSL for the WebGL 2 fallback).

20.2 WebGPURenderer in three.js

Topic	WebGLRenderer	WebGPURenderer
Import	`three`	`three/webgpu` (+ `three/tsl` for nodes)
Initialisation	Synchronous	Asynchronous: `await renderer.init()` or `setAnimationLoop`
Backends	WebGL 2	WebGPU, automatic WebGL 2 fallback (`forceWebGL: true` to test)
Built-in materials	Yes	Yes (converted to node materials internally)
Custom shaders	`ShaderMaterial`, `onBeforeCompile`	Node materials + TSL only
Post-processing	`EffectComposer`	`RenderPipeline` + TSL nodes (MRT built in)
Compute	No	`renderer.compute(node)`
Many lights	Forward, all lights per fragment	Clustered (Forward+) lighting since r185
WebXR	Yes	Yes, since r185
Depth	Standard; reversed optional	Reversed depth supported (r183)

⚠ PITFALL · **Not everything ports automatically**

Custom `ShaderMaterial`s, `onBeforeCompile` hacks, `EffectComposer` passes and libraries built on them (including `@react-three/postprocessing`) do not run on `WebGPURenderer`. Move them to TSL. Built-in materials, loaders, controls, animation and physics work unchanged. WebGPU is not automatically faster: it shines with many draw calls and compute, so profile before migrating for speed alone.

20.3 TSL: the three.js shading language

DEFINITION · **TSL (Three.js Shading Language)**

A way to write shaders as JavaScript function calls that build a **node graph**. Nothing runs in JavaScript at draw time: three.js compiles the graph to WGSL (WebGPU) or GLSL (WebGL 2). Because it is JavaScript, shader code can be composed, imported, parameterised and tree-shaken like any other module, and the same code runs on both backends.

GLSL	TSL
`float a = 1.0;`	`const a = float(1)`
`vec3 c = vec3(1, 0, 0);`	`const c = vec3(1, 0, 0)` or `color("#ff0000")`
`a + b`, `a * b`, `a / b`	`a.add(b)`, `a.mul(b)`, `a.div(b)`
`x += y;`	`x.addAssign(y)` (inside `Fn`)
`mix(a, b, t)`, `smoothstep(e0, e1, x)`	`mix(a, b, t)`, `smoothstep(e0, e1, x)` (also as methods)
`uniform float uTime;`	`const uTime = uniform(0)`; set `uTime.value` from JS
`texture(map, vUv)`	`texture(map, uv())`
`if (c) {…}`	`If(c, () ⇒ {…})` inside `Fn`
`for (…)`	`Loop(count, ({ i }) ⇒ {…})`
a GLSL function	`const f = Fn(([a, b]) ⇒ { … return … })`
`gl_FragColor = …`	`material.colorNode = …` / `material.outputNode = …`
varyings	automatic: a vertex-stage value used in the fragment stage becomes a varying (or use `varying(node)`)

Useful built-in nodes: `positionLocal`, `positionWorld`, `normalLocal`, `normalWorld`, `uv()`, `time`, `cameraPosition`, `screenUV`, `instanceIndex`, `vertexIndex`, `modelWorldMatrix`, plus noise (`mx_noise_float`, `hash`), and effect helpers.

20.3.1 Node material slots

Slot	Replaces	Example
`colorNode`	Base colour	Gradients, procedural patterns, texture blends
`positionNode`	Vertex position (local space)	Waves, wind, morphing, billboards
`normalNode`	Shading normal	Procedural bumps
`emissiveNode`, `roughnessNode`, `metalnessNode`, `opacityNode`	Those PBR inputs	Glowing patterns, wet streaks
`outputNode` / `fragmentNode`	The final colour / the whole fragment stage	Full custom shading

three.js (TSL) — a lit material with procedural colour and a vertex wave — JAVASCRIPT

```
import * as THREE from "three/webgpu";
import { color, mix, uv, sin, time, positionLocal, normalLocal } from "three/tsl";

const material = new THREE.MeshStandardNodeMaterial({ roughness: 0.3, metalness: 0.2 });
material.colorNode = mix(color("#4f46e5"), color("#10b981"), uv().y);          // fragment stage
const wave = sin(positionLocal.y.mul(8).add(time.mul(3))).mul(0.08);
material.positionNode = positionLocal.add(normalLocal.mul(wave));               // vertex stage
// PBR lighting, shadows, fog and tone mapping still come from three.js
```

WORKED EXAMPLE · What that graph computes for one vertex

At $t = 0.5$ s, a vertex with local $y = 0.2$ and normal $(0, 0, 1)$: $\sin(0.2 \times 8 + 0.5 \times 3) = \sin(3.1) = 0.0416$, times 0.08 gives 0.0033, so the vertex moves 3.3 mm outwards along its normal. Every vertex evaluates the same graph in parallel on the GPU.

NEW IN RECENT THREE.JS · TSL changes to know (r181–r186)

`PI2` was renamed `TWO_PI` (r181). `directionToColor` / `colorToDirection` became `packNormalToRGB` / `unpackRGBToNormal` (r185). In a `positionNode`, `positionLocal` no longer includes skinning; use `positionGeometry` for the raw vertex (r185). `rangeFog` and `viewportResolution` were removed (use `fog(color, rangeFogFactor(near, far))` and `screenSize`) in r186. Node materials and TSL also run on `WebGLRenderer` through a compatibility layer since r184.

20.4 Compute shaders

DEFINITION · Compute shader

A GPU program that does not draw: it runs a function once for each element of a problem (each particle, pixel, cell, boid), in parallel, reading and writing **storage buffers** or textures. The results stay on the GPU and can be drawn directly, so the CPU never touches the data.

DEFINITION · Workgroups and invocations

A compute dispatch is split into **workgroups** of a fixed size (often 64 invocations). Each invocation gets a global index (`instanceIndex` in TSL, `global_invocation_id` in WGSL). Invocations within a workgroup can share fast workgroup memory and synchronise with barriers; different workgroups cannot.

WORKED EXAMPLE · Dispatching 100,000 particles

With a workgroup size of 64 (three.js's default), $\lceil 100{,}000/64 \rceil = 1{,}563$ workgroups run $1{,}563 \times 64 = 100{,}032$ invocations; the last 32 must do nothing (an index check, which three.js adds for you). Each particle stores position, velocity and colour as `vec3` (16 bytes each with padding): $100{,}000 \times 48$ bytes $= 4.8$ MB of storage buffers, all GPU-resident.

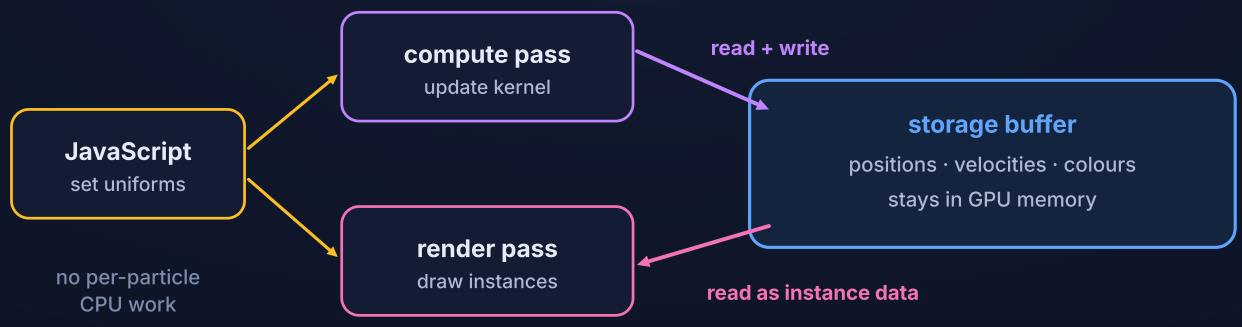


Figure 20.2 The compute loop. Each frame JavaScript only updates a few uniforms and asks the GPU to run the update kernel; the render pass then reads the same storage buffer as instance positions.

three.js (TSL) — GPU particles: init once, update every frame JAVASCRIPT

```
import { Fn, If, uniform, float, vec3, hash, instancedArray, instanceIndex } from "three/tsl";

const COUNT = 100_000;
const positions = instancedArray(COUNT, "vec3");          // storage buffers, GPU only
const velocities = instancedArray(COUNT, "vec3");
const gravity = uniform(-0.00098), bounce = uniform(0.8);

const init = Fn(() => {
  const p = positions.element(instanceIndex);
  p.x = hash(instanceIndex).sub(0.5).mul(40);
  p.z = hash(instanceIndex.add(1)).sub(0.5).mul(40);
  p.y = hash(instanceIndex.add(2)).mul(8);
```

```
})().compute(COUNT);

const update = Fn(() => {
  const p = positions.element(instanceIndex), v = velocities.element(instanceIndex);
  v.addAssign(vec3(0, gravity, 0));
  p.addAssign(v);
  If(p.y.lessThan(0), () => { p.y = 0; v.y = v.y.negate().mul(bounce); });
})().compute(COUNT);

renderer.compute(init);                                  // once
const sprites = new THREE.SpriteNodeMaterial();
sprites.positionNode = positions.toAttribute();          // read the buffer per instance
const particles = new THREE.Sprite(sprites);
particles.count = COUNT;
scene.add(particles);
renderer.setAnimationLoop(() => { renderer.compute(update); renderer.render(scene, camera); });
```

TIP · **The WebGL fallback**

When the browser has no WebGPU, three.js runs simple compute kernels on WebGL 2 with **transform feedback**, so many examples still work, more slowly and with limits (no atomics or workgroup memory). Check `renderer.backend.isWebGPUBackend` and reduce particle counts on the fallback.

Compute use case	Idea
Particles, sparks, rain, galaxies	Integrate positions and velocities per particle
Flocking (boids)	Separation, alignment, cohesion; spatial hashing for neighbours
Cloth and soft bodies	Verlet + distance constraints on the GPU
Fluids (SPH, grid-based)	Neighbour sums, pressure solve, advection
GPU culling and LOD	Test instances against the frustum, write a compact visible list for indirect draws
Image processing	Blur, histograms (atomics), tone curves on textures
Skinning, morphing	Pre-transform vertices once, reuse across passes
ML inference	Matrix multiplications in compute (Transformers.js, ONNX Runtime Web use WebGPU)

20.5 Migrating a project from WebGL

MIGRATION CHECKLIST

1. Swap imports to `three/webgpu` and initialise with `setAnimationLoop` or `await renderer.init()`
2. Run the app: built-in materials, loaders, controls and animation should already work
3. Port each `ShaderMaterial` / `onBeforeCompile` to a node material with TSL slots
4. Replace `EffectComposer` passes with a `RenderPipeline` and TSL effect nodes (bloom, GTAO, SMAA/TRAA, DOF…)
5. Replace `WebGLCubeRenderTarget` with `CubeRenderTarget`; remove `PCFSoftShadowMap`; re-tune shadow bias
6. Pre-compile with `compileAsync()` / `compileComputeAsync()` during loading to avoid hitches
7. Profile both backends (timestamp queries, the Inspector addon) before declaring victory

20.6 WebGPU in React Three Fiber

R3F — WebGPURenderer with node materials — JSX

```jsx
import * as THREE from "three/webgpu";
import { color, mix, uv } from "three/tsl";
import { Canvas, extend } from "@react-three/fiber";
import { useMemo } from "react";

extend(THREE);                        // register every three/webgpu class (node materials
included) as JSX

function Knot() {
  const colorNode = useMemo(() => mix(color("#4f46e5"), color("#10b981"), uv().y), []);
  return (
    <mesh>
      <torusKnotGeometry args={[0.8, 0.28, 256, 48]} />
      <meshStandardNodeMaterial colorNode={colorNode} roughness={0.3} />
    </mesh>
  );
}

export default function App() {
  return (
    <Canvas gl={async (props) => {              // R3F 9: async factory receives default props
      const renderer = new THREE.WebGPURenderer(props);
      await renderer.init();
      return renderer;
    }}>
      <ambientLight intensity={0.5} />
      <directionalLight position={[2, 3, 4]} intensity={2.5} />
```

```
      <Knot />
    </Canvas>
  );
}
```

For compute in R3F, build the buffers and kernels in `useMemo`, run the init kernel once the renderer is ready (`useEffect(() => gl.compute(init), [gl])`), and dispatch the update kernel in `useFrame((state) => state.gl.compute(update))` . For post-processing, build a `RenderPipeline` in `useMemo` and render it from a `useFrame` with **priority 1**, which tells R3F not to render the scene itself.

✓ KEY POINTS

- ✓ WebGPU: adapter → device → buffers/textures → bind groups → pipelines → command encoder → queue; validation happens at creation.
- ✓ `WebGPURenderer` (`three/webgpu`): async init, automatic WebGL 2 fallback, node materials, TSL, compute, `RenderPipeline` , clustered lighting, WebXR.
- ✓ TSL builds a node graph in JavaScript that compiles to WGSL or GLSL; maths is method chaining (`a.mul(b).add(c)`), logic uses `Fn` , `If` , `Loop` .
- ✓ Material slots: `colorNode` , `positionNode` , `normalNode` , `emissiveNode` , `opacityNode` , `outputNode` .
- ✓ Compute: `instancedArray` storage buffers + `Fn(...)().compute(count)` + `renderer.compute()` ; workgroups of 64; data stays on the GPU.
- ✓ `ShaderMaterial` , `onBeforeCompile` and `EffectComposer` must be ported; profile before migrating for speed.
- ✓ R3F 9: async `gl` factory, `extend(THREE)` from `three/webgpu` , `useFrame` for compute and pipelines.

20.7 Interview questions

Q What does WebGPU offer over WebGL? MID

A Lower CPU overhead per draw (state is baked into pre-validated pipeline objects and resources into bind groups, and commands are recorded and submitted in batches), compute shaders with storage buffers and atomics for general GPU work, a modern shading language (WGSL), better-defined resource management, and features like timestamp queries and indirect drawing. In three.js this enables GPU particles and simulations, GPU culling, clustered lighting and a node-based post-processing stack. It is not automatically faster for every scene.

Q **What is TSL and why did three.js create it?** MID}

A TSL is three.js's shading language: you compose shader logic with JavaScript functions that build a node graph, which three.js compiles to WGSL for WebGPU or GLSL for WebGL. It lets one shader source target both backends, makes shaders composable and type-checked like normal modules (import a noise function, parameterise it, tree-shake it), and lets you override just one part of a material (`colorNode` , `positionNode`) while keeping three.js's lighting, shadows and fog, which replaces fragile string-replacement hacks like `onBeforeCompile` .

Q **How would you simulate a million particles in the browser?** SENIOR}

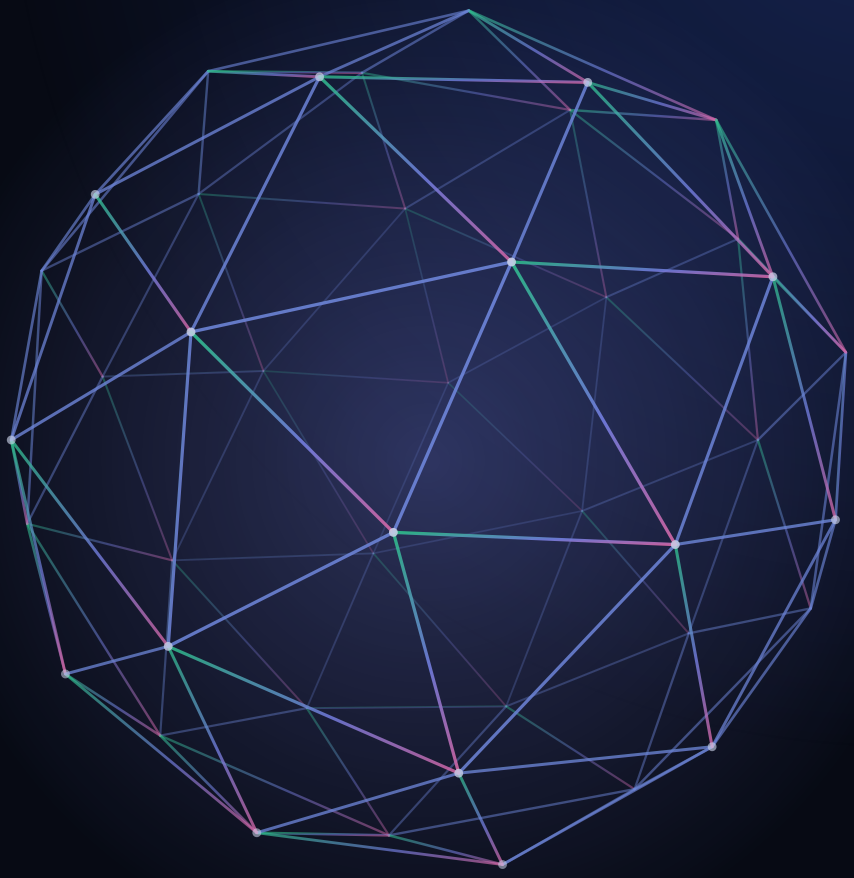
A Keep the data on the GPU. Store positions and velocities in storage buffers (`instancedArray`), write an init and an update compute kernel in TSL (forces, collisions, noise fields), dispatch the update each frame with `renderer.compute()` (about 15,600 workgroups of 64), and render the particles as instanced sprites or points whose vertex stage reads the same buffer (`positions.toAttribute()`). The CPU only updates a few uniforms (time, mouse). Use additive blending without depth writes for glowing particles, small sprites to limit overdraw, and fall back to fewer particles on the WebGL backend.

Q **A WebGL app uses ShaderMaterial and EffectComposer. What does moving to WebGPURenderer involve?** SENIOR}

A Switch imports to `three/webgpu` and handle async initialisation. Port each `ShaderMaterial` to a node material: vertex logic to `positionNode` , colour logic to `colorNode` or `outputNode` , uniforms to `uniform()` nodes, GLSL functions to `Fn` . Port `onBeforeCompile` injections to the matching node slots. Rebuild post-processing as a `RenderPipeline` from `pass()` and TSL effect nodes. Replace `WebGLCubeRenderTarget` , re-tune shadow bias, pre-compile pipelines, then profile both backends to confirm the move is worth it.

Q **Why might the first frame that shows a new material stutter on WebGPU, and what can you do?** MID}

A A new material variant needs a new render pipeline, which means generating and compiling its shader and creating the pipeline object, potentially taking tens of milliseconds. Pre-warm during loading with `renderer.compileAsync(scene, camera)` (non-blocking since r184) and `compileComputeAsync()` for compute kernels (r186), reuse materials instead of creating variants at runtime, and avoid toggling features that change the shader (switch between pre-built materials instead).



PART

React Three Fiber

React Three Fiber in Depth

21

How R3F works inside (a React reconciler whose host objects are three.js objects), the exact rules that map JSX to three.js (args, props, dash-piercing, attach, primitive, extend), the Canvas store, the frame loop and its priorities, the hooks, the event system, state management without re-renders, performance rules and TypeScript.

IN THIS CHAPTER

21.1 A React renderer for three.js
21.2 How JSX maps to three.js
21.3 The Canvas and its store
21.4 The event system
21.5 State management without re-renders
21.6 Performance rules
21.7 TypeScript and testing
21.8 Interview questions

21.1 A React renderer for three.js

DEFINITION · **Reconciler**

The part of React that compares the previous and next component trees and decides what to create, update or remove. `react-dom` plugs DOM operations into it; `@react-three/fiber` plugs in three.js operations: `createInstance` calls `new THREE.X(...args)`, `appendChild` calls `parent.add(child)` (or attaches), `commitUpdate` sets properties, `removeChild` removes and disposes.

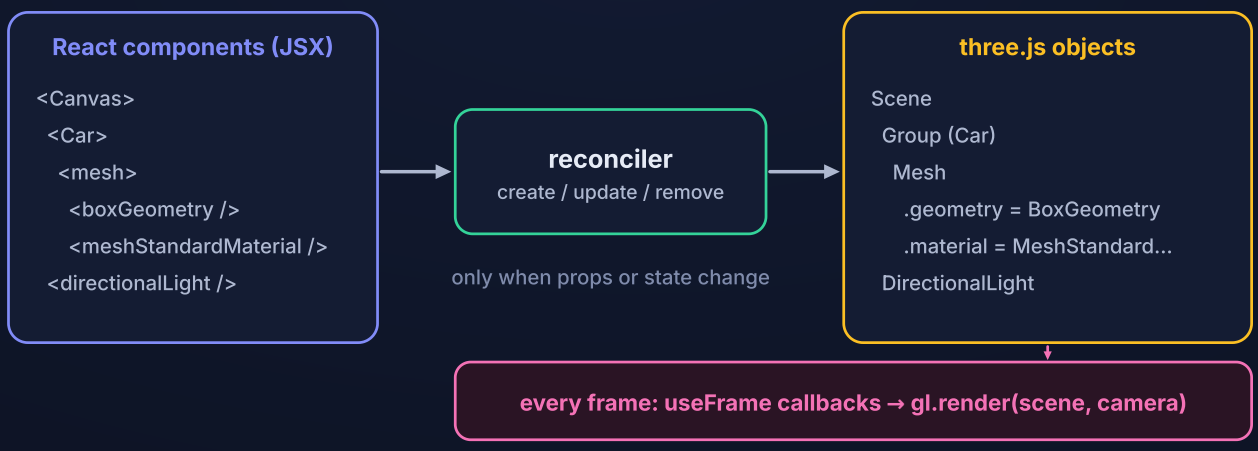


Figure 21.1 What R3F does. React components produce a tree of JSX elements; the reconciler turns them into a tree of three.js objects and applies only the changes on each React render. The frame loop (useFrame) runs outside React, every frame.

Two consequences follow, and they explain most R3F advice:

1. **React renders are not frames.** A React render happens only when state or props change; three.js renders 60–120 times a second regardless. Per-frame work belongs in `useFrame`, not in React state.
2. **There is no overhead per frame.** Once the objects exist, rendering is plain three.js. R3F costs time only when the React tree changes (mounting thousands of elements, frequent re-renders).

21.2 How JSX maps to three.js

JSX	three.js equivalent
`<mesh />`	`new THREE.Mesh()` (lower-camel-case name of any class in the catalogue)
`<boxGeometry args={[2, 1, 1]} />`	`new THREE.BoxGeometry(2, 1, 1)`, attached to the parent's `geometry`
`<meshStandardMaterial color="hotpink" />`	`new THREE.MeshStandardMaterial()`, then `material.color.set("hotpink")`, attached to `material`
`position={[1, 2, 3]}`	`object.position.set(1, 2, 3)` (properties with `.set` are set, not replaced)
`scale={2}`	`object.scale.setScalar(2)`
`rotation-y={Math.PI}`	`object.rotation.y = Math.PI` (dash = nested property, "piercing")
`material-roughness={0.2}`	`object.material.roughness = 0.2`
`<color attach="background" args={["#000"]} />`	`scene.background = new THREE.Color("#000")`
`<bufferAttribute attach="attributes-position" args={[arr, 3]} />`	`geometry.attributes.position = new THREE.BufferAttribute(arr, 3)`
`<primitive object={gltf.scene} />`	Insert an **existing** object as is
`extend({ OrbitControls })` then `<orbitControls />`	Register a class that is not in the THREE namespace

DEFINITION · **attach**

Says **where** a child goes in its parent instead of `parent.add(child)`. Geometries and materials attach to `geometry` and `material` automatically. Use a string path (`attach="material-0"`, `attach="shadow-camera"`) for anything else, or a function `attach={(parent, self) ⇒ { …; return () ⇒ detach }}` for custom logic.

⚠ PITFALL · **Changing args re-creates the object**

`args` are constructor arguments. When they change, R3F must **destroy and re-create** the object (a new geometry, re-uploaded to the GPU, and a new material means a new shader program). `<sphereGeometry args={[radius, 64, 32]} />` with an animated `radius` rebuilds geometry every render. Animate `scale` instead, or change properties (not constructor args), and memoise `args` arrays that do not change.

R3F — all the mapping rules in one component JSX

```jsx
import { extend, useFrame } from "@react-three/fiber";
import { RoundedBoxGeometry } from "three/addons/geometries/RoundedBoxGeometry.js";
extend({ RoundedBoxGeometry });                    // now <roundedBoxGeometry /> exists

function Card({ hovered }) {
  return (
    <group position={[0, 1, 0]} rotation-y={0.3}>
      <mesh castShadow scale={hovered ? 1.1 : 1}>
        <roundedBoxGeometry args={[1.6, 1, 0.08, 4, 0.06]} />
        <meshPhysicalMaterial color="#e2e8f0" clearcoat={1} roughness={0.25} />
      </mesh>
      <spotLight position={[2, 3, 2]} angle={0.4} penumbra={0.6} castShadow
                 shadow-mapSize={[1024, 1024]} shadow-bias={-0.0004} />
    </group>
  );
}
```

21.3 The Canvas and its store

`<Canvas>` creates the renderer, scene, camera, raycaster and a resize observer, starts the frame loop, and holds everything in a **zustand store** that every component inside can read with hooks.

DEFINITION · **useThree**

Reads the store: `gl`, `scene`, `camera`, `size` (CSS pixels), `viewport` (world units at the camera's target distance, plus `dpr` and `aspect`), `pointer` (NDC), `raycaster`, `clock`, `invalidate`, `advance`, `setDpr`, `setFrameloop`, `get` (non-reactive read). Pass a **selector** to subscribe only to what you need: `useThree((s) ⇒ s.viewport)` re-renders on resize, not on every store change.

DEFINITION · **useFrame**

`useFrame((state, delta, xrFrame) ⇒ {…}, priority?)` subscribes a callback to the render loop. Callbacks run before rendering, sorted by priority (lower first). If **any** subscriber has a priority greater than 0, R3F assumes you render manually (for example, a post-processing pipeline) and no longer calls `gl.render(scene, camera)` itself. Unsubscription happens automatically when the component unmounts.

WORKED EXAMPLE · **Viewport units**

Camera FOV 50°, distance 5 to the origin, canvas 1200 × 800 px. `viewport.height` $= 2 \times 5 \times \tan 25^\circ = 4.66$ world units and `viewport.width` $= 4.66 \times 1.5 = 6.99$. `viewport.factor` (pixels per world unit) $= 800/4.66 = 171.6$. A plane of `args={[viewport.width, viewport.height]}` at the origin fills the screen exactly.

Hook	Use
`useFrame`	Per-frame animation by mutating refs
`useThree`	Renderer, camera, size, viewport, invalidate
`useLoader(Loader, url, extensions?)`	Suspending, cached loading; `useLoader.preload`, `useLoader.clear`
`useGraph(object)`	Flattened `{ nodes, materials }` of an object tree
`createPortal(children, target)`	Render JSX into another scene or object (HUDs, render-to-texture)
`useInstanceHandle(ref)`	Low-level access to R3F's instance data

21.4 The event system

R3F raycasts only against objects that **have event handlers**, once per pointer event (and per frame for hover state). Handlers receive a three.js-flavoured event (`point`, `distance`, `face`, `uv`, `object`, `eventObject`, `intersections`, `ray`, `stopPropagation`) (Chapter 14).

Behaviour	Rule
Ordering	Nearest intersection first, then farther objects (the ray passes through everything)
Bubbling	An event bubbles from the hit object up through its parents (`eventObject` changes)
`stopPropagation()`	Stops delivery to parents **and** to objects behind
`onPointerOver/Out` vs `onPointerEnter/Leave`	Over/Out bubble; Enter/Leave do not
Missed clicks	`onPointerMissed` on objects or the Canvas fires when a click hits none of them
Capture	`e.target.setPointerCapture(e.pointerId)` keeps events flowing during drags
Overlays	`eventSource` lets a DOM element that wraps both the canvas and HTML UI receive events

21.5 State management without re-renders

Kind of state	Changes	Where to keep it
UI state (selected colour, menu open)	Rarely	React state or a store with selectors; re-render is fine
Shared app state (cart, level, score)	Occasionally	zustand/jotai store, read with selectors
Fast state (positions, velocities, pointer, time)	Every frame	Refs, three.js objects, or a store read **transiently** in `useFrame`

R3F — transient store reads (no re-render) JSX

```jsx
import { create } from "zustand";
const useGame = create((set) => ({ speed: 1, score: 0, setSpeed: (speed) => set({ speed }) }));

function Spinner() {
  const ref = useRef();
  useFrame((_, dt) => {
    const { speed } = useGame.getState();      // read without subscribing: no React render
    ref.current.rotation.y += speed * dt;
  });
  return <mesh ref={ref}><boxGeometry /><meshNormalMaterial /></mesh>;
}

function Score() {
  const score = useGame((s) => s.score);       // subscribe to one slice: re-renders only when score changes
  return <div className="hud">{score}</div>;
}
```

21.6 Performance rules

Rule	Why
Never `setState` in `useFrame` or fast events	Re-renders the component (and children) 60–120 times a second
Mutate refs in `useFrame`; multiply by `delta`	No React work; frame-rate independent
Do not allocate in `useFrame`	Reuse vectors (`useMemo`/module scope); avoids GC pauses
Share geometries and materials	One upload, one program; pass them as props (`geometry={geo}`)
Toggle `visible` instead of mounting/unmounting	Mounting re-creates objects and may compile shaders
Use instancing for many similar objects	drei `<Instances>` / `<Merged>` or `<instancedMesh>`: one draw call
Avoid thousands of JSX elements	Each element is a reconciler instance; mount cost grows
`useLoader` / `useGLTF` instead of manual loaders	Caching and Suspense; no duplicate downloads

Rule	Why
`frameloop="demand"` for static scenes	Render only on change
Adaptive quality	`<PerformanceMonitor>`, `<AdaptiveDpr>`, `performance={{ min: 0.5 }}` + `regress()`
`startTransition` for heavy updates	Keeps input responsive while React works

WORKED EXAMPLE · **Why setState in useFrame hurts**

A component tree of 40 meshes re-rendered by `setState` each frame at 120 Hz: 4,800 component renders per second, each diffing props and possibly re-creating objects, all on the main thread that also runs three.js. The same animation with a ref mutation costs 120 property assignments per second.

R3F — instancing with drei JSX

```jsx
import { Instances, Instance } from "@react-three/drei";

function Forest({ trees }) {                    // 5,000 trees, one draw call
  return (
    <Instances limit={trees.length} castShadow receiveShadow>
      <coneGeometry args={[0.5, 1.6, 8]} />
      <meshStandardMaterial color="#2f6b3a" />
      {trees.map((t, i) => <Instance key={i} position={t.position} scale={t.scale} rotation-y={t.angle} />)}
    </Instances>
  );
}
```

TIP · **Instances versus instancedMesh**

drei's `<Instance>` components are convenient (each can have events and props) but each is still a React element. For tens of thousands of static instances, write the matrices once into an `<instancedMesh args={[geo, mat, count]}>` in a `useLayoutEffect` (`setMatrixAt`, then `instanceMatrix.needsUpdate = true`) to avoid the React overhead entirely.

21.7 TypeScript and testing

R3F 9 — typing custom elements and props TSX

```tsx
import { type ThreeElement, type ThreeElements, extend } from "@react-three/fiber";
import { OrbitControls } from "three/addons/controls/OrbitControls.js";

declare module "@react-three/fiber" {
  interface ThreeElements { orbitControls: ThreeElement<typeof OrbitControls> }
}
extend({ OrbitControls });

type MeshProps = ThreeElements["mesh"];           // was MeshProps in v8 (removed in v9)
export function Box(props: MeshProps) { return <mesh {...props}><boxGeometry />
<meshStandardMaterial /></mesh>; }
```

`@react-three/test-renderer` renders a tree without a browser canvas so you can assert on the resulting three.js graph and fire events; React 19 exports `act` from `react` itself. Visual regressions are better caught with Playwright screenshots of the real canvas.

✓ KEY POINTS

- ✓ R3F is a React reconciler whose host objects are three.js objects; React renders on change, three.js renders every frame.
- ✓ JSX rules: lower-camel-case classes, `args` = constructor arguments (changing them re-creates the object), `.set()` for vector props, dash-piercing for nested properties, `attach` for non-child slots, `<primitive>` for existing objects, `extend` for custom classes.
- ✓ `useFrame` with priority > 0 takes over rendering; `useThree` with selectors; `viewport` gives world units.
- ✓ Events raycast only objects with handlers; they bubble and pass through objects unless stopped.
- ✓ Fast state lives in refs or transient store reads; React state is for rare changes.
- ✓ Instancing, shared resources, visibility toggling, demand frameloop and adaptive DPR are the main performance tools.
- ✓ R3F 9 types: `ThreeElements["mesh"]`, `ThreeElement<typeof X>`.

21.8 Interview questions

Q How does React Three Fiber work under the hood? MID}

A It is a custom React renderer built with `react-reconciler`. Its host config maps JSX element types to three.js constructors (`<mesh>` → `new THREE.Mesh`), applies props as property assignments (with `.set()` for maths objects and dash-piercing for nested properties), adds children with `add()` or `attach`, and removes and disposes objects on unmount. The `<Canvas>` component creates the renderer, scene, camera and a zustand store, runs the render loop (calling `useFrame` subscribers, then `gl.render`), and implements pointer events by raycasting against objects with handlers. After mounting, each frame is plain three.js.

Q Why shouldn't you use setState to animate in R3F? What do you do instead? JUNIOR}

A setState schedules a React render; doing it every frame re-renders the component and its children 60–120 times per second, diffing props and possibly re-creating objects, on the same thread three.js needs. Instead, keep a ref to the three.js object and mutate it in `useFrame`, using `delta` for frame-rate independence. For values shared across components, store them in refs or a zustand store read transiently with `getState()` inside `useFrame`.

Q What happens when the `args` of a geometry change? MID}

A `args` are constructor arguments, so R3F cannot apply them to the existing object: it disposes the old geometry and constructs a new one, which then has to be uploaded to the GPU (for materials, a new shader program may compile). Doing this every frame or on every slider tick is expensive. Prefer animating properties like `scale`, memoising `args`, or building the geometry once in `useMemo` and updating its attributes.

Q How do you render 10,000 interactive objects efficiently in R3F? SENIOR}

A Use instancing: one `instancedMesh` (or drei `<Instances>` for moderate counts) with per-instance matrices and colours, so it is one draw call. Avoid one JSX element per object at very high counts; write instance matrices in an effect or `useFrame`. For interaction, R3F events on an instanced mesh report `e.instanceId`, so one handler serves all instances; accelerate raycasting with a BVH if the geometry is complex. Keep per-instance state in typed arrays, not React state.

Q When does useFrame's priority matter? MID

A Callbacks run in ascending priority each frame, so a camera rig (priority 0) can update before an effect that reads the camera (priority 1). Any positive priority also tells R3F that you will call render yourself, so R3F stops its own `gl.render`. That is how post-processing pipelines (`EffectComposer`, a WebGPU `RenderPipeline`) and multi-pass renders take over rendering; forgetting it leads to rendering twice or not at all.

Q How do you share state between a 3D scene and DOM UI in an R3F app? MID

A Use a store outside React's tree, typically zustand: DOM components and 3D components both import the hook and subscribe to the slices they need with selectors, so updates only re-render consumers. Fast-changing values are read transiently in `useFrame` (`useStore.getState()`) or kept in refs. React context also works, but context does not cross renderer boundaries automatically in older versions and re-renders all consumers on every change, so a store is the common choice.

The R3F Ecosystem and App Architecture

22

A map of the pmndrs ecosystem (drei, postprocessing, rapier, xr, uikit, leva, zustand, react-spring, maath, gltfjsx) with what each piece is for, then how to structure a real R3F application: folders, scene versus DOM UI, multiple views in one canvas, HUDs and portals, code splitting, Next.js and SSR, accessibility, and a production checklist.

IN THIS CHAPTER

22.1 The ecosystem at a glance
22.2 drei, by job
22.3 Structuring a real application
22.4 Production checklist
22.5 Interview questions

22.1 The ecosystem at a glance

Package	What it gives you
`@react-three/fiber`	The renderer: `<Canvas>`, hooks, events
`@react-three/drei`	Hundreds of ready-made helpers (below)
`@react-three/postprocessing`	Merged post-processing effects (WebGL)
`@react-three/rapier`	Physics with Rapier (Chapter 17)
`@react-three/xr`	VR/AR sessions, controllers, hands, teleport, hit testing (Chapter 25)
`@react-three/uikit`	Flexbox UI rendered in 3D (panels, buttons, text) for XR and in-scene interfaces
`@react-three/a11y`	Accessibility: focus, keyboard and screen-reader support for 3D objects
`@react-three/test-renderer`	Unit-testing R3F trees without a browser canvas
`zustand`	Tiny global store; transient reads in `useFrame`
`leva`	A debug control panel (sliders, colour pickers) bound to state
`@react-spring/three`, `maath`	Spring physics animation; maths and easing (`easing.damp3`)
`gltfjsx`	Turns a `.glb` into a typed JSX component, optionally optimising it
`r3f-perf`	A performance overlay: FPS, draw calls, triangles, GPU time

22.2 drei, by job

Job	drei components and hooks
Cameras and controls	`OrbitControls`, `CameraControls`, `MapControls`, `PointerLockControls`, `PresentationControls`, `KeyboardControls`, `PerspectiveCamera`, `OrthographicCamera`, `GizmoHelper`
Staging and lighting	`Environment`, `Lightformer`, `Stage`, `ContactShadows`, `AccumulativeShadows`, `SoftShadows`, `Sky`, `Stars`, `Sparkles`, `Center`, `Bounds`, `Float`
Loading	`useGLTF`, `useTexture`, `useKTX2`, `useVideoTexture`, `useFBX`, `Preload`, `useProgress`, `Loader`
Text and HTML	`Text` (SDF text via troika), `Text3D`, `Html` (DOM in 3D, with occlusion), `Billboard`
Shapes and lines	`RoundedBox`, `Line`, `QuadraticBezierLine`, `Edges`, `Outlines`, `Grid`
Materials	`MeshTransmissionMaterial`, `MeshReflectorMaterial`, `MeshDistortMaterial`, `MeshWobbleMaterial`, `shaderMaterial`
Performance	`Instances` / `Instance`, `Merged`, `Detailed` (LOD), `Bvh`, `AdaptiveDpr`, `AdaptiveEvents`, `PerformanceMonitor`, `BakeShadows`
Interaction	`DragControls`, `PivotControls`, `TransformControls`, `Select`
Layout and composition	`View`, `Hud`, `RenderTexture`, `ScrollControls` + `Scroll` + `useScroll`
Utilities	`useHelper`, `Clone`, `useAnimations`, `Stats`

R3F — a product stage in a dozen lines — JSX

```jsx
import { Canvas } from "@react-three/fiber";
import { Stage, PresentationControls, ContactShadows, Html, useGLTF } from "@react-three/drei";

function Shoe() {
  const { scene } = useGLTF("/shoe.glb");
  return <primitive object={scene} />;
}

export default function Viewer() {
  return (
    <Canvas dpr={[1, 2]} shadows camera={{ fov: 40, position: [0, 0, 4] }} frameloop="demand">
      <PresentationControls global polar={[-0.2, 0.3]} azimuth={[-1, 1]} snap>
        <Stage environment="city" intensity={0.6} adjustCamera={1.2}>
          <Shoe />
        </Stage>
      </PresentationControls>
      <ContactShadows position-y={-0.8} opacity={0.5} blur={2.5} />
      <Html position={[0.6, 0.4, 0]} occlude className="hotspot">Breathable mesh</Html>
    </Canvas>
  );
}
```

22.3 Structuring a real application

A TYPICAL R3F PROJECT LAYOUT

1. `app/` or `src/pages/` : routes and DOM UI (React as usual)
2. `src/scene/` : the 3D world: `Experience.tsx` (lights, environment, camera rig), feature components (`Car.tsx`, `Showroom.tsx`)
3. `src/models/` : gltfjsx-generated components, one per asset, with `useGLTF.preload`
4. `src/materials/` and `src/shaders/` : shared materials, `shaderMaterial`s, TSL node functions
5. `src/state/` : zustand stores (UI state, configuration, game state)
6. `src/hooks/` : reusable behaviour (`useFollowCamera`, `useKeyboardMove`)
7. `public/` : optimised `.glb`, `.ktx2`, `.hdr` assets (built by a gltf-transform script)

TIP · **Keep the DOM and the scene separate but connected**

Render menus, forms and text in normal HTML outside (or over) the canvas, and put only 3D in the scene. Connect them through a store: a DOM colour picker writes `useConfig.setState({ color })`; the 3D component subscribes with a selector and updates its material. DOM UI stays accessible, indexable and easy to style; the 3D scene stays fast.

22.3.1 Many views, one canvas

Browsers limit live WebGL contexts (often 16 per page), and each `<Canvas>` creates one. A page with a grid of product thumbnails or several viewports should use **one** canvas with drei's `<View>`: each view tracks a DOM element and renders its own scene into that rectangle with the scissor test (Chapter 7).

R3F — several 3D views in one WebGL context JSX

```jsx
import { View, OrbitControls, Environment } from "@react-three/drei";

export default function Gallery({ products }) {
  const container = useRef();
  return (
    <div ref={container} className="grid">
      {products.map((p) => (
        <View key={p.id} className="tile">          {/* a normal DOM element in the grid */}
          <Environment preset="studio" />
          <ProductModel url={p.url} />
          <OrbitControls makeDefault />
        </View>
      ))}
      <Canvas eventSource={container} className="canvas-overlay">  {/* one fixed canvas behind/over the page */}
        <View.Port />
```

```
      </Canvas>
    </div>
  );
}
```

22.3.2 HUDs, portals and render-to-texture

- `<Hud>` renders a second scene on top of the main one with its own camera (minimaps, gizmos, crosshairs) in the same frame.
- `createPortal(children, scene)` renders JSX into another scene or object, for example a scene that is only rendered into a texture.
- `<RenderTexture>` renders a scene into a texture you can attach as a material map: a TV screen showing another 3D world, a screen inside a laptop model.

22.3.3 Code splitting and loading

Split heavy 3D experiences from the rest of the page: `const Experience = lazy(() => import("./scene/Experience"))` inside `<Suspense>`. Preload the assets of the next likely view (`useGLTF.preload`), show a lightweight placeholder (a poster image or a low-poly proxy), and use `<Preload all />` to upload textures to the GPU before the first frame so the camera move does not stutter.

22.3.4 Next.js and server rendering

WebGL needs a browser, so the canvas must render only on the client: mark scene files with `"use client"` in the App Router, or load the canvas with `dynamic(() => import("./Scene"), { ssr: false })`. Everything around it (text, SEO content, layout) still benefits from server rendering. Do not import `three` addons that touch `window` at module level in server components.

22.3.5 Accessibility

3D content is invisible to screen readers and keyboard users by default. Provide the important information and actions in DOM (text descriptions, buttons for configurator options), make the canvas focusable with a label, offer a reduced-motion mode (`prefers-reduced-motion`: disable auto-rotation and parallax), keep contrast high for overlaid text, and use `@react-three/a11y` for focusable, announced 3D objects where interaction lives inside the scene.

22.4 Production checklist

Area	Check
Assets	glTF optimised (Meshopt/Draco, KTX2, resized textures); total download budget per page; preloading
Rendering	DPR capped (`dpr={[1, 2]}`); shadows only where needed; environment map for PBR; tone mapping chosen deliberately
Performance	Draw calls and triangles measured (`r3f-perf`, `renderer.info`); instancing; `frameloop="demand"` when static; `PerformanceMonitor` + `AdaptiveDpr`

Area	Check
Memory	Disposal of unmounted primitives and cached assets; no growth when navigating back and forth
Robustness	WebGL context-loss handling; fallback image when WebGL/WebGPU is unavailable; error boundaries around the canvas
Mobile	Tested on a mid-range Android and an iPhone; touch controls; low-power GPU paths
Accessibility	DOM alternatives, keyboard paths, reduced motion
SEO and sharing	Server-rendered text around the canvas; poster images for social cards
Monitoring	Real-user FPS and load-time metrics; error reporting from the canvas

✓ KEY POINTS

- ✓ drei covers controls, staging, loading, text, materials, performance and composition; check it before writing a helper.
- ✓ Organise by feature: scene components, generated model components, shared materials/shaders, stores, hooks, optimised assets.
- ✓ DOM UI outside the canvas, connected through a store; 3D for 3D.
- ✓ One canvas with `<View>` instead of many canvases; `<Hud>`, portals and `<RenderTexture>` for layered and nested scenes.
- ✓ Lazy-load the 3D experience, preload next assets, client-only rendering in Next.js.
- ✓ Treat accessibility, mobile testing, context loss and memory as release criteria.

22.5 Interview questions

Q **A page shows 30 product thumbnails, each a small 3D viewer, and crashes on some devices. Why, and what would you do?** MID

A Each `<Canvas>` creates its own WebGL context, and browsers cap live contexts (commonly 16); older contexts are lost and memory is duplicated (every viewer loads its own copies of textures and shaders). Use a single canvas with drei's `<View>` components, which render each tile's scene into its DOM rectangle with scissoring, sharing one context and the asset cache. Render only visible views, use `frameloop="demand"`, and consider static poster images until a tile is hovered.

Q **How would you structure a 3D product configurator in React?** MID}

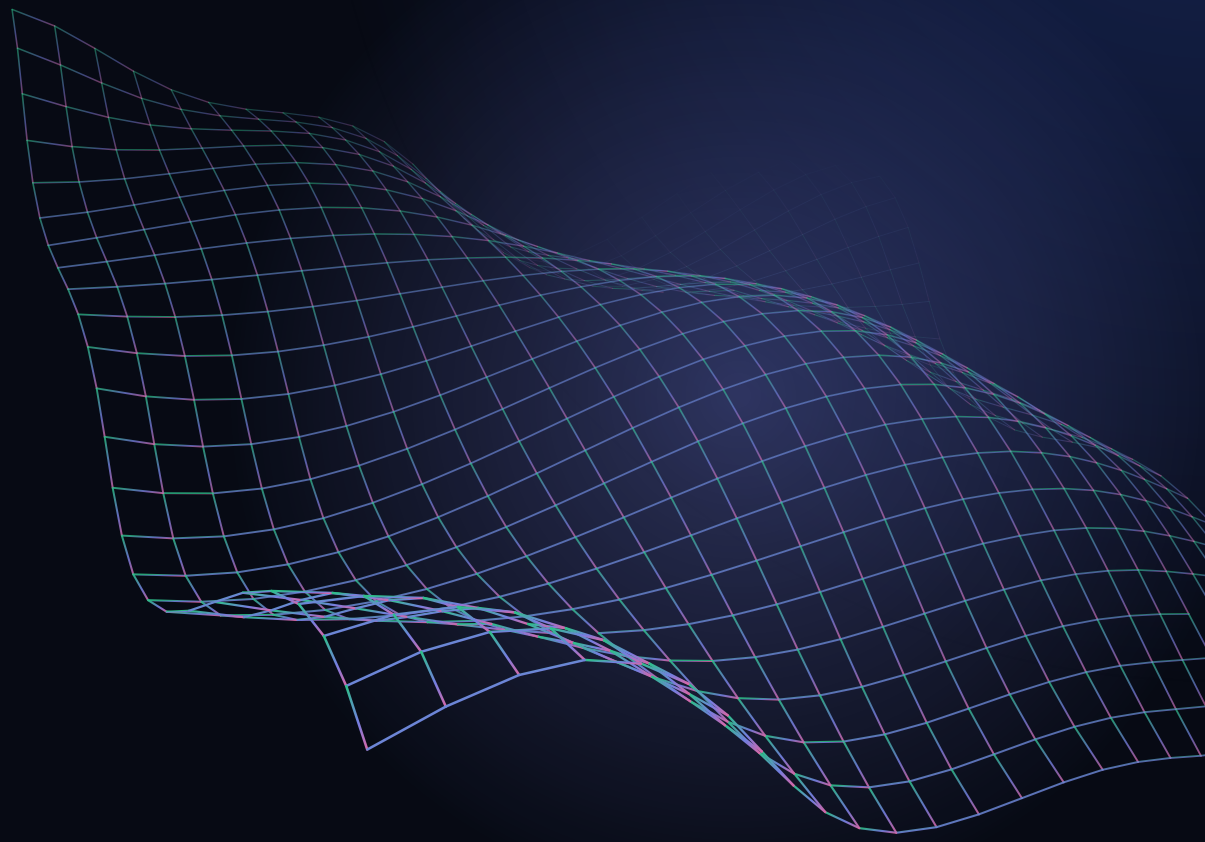
A Configuration state (colour, material, options, price) lives in a zustand store; DOM components render the option pickers and summary and write to the store; the 3D component (generated with gltfjsx) subscribes to the relevant slices and swaps pre-created materials or toggles node visibility. Assets are optimised and preloaded; materials for all options are created once to avoid shader compiles on click; the canvas uses `frameloop="demand"` and Neutral tone mapping for accurate colours. Snapshots for sharing come from `gl.domElement.toBlob` after rendering, and the configuration is serialised into the URL.

Q **How do you use React Three Fiber with Next.js?** JUNIOR}

A The canvas must run only in the browser, so make the scene a client component (`"use client"`) or load it with `next/dynamic` and `ssr: false` . Keep page text and metadata server-rendered for SEO, lazy-load the 3D bundle so it does not block first paint, put optimised models in `public/` , and wrap the canvas in `<Suspense>` with a fallback. Avoid importing browser-only three.js addons into server components.

Q **What is drei's `<Html>` component and what are its costs?** MID}

A It renders real DOM elements positioned at a 3D point by projecting the point to screen space each frame (like `CSS2DRenderer`), optionally scaling with distance (`transform`) and hiding when occluded (`occlude` , which raycasts or uses depth). It is ideal for labels, tooltips and forms because the content stays accessible and crisp. Costs: per-frame projection and DOM style updates for every instance, layout work in the browser, and occlusion raycasts; hundreds of them hurt performance, where SDF `<Text>` or sprites are cheaper.



PART

Advanced Rendering and Production

Performance and Optimisation

23

A method, not a list of tricks. Measure first, find whether the CPU or GPU limits you, then apply the right fix: draw-call reduction (instancing, BatchedMesh, merging), culling and LOD with the screen-size maths, fill-rate and overdraw, shader and post-processing cost, memory budgets, dynamic resolution, workers and OffscreenCanvas, and mobile realities.

IN THIS CHAPTER

23.1 Measure first
23.2 Budgets
23.3 Reducing draw calls (CPU)
23.4 Culling and level of detail
23.5 Fill rate and overdraw (GPU)
23.6 Memory
23.7 Dynamic resolution and adaptive quality
23.8 Off the main thread
23.9 Mobile realities
23.10 Interview questions

23.1 Measure first

DEFINITION · **CPU-bound and GPU-bound**

A frame is **CPU-bound** when JavaScript (scene updates, physics, draw-call submission) takes longer than the GPU's work; it is **GPU-bound** when the GPU (vertex and fragment work, bandwidth) is the slower side. Fixes for one do nothing for the other, so diagnose before optimising.

Experiment	If the frame time drops a lot…
Shrink the canvas or set `setPixelRatio(0.5)`	You are **fragment / fill-rate bound** (GPU)
Hide all post-processing	Post-processing passes are the cost (GPU bandwidth)
Replace all materials with `MeshBasicMaterial`	Shading / lighting cost (GPU)
Disable shadows	Shadow passes (GPU, plus CPU for the extra draw calls)
Hide half the objects (keep resolution)	Draw calls or vertex count (CPU or vertex-bound GPU)
Pause physics / animation logic	JavaScript simulation (CPU)

Tool	Shows
`renderer.info`	Draw calls (`render.calls` , or `render.drawCalls` on WebGPU), triangles, geometries, textures, programs
stats-gl, r3f-perf	FPS, CPU time, **GPU time** (timer queries), draw calls
Chrome Performance panel	JavaScript, garbage collection, layout, GPU tasks on a timeline
Spector.js	Every WebGL call in a frame, state, textures, shaders
WebGPU timestamp queries, three.js Inspector addon	Per-pass GPU timings on `WebGPURenderer`
Safari Web Inspector (connected iPhone/iPad)	Real mobile timings and memory

23.2 Budgets

These are rules of thumb for a smooth experience on typical hardware; measure on your own targets.

Resource	Mid-range phone	Laptop (integrated GPU)	Desktop (discrete GPU)
Draw calls per frame	100–200	300–1,000	1,000–5,000+
Triangles on screen	100k–500k	1–3 M	5–20 M
Texture VRAM	100–300 MB	500 MB–1 GB	Several GB
Pixels shaded (canvas × DPR²)	≈ 1–2 M	≈ 4–8 M	≈ 8–16 M
Full-screen post passes	1–2	3–5	5–10

23.3 Reducing draw calls (CPU)

Each mesh with each material is at least one draw call; shadows add one per caster per shadow map; multi-material meshes add one per material group.

Technique	Draw calls	Constraints
Share materials and geometries	Fewer state changes, programs	Needed for everything below
Merge static geometry (`mergeGeometries`)	1 per merged group	Static; culled as a whole
`InstancedMesh`	1 per geometry+material	Same geometry; per-instance matrix, colour (and custom attributes)
`BatchedMesh`	1 multi-draw	Different geometries, one material; per-instance matrix, colour, visibility, opacity (r183)
Texture atlases / array textures	Lets different-looking objects share a material	UV remapping or a layer index per instance
Fewer shadow casters	Fewer shadow-pass draws	`castShadow` only where it shows

three.js — BatchedMesh: different shapes, one draw call — JAVASCRIPT

```
const batch = new THREE.BatchedMesh(1000, 200_000, 400_000, material);   // max instances, vertices, indices
const rockA = batch.addGeometry(rockGeoA);       // register each geometry once
const rockB = batch.addGeometry(rockGeoB);
const m = new THREE.Matrix4();
for (let i = 0; i < 1000; i++) {
  const id = batch.addInstance(i % 2 ? rockA : rockB);
  batch.setMatrixAt(id, m.compose(randomPos(), randomQuat(), randomScale()));
  batch.setColorAt(id, new THREE.Color().setHSL(0.08, 0.3, 0.3 + Math.random() * 0.2));
}
scene.add(batch);   // per-instance frustum culling and optional sorting happen inside
```

TIP · Keep matrices static when you can

For static objects set `matrixAutoUpdate = false` and call `updateMatrix()` once; for a whole static subtree, also set `matrixWorldAutoUpdate = false` on the root and update manually after edits. In a scene of 20,000 objects, recomputing every matrix each frame is pure waste.

23.4 Culling and level of detail

Culling	What it skips	In three.js
Frustum culling	Objects outside the view	Automatic (bounding spheres); per instance in `BatchedMesh`
Distance culling	Small or far objects	`camera.far`, layers, manual `visible = false` by distance
Occlusion culling	Objects hidden behind others	Not built in; portals/rooms for interiors, occlusion queries, or precomputed visibility
Back-face culling	Triangles facing away	`side: FrontSide` (default)

DEFINITION · **Level of detail (LOD)**

Swapping a detailed mesh for simpler versions as it gets smaller on screen. `THREE.LOD` (`lod.addLevel(mesh, distance)`) and drei's `<Detailed distances={[0, 15, 40]}>` switch by camera distance; `SimplifyModifier` (meshoptimizer-based since r186, async) or `gltf-transform simplify` produce the levels; at the far end an **impostor** (a camera-facing quad with a pre-rendered image) can replace the model entirely.

FORMULA · **Screen-space size**

An object with bounding radius r at distance d covers about

$$h_{\text{px}} \approx \frac{2r}{2d\tan(\text{fov}/2)} \cdot H_{\text{px}} = \frac{r\,H_{\text{px}}}{d\tan(\text{fov}/2)}$$

pixels of a canvas H_{px} tall. Choose LOD distances so each level switches at a target on-screen size.

WORKED EXAMPLE · **Where to switch LOD levels**

A tree with $r = 1$ m, FOV 50°, a 1080 px canvas: at 10 m it is $1080/(10 \times 0.466) = 232$ px tall; at 50 m, 46 px. To switch to the low-poly version when it is under 100 px: $d = 1080/(100 \times 0.466) = 23.2$ m. A 5,000-triangle tree that is 46 px tall has about one triangle per pixel, which is wasted work (and tiny triangles are inefficient for GPUs, which shade in 2 × 2 pixel quads).

23.5 Fill rate and overdraw (GPU)

DEFINITION · **Overdraw**

Shading the same pixel more than once in a frame because several surfaces cover it: transparent layers, particles, foliage cards, full-screen effects, and opaque objects drawn back to front. The cost is (pixels) × (average layers) × (fragment shader cost).

WORKED EXAMPLE · **Fragment shader invocations**

1920 × 1080 at an average overdraw of 3 and 60 Hz: $2.07\text{M} \times 3 \times 60 = 373$ million fragment-shader runs per second. Halve the overdraw or render at 0.75 resolution (0.56× the pixels) and you remove a large fraction of GPU time.

Fix	Effect
Cap DPR at 2 (or lower adaptively)	Pixel count scales with DPR²
Opaque objects front to back (three.js does)	Early-z rejects hidden fragments
Fewer, smaller transparent particles; additive without depth writes	Less blended overdraw
Tight geometry around alpha-tested foliage cards	Fewer discarded fragments
Half-resolution buffers for AO, bloom, volumetrics	¼ of the fragment work for those passes
Simpler materials for distant or small objects	Cheaper per-pixel shading
Bake lighting and AO for static scenes	No real-time shadows or AO passes

23.6 Memory

Item	Size rule	Example
Texture	$w \times h \times$ bytes per texel $\times\ 4/3$	4K RGBA8: 85 MiB; 4K BC7: 21 MiB
Render target	$w \times h \times$ bytes $\times$ (MSAA samples)	1440p half-float RGBA ×4 MSAA: 118 MB
Shadow map	$\text{size}^2 \times 4$ (× 6 for point lights)	2048^2: 16.8 MB
Geometry	vertices × bytes per vertex + indices	1 M vertices × 32 B = 32 MB

Free what you no longer render (`dispose()`, Chapter 7), compress textures (KTX2), quantise geometry, share resources, and stream large worlds in and out by tiles.

23.7 Dynamic resolution and adaptive quality

FORMULA · **Dynamic resolution scale**

To bring a GPU-bound frame time t_{measured} down to a target t_{target}, scale the pixel count by $t_{\text{target}}/t_{\text{measured}}$, i.e. scale the pixel ratio by

$$k = \sqrt{\frac{t_{\text{target}}}{t_{\text{measured}}}}$$

(assuming fragment work dominates and scales with pixels). Smooth the measurement over several frames and clamp the result.

WORKED EXAMPLE · **Recovering a slow frame**

GPU time 22 ms, target 14 ms (leaving headroom under 16.7 ms): pixel scale $14/22 = 0.636$, so the ratio factor $k = \sqrt{0.636} = 0.80$. At DPR 2, render at $2 \times 0.80 = 1.6$. Upscaling with FSR 1 or TAAU (WebGPU nodes since r184) recovers much of the lost sharpness.

R3F — adaptive quality JSX

```jsx
import { PerformanceMonitor, AdaptiveDpr, AdaptiveEvents } from "@react-three/drei";

function App() {
  const [dpr, setDpr] = useState(1.5);
  return (
    <Canvas dpr={dpr} performance={{ min: 0.5 }}>       {/* regress() can drop to half quality briefly */}
      <PerformanceMonitor onDecline={() => setDpr(1)} onIncline={() => setDpr(2)} flipflops={3} onFallback={() => setDpr(1)} />
      <AdaptiveDpr pixelated />                         {/* lowers DPR while regressing (for example, during camera moves) */}
      <AdaptiveEvents />                                {/* skips raycasting while regressing */}
      <Scene />
    </Canvas>
  );
}
```

23.8 Off the main thread

- **Web Workers** for physics, procedural generation, pathfinding, parsing (loaders already decode Draco/Basis in workers).
- **OffscreenCanvas**: transfer the canvas to a worker and run the whole three.js renderer there (`canvas.transferControlToOffscreen()`), leaving the main thread for the UI. Input events must be forwarded to the worker; R3F has `@react-three/offscreen` for this.
- **WASM** for heavy maths (Rapier, Jolt, meshoptimizer, Draco).
- **Compute shaders** for massively parallel work (Chapter 20).

23.9 Mobile realities

PITFALL · **Thermal throttling**

Phones and thin laptops reduce GPU and CPU clocks after a minute or two of sustained load, so a scene that runs at 60 fps in a quick test can drop to 30 fps later. Test for five minutes, leave headroom (aim for 10–12 ms of work at 60 Hz), render on demand when nothing moves, pause when the tab is hidden, and lower quality on battery-saver signals where available.

Mobile-specific tip	Why
DPR ≤ 2, often 1.5	Pixel count is the biggest GPU cost
Avoid real-time shadows from many lights; bake or use contact shadows	Shadow passes multiply draw calls
KTX2 textures, ≤ 2K	VRAM limits (iOS Safari kills tabs)
Fewer post passes; no SSR/SSGI	Bandwidth is limited on tile-based GPUs
Prefer `alphaTest` over blending	Tile-based GPUs pay heavily for overdraw and blending
Watch `highp` precision in heavy fragment shaders	Mobile GPUs may run `mediump` faster

✓ KEY POINTS

- ✓ Diagnose first: shrink resolution (fragment-bound?), hide objects (draw calls?), pause logic (CPU?).
- ✓ Draw calls: share materials, merge static meshes, instance repeats, batch varied shapes with `BatchedMesh`.
- ✓ Static matrices: `matrixAutoUpdate = false`; cull with frustum, distance and portals; LOD by on-screen size $rH/(d\tan(\text{fov}/2))$.
- ✓ Fragment cost = pixels × overdraw × shader cost: cap DPR, reduce transparency, half-resolution effects, bake.
- ✓ Memory: compress, share, dispose, stream.
- ✓ Adaptive quality: DPR scale $\sqrt{t_{\text{target}}/t}$, `PerformanceMonitor`, `AdaptiveDpr`, upscalers.
- ✓ Workers, OffscreenCanvas, WASM and compute shaders move work off the main thread.
- ✓ Test on real phones for minutes, not seconds.

23.10 Interview questions

Q **A scene runs at 60 fps on desktop and 25 fps on a mid-range phone. How do you diagnose it?** MID

A Measure on the phone (remote debugging), then isolate. Reduce the pixel ratio or canvas size: if FPS jumps, it is fragment/fill-rate bound, so cap DPR, cut post-processing and overdraw, simplify shaders, bake lighting. If not, check `renderer.info.render.calls` and CPU time in the profiler: too many draw calls means instancing, batching, merging and fewer shadow casters; heavy JavaScript means moving physics or logic to workers and avoiding per-frame allocations. Also check texture memory (KTX2, smaller sizes) and thermal throttling over a few minutes.

Q When would you use InstancedMesh versus BatchedMesh versus merging? MID}

A `InstancedMesh` when many objects share exactly one geometry and material but need individual transforms or colours (trees, bolts, crowds, particles). `BatchedMesh` when objects have different geometries but share a material and still need individual transforms, visibility or per-instance culling (varied props, CAD assemblies, city buildings). Merging when objects are static and never need to move or be culled individually; it is the simplest and gives one draw call, at the cost of flexibility.

Q How do you implement level of detail, and how do you choose switch distances? MID}

A Create simplified versions of the mesh (meshoptimizer via `gltf-transform simplify`, `SimplifyModifier`, or artist-made), add them to `THREE.LOD` (or drei's `<Detailed>`) with distances, and let three.js pick the level per frame. Choose distances from the target on-screen size: the projected height is about $rH/(d\tan(\text{fov}/2))$ pixels, so switch when the object falls below, for example, 100–200 pixels for the mid level, and use an impostor or nothing below a few dozen pixels. Hide pops with a small hysteresis or dithered cross-fade.

Q What is overdraw and how do you reduce it? MID}

A Shading a pixel multiple times per frame because several surfaces overlap: transparent layers, particles, foliage, full-screen effects, unsorted opaque geometry. Each extra layer costs a full fragment-shader run. Reduce it by keeping opaque objects sorted front to back (three.js does), using fewer and smaller transparent particles, preferring alpha-tested cut-outs, trimming card geometry to the visible shape, merging post-processing passes and running blurry effects at lower resolution, and capping the pixel ratio.

Q How would you keep a heavy 3D page responsive while it loads? SENIOR}

A Split the bundle (lazy-load the 3D experience), show HTML content and a lightweight poster immediately, stream optimised assets (Meshopt/KTX2, smallest first), decode in workers, preload the next assets, pre-compile shaders and pre-upload textures before revealing the scene (`renderer.compileAsync`, drei `<Preload all />`), avoid long synchronous tasks by chunking work, and consider OffscreenCanvas so rendering never blocks input. Measure with Lighthouse and real-user metrics (Interaction to Next Paint, long tasks).

Advanced Rendering Techniques

24

The wider world of real-time graphics, explained from first principles with the maths. Forward, deferred and clustered (Forward+) rendering; global illumination with lightmaps, light probes and spherical harmonics; reflections; order-independent transparency; fog, volumetrics and Beer–Lambert; raymarching signed distance fields; Monte Carlo path tracing; 3D Gaussian splatting; virtualised geometry; and precision in very large worlds.

IN THIS CHAPTER

24.1 Rendering architectures
24.2 Global illumination
24.3 Reflections
24.4 Transparency without sorting
24.5 Fog, participating media and volumetrics
24.6 Raymarching signed distance fields
24.7 Path tracing
24.8 3D Gaussian splatting
24.9 Virtualised geometry and GPU-driven rendering
24.10 Large worlds and precision
24.11 Interview questions

24.1 Rendering architectures

How a renderer organises lighting work is its biggest design decision, because lighting cost multiplies with both scene complexity and the number of lights.

DEFINITION · **Forward rendering**

Each object is drawn once and its fragment shader evaluates **every light** that might affect it. Simple, works with transparency and MSAA, and is what three.js does by default. Cost grows with (fragments shaded) × (lights), and overdraw wastes lighting work on hidden fragments.

DEFINITION · **Deferred rendering**

Two stages. The **geometry pass** draws all objects once into a **G-buffer** (several textures holding albedo, normal, roughness/metalness, depth for every pixel). The **lighting pass** then shades each screen pixel once per light that touches it, reading the G-buffer. Lighting cost no longer depends on overdraw, and hundreds of lights are affordable. Costs: large memory bandwidth for the G-buffer, no hardware MSAA, transparency needs a separate forward pass, and every material must fit the G-buffer layout.

DEFINITION · **Clustered (Forward+) shading**

Divide the view frustum into a 3D grid of **clusters** (screen tiles × depth slices). A cheap pass assigns each light to the clusters its sphere of influence touches. When shading a fragment, look up its cluster and evaluate **only that cluster's lights**. Works with forward rendering (keeping MSAA and transparency) and with deferred. three.js offers it for `WebGPURenderer` as `ClusteredLighting` (r185).

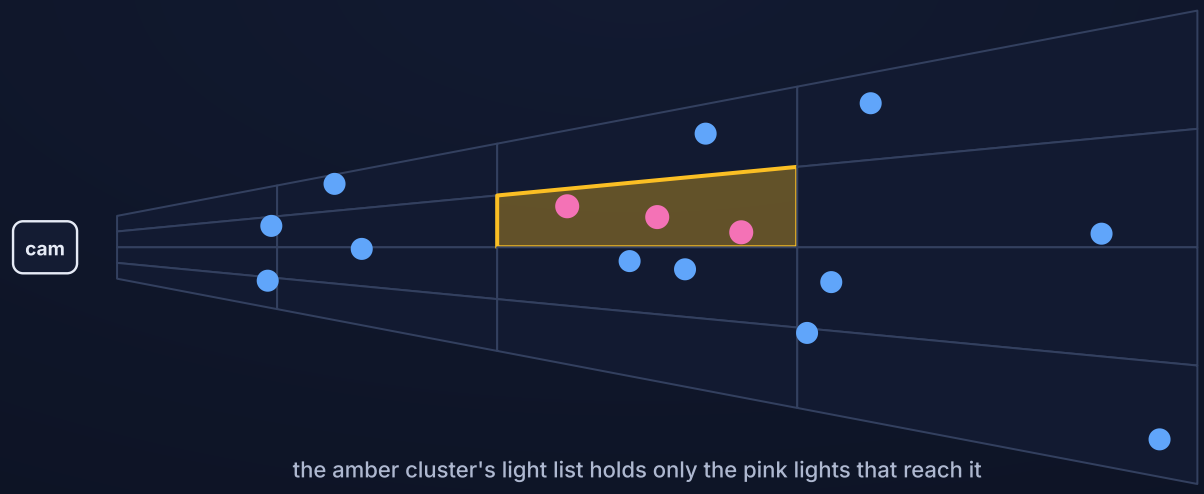


Figure 24.1 Clustered shading. The view frustum is cut into clusters (here 2D tiles × depth slices). Each cluster stores the short list of lights that reach it, so a fragment evaluates three lights instead of all twelve.

WORKED EXAMPLE · **Light evaluations per frame**

A 1080p frame (2.07 M pixels) with average overdraw 1.5 and 50 point lights:

- Forward, all lights everywhere: $2.07\text{M} \times 1.5 \times 50 \approx 155$ million light evaluations per frame.
- Clustered, about 4 lights per cluster on average: $2.07\text{M} \times 1.5 \times 4 \approx 12$ million, about **13× less**. The cluster build costs a little compute, which is negligible by comparison.

three.js — clustered lighting on WebGPURenderer (r185+) JAVASCRIPT

```
import { ClusteredLighting } from "three/addons/lighting/ClusteredLighting.js";
renderer.lighting = new ClusteredLighting(1024 /* max lights */, 32 /* tile px */, 24 /* depth slices */, 64 /* max per cluster */);
for (let i = 0; i < 500; i++) {
  const light = new THREE.PointLight(randomColour(), 4, 3);   // intensity 4 cd, distance 3 m
  light.position.copy(randomPos());
  scene.add(light);
}
// give each light a finite distance: it bounds the light's sphere of influence, so it touches few clusters
```

	Forward	Deferred	Clustered forward (Forward+)
Lights	Few	Hundreds	Hundreds
Overdraw sensitivity	High	Low	Medium (a depth pre-pass helps)
Transparency	Native	Separate forward pass	Native
MSAA	Yes	Hard	Yes
Material variety	Unlimited	Limited by G-buffer	Unlimited
Memory bandwidth	Low	High	Medium

24.2 Global illumination

DEFINITION · **Global illumination (GI)**

Light that arrives **indirectly**, after bouncing off other surfaces: the red glow on a white wall next to a red sofa, soft light filling a room from a sunlit floor, darkening in corners. Direct lighting (Chapters 9–13) ignores it; GI techniques approximate it, trading quality, cost and how dynamic the scene may be.

Technique	Dynamic?	Cost at runtime	Notes
Baked lightmaps (Blender, Unity, Bakery)	Static geometry and lights	Free (a texture lookup)	Highest quality per cost; needs a second UV set
Light probes (spherical harmonics)	Dynamic objects in static lighting	Very cheap	`LightProbe`; probe grids via `LightProbeGridWebGL` (added r184, renamed r186)
Irradiance volumes / probe grids	Moving objects in large spaces	Cheap	Interpolate between probes in a 3D grid
Reflection probes (cube maps)	Mostly static	Cheap	`CubeCamera` / PMREM per area
Screen-space GI (SSGI)	Fully dynamic	Moderate–high	Only what is on screen; `SSGINode` (WebGPU)
Voxel / SDF GI	Dynamic	High	Used in large engines
Path tracing	Everything	Very high	Offline-like quality; progressive (below)

DEFINITION · **Spherical harmonics (SH)**

A set of basis functions on the sphere, the spherical equivalent of a Fourier series. Low-frequency lighting around a point (how much light arrives from each direction, blurred) can be stored as a few SH coefficients: **order 2 (L2) uses 9 coefficients per colour channel**, 27 numbers in total. Evaluating irradiance for a normal is then a short polynomial in the normal's components, with no texture lookup.

FORMULA · **Irradiance from L2 spherical harmonics**

$$E(\hat{\mathbf{n}}) \approx \sum_{l=0}^{2} \sum_{m=-l}^{l} \hat{A}_l \, L_{lm} \, Y_{lm}(\hat{\mathbf{n}})$$

L_{lm} are the stored coefficients, Y_{lm} the basis functions (constants, x, y, z, xy, yz, $3z^2 - 1$, xz, $x^2 - y^2$ up to normalisation), and $\hat{A}_0 = \pi$, $\hat{A}_1 = 2\pi/3$, $\hat{A}_2 = \pi/4$ the cosine-lobe convolution factors (Ramamoorthi and Hanrahan, 2001).

WORKED EXAMPLE · **The size of a probe grid**

A building lit by a $16 \times 8 \times 16$ grid of probes: $2{,}048$ probes $\times$ 27 floats $\times$ 4 bytes $=$ 221 KB. That small table gives every moving character believable bounce light by trilinearly blending the eight surrounding probes.

24.3 Reflections

Method	Accuracy	Cost	Limits
Environment map (IBL)	Distant surroundings only	Very low	No nearby objects, no self-reflection
Box-projected / local cube maps	Good inside rooms	Low	Assumes a box-shaped room
Planar reflection (`Reflector`, `MeshReflectorMaterial`)	Exact for flat mirrors and water	Re-renders the scene	One plane per extra render
Screen-space reflections (SSR)	Good for what is visible	Moderate	Missing data off-screen or behind objects
Ray-traced reflections	Exact	High	Path tracers, offline

24.4 Transparency without sorting

Sorting transparent objects (Chapter 2) fails for intersecting or self-overlapping geometry. **Order-independent transparency (OIT)** removes the dependence on order.

Σ FORMULA · **Weighted blended OIT (McGuire and Bavoil, 2013)**

$$C = \frac{\sum_i c_i \, \alpha_i \, w_i}{\sum_i \alpha_i \, w_i} \left(1 - \prod_i (1 - \alpha_i)\right) + C_{\text{bg}} \prod_i (1 - \alpha_i)$$

Each transparent fragment adds into two accumulation targets in any order; a weight w_i that decreases with depth favours nearer surfaces. One extra composite pass resolves the image. Approximate, but cheap and stable. three.js's WebGPU stack includes an `OITPassNode`; `alphaHash` (stochastic transparency + TAA) is another order-independent option.

Exact alternatives: **depth peeling** (render several layers, peeling off the nearest each pass: exact, but costs one pass per layer) and per-pixel linked lists (exact, needs compute and atomics).

24.5 Fog, participating media and volumetrics

DEFINITION · **Participating medium**

A volume that interacts with light along the way: fog, smoke, clouds, murky water, dusty air. Light is absorbed and scattered as it travels, so distant objects fade towards the medium's colour and beams of light become visible.

Σ FORMULA · **Beer–Lambert transmittance and three.js fog**

$$T(d) = e^{-\sigma_t d}$$

The fraction of light surviving a path of length d through a uniform medium with extinction coefficient σ_t. three.js fog blends the surface colour towards the fog colour with a factor f:

$$f_{\text{linear}} = \frac{d - \text{near}}{\text{far} - \text{near}}, \qquad f_{\text{exp2}} = 1 - e^{-(\rho d)^2}$$

(`THREE.Fog(color, near, far)` and `THREE.FogExp2(color, density)`.)

WORKED EXAMPLE · **How foggy is 20 metres?**

- `Fog(color, 10, 50)` at 20 m: $f = (20 - 10)/40 = 0.25$.
- `FogExp2(color, 0.05)` at 20 m: $f = 1 - e^{-(0.05 \times 20)^2} = 1 - e^{-1} = 0.632$.
- A physical medium with $\sigma_t = 0.1\ \text{m}^{-1}$: $T(10) = e^{-1} = 0.37$, so 63% of the light from a wall 10 m away is lost before it reaches you.

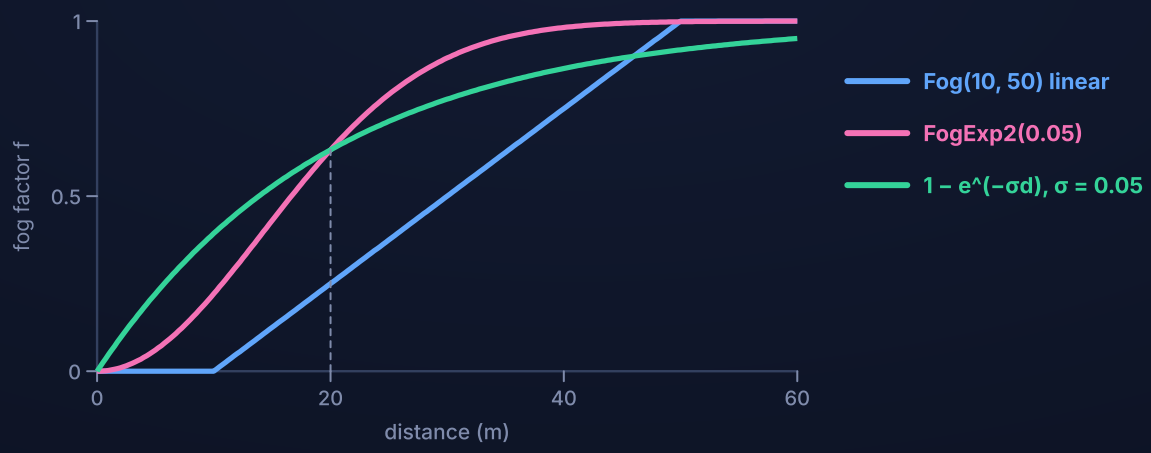


Figure 24.2 Fog factor against distance: linear fog ramps evenly between near and far; exponential-squared fog stays clear close up and closes in quickly; physical transmittance falls off exponentially.

DEFINITION · **Raymarched volumetrics**

For clouds, smoke and light shafts, march a ray through the volume in small steps; at each step sample density (from noise or a 3D texture), accumulate transmittance with Beer–Lambert and add light scattered towards the eye (with a phase function such as Henyey–Greenstein and a shadow ray towards the sun). Expensive, so done at reduced resolution with temporal accumulation. Screen-space **god rays** (`GodraysNode`) are a cheaper approximation from the depth buffer.

24.6 Raymarching signed distance fields

Chapter 18 used 2D SDFs. In 3D, a scene described entirely by an SDF can be rendered without triangles by **sphere tracing**.

DEFINITION · **Sphere tracing**

March along each pixel's ray. At the current point, the SDF value d is the distance to the nearest surface in **any** direction, so it is safe to step forward by exactly d without passing through anything. Repeat until d is below a small epsilon (a hit) or the ray travels too far (a miss). Normals come from the SDF's gradient (central differences).

Σ FORMULA · **Sphere tracing and SDF normals**

$$t_{k+1} = t_k + f(\mathbf{o} + t_k \hat{\mathbf{d}}), \qquad \hat{\mathbf{n}} = \operatorname{normalise}\big(\nabla f\big) \approx \operatorname{normalise}\begin{pmatrix} f(\mathbf{p}+\epsilon\hat{\mathbf{x}}) - f(\mathbf{p}-\epsilon\hat{\mathbf{x}}) \\ \cdots \\ \cdots \end{pmatrix}$$

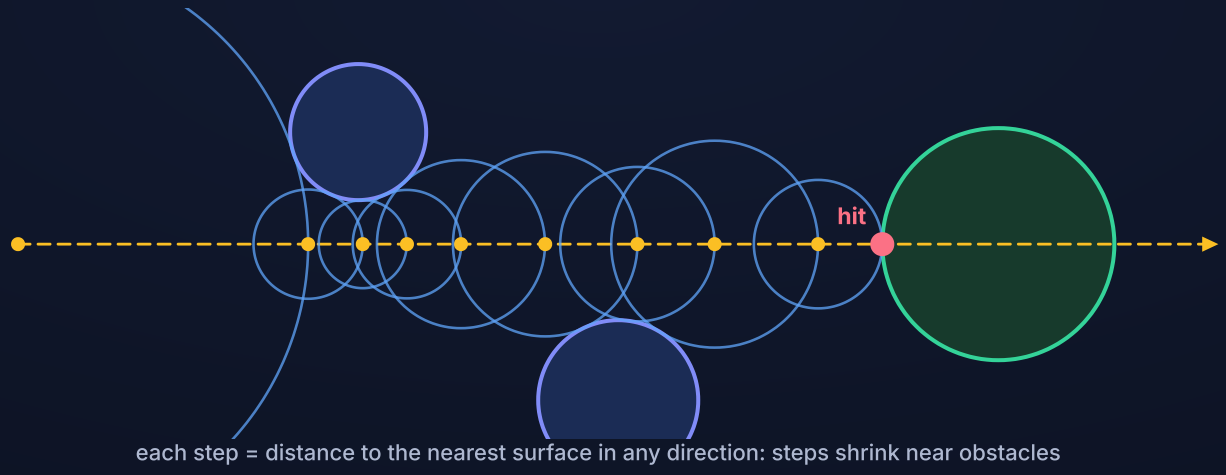


Figure 24.3 Sphere tracing in 2D. Each circle is the safe step given by the distance field at that point; steps shrink as the ray approaches the surface and converge on the hit.

WORKED EXAMPLE · **Tracing the sphere from Chapter 14**

The Chapter 14 sphere, mirrored: a ray from the origin along $+z$ towards a sphere at $(0.5, 0, 5)$ with $r = 1$, so $f(\mathbf{p}) = \|\mathbf{p} - \mathbf{c}\| - 1$.

Step	t	f
0	0	4.025
1	4.025	0.096
2	4.121	0.012
3	4.132	0.0015
4	4.134	0.0002

Converged to $t = 4.134$ in five steps: the same answer as the analytic ray–sphere intersection in Chapter 14. Rays passing close to surfaces without hitting need many small steps, which is the main cost of the technique.

SDF raymarching powers Shadertoy-style demos, fractals, soft shadows and ambient occlusion from distance fields, metaballs, and medical or scientific volume rendering. In three.js it is a full-screen quad with a custom shader (or a TSL `Fn` with `Loop`).

24.7 Path tracing

DEFINITION · **Path tracing**

Solving the rendering equation by **Monte Carlo** integration: from each pixel, trace random light paths that bounce around the scene (sampling the BRDF at each hit) until they reach a light, and average many such samples. It produces all effects at once (soft shadows, GI, caustics, glossy reflections), at the cost of noise that only fades as samples accumulate.

Σ FORMULA · **Monte Carlo estimate and its error**

$$\int f(x)\,dx \approx \frac{1}{N}\sum_{i=1}^{N}\frac{f(x_i)}{p(x_i)}, \qquad \text{error} \propto \frac{1}{\sqrt{N}}$$

p is the probability density used to choose samples. **Importance sampling** (choosing directions where the BRDF or the lights are strong) lowers the variance; denoisers remove much of what remains.

1 sample / pixel | 4 samples / pixel | 16 samples / pixel | 64 samples / pixel

Figure 24.4 Monte Carlo ambient occlusion of a sphere on a floor, computed per pixel with 1, 4, 16 and 64 random rays. Noise falls as 1/√N: sixteen times the samples give a four times cleaner image.

WORKED EXAMPLE · **Samples versus noise**

To halve the noise you need **four times** the samples. Going from 16 to 1,024 samples per pixel ($64\times$ more work) reduces noise by only $8\times$. Interactive path tracers (such as three-gpu-pathtracer for three.js) therefore render progressively: a noisy image while the camera moves, refining when it stops, plus a denoiser.

24.8 3D Gaussian splatting

DEFINITION · **3D Gaussian splatting (3DGS)**

A scene represented not by triangles but by millions of small, semi-transparent, coloured 3D Gaussians (“splats”), each with a position, a shape (3D covariance from a scale and a rotation), an opacity and a view-dependent colour (spherical-harmonic coefficients). They are **optimised from photographs** of a real place or object, giving photoreal captures that render in real time. three.js r186 added `GaussianSplat` with PLY, `.splat`, `.ksplat` and glTF loaders (on `WebGPURenderer`); World Labs’ **Spark** library is a full-featured splat renderer for three.js.

Σ FORMULA · Splat geometry and blending

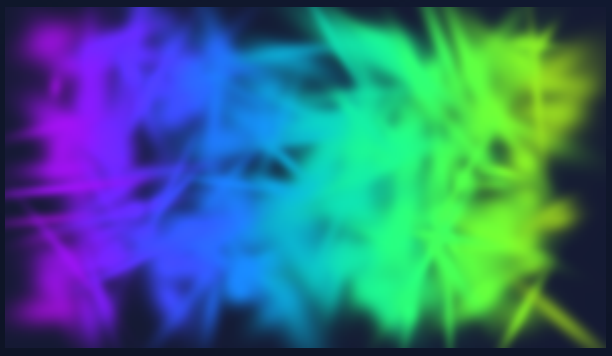
Each Gaussian's 3D covariance is $\Sigma = R\,S\,S^T R^T$ (rotation R from a quaternion, scale S diagonal). Projected to the screen it becomes a 2D ellipse with covariance $\Sigma' = J\,W\,\Sigma\,W^T J^T$ (W the view rotation, J the Jacobian of the perspective projection). Splats are **sorted by depth** and composited front to back:

$$C = \sum_i c_i\,\alpha_i \prod_{j<i}(1-\alpha_j)$$

where α_i is the splat's opacity times its Gaussian falloff at the pixel.

WORKED EXAMPLE · Three splats over one pixel

Front to back, opacities $\alpha = 0.7, 0.9, 0.5$: - Splat 1 contributes 0.7 of its colour; remaining transmittance 0.3. - Splat 2 contributes $0.3 \times 0.9 = 0.27$; remaining $0.3 \times 0.1 = 0.03$. - Splat 3 contributes $0.03 \times 0.5 = 0.015$. After two splats 97% of the pixel is decided, which is why renderers stop early once transmittance is tiny.



each splat: position, scale + rotation, opacity, colour (SH)

$C = \Sigma\, c_i\, \alpha_i\, \Pi(1 - \alpha_j)$

sorted front to back;
stop when transmittance ≈ 0

260 splats, rendered per pixel

Figure 24.5 Gaussian splats are anisotropic, semi-transparent ellipses. Sorted front to back and alpha-blended, millions of them reconstruct a photoreal scene.

three.js — loading a Gaussian splat capture (r186, WebGPURenderer) JAVASCRIPT

```javascript
import * as THREE from "three/webgpu";
import { GaussianSplat } from "three/addons/objects/GaussianSplat.js";
import { GaussianSplatPLYLoader } from "three/addons/loaders/GaussianSplatPLYLoader.js";

const geometry = await new GaussianSplatPLYLoader().loadAsync("/captures/garden.ply");
const splats = new GaussianSplat(geometry);       // sorts splats back to front automatically (autoSort)
scene.add(splats);
```

	Meshes	Gaussian splats
Source	Modelled or scanned and retopologised	Optimised from photos or video
Look	Depends on materials and lighting	Photoreal, lighting baked in
Editing and relighting	Easy	Hard (research area)
Physics and collisions	Easy	Needs a proxy mesh
Size	Small	Large (tens to hundreds of MB; compressed formats such as `.spz` help)
Rendering	Rasterised triangles, depth buffer	Sorted, blended quads; sorting is the bottleneck

24.9 Virtualised geometry and GPU-driven rendering

Film-quality assets have millions of triangles. **Virtualised geometry** (popularised by Unreal's Nanite) splits meshes into small **clusters (meshlets)** of about 64–128 triangles, builds a hierarchy of simplified cluster groups, and each frame selects, per region of the screen, the coarsest clusters whose error is under a pixel. Culling and LOD selection run on the GPU (compute shaders), which then draw everything with indirect draw calls. On the web this is experimental, enabled by WebGPU compute and indirect drawing; three.js ships meshoptimizer's clusteriser and simplifier in `examples/jsm/libs` as building blocks.

24.10 Large worlds and precision

⚠ PITFALL · **Floating-point precision far from the origin**

GPUs compute positions in 32-bit floats, which have 24 bits of precision. The spacing between representable values grows with magnitude: at 10 km from the origin it is about 1 mm, at 1,000 km 6 cm, at Earth's radius (6,371 km) **0.5 m**. Vertices snap to that grid, so geometry far from the origin jitters as the camera moves.

WORKED EXAMPLE · **Precision at a distance**

A float32 with exponent e has spacing 2^{e-23}. At 10,000 m, $e = 13$: $2^{-10} = 0.98$ mm. At 10^6 m, $e = 19$: $2^{-4} = 0.0625$ m. At 6,371 km, $e = 22$: $2^{-1} = 0.5$ m.

Fixes: **floating origin** (keep the camera near (0, 0, 0) and move the world instead, re-basing when the camera travels far), relative-to-eye rendering (subtract the camera position in double precision on the CPU before uploading), tiled worlds with local origins, and logarithmic or reversed depth for the huge

depth ranges (Chapter 6). Geospatial three.js work typically uses `3d-tiles-renderer` (NASA-AMMOS) for streamed, LOD-managed Cesium 3D Tiles and Google Photorealistic 3D Tiles.

✓ KEY POINTS

- ✓ Forward: lights per fragment; deferred: G-buffer then lighting per pixel; clustered: only the lights in each cluster (`ClusteredLighting` in r185).
- ✓ GI: bake when static; light probes with L2 spherical harmonics (27 numbers) for dynamic objects; SSGI and path tracing for dynamic, expensive cases.
- ✓ Reflections range from environment maps to planar mirrors, SSR and ray tracing.
- ✓ OIT (weighted blended, depth peeling, stochastic `alphaHash`) avoids sorting failures.
- ✓ Fog: linear and exp² in three.js; physically $T = e^{-\sigma_t d}$; volumetrics by raymarching.
- ✓ Sphere tracing steps by the SDF value; normals from the SDF gradient.
- ✓ Path tracing is Monte Carlo: noise $\propto 1/\sqrt{N}$, so importance sampling and denoising matter.
- ✓ Gaussian splats: covariance RSS^TR^T projected with JW, sorted and alpha-blended; `GaussianSplat` (r186) and Spark.
- ✓ Float32 spacing 2^{e-23}: use floating origins for large worlds.

24.11 Interview questions

Q **Compare forward, deferred and clustered rendering.** SENIOR}

A Forward shading evaluates all relevant lights in each object's fragment shader: simple, supports MSAA, transparency and any material, but cost scales with lights × shaded fragments, including overdraw. Deferred writes surface attributes to a G-buffer first and lights each screen pixel once per affecting light, decoupling lighting from overdraw and handling many lights, at the cost of bandwidth, no hardware MSAA, a separate forward path for transparency and a fixed material model. Clustered (Forward+) keeps forward shading but culls lights into 3D clusters of the frustum so each fragment evaluates only nearby lights, giving many-light performance while keeping MSAA and transparency; it works with deferred too.

Q **What are spherical harmonics used for in real-time rendering?** SENIOR}

A To store low-frequency lighting compactly: the irradiance arriving at a point from all directions is projected onto the first few SH basis functions (9 coefficients per channel for L2). Evaluating diffuse lighting for any normal is then a cheap polynomial. They are used for light probes and probe grids giving dynamic objects baked bounce light, for the diffuse part of environment lighting, and in Gaussian splats for view-dependent colour. They cannot represent sharp detail, which is why specular reflections use prefiltered cube maps instead.

Q **What is 3D Gaussian splatting and how does it differ from meshes?** MID

A A capture representation: millions of anisotropic 3D Gaussians, each with position, covariance (scale and rotation), opacity and view-dependent colour, optimised from photographs so that rendering them reproduces the photos. Rendering projects each Gaussian to a 2D ellipse, sorts them by depth and alpha-blends them front to back. It produces photoreal results from a phone video without modelling, but the lighting is baked in, editing and collision need extra work, files are large, and depth sorting is the main performance cost. three.js r186 has a `GaussianSplat` addon; Spark is a full renderer.

Q **A planet-scale scene jitters when the camera is far from the origin. Why and how do you fix it?** MID

A 32-bit floats have about seven significant digits, so at millions of metres from the origin the spacing between representable positions reaches tens of centimetres; vertex positions and the camera snap to that coarse grid and jitter as the camera moves. Fix it with a floating origin (keep the camera at or near the origin and translate the world, re-basing periodically), relative-to-eye transforms computed in double precision on the CPU, local tile origins, and a logarithmic or reversed depth buffer for the huge depth range.

Q **How does sphere tracing work, and when is it slow?** MID

A For each pixel a ray is marched through a signed distance field. The SDF value at the current point is the radius of an empty sphere around it, so the ray can safely advance by that amount; it repeats until the distance is below a threshold (hit) or a maximum distance or step count is reached (miss). Normals come from the SDF gradient. It is slow when rays graze surfaces (the distance stays small, so steps stay small), for complex SDFs that are expensive to evaluate per step, and at high resolution, since every pixel runs the loop.

WebXR — VR and AR on the Web

25

How immersive sessions, reference spaces, poses and per-eye views work; VR in three.js with controllers, rays and hands; AR hit testing and anchors; the maths of teleport arcs and readable text in headsets; comfort and performance budgets; and @react-three/xr.

IN THIS CHAPTER

25.1 The WebXR model
25.2 VR in three.js
25.3 Locomotion and comfort
25.4 AR: hit testing and anchors
25.5 Performance in XR
25.6 WebXR in React Three Fiber
25.7 Interview questions

25.1 The WebXR model

DEFINITION · **WebXR Device API**

The browser standard for VR and AR. A page requests an **XR session** (only from a user gesture such as a click), the browser takes over the display (headset, or a phone's camera view for AR), and on every frame it supplies the device's **pose** and one **view** per eye, each with its own view and projection matrix.

Session mode	Device	What you get
`immersive-vr`	Headsets (Quest, Vision Pro, PC VR)	Full virtual environment, stereo rendering
`immersive-ar`	AR phones (Android Chrome), passthrough headsets	Your content composited over the real world
`inline`	Any browser	A non-immersive view on the page (rarely used directly)

DEFINITION · **Reference space**

The coordinate system poses are expressed in. `viewer` (fixed to the head), `local` (origin at the starting head position), `local-floor` (origin on the floor below the start position: y = 0 is the floor, which is what three.js requests by default), `bounded-floor` (with a play-area boundary) and `unbounded` (large-scale AR). Units are metres.

DEFINITION · **Pose and views**

An `XRFrame` gives the **viewer pose** (head position and orientation) and an array of **views**: two for stereo headsets (left and right eye, offset by the interpupillary distance, about 63 mm), one for phone AR. Each view has a transform and a projection matrix, often asymmetric. three.js builds an `ArrayCamera` with one sub-camera per view and renders the scene once per eye (or once with the multiview extension).

Optional feature	Enables
`hand-tracking`	Joint poses for both hands (25 joints each)
`hit-test`	Rays against real-world surfaces (AR)
`anchors`	Points that stay fixed to the real world as tracking improves
`plane-detection`, `mesh-detection`	Detected floors, walls, tables and room meshes
`dom-overlay`	HTML UI on top of phone AR
`layers`	Compositor layers (sharp text and video, cheaper rendering)
`depth-sensing`, `light-estimation`	Real-world depth for occlusion; real lighting for matching

25.2 VR in three.js

three.js — VR with controllers, rays and grabbing JAVASCRIPT

```
import { VRButton } from "three/addons/webxr/VRButton.js";
import { XRControllerModelFactory } from "three/addons/webxr/XRControllerModelFactory.js";

renderer.xr.enabled = true;
document.body.appendChild(VRButton.createButton(renderer, { optionalFeatures: ["hand-tracking"]
}));

const factory = new XRControllerModelFactory();
const raycaster = new THREE.Raycaster();
for (let i = 0; i < 2; i++) {
  const ctrl = renderer.xr.getController(i);              // the "target ray" space: where the
pointer aims
  ctrl.addEventListener("selectstart", () => {
    raycaster.setFromXRController(ctrl);                  // ray from the controller's pose
    const hit = raycaster.intersectObjects(grabbables, false)[0];
    if (hit) { ctrl.attach(hit.object); ctrl.userData.held = hit.object; }   // attach keeps the
world pose
  });
  ctrl.addEventListener("selectend", () => {
    const held = ctrl.userData.held;
    if (held) { scene.attach(held); ctrl.userData.held = null; }
  });
  scene.add(ctrl);
  const grip = renderer.xr.getControllerGrip(i);          // the "grip" space: where the hand
holds it
  grip.add(factory.createControllerModel(grip));          // the real controller model, loaded
```

```
  on connect
    scene.add(grip);
  }
  renderer.setAnimationLoop(render);                    // required: XR drives the frame loop
```

⚠ PITFALL · **The camera belongs to the headset**

In a session three.js overwrites the camera's pose every frame from the headset. To move the player (teleport, smooth locomotion), move a parent **rig** group containing the camera and controllers, not the camera itself. Content is in metres, with the floor at y = 0 in `local-floor`.

25.2.1 Hands

`renderer.xr.getHand(i)` gives a group whose `joints` (wrist, finger tips…) follow the tracked hand; `XRHandModelFactory` renders them as spheres, boxes or a skinned mesh. A **pinch** is detected when the thumb tip and index tip are closer than about 1–2 cm, and the browser also emits `select` events for pinches, so controller code often works for hands unchanged.

25.3 Locomotion and comfort

DEFINITION · **Simulator sickness**

Discomfort caused when the eyes see motion the inner ear does not feel (vection), or when the image lags head movement. Main causes: smooth artificial movement and rotation, acceleration, dropped frames and latency, and wrong scale. Rules: never move the camera without the user's intent, keep the frame rate at the display rate, and prefer teleportation and snap turning.

Technique	Comfort	Notes
Teleport (arc + target)	High	The standard default
Snap turning (30–45° steps)	High	Instead of smooth rotation
Smooth locomotion	Low–medium	Offer as an option, add a comfort vignette
Room-scale walking	Highest	Limited by the play area

Σ FORMULA · **A teleport arc**

The pointer arc is a projectile path from the controller position $\mathbf{p}_0$ with launch speed v along the controller's forward direction:

$$\mathbf{p}(t) = \mathbf{p}_0 + \mathbf{v}\,t + \tfrac{1}{2}\mathbf{g}\,t^2$$

Sample it at small time steps, raycast between successive points, and the first hit on a valid floor is the teleport target.

WORKED EXAMPLE · **How far does the arc reach?**

Hand at height 1.2 m, $v = 7$ m/s at 45°: $v_x = v_y = 4.95$ m/s. Landing when $1.2 + 4.95t - 4.905t^2 = 0$: $t = (4.95 + \sqrt{4.95^2 + 2 \times 9.81 \times 1.2})/9.81 = 1.21$ s, so the target is $4.95 \times 1.21 = 6.0$ m away. A faster launch speed reaches farther but makes precise aiming harder.

25.3.1 Readable text and UI in a headset

Σ FORMULA · **Size from visual angle**

$$h = 2\,d\,\tan\frac{\theta}{2}$$

An object of height h at distance d subtends visual angle θ. Size UI in angles, not pixels: headset pixels are large, so text needs to subtend roughly 1–1.5° for comfortable reading.

WORKED EXAMPLE · **Sizing a menu label**

At 1.5 m, 1° of visual angle is $2 \times 1.5 \times \tan 0.5^\circ = 2.6$ cm, and 1.5° is 3.9 cm. So a label meant to be read at arm's-length-plus (1.5 m) needs letters about 3–4 cm tall, far larger than "desktop" proportions. Place panels 1–2 m away, slightly below eye level, and curve wide panels around the user.

25.4 AR: hit testing and anchors

DEFINITION · **Hit test**

A ray cast against the device's understanding of the **real world** (detected planes and meshes), returning poses where the ray meets real surfaces. The classic AR placement loop: each frame, cast a ray from the screen centre (phone) or the controller, show a **reticle** at the hit pose, and place the object there when the user taps.

three.js — an AR placement reticle JAVASCRIPT

```
import { ARButton } from "three/addons/webxr/ARButton.js";
renderer.xr.enabled = true;
document.body.appendChild(ARButton.createButton(renderer, { requiredFeatures: ["hit-test"] }));

let hitSource = null;
renderer.xr.addEventListener("sessionstart", async () ⇒ {
  const session = renderer.xr.getSession();
  const viewerSpace = await session.requestReferenceSpace("viewer");
  hitSource = await session.requestHitTestSource({ space: viewerSpace });   // rays from the
screen centre
});
renderer.xr.getController(0).addEventListener("select", () ⇒ {
  if (reticle.visible) { const chair = chairModel.clone();
chair.position.setFromMatrixPosition(reticle.matrix); scene.add(chair); }
});
renderer.setAnimationLoop((t, frame) ⇒ {
  if (frame && hitSource) {
```

```
    const hits = frame.getHitTestResults(hitSource);
    reticle.visible = hits.length > 0;
    if (hits.length)
reticle.matrix.fromArray(hits[0].getPose(renderer.xr.getReferenceSpace()).transform.matrix);
  }
  renderer.render(scene, camera);
});
// reticle.matrixAutoUpdate = false, so the pose matrix is used as is
```

DEFINITION · **Anchor**

A pose the AR system keeps attached to the real world, updating it as tracking improves (drift correction). Objects that must stay put for a long session (a virtual sofa, a measurement, a note on a machine) should be parented to an anchor created from a hit-test result rather than placed at fixed coordinates.

25.5 Performance in XR

WORKED EXAMPLE · **Frame budgets in headsets**

72 Hz: 13.9 ms. 90 Hz: 11.1 ms. 120 Hz: 8.3 ms. And the scene is rendered **twice** (once per eye) at high resolution, on a mobile GPU for standalone headsets. Dropped frames are not just stutter here; they cause discomfort, and the system's reprojection fills gaps with warped images.

Technique	Effect
Multiview (`OVR_multiview2`)	Render both eyes in one pass (three.js uses it when available)
Fixed foveated rendering (`renderer.xr.setFoveation(1)`)	Lower resolution in the periphery
`renderer.xr.setFramebufferScaleFactor(0.8-1.0)`	Trade sharpness for speed
Compositor layers	Text and video at full sharpness, cheaply
Budget like mobile	Few draw calls, instancing, baked lighting, no heavy post-processing, KTX2
Avoid transparency and full-screen effects	Overdraw at stereo resolution is expensive

25.6 WebXR in React Three Fiber

R3F — @react-three/xr v6 JSX

```
import { Canvas } from "@react-three/fiber";
import { XR, createXRStore, XROrigin, TeleportTarget } from "@react-three/xr";

const store = createXRStore({ hand: { teleportPointer: true }, controller: { teleportPointer:
true } });
```

```
export default function App() {
  const [origin, setOrigin] = useState([0, 0, 0]);
  return (
    <>
      <button onClick={() => store.enterVR()}>Enter VR</button>
      <button onClick={() => store.enterAR()}>Enter AR</button>
      <Canvas>
        <XR store={store}>
          <XROrigin position={origin} />                    {/* the player rig */}
          <TeleportTarget onTeleport={(p) => setOrigin([p.x, 0, p.z])}>
            <mesh rotation-x={-Math.PI / 2}><planeGeometry args={[20, 20]} />
<meshStandardMaterial /></mesh>
          </TeleportTarget>
          <Gallery />                                        {/* ordinary R3F content; onClick
works with rays and pinches */}
        </XR>
      </Canvas>
    </>
  );
}
```

`createXRStore()` provides controller and hand models, ray and grab pointers, teleport pointers when enabled, Vision Pro's transient pointer and phone screen taps. It also injects the **IWER emulator** on localhost, so VR and AR can be tested on a laptop. `useXRHitTest` and `XRHitTest` handle AR hit testing; `IfInSessionMode` renders content only in VR or AR; `@react-three/uikit` builds flexbox UI panels for headsets.

✓ KEY POINTS

- ✓ Sessions: `immersive-vr`, `immersive-ar`, `inline`, requested from a user gesture.
- ✓ `local-floor` reference space: metres, floor at y = 0; one view (and projection) per eye.
- ✓ three.js: `renderer.xr.enabled`, `VRButton` / `ARButton`, `getController` (ray) and `getControllerGrip` (model), `setAnimationLoop`, move a rig not the camera.
- ✓ Comfort: teleport and snap turning, steady frame rate, correct scale; arcs are projectile paths.
- ✓ Size UI by visual angle: $h = 2d\tan(\theta/2)$; 1° ≈ 2.6 cm at 1.5 m.
- ✓ AR: hit-test sources from the viewer space → reticle → place; anchors for long-lived placement.
- ✓ Budgets: 8.3–13.9 ms rendered twice; multiview, foveation, layers, mobile-class content.
- ✓ R3F: `createXRStore` + `<XR>` + `XROrigin`, `TeleportTarget`, emulator on localhost.

25.7 Interview questions

Q What changes in a three.js app when you add VR? MID

A Enable `renderer.xr`, offer a `VRButton` (sessions need a user gesture), and drive rendering with `setAnimationLoop` because the headset controls the frame rate. The camera's pose now comes from the headset each frame, so player movement must move a parent rig. Scale must be real (metres, floor at y = 0). Input comes from controllers or hands (select/squeeze events, target-ray and grip spaces). And the scene renders twice per frame at 72–120 Hz on mobile hardware, so the performance budget becomes much tighter.

Q How does AR placement with hit testing work? MID

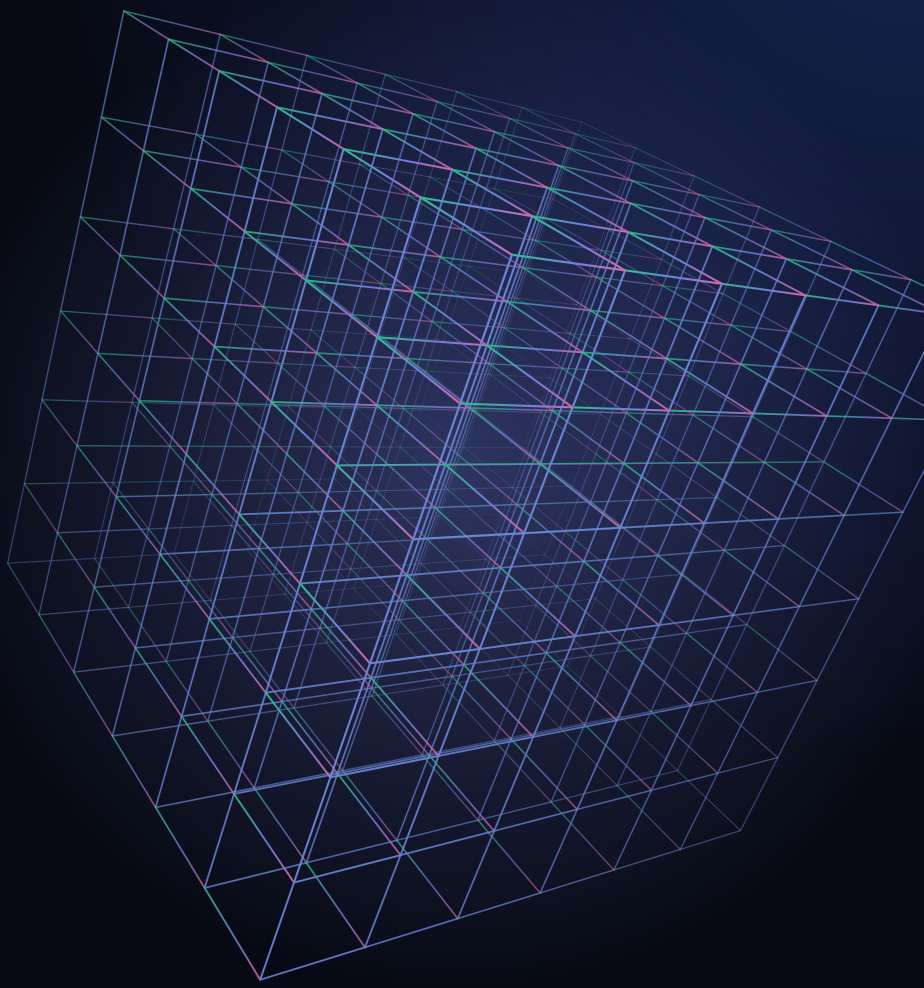
A Request an `immersive-ar` session with the `hit-test` feature, create a hit-test source from the viewer reference space (a ray from the screen centre or a controller), and each frame read `frame.getHitTestResults(source)`. The first result's pose (in the session's reference space) positions a reticle on the real surface; on select, place the object at the reticle's pose, ideally attached to an anchor created from that hit result so it stays fixed as tracking refines.

Q Why is smooth locomotion uncomfortable and what are the alternatives? JUNIOR

A The eyes perceive self-motion while the vestibular system feels none, a sensory mismatch that causes simulator sickness, worst with acceleration and rotation. Alternatives: teleportation (aim an arc, fade or blink, appear at the target), snap turning in fixed steps, comfort vignettes that narrow the view while moving, and designing experiences around room-scale walking.

Q How would you size text for a VR menu? MID

A By visual angle rather than pixels: height $h = 2d\tan(\theta/2)$. For about 1–1.5° at a reading distance of 1.5 m, letters need to be roughly 2.6–3.9 cm tall. Place panels 1–2 m away, slightly below eye level, facing the user (or curved), with high contrast; use SDF text (drei `Text`, uikit) or compositor layers so glyphs stay sharp, and avoid thin fonts that shimmer at headset resolution.



PART

Interview Preparation

The Interview Question Bank

26

Everything you need on the day. How to structure answers, a rapid-fire revision table, coding questions with complete code, debugging scenarios, system-design questions for 3D web products, senior-level conceptual questions, and behavioural preparation. Together with the questions at the end of each chapter, this notebook holds over 200 worked answers.

IN THIS CHAPTER

26.1 How to answer a 3D interview question
26.2 Rapid-fire revision
26.3 Coding questions
26.4 Debugging scenarios
26.5 System design questions
26.6 Senior-level conceptual questions
26.7 Behavioural and portfolio preparation

26.1 How to answer a 3D interview question

A RELIABLE ANSWER STRUCTURE

1. **Define** the thing in one sentence (what it is)
2. Say **why it exists**: the problem it solves
3. Explain **how it works**, with the maths or the pipeline stage if relevant
4. Show **how you do it in three.js / R3F**: the class, the property, the hook
5. Name the **trade-offs** and failure modes
6. Give a **real example**: something you built, measured or fixed

TIP · **What interviewers listen for**

Juniors are expected to know the scene/camera/renderer model, materials, lights, loading and the render loop. Mid-level developers should talk about draw calls, disposal, instancing, raycasting edge cases and shaders. Seniors are judged on diagnosing performance with evidence, rendering-pipeline knowledge (colour management, shadows, post-processing, WebGPU), architecture of large apps, and trade-offs. Saying "I would measure first with `renderer.info` and a GPU timer" is often worth more than listing tricks.

26.2 Rapid-fire revision

Question	Short answer
Three objects every three.js app needs?	Scene, camera, renderer (plus a mesh = geometry + material)
Units and up axis?	Metres by convention; right-handed, +Y up, camera looks down −Z
`fov` units?	Degrees, vertical; everything else uses radians
What is a draw call?	One CPU command to draw a geometry with a material; ~1 per visible mesh
How to reduce draw calls?	Instancing, BatchedMesh, merging, shared materials, atlases
Why multiply by delta?	Frame-rate independence
Default material that needs no light?	`MeshBasicMaterial`
Why is my model black?	No lights / no environment; metals need an environment
sRGB textures?	Colour maps only: `texture.colorSpace = SRGBColorSpace`
Default output colour space?	`SRGBColorSpace`
What is tone mapping?	Compressing HDR values into displayable [0,1]
Tone mapper for product colours?	`NeutralToneMapping`
Lambert's law?	Diffuse ∝ max(0, n·l)
Blinn–Phong half vector?	h = normalise(l + v)
GGX roughness parameter?	α = roughness²
F0 of plastic/glass?	≈ 0.04
Metals and diffuse?	Metals have no diffuse; base colour is F0
What is PMREM?	Pre-filtered, mip-mapped environment map for roughness-based reflections
Model matrix order?	M = T·R·S (scale, then rotate, then translate)
Local vs world matrix?	`matrix` vs `matrixWorld` = parent.matrixWorld · matrix
Normal matrix?	Inverse-transpose of the model-view 3×3
Gimbal lock?	Euler middle angle ±90° aligns two axes
Quaternion for θ about axis k?	(k sin θ/2, cos θ/2)
Slerp vs lerp?	Constant angular speed on the arc vs uneven speed on the chord
Why does z-fighting happen?	Depth precision; raise `near` first
Reversed-Z?	Float depth with near = 1, far = 0: near-uniform precision

Question	Short answer
Frustum culling uses?	Bounding spheres vs six planes
Mip level?	log2(texels per pixel)
Mipmap memory overhead?	+1/3
4K RGBA texture VRAM?	≈ 85 MiB with mips
KTX2/Basis?	One file transcoded to the GPU's compressed format
Raycasting cost?	Bounding sphere/box then every triangle; BVH for big meshes
R3F events pass through objects?	Yes, call `e.stopPropagation()`
setState in useFrame?	Never; mutate refs
useFrame priority > 0?	You render manually; R3F stops its own render
Changing `args`?	Re-creates the object
Fixed timestep?	Accumulator + interpolation for stable physics
Semi-implicit Euler?	Update v, then x with new v; stable
Shadow acne fix?	Small bias + normalBias; tight shadow camera
Point light shadows cost?	Six renders (cube)
Bloom steps?	Bright pass → mip-chain blur → add
MSAA with post-processing?	Needs a multisampled target or SMAA/TAA
Separable blur cost?	2N instead of N^2 samples
TSL?	Shaders as JavaScript node graphs → WGSL/GLSL
WebGPU compute workgroup default in three.js?	64
What replaced Clock?	`THREE.Timer` (core since r179; Clock deprecated r183)
What replaced PostProcessing class?	`RenderPipeline` (r183)
What replaced RGBELoader?	`HDRLoader` (r180)
PCFSoftShadowMap?	Removed in r186; PCFShadowMap is soft
New light in r186?	`SunLight` with cascaded shadows
Gaussian splats in three.js?	`GaussianSplat` addon (r186, WebGPURenderer)
WebXR frame loop?	`renderer.setAnimationLoop`
Move the player in VR?	Move a rig parent, not the camera

26.3 Coding questions

Q **Set up three.js inside a plain React component (no R3F) without leaks or duplicate loops.** MID

A Create everything in an effect, start one animation loop, and undo everything in the cleanup (React 19 Strict Mode runs effects twice in development, which exposes missing cleanup).

React + three.js — correct lifecycle JSX

```jsx
function Viewer() {
  const host = useRef(null);
  useEffect(() => {
    const el = host.current;
    const renderer = new THREE.WebGLRenderer({ antialias: true });
    renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
    el.appendChild(renderer.domElement);
    const scene = new THREE.Scene();
    const camera = new THREE.PerspectiveCamera(50, 1, 0.1, 100);
    camera.position.set(0, 1, 4);
    const mesh = new THREE.Mesh(new THREE.TorusKnotGeometry(0.8, 0.25, 128, 16), new
THREE.MeshNormalMaterial());
    scene.add(mesh);

    const ro = new ResizeObserver(([e]) => {
      const { width, height } = e.contentRect;
      renderer.setSize(width, height, false);
      camera.aspect = width / height; camera.updateProjectionMatrix();
    });
    ro.observe(el);
    const timer = new THREE.Timer();
    renderer.setAnimationLoop((t) => { timer.update(t); mesh.rotation.y +=
timer.getDelta(); renderer.render(scene, camera); });

    return () => {                                  // runs on unmount (and between
Strict Mode double-mounts)
      renderer.setAnimationLoop(null);
      ro.disconnect();
      mesh.geometry.dispose(); mesh.material.dispose();
      renderer.dispose();
      el.removeChild(renderer.domElement);
    };
  }, []);
  return <div ref={host} style={{ width: "100%", height: 400 }} />;
}
```

Q Render a 20×20×20 grid of cubes in one draw call, coloured by position, and highlight the cube under the mouse. MID}

A

three.js — instanced grid with per-instance hover JAVASCRIPT

```
const N = 20, COUNT = N ** 3;
const mesh = new THREE.InstancedMesh(new THREE.BoxGeometry(0.8, 0.8, 0.8), new
THREE.MeshStandardMaterial(), COUNT);
const m = new THREE.Matrix4(), c = new THREE.Color();
let i = 0;
for (let x = 0; x < N; x++) for (let y = 0; y < N; y++) for (let z = 0; z < N; z++,
i++) {
  mesh.setMatrixAt(i, m.makeTranslation(x - N / 2, y - N / 2, z - N / 2));
  mesh.setColorAt(i, c.setRGB(x / N, y / N, z / N));        // linear values
}
mesh.computeBoundingSphere();                                 // so culling and
raycasting see all instances
scene.add(mesh);

let hovered = -1;
const base = new THREE.Color(), white = new THREE.Color(1, 1, 1);
canvas.addEventListener("pointermove", (e) => {
  raycaster.setFromCamera(toNdc(e), camera);
  const hit = raycaster.intersectObject(mesh)[0];
  const id = hit ? hit.instanceId : -1;
  if (id === hovered) return;
  if (hovered >= 0) { mesh.setColorAt(hovered, base); }      // restore the previous
colour
  if (id >= 0) { mesh.getColorAt(id, base); mesh.setColorAt(id, white); }
  hovered = id;
  mesh.instanceColor.needsUpdate = true;
});
```

One draw call for 8,000 cubes; hover costs one raycast per pointer move (add three-mesh-bvh or a coarser proxy if the geometry were complex).

Q **Write a fragment shader that draws an anti-aliased ring.** MID}

A

GLSL — ring with fwidth anti-aliasing GLSL

```glsl
varying vec2 vUv;
uniform vec3 uColor;
uniform float uRadius;      // e.g. 0.35
uniform float uThickness;   // e.g. 0.05
void main() {
  vec2 p = vUv - 0.5;
  float d = abs(length(p) - uRadius) - uThickness * 0.5;   // SDF of a ring
  float aa = fwidth(d);
  float alpha = 1.0 - smoothstep(-aa, aa, d);
  if (alpha < 0.01) discard;
  gl_FragColor = vec4(uColor, alpha);
}
```

The ring is the set of points whose distance from the circle of radius `uRadius` is under half the thickness; `fwidth` makes the edge one pixel wide at any scale. Use `transparent: true` on the material.

Q **Build a custom BufferGeometry for a parametric surface with correct normals and UVs.** SENIOR}

A

three.js — parametric surface (a wavy sheet) JAVASCRIPT

```javascript
function waveSurface(segU = 64, segV = 64, f = (u, v) => new THREE.Vector3(u * 4 - 2,
Math.sin(u * 6) * Math.cos(v * 6) * 0.2, v * 4 - 2)) {
  const pos = [], uv = [], idx = [];
  for (let j = 0; j ⩽ segV; j++) for (let i = 0; i ⩽ segU; i++) {
    const u = i / segU, v = j / segV, p = f(u, v);
    pos.push(p.x, p.y, p.z); uv.push(u, v);
  }
  const row = segU + 1;
  for (let j = 0; j < segV; j++) for (let i = 0; i < segU; i++) {
    const a = j * row + i, b = a + 1, c = a + row, d = c + 1;
    idx.push(a, c, b, b, c, d);              // counter-clockwise when seen from +Y
  }
  const g = new THREE.BufferGeometry();
  g.setAttribute("position", new THREE.Float32BufferAttribute(pos, 3));
  g.setAttribute("uv", new THREE.Float32BufferAttribute(uv, 2));
  g.setIndex(idx);
  g.computeVertexNormals();                  // smooth normals from the shared,
indexed vertices
  g.computeBoundingSphere();
  return g;
}
```

Check the winding: for the cell corners $a = (u, v)$, $c = (u, v + \Delta)$, $b = (u + \Delta, v)$, the normal $(c - a) \times (b - a)$ points along $+Y$ for a flat sheet (here $+Z$ in v maps to $+z$ and u to $+x$: $\hat{\mathbf{z}} \times \hat{\mathbf{x}} = \hat{\mathbf{y}}$).

Q **Implement drag-and-drop of objects on the floor in R3F.** MID}

A

R3F — draggable object on the ground plane JSX

```jsx
const floor = new THREE.Plane(new THREE.Vector3(0, 1, 0), 0);
const hit = new THREE.Vector3(), offset = new THREE.Vector3();

function Draggable({ children, ...props }) {
  const ref = useRef();
  const controls = useThree((s) => s.controls);     // set by <OrbitControls
makeDefault />
  const [dragging, setDragging] = useState(false);
  return (
    <group ref={ref} {...props}
      onPointerDown={(e) => {
        e.stopPropagation();
        e.target.setPointerCapture(e.pointerId);    // keep receiving moves
        e.ray.intersectPlane(floor, hit);
        offset.copy(ref.current.position).sub(hit);
        if (controls) controls.enabled = false;
        setDragging(true);                           // a rare state change: fine
      }}
      onPointerMove={(e) => {
        if (!dragging) return;
        if (e.ray.intersectPlane(floor, hit))
ref.current.position.copy(hit).add(offset);   // mutate, no setState
      }}
      onPointerUp={(e) => { e.target.releasePointerCapture(e.pointerId); if (controls)
controls.enabled = true; setDragging(false); }}>
      {children}
    </group>
  );
}
```

Q Write a camera shake that decays smoothly and does not depend on frame rate. MID}

A

three.js — trauma-based camera shake JAVASCRIPT

```
let trauma = 0;                                   // 0..1, add on hits: trauma =
Math.min(1, trauma + 0.5)
const base = new THREE.Vector3(), jitter = new THREE.Vector3();   // allocated once,
reused every frame
function updateShake(camera, t, dt) {
  trauma = Math.max(0, trauma - dt * 1.5);        // decays in about 0.7 s
  const s = trauma * trauma;                      // squared: small hits barely
shake, big ones a lot
  jitter.set(
    s * 0.15 * Math.sin(t * 37.1),                // incommensurate frequencies
avoid an obvious loop
    s * 0.15 * Math.sin(t * 43.7 + 1.3),
    0);
  camera.position.copy(base).add(jitter);
  camera.rotation.z = s * 0.05 * Math.sin(t * 29.3 + 2.1);
}
```

Use smooth noise (simplex) instead of sines for a more organic feel; drei's `<CameraShake>` packages the same idea for R3F.

Q Make an object always face the camera (a billboard) but stay upright. JUNIOR}

A

three.js — cylindrical billboard JAVASCRIPT

```
const target = new THREE.Vector3();
function faceCamera(obj, camera) {
  target.copy(camera.position);
  target.y = obj.position.y;          // ignore height difference: rotate about Y
only
  obj.lookAt(target);                 // Object3D.lookAt points +Z at the target for
non-camera objects
}
```

For a full (spherical) billboard copy the camera's quaternion: `obj.quaternion.copy(camera.quaternion)`. `THREE.Sprite`, drei's `<Billboard>` and TSL's `billboarding()` do this for you; in shaders, building the quad from the camera's right and up vectors is cheapest for particles.

Q Take a high-resolution screenshot of the canvas. MID}

A

three.js — screenshot at 2× without changing the visible canvas JAVASCRIPT

```
function screenshot(scale = 2) {
  const size = renderer.getSize(new THREE.Vector2());
  const dpr = renderer.getPixelRatio();
  renderer.setPixelRatio(dpr * scale);
  renderer.setSize(size.x, size.y, false);
  renderer.render(scene, camera);                    // render and read in the same
task
  const url = renderer.domElement.toDataURL("image/png");
  renderer.setPixelRatio(dpr);
  renderer.setSize(size.x, size.y, false);
  return url;
}
```

The drawing buffer is cleared after compositing, so read it immediately after rendering (or set `preserveDrawingBuffer: true`, which costs performance). Very large shots may exceed the maximum texture size; render tiles with `camera.setViewOffset` and stitch them.

26.4 Debugging scenarios

Symptom	Likely causes	First checks
Blank / black canvas	Camera inside or looking away from objects; object outside `near` – `far`; no `render()` call; canvas size 0	Add an `AxesHelper`; log `camera.position`; check `renderer.domElement` size; console errors
Model loads but is invisible	Wrong scale (centimetres → 100× too big, camera inside it); far away; `visible` false; culled by stale bounds	`Box3().setFromObject(model)` size and centre; frame the camera to it
Model is black	No light, no environment; metal without environment; normals missing or inverted	Add `scene.environment`; `MeshNormalMaterial` to inspect normals
Colours washed out or too dark	Colour textures not sRGB; double conversion with post-processing; custom shader missing colorspace chunk	`texture.colorSpace`; only one `OutputPass`
Texture upside down on a glTF	`flipY` mismatch	glTF textures need `flipY = false`
Flickering surfaces	Z-fighting	Raise `near`; separate geometry; `polygonOffset`
Shadows missing	`renderer.shadowMap.enabled`, `light.castShadow`, `mesh.castShadow/receiveShadow`; object outside shadow camera	`CameraHelper(light.shadow.camera)`

Symptom	Likely causes	First checks
Stripy shadows	Shadow acne	Bias / normalBias; tighter shadow box
Transparent objects vanish or flicker	Sorting; `depthWrite`	`depthWrite: false`; `renderOrder`; `alphaTest`
Frame rate drops over time	Memory leak (no dispose), growing arrays, listeners added repeatedly	`renderer.info.memory` over time; heap snapshots
Stutter when something first appears	Shader compilation, texture upload	`compileAsync`, pre-upload (`initTexture`), drei `<Preload all />`
R3F hook error "R3F: Hooks can only be used within the Canvas component"	`useThree` / `useFrame` used outside `<Canvas>`	Move the component inside, or pass data via a store
R3F clicks hit the wrong object	No `stopPropagation`; invisible objects with handlers; displaced geometry	Inspect `e.intersections`
Everything renders twice per frame	Manual render plus R3F's render	Positive `useFrame` priority when rendering yourself
Works on desktop, crashes on iPhone	VRAM (textures), too many contexts, WebGL feature differences	KTX2, smaller textures, one canvas, test on device

26.5 System design questions

Q **Design a 3D product configurator for a furniture e-commerce site (millions of visitors).** SENIOR}

A **Goals**: fast first view, accurate colours, smooth option changes, works on mid-range phones, shareable configurations.

Assets: models built in Blender with clean UVs and separate material slots; an automated pipeline (gltf-transform: dedupe, weld, simplify, Meshopt, KTX2 with UASTC for normals) producing per-product `.glb` files under 3–5 MB; material swatches as separate small KTX2 sets loaded on demand; CDN with long cache headers and content hashing.

Rendering: one canvas; `NeutralToneMapping`; a neutral studio HDR (small, prefiltered once); contact shadows instead of real-time shadows; `frameloop="demand"`; DPR capped at 2 and adaptive. Pre-create materials for the current options and compile them during idle time to avoid hitches.

State: configuration in a store (and the URL for sharing); the DOM UI (option pickers, price) is server-rendered and accessible; the 3D view subscribes to the store.

Robustness: poster image fallback if WebGL fails; context-loss handling; analytics on load time and FPS; snapshot rendering server-side or client-side for carts and social sharing; AR via `USDZExporter` /Quick Look on iOS and WebXR or Scene Viewer on Android.

Q **Design a web digital twin of a factory floor with live sensor data.** SENIOR}

A **Scene**: the building as merged static geometry with baked lighting; machines as instanced or batched meshes by type; detailed models only for the selected machine (LOD and on-demand loading). **Data**: a WebSocket or MQTT-over-WebSocket feed into a normalised store keyed by machine ID; updates at sensor rates are buffered and applied once per frame (no React re-render per message); status drives instance colours and small HTML labels only for visible, nearby machines. **Interaction**: picking via `instanceId`, a BVH for complex meshes, a details panel in DOM. **Time**: a timeline scrubber that replays history from a time-series API, interpolating states. **Scale**: frustum culling per instance (`BatchedMesh`), distance-based label culling, workers for parsing and aggregation. **Ops**: role-based access, offline mode, alarms highlighted with emissive pulses and audio.

Q **Design a viewer for a 200-million-point LiDAR scan in the browser.** SENIOR}

A You cannot load 200 M points (≈ 3 GB as float positions and colours) at once. Preprocess into a spatial hierarchy (an octree with progressively denser levels, as in Potree, or Cesium 3D Tiles / COPC), stored as compressed chunks on a CDN. At runtime, traverse the hierarchy each frame, choosing nodes by projected screen size within a point budget (for example 3–5 M points), stream them with priority by screen-space error, decode in workers, and render as `Points` with a custom shader (size attenuation, eye-dome lighting for depth perception, colour by height or intensity). Evict least recently used nodes to cap memory. Picking uses the hierarchy plus a GPU ID pass. WebGPU compute can rasterise points faster than the point primitive.

Q **Design a small multiplayer 3D game in the browser.** SENIOR}

A Authoritative server (Node or a dedicated game server) running the simulation at a fixed tick (for example 20–30 Hz); clients send inputs, not positions. Clients predict their own player locally (client-side prediction) and reconcile with server snapshots; remote players are rendered with **interpolation** about 100 ms in the past between received snapshots, with extrapolation only for short gaps. Transport: WebSockets (or WebTransport/WebRTC data channels for unreliable, unordered updates). Physics deterministic on the server (Rapier), cosmetic physics on clients. Interest management sends only nearby entities. Rendering: instanced characters, compressed assets, a fixed-step client loop for prediction with interpolated rendering.

26.6 Senior-level conceptual questions

Q How does instanced rendering work on the GPU? SENIOR}

A One draw call is issued with an instance count. The vertex shader runs for every vertex of every instance; per-instance data (a transform matrix, a colour, any custom value) comes from instanced attributes that advance once per instance instead of once per vertex (or from a storage buffer indexed by `gl_InstanceID` / `instanceIndex`). three.js's `InstancedMesh` stores a 4×4 matrix per instance (`instanceMatrix`, 64 bytes) and optional `instanceColor`, and multiplies the vertex by the instance matrix before the model-view-projection. The CPU cost is one call regardless of count; the GPU still processes every vertex.

Q What do the different needsUpdate flags do? MID}

A They tell three.js to re-upload or recompile something on the next render. `geometry.attributes.x.needsUpdate` re-uploads that buffer (optionally only `updateRanges`). `texture.needsUpdate` re-uploads the image (after drawing on a canvas, changing format or wrapping on an existing texture). `material.needsUpdate` forces the shader program to be re-evaluated and recompiled if its defines changed (adding a map, toggling flat shading or vertex colours). `instanceMatrix.needsUpdate` / `instanceColor.needsUpdate` re-upload instance data. Shadow maps use `renderer.shadowMap.needsUpdate` when `autoUpdate` is off.

Q What is the difference between `visible = false`, removing from the scene, and using layers? MID}

A `visible = false` skips rendering (and raycasting by default) of the object and its children but keeps it in the graph with its GPU resources, and its matrices still update: ideal for toggling quickly. Removing it detaches it from the graph (no matrix updates, no rendering), but GPU resources stay until disposed, so it suits objects that may return later. Layers filter per camera or raycaster: the object stays visible to cameras with matching layers, enabling selective rendering (minimaps, reflections, selection passes) without changing the scene.

Q How does three.js decide when to compile a new shader program? SENIOR}

A For each material it computes a program key (cache key) from the material type and every parameter that affects the generated code: which maps are present, fog, vertex colours, skinning and morphs, number and types of lights and shadows, tone mapping, output colour space, precision, custom `onBeforeCompile` keys. Materials with the same key share one program; a new key means generating, compiling and linking a program, which is slow the first time. That is why adding a light to a scene recompiles every lit material, and why pre-warming with `renderer.compile` / `compileAsync` during loading avoids hitches.

Q How do you stay current with three.js? JUNIOR}

A Read each release's notes and the migration guide (releases arrive every couple of months and deprecations last about ten releases), follow the forum and examples (especially the WebGPU ones), test upgrades against screenshots because PBR and shadows changed in recent releases, and use the official `llms.txt` documentation when working with AI assistants, which otherwise suggest deprecated APIs such as `Clock` or `PostProcessing`.

26.7 Behavioural and portfolio preparation

- Prepare **three stories** with numbers: a performance win ("draw calls 1,200 → 40 with BatchedMesh; mid-range Android 24 → 58 fps"), a hard bug ("textures leaked on route changes; added disposal and a memory test"), and a product impact ("configurator raised add-to-cart by X%").
- Bring a **portfolio** of live demos that load fast on a phone; show the code for one and be ready to explain every line, including what you would improve.
- Know your **limits** honestly: if you have not written WGSL, say so and explain how you would approach it (TSL, docs, examples).
- Ask the interviewer about their **stack** (WebGL or WebGPU, R3F or vanilla, asset pipeline, target devices); it shows you think about real constraints.

✓ KEY POINTS

- ✓ Answer with: definition → why → how → three.js/R3F → trade-offs → example.
- ✓ Revise with the rapid-fire table; practise the coding questions by typing them out.
- ✓ For debugging questions, describe the evidence you would collect before the fix.
- ✓ For system design: assets pipeline, rendering choices, state and data flow, performance budgets, robustness and fallbacks.

Cheat Sheets and Formula Sheet

27

The whole notebook compressed for revision. Every key formula on a few pages, the numbers worth memorising, three.js, R3F and TSL quick references, the recent-release changes, a debugging checklist, and the best places to keep learning.

IN THIS CHAPTER

27.1 Formula sheet
27.2 Numbers to remember
27.3 three.js quick reference
27.4 R3F quick reference
27.5 TSL quick reference
27.6 What changed recently (r180–r186)
27.7 Debugging checklist
27.8 Where to keep learning

27.1 Formula sheet

27.1.1 Vectors and transforms

Concept	Formula
Length, normalise	$\|\mathbf{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$, $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$
Dot product	$\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i = \|\mathbf{a}\|\|\mathbf{b}\| \cos\theta$
Cross product	$(a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x)$, length $= \|\mathbf{a}\|\|\mathbf{b}\| \sin\theta$
Triangle normal, area	$\hat{\mathbf{n}} = \text{norm}((B-A) \times (C-A))$, area $= \frac{1}{2}\|(B-A) \times (C-A)\|$
Projection onto $\hat{\mathbf{n}}$ / plane	$(\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$ / $\mathbf{v} - (\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$
Reflection	$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$
Lerp, smoothstep	$a + t(b-a)$; $t^2(3-2t)$
Frame-rate-independent damping	$x \leftarrow \text{lerp}(x, x_t, 1 - e^{-\lambda \Delta t})$
Point–plane distance	$\hat{\mathbf{n}} \cdot \mathbf{p} + d$
Model matrix	$M = T\,R\,S$; world $= M_{\text{parent}} M_{\text{local}}$
Inverse TRS	$S^{-1} R^T T^{-1}$
Normal matrix	$(M_{3\times3}^{-1})^T$
Clip position	$P\,V\,M\,\mathbf{p}$

27.1.2 Rotations

Concept	Formula
Rotation about Y	$R_y = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}$
Rodrigues	$\mathbf{v}\cos\theta + (\hat{\mathbf{k}} \times \mathbf{v})\sin\theta + \hat{\mathbf{k}}(\hat{\mathbf{k}} \cdot \mathbf{v})(1 - \cos\theta)$
Unit quaternion	$q = (\hat{\mathbf{k}}\sin\frac{\theta}{2}, \cos\frac{\theta}{2})$; q and $-q$ equal
Product	$(w_1\mathbf{u}_2 + w_2\mathbf{u}_1 + \mathbf{u}_1 \times \mathbf{u}_2,\ w_1w_2 - \mathbf{u}_1 \cdot \mathbf{u}_2)$
Rotate a vector	$\mathbf{v} + 2w(\mathbf{u} \times \mathbf{v}) + 2\mathbf{u} \times (\mathbf{u} \times \mathbf{v})$
Slerp	$\frac{\sin((1-t)\Omega)}{\sin\Omega}q_0 + \frac{\sin(t\Omega)}{\sin\Omega}q_1$, $\cos\Omega = q_0 \cdot q_1$
Look-at basis	$\hat{\mathbf{z}} = \mathrm{norm}(E - T)$, $\hat{\mathbf{x}} = \mathrm{norm}(\mathbf{up} \times \hat{\mathbf{z}})$, $\hat{\mathbf{y}} = \hat{\mathbf{z}} \times \hat{\mathbf{x}}$

27.1.3 Cameras and projection

Concept	Formula
Visible height at distance d	$h = 2d\tan(\mathrm{fov}/2)$, $w = h \cdot \mathrm{aspect}$
Fit a sphere	$d = r/\sin(\mathrm{fov}/2)$
Horizontal FOV	$\tan\frac{\mathrm{fov}_h}{2} = \mathrm{aspect}\tan\frac{\mathrm{fov}_v}{2}$
Perspective P	$\mathrm{diag}(s/a, s)$, z row $\left(\frac{f+n}{n-f}, \frac{2fn}{n-f}\right)$, $w = -z$, $s = 1/\tan(\mathrm{fov}/2)$
NDC depth	$z_{\mathrm{ndc}} = \frac{(f+n)d - 2fn}{(f-n)d}$ (hyperbolic)
Pointer → NDC	$x = 2u - 1$, $y = 1 - 2v$
Spherical coordinates	$(r\sin\phi\sin\theta,\ r\cos\phi,\ r\sin\phi\cos\theta)$
On-screen size	$h_{\mathrm{px}} \approx rH/(d\tan(\mathrm{fov}/2))$

27.1.4 Rasterisation and colour

Concept	Formula
Barycentrics	λ_i = opposite sub-triangle area / total area
Perspective-correct varying	$\frac{\sum \lambda_i a_i/w_i}{\sum \lambda_i/w_i}$
Alpha blending	$\alpha C_s + (1-\alpha)C_d$
sRGB encode	$1.055L^{1/2.4} - 0.055$ ($12.92L$ below 0.0031308)
Exposure	$L \cdot 2^{\mathrm{EV}}$

Concept	Formula
Reinhard	$x/(1+x)$
Mip level	$\log_2(\text{texels per pixel})$
Texture VRAM	$w \cdot h \cdot \text{bytes} \cdot \frac{4}{3}$
Normal map decode	$\mathbf{n} = 2\mathbf{c} - 1$, world $= [T\ B\ N]\,\mathbf{n}$

27.1.5 Lighting and PBR

Concept	Formula
Lambert	$k_d E \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$; BRDF $= c/\pi$
Phong / Blinn–Phong	$(\hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^{\alpha}$ / $(\hat{\mathbf{n}} \cdot \hat{\mathbf{h}})^{\alpha'}$, $\hat{\mathbf{h}} = \text{norm}(\hat{\mathbf{l}} + \hat{\mathbf{v}})$
Rendering equation	$L_o = L_e + \int_\Omega f_r L_i (\hat{\mathbf{n}} \cdot \boldsymbol{\omega}_i) d\boldsymbol{\omega}_i$
Cook–Torrance	$\frac{DFG}{4(\hat{\mathbf{n}} \cdot \hat{\mathbf{l}})(\hat{\mathbf{n}} \cdot \hat{\mathbf{v}})}$
GGX	$\frac{\alpha^2}{\pi((\hat{\mathbf{n}} \cdot \hat{\mathbf{h}})^2(\alpha^2 - 1) + 1)^2}$, $\alpha = r^2$
Schlick Fresnel	$F_0 + (1 - F_0)(1 - \hat{\mathbf{v}} \cdot \hat{\mathbf{h}})^5$, $F_0 = \left(\frac{n-1}{n+1}\right)^2$
Metal–roughness	$F_0 = \text{mix}(0.04, \text{base}, m)$, diffuse $= \text{base}(1 - m)$
Point-light falloff (three.js)	$1/d^{\text{decay}} \times \text{saturate}(1 - (d/d_{\max})^4)^2$
Spot cone	$\text{smoothstep}(\cos a, \cos(a(1-p)), \cos\theta)$
Hemisphere light	mix(ground, sky, $\frac{1}{2}n_y + \frac{1}{2}$)
VSM (Chebyshev)	$\sigma^2/(\sigma^2 + (t - \mu)^2)$
CSM split	$\lambda n(f/n)^{i/N} + (1-\lambda)(n + (f-n)i/N)$
Fog	linear $\frac{d - \text{near}}{\text{far} - \text{near}}$; exp² $1 - e^{-(\rho d)^2}$; Beer–Lambert $e^{-\sigma d}$

27.1.6 Intersections, physics and animation

Concept	Formula
Ray–plane	$t = -(\hat{\mathbf{n}} \cdot \mathbf{o} + d)/(\hat{\mathbf{n}} \cdot \hat{\mathbf{d}})$
Ray–sphere	$t = -b \pm \sqrt{b^2 - c}$, $b = \mathbf{m} \cdot \hat{\mathbf{d}}$, $c = \mathbf{m} \cdot \mathbf{m} - r^2$
Slab test	$t_{\text{near}} = \max\min(t_1, t_2)$, $t_{\text{far}} = \min\max(t_1, t_2)$
Möller–Trumbore	$u = \frac{\mathbf{s} \cdot \mathbf{p}}{\det}$, $v = \frac{\hat{\mathbf{d}} \cdot \mathbf{q}}{\det}$, $t = \frac{\mathbf{e}_2 \cdot \mathbf{q}}{\det}$
Semi-implicit Euler	$v \mathrel{+}= a\,h$; $x \mathrel{+}= v\,h$

Concept	Formula
Collision impulse	$j = \frac{(1+e)v_n}{1/m_1 + 1/m_2}$
Torque	$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$
Spring	$\ddot{x} = -k(x - x_t) - c\dot{x}$, critical $c = 2\sqrt{k}$
Linear blend skinning	$\sum w_i M_i B_i^{-1} \mathbf{v}$
Two-bone IK	$\cos C = \frac{a^2 + b^2 - d^2}{2ab}$
Teleport arc	$\mathbf{p}_0 + \mathbf{v}t + \frac{1}{2}\mathbf{g}t^2$
Monte Carlo error	$\propto 1/\sqrt{N}$
Splat blending	$\sum c_i \alpha_i \prod_{j<i}(1 - \alpha_j)$

27.2 Numbers to remember

Number	Meaning
16.67 / 11.11 / 8.33 ms	Frame budget at 60 / 90 / 120 Hz
1 unit = 1 m	three.js, glTF, WebXR and physics convention
0.04	F_0 of most dielectrics (IOR 1.5)
0.214 / 0.735	sRGB 0.5 → linear; linear 0.5 → sRGB
1/π ≈ 0.318	White Lambertian BRDF
4/3	Mipmap memory factor
85 MiB	A 4K RGBA texture with mips
16.8 MB	A 2048² shadow map
4	Max bone influences per vertex
24	Vertices in a `BoxGeometry`
64	Default compute workgroup size in three.js
≈16	Live WebGL contexts per page (typical limit)
100–200	Draw calls per frame on a mid-range phone
≤ 2	Sensible device pixel ratio cap
63 mm	Typical interpupillary distance
1° ≈ 2.6 cm at 1.5 m	Visual-angle sizing in VR
9 × 3	L2 spherical-harmonic coefficients (RGB)

Number	Meaning
$2^{-23} \times 2^{e}$	Float32 spacing at magnitude 2^{e}

27.3 three.js quick reference

Task	Code
Renderer	`new THREE.WebGLRenderer({ antialias: true })` ; `setPixelRatio(Math.min(devicePixelRatio, 2))` ; `setAnimationLoop(fn)`
WebGPU	`import * as THREE from "three/webgpu"` ; `new THREE.WebGPURenderer()` ; `await renderer.init()`
Camera	`new THREE.PerspectiveCamera(fov, aspect, near, far)` ; `updateProjectionMatrix()` after changes
Time	`const timer = new THREE.Timer(); timer.update(t); timer.getDelta()`
Mesh	`new THREE.Mesh(geometry, material)` ; `castShadow` , `receiveShadow`
Environment	`scene.environment = hdrTexture` (equirect + `EquirectangularReflectionMapping`)
Load glTF	`new GLTFLoader().setDRACOLoader(d).setKTX2Loader(k).setMeshoptDecoder(M).loadAsync(url)`
Animation	`const mixer = new THREE.AnimationMixer(root); mixer.clipAction(clip).play(); mixer.update(dt)`
Raycast	`raycaster.setFromCamera(ndc, camera); raycaster.intersectObjects(list, true)`
Instancing	`new THREE.InstancedMesh(geo, mat, n); setMatrixAt(i, m); instanceMatrix.needsUpdate = true`
Batching	`new THREE.BatchedMesh(maxInst, maxVerts, maxIdx, mat); addGeometry; addInstance; setMatrixAt`
Shadows	`renderer.shadowMap.enabled = true; light.castShadow = true; light.shadow.mapSize.set(2048, 2048)`
Colour space	`texture.colorSpace = THREE.SRGBColorSpace` (colour maps only)
Tone mapping	`renderer.toneMapping = THREE.ACESFilmicToneMapping` / `AgXToneMapping` / `NeutralToneMapping`
Post (WebGL)	`EffectComposer` + `RenderPass` + effects + `OutputPass`
Post (WebGPU)	`new THREE.RenderPipeline(renderer); pipeline.outputNode = pass(scene, camera)…`
Dispose	`geometry.dispose(); material.dispose(); texture.dispose(); renderTarget.dispose()`
Stats	`renderer.info.render.calls` / `.triangles` ; `renderer.info.memory`

27.4 R3F quick reference

Task	Code
Canvas	`<Canvas camera={{ position: [0, 1, 5], fov: 50 }} dpr={[1, 2]} shadows frameloop="demand">`
Element	`<mesh position={[x, y, z]} rotation-y={a} scale={2}>`
Constructor args	`<boxGeometry args={[1, 1, 1]} />` (changing args re-creates)
Attach	`<color attach="background" args={["#000"]} />`
Existing object	`<primitive object={gltf.scene} />`
Custom class	`extend({ MyThing })` → `<myThing />`
Per frame	`useFrame((state, delta) ⇒ { ref.current.rotation.y += delta; })`
Store	`const camera = useThree((s) ⇒ s.camera)`
Loading	`useGLTF(url)`, `useTexture(url)`, `useLoader(Loader, url)` in `<Suspense>`
Events	`onClick`, `onPointerOver`, `onPointerMissed`; `e.stopPropagation()`; `e.point`, `e.instanceId`
Manual render	`useFrame(() ⇒ pipeline.render(), 1)`
WebGPU	`gl={async (props) ⇒ { const r = new THREE.WebGPURenderer(props); await r.init(); return r; }}`
Types (v9)	`ThreeElements["mesh"]`, `ThreeElement<typeof X>`

27.5 TSL quick reference

Task	TSL
Imports	`import { float, vec3, color, uniform, uv, time, mix, sin, Fn, If, Loop } from "three/tsl"`
Maths	`a.add(b).mul(c).sub(d).div(e)` , `.pow()` , `.clamp()` , `.length()` , `.normalize()`
Uniform	`const u = uniform(0); u.value = 1`
Material slots	`colorNode` , `positionNode` , `normalNode` , `emissiveNode` , `opacityNode` , `outputNode`
Built-ins	`positionLocal` , `positionWorld` , `normalWorld` , `uv()` , `time` , `cameraPosition` , `screenUV` , `instanceIndex`
Function	`const f = Fn(([a, b]) ⇒ a.mul(b))`
Compute	`const k = Fn(() ⇒ {…})().compute(count); renderer.compute(k)`
Storage	`instancedArray(count, "vec3")` , `.element(instanceIndex)` , `.toAttribute()`

27.6 What changed recently (r180–r186)

Release	Change
r179	`Timer` moved into core
r180	`RGBELoader` → `HDRLoader` ; `RGBMLoader` removed; new DOF node
r181	Better PBR energy conservation (rough materials brighter); `renderAsync` family deprecated; `PI2` → `TWO_PI`
r182	`PCFShadowMap` soft; `PCFSoftShadowMap` deprecated; `colorBufferType` → `outputBufferType`
r183	`PostProcessing` → `RenderPipeline` ; `Clock` deprecated; reversed depth on WebGPU; improved WebGPU shadows
r184	Node materials/TSL on `WebGLRenderer` ; FSR 1 and TAAU nodes; `compileAsync` non-blocking; env rotation aligned
r185	Clustered lighting; WebXR on WebGPU; premultiplied-alpha change; `positionLocal` excludes skinning in `positionNode`
r186	`SunLight` with cascaded shadows; Gaussian splats; `compileComputeAsync` ; reworked energy conservation; `retroreflectivity` ; `PCFSoftShadowMap` removed; `Object3D.dispose()` ; `Source` → `TextureSource` ; minified builds removed

27.7 Debugging checklist

WHEN THE SCENE LOOKS WRONG

1. Is anything rendering? Add `AxesHelper` and `GridHelper`, check console errors and canvas size
2. Is the camera looking at it? Log positions; frame the object with its bounding box
3. Is it lit? Lights and `scene.environment`; check normals with `MeshNormalMaterial`
4. Are colours right? sRGB on colour maps only; one output conversion; tone mapping chosen
5. Is it sorted and depth-tested correctly? Transparency settings, `renderOrder`, near/far
6. Are shadows configured end to end? Renderer, light, caster, receiver, shadow camera box
7. Is it fast enough? `renderer.info`, GPU timer, resolution test, draw-call test
8. Is memory stable? Dispose and watch `renderer.info.memory` over time

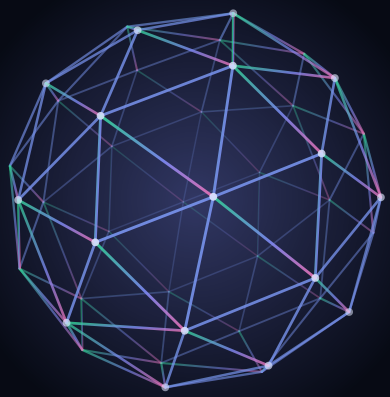
27.8 Where to keep learning

Resource	Why
three.js documentation, manual and examples: threejs.org/docs, /manual, /examples	The reference; examples show every feature in working code
three.js migration guide (GitHub wiki)	Exactly what changed in each release
threejs.org/llms.txt	Up-to-date docs for AI coding assistants
React Three Fiber docs: r3f.docs.pmnd.rs; drei: drei.docs.pmnd.rs; XR: pmndrs.github.io/xr/docs	The R3F ecosystem
discourse.threejs.org	The official forum
LearnOpenGL (learnopengl.com), especially the PBR chapters	Clear graphics theory with code
Physically Based Rendering (pbr-book.org)	The definitive free book on rendering theory
SIGGRAPH 2013 “Physically Based Shading” course (blog.selfshadow.com)	Where the real-time PBR model comes from
Inigo Quilez’s articles (iquilezles.org)	SDFs, noise, raymarching
The Book of Shaders (thebookofshaders.com)	Learning fragment shaders step by step
NVIDIA “Visualizing Depth Precision”	Depth buffers and reversed-Z
glTF (khronos.org/gltf) and gltf-transform (gltf-transform.dev)	The asset format and its pipeline

Resource	Why
three-mesh-bvh, Rapier (rapier.rs), Spark (sparkjs.dev)	Key libraries for raycasting, physics and splats
WebGPU (w3.org/TR/webgpu) and WebXR (immersive-web.github.io/webxr) specifications	The platform itself

✓ KEY POINTS

- ✓ Revise the formula sheet until you can derive each line, not just recite it.
- ✓ Memorise the numbers: frame budgets, F0 = 0.04, sRGB 0.214/0.735, 4/3 mip factor, DPR ≤ 2.
- ✓ Know what changed from r180 to r186; interviewers check whether you use current APIs.
- ✓ Work through the debugging checklist in order; most bugs are found in the first four steps.



ThreeLab Notebook · Edition 1 · October 2026 · three.js r186 · React Three Fiber 9

Part of the Techbook ThreeLab course. Study it, scribble on it, and build things.